Pandas at Work

A practical book on pandas 3.0, with one data set of call records

24 chapters · 51,825 words

Code that ships with the book printed every number in it.

Contents

Chapters

1. Series and DataFrame 4
2. Input and output 12
3. Selection 22
4. Types and memory 32
5. Missing data 42
6. Text and date cleanup 51
7. New columns 63
8. Group by 73
9. Joins 82
10. Pivot and melt 90
11. The index and MultiIndex 99
12. Dates and resampling 108
13. Windows, shifts and gaps 116
14. Ranks, running totals and sessions 124
15. Copy-on-Write and traps 132
16. Method chains and pipelines 139
17. Data validation 147
18. Performance 155
19. Data larger than memory 163
20. SQL and pandas 171
21. Plots 179
22. JSON and nested records 187
23. Features and evaluation 195
24. Field guide 203

Appendices

Appendix A. Described, not built 210
Appendix B. What Python already gives you 212

Exercises

Exercises for Chapter 1 213
Exercises for Chapter 2 214
Exercises for Chapter 3 215
Exercises for Chapter 4 216
Exercises for Chapter 5 217
Exercises for Chapter 6 218
Exercises for Chapter 7 219
Exercises for Chapter 8 220
Exercises for Chapter 9 221
Exercises for Chapter 10 222

Exercises for Chapter 11 223
Exercises for Chapter 12 224
Exercises for Chapter 13 225
Exercises for Chapter 14 226
Exercises for Chapter 15 227
Exercises for Chapter 16 228
Exercises for Chapter 17 229
Exercises for Chapter 18 230
Exercises for Chapter 19 231
Exercises for Chapter 20 232
Exercises for Chapter 21 233
Exercises for Chapter 22 234
Exercises for Chapter 23 235
Exercises for Chapter 24 236

Worked solutions

Solutions for Chapter 1 237
Solutions for Chapter 2 239
Solutions for Chapter 3 241
Solutions for Chapter 4 243
Solutions for Chapter 5 245
Solutions for Chapter 6 247
Solutions for Chapter 7 249
Solutions for Chapter 8 253
Solutions for Chapter 9 255
Solutions for Chapter 10 257
Solutions for Chapter 11 260
Solutions for Chapter 12 262
Solutions for Chapter 13 264
Solutions for Chapter 14 267
Solutions for Chapter 15 269
Solutions for Chapter 16 272
Solutions for Chapter 17 275
Solutions for Chapter 18 278
Solutions for Chapter 19 281
Solutions for Chapter 20 284
Solutions for Chapter 21 286
Solutions for Chapter 22 288
Solutions for Chapter 23 290
Solutions for Chapter 24 293

1. Series and DataFrame

> **Build** — `first_look(df)` : a function that tells you, for any table, what each column holds, how much is missing and how much memory the table uses.
>
> **Theory kept** — A Series is values plus an index. A DataFrame is a set of Series that share one index. The index is a set of labels, not a position.
>
> **Source** — New. Data: `data/calls.csv` , made by `pdbook/datasets.py` .
>
> **Run this first** — `uv run python -m examples.ch01_first_look` , which is `examples/ch01_first_look.py` . Exercises: `exercises/01.md` . Solutions: `exercises/solutions/01.md` .

What pandas is for

pandas holds a table in memory and gives each row and each column a label. You use it when the data fit in memory and the question is about rows and columns: how many calls failed, which cell has the longest calls, what changed between Monday and Tuesday.

Underneath, pandas stores each column as an array (NumPy or Arrow). Operations on a column run in compiled code, not in a Python loop. This is the reason pandas is fast enough for millions of rows. It is also the reason you must learn to write operations on whole columns. Chapter 7 measures the difference.

This book uses pandas 3.0. Most tutorials on the web describe pandas 1.x or 2.x. Where 3.0 behaves differently, the text says so and shows the output. Chapter 24 lists, in one table, the differences that this book ran.

The Series: values and labels

A Series is a one-dimensional array with a label for each value. It has four parts: the values, the index, a name, and a dtype (the type of every value).

```
import pandas as pd

fee = pd.Series([12.5, 24.0, 45.0],
                index=["basic", "plus", "business"],
                name="monthly_fee_eur")
print(fee)
print(fee.dtype)
print(list(fee.index))
print(fee.to_numpy())
```

```
basic       12.5
plus        24.0
business    45.0
Name: monthly_fee_eur, dtype: float64
float64
['basic', 'plus', 'business']
[12.5 24.  45. ]
```

The values are a plain NumPy array. The index holds the labels. You can read a value by its label or by its position:

```
import pandas as pd

fee = pd.Series([12.5, 24.0, 45.0], index=["basic", "plus", "business"])
print(fee["plus"])
print(fee.iloc[1])
```

```
24.0
24.0
```

Use `.loc` for labels and `.iloc` for positions. Chapter 3 explains both. For now, remember one fact: the label is part of the data, and it does not change when you reorder the Series.

The index takes part in arithmetic

NumPy arrays add by position. A Series adds by label. This is the first difference that surprises people, and it is the source of many silent errors.

```
import pandas as pd

a = pd.Series([1, 2, 3], index=["x", "y", "z"])
b = pd.Series([10, 20, 30], index=["y", "z", "w"])
print(a + b)
```

```
w     NaN
x     NaN
y    12.0
z    23.0
dtype: float64
```

Only `y` and `z` appear in both Series, so only those two labels get a sum. The labels `x` and `w` appear in one Series only, and the result for them is `NaN` (not a number, the marker for a missing value). Notice also that the dtype changed from integer to `float64`, because an integer array has no place to store `NaN`. Chapter 5 returns to this.

Alignment is a feature. When you divide one column by another from a different table, pandas matches the right rows without a join in your code. It is also a trap, and Chapter 15 lists the ways it fails.

The DataFrame: Series that share an index

A DataFrame is a table. Each column is a Series. All the columns share one index, which labels the rows. Each column has its own dtype.

```
import pandas as pd

plans = pd.DataFrame({
    "monthly_fee_eur": [0.0, 12.5, 24.0, 45.0],
    "included_minutes": [0, 100, 500, 2000],
}, index=["prepaid", "basic", "plus", "business"])
print(plans)
print(plans.dtypes)
print(type(plans["included_minutes"]).__name__)
print(type(plans[["included_minutes"]]).__name__)
```

```
          monthly_fee_eur  included_minutes
prepaid               0.0                 0
basic                12.5               100
plus                 24.0               500
business             45.0              2000
monthly_fee_eur     float64
included_minutes      int64
dtype: object
Series
DataFrame
```

One pair of square brackets with a string gives a Series. Two pairs (a list of names) give a DataFrame. This difference matters: many methods exist on both, but they return different shapes.

Read the call records

The file `data/calls.csv` holds 20,000 call detail records for September 2026. One row is one call attempt.

```
import pandas as pd

calls = pd.read_csv("data/calls.csv")
print(calls.shape)
print(calls.index)
print(calls.head(3).to_string())
```

```
(20000, 9)
RangeIndex(start=0, stop=20000, step=1)
   call_id                start caller       callee  duration_s    status  cell_id codec
roaming
0        1  2026-08-31T22:13:30Z  S0125  32434086075        24.0  answered  BRU-003   EVS
False
1        2  2026-08-31T22:15:27Z  S0108  32491633063         0.0      busy  GHE-001   NaN
False
2        3  2026-08-31T22:51:30Z  S0414  32447458698       174.0  answered  BRU-003   EVS
False
```

`shape` is a tuple of rows and columns. The index is a `RangeIndex` that pandas made: 0, 1, 2 and so on. The file has no index column, so pandas labels the rows by position.

The `info` method gives the dtypes and the number of non-null values for each column in one view:

```
import pandas as pd

calls = pd.read_csv("data/calls.csv")
calls.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999
Data columns (total 9 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   call_id     20000 non-null  int64
 1   start       20000 non-null  str
 2   caller      20000 non-null  str
 3   callee      20000 non-null  int64
 4   duration_s  18967 non-null  float64
 5   status      20000 non-null  str
 6   cell_id     19723 non-null  str
 7   codec       15526 non-null  str
 8   roaming     20000 non-null  bool
dtypes: bool(1), float64(1), int64(2), str(5)
memory usage: 2.1 MB
```

Look at the column `callee`. The file holds phone numbers such as `+32434086075`. pandas read them as integers, because a plus sign at the start is a valid sign for a number. A phone number is an identifier, not a quantity, so integers are the wrong type. The sum of two phone numbers is not useful in the real world. Chapter 2 shows how to prevent this.

Look also at the text columns. In pandas 3.0 they have the dtype `str`. In pandas 2.x they had the dtype `object`, which stored Python string objects. The new dtype is faster and uses less memory. Chapter 4 measures both.

Build: first_look

The standard `info` output is for the eye. It does not return a value that you can test, save or compare. `first_look` returns a DataFrame with one row per column.

```
import pandas as pd


def first_look(df: pd.DataFrame) -> pd.DataFrame:
    """One row per column: dtype, null count, distinct count, first value."""
    return pd.DataFrame({
        "dtype": df.dtypes.astype(str),
        "nulls": df.isna().sum(),
        "unique": df.nunique(),
        "first": df.iloc[0].astype(str),
    })


calls = pd.read_csv("data/calls.csv")
print(first_look(calls))
```

```
             dtype  nulls  unique                first
call_id      int64      0   20000                    1
start          str      0   19881  2026-08-31T22:13:30Z
caller         str      0     600                 S0125
callee       int64      0   14617           32434086075
duration_s  float64   1033     858                  24.0
status         str      0       4              answered
cell_id        str    277      12               BRU-003
codec          str   4474       3                   EVS
roaming       bool      0       2                 False
```

The function builds a DataFrame from a dict of four Series. Each Series has the column names of `df` as its index. Because the indexes are equal, pandas aligns them without any extra code. This is the alignment from the previous section, and here it saves code.

The table tells you five things at once:

1. `call_id` has 20,000 distinct values in 20,000 rows. It is a key.
2. `caller` has 600 distinct values. It repeats, so it identifies a subscriber, not a call.
3. `duration_s` has 1,033 nulls and the dtype `float64`. An integer column cannot hold nulls, so a count of seconds became a float.
4. `cell_id` and `codec` have nulls. They do not have the same cause: the next chapters find out why.
5. `status` has 4 distinct values. A column with few distinct values is a candidate for the `category` dtype (Chapter 4).

The module in `pdbook/looks.py` holds this function and `headline`, which returns one line with the row count and the memory:

```
import pandas as pd

from pdbook.looks import headline

print(headline(pd.read_csv("data/calls.csv")))
```

```
20,000 rows x 9 columns, 2.2 MB
```

The memory figure is the deep figure: it counts the bytes of the strings, not only of the pointers to them. Use `memory_usage(deep=True)` whenever you compare two representations of the same table.

How it works

A Series is two objects held together: a NumPy array of values and an `Index` object that holds the labels. A DataFrame is an ordered set of such arrays. Each column is its own array with its own dtype, and all columns share one `Index` for the rows. There is no single block of memory with rows in it, so a column operation reads one contiguous array.

Arithmetic between two Series has three steps. First, pandas builds the union of the two indexes. Second, it reindexes each Series to that union, and a label that the Series lacks becomes `NaN`. Third, it adds the two arrays by position. The alignment of this chapter is steps one and two. You can do the three steps yourself and compare:

```
import numpy as np
import pandas as pd

from pdbook.io import load_calls

a = pd.Series([1, 2, 3], index=["x", "y", "z"])
b = pd.Series([10, 20, 30], index=["y", "z", "w"])

labels = a.index.union(b.index)
by_hand = a.reindex(labels).to_numpy() + b.reindex(labels).to_numpy()
print(labels.tolist())
print(by_hand.tolist())
print(bool(np.array_equal((a + b).to_numpy(), by_hand, equal_nan=True)))
print(type(a.index).__name__, type(a.to_numpy()).__name__)

calls = load_calls()
print({name: type(calls[name].to_numpy()).__name__ for name in ["call_id", "status"]})
print(calls["call_id"].to_numpy().dtype, calls["duration_s"].to_numpy().dtype)
```

```
['w', 'x', 'y', 'z']
[nan, nan, 12.0, 23.0]
True
Index ndarray
{'call_id': 'ndarray', 'status': 'ndarray'}
int64 float64
```

The union of the labels is sorted: `w`, `x`, `y`, `z`. The hand-made sum has `nan` at `w` and `x`, because each of those labels is in only one Series. The third line is `True`:

`a + b` has the same values, `NaN` included. The last lines show two things. The index and the values are separate objects. The columns of the table are separate arrays with different types.

- **Invariant.** The result of an operation on two Series has the union of their indexes as labels.
- **Failure.** A label on one side only gives `NaN`, and `int64` becomes `float64`.
- **Cost.** The reindex looks up each label and makes a new array. This is why arithmetic on two different indexes is slower than on two arrays.

Reference

The signatures below are printed from the installed code, with the parameters that this chapter uses.

```
import pandas as pd

from pdbook.looks import first_look, headline
from pdbook.sigs import show_signature

show_signature(pd.Series, ["data", "index", "dtype", "name"])
show_signature(pd.DataFrame, ["data", "index", "columns", "dtype"])
show_signature(pd.read_csv, ["filepath_or_buffer", "dtype", "nrows"])
show_signature(first_look)
show_signature(headline)
```

```
Series(
    data=None
    index=None
    dtype: 'Dtype | None' = None
    name=None
)  # 1 more parameters
DataFrame(
    data=None
    index: 'Axes | None' = None
    columns: 'Axes | None' = None
    dtype: 'Dtype | None' = None
)  # 1 more parameters
read_csv(
    filepath_or_buffer: 'FilePath | ReadCsvBuffer[bytes] | ReadCsvBuffer[str]'
    dtype: 'DtypeArg | None' = None
    nrows: 'int | None' = None
)  # 41 more parameters
first_look(
    df: 'pd.DataFrame'
)
headline(
    df: 'pd.DataFrame'
)
```

The parameters that matter in this chapter:

Parameter	Default	What it does here	What goes wrong
`Series(data=...)`	`None`	the values: a list, an array or a dict	a dict puts its keys in the index
`Series(index=...)`	`None`	the labels, which also drive alignment	labels on one side only give `NaN`, and `int64` becomes `float64` (this chapter)
`Series(dtype=...)`	`None`	forces the type of every value	—
`Series(name=...)`	`None`	the name, which becomes the column name	—
`DataFrame(data=..., index=..., columns=...)`	`None`	a dict of columns, or rows and column names	—
`read_csv(filepath_or_buffer)`	required	the file or buffer to read	—
`read_csv(nrows=...)`	`None`	read only the first n rows, to look at a large file	—
`first_look(df)`	required	one row for each column: dtype, nulls, distinct values, first value	—
`headline(df)`	required	one line: rows, columns and deep memory	—

`first_look` returns a DataFrame indexed by column name. `headline` returns a string.

What you have now

You can read a CSV file into a DataFrame, and you know that:

- Every column is a Series with its own dtype.
- The index labels the rows and takes part in arithmetic.
- One bracket pair gives a Series. A list gives a DataFrame.
- The dtype `float64` in a column of whole numbers means that nulls are present.

The next chapter reads and writes data in a controlled way. The phone numbers in `callee` need it.

Next: Chapter 2, Input and output.

2. Input and output

> **Build** — `load_calls()`: a loader that reads the call records with the right type for every column, plus a round-trip test that proves a file format keeps those types.
>
> **Theory kept** — A CSV file has no types. `read_csv` guesses them, and every guess can be wrong without an error message. You must state the types that matter. Parquet stores the types, so it does not guess.
>
> **Source** — New. Loader: `pdbook/io.py`.
>
> **Run this first** — `uv run python -m examples.ch02_input_output`, which is `examples/ch02_input_output.py`. Exercises: `exercises/02.md`. Solutions: `exercises/solutions/02.md`.

A file is text, and types are guesses

A CSV file contains characters. The characters `24.0` can be a duration, a price or a version number. The function `read_csv` looks at the characters in each column and chooses a dtype. The choice is correct often enough that you stop checking, and that is the danger.

Chapter 1 found one wrong guess already: `callee` came back as `int64`. Here is the same problem on four lines of text:

```
import io

import pandas as pd

text = "phone\n0477123456\n+32477123456\n"
print(pd.read_csv(io.StringIO(text))["phone"].tolist())

text = "phone\n0477123456\n0477123457\n"
print(pd.read_csv(io.StringIO(text))["phone"].tolist())
print(pd.read_csv(io.StringIO(text), dtype={"phone": "str"})["phone"].tolist())
```

```
[477123456, 32477123456]
[477123456, 477123457]
['0477123456', '0477123457']
```

Without a dtype, the number `0477123456` loses its leading zero. Nothing warns you. The value `+32477123456` becomes the integer 32477123456, which is a different string. With `dtype={"phone": "str"}`, the characters stay as they are.

The rule: any column that is an identifier must be read as text. Identifiers are phone numbers, postcodes, account numbers, product codes and IMEI numbers. You do not do arithmetic on them, so integers give you nothing and can destroy information.

Say what you know

The arguments of `read_csv` that you use every day are few:

Argument	What it does
`dtype`	A dict of column name to dtype. Use it for identifiers.
`parse_dates`	A list of columns to convert to timestamps.
`usecols`	Read only these columns. Faster and smaller.
`nrows`	Read only the first n rows. Use it to look at a large file.
`na_values`	More strings that mean "missing".
`index_col`	Use a column as the index.

Read the call records with the first three:

```
import pandas as pd

calls = pd.read_csv("data/calls.csv",
                    dtype={"caller": "str", "callee": "str"},
                    parse_dates=["start"])
print(calls[["start", "caller", "callee"]].dtypes.astype(str).to_dict())
print(calls["start"].iloc[0])
print(calls["callee"].iloc[0])
```

```
{'start': 'datetime64[us, UTC]', 'caller': 'str', 'callee': 'str'}
2026-08-31 22:13:30+00:00
+32434086075
```

The column `start` now has the dtype `datetime64[us, UTC]`. The `UTC` is there because every string in the file ends with `Z`, which means UTC. If the strings had no zone, the column has no zone, and Chapter 12 shows why that matters. The `us` means microsecond resolution. pandas 2.x used nanoseconds for every timestamp, and nanoseconds cannot hold a date before the year 1678. pandas 3.0 picks the resolution from the data:

```
import pandas as pd

for text in ["1500-01-01", "2026-10-12 10:00:00", "2026-10-12 10:00:00.123456789"]:
    print(text, "->", pd.to_datetime([text]).dtype)

try:
    pd.Timestamp("1500-01-01").as_unit("ns")
except Exception as error:
    print("as nanoseconds:", type(error).__name__)
```

```
1500-01-01 -> datetime64[us]
2026-10-12 10:00:00 -> datetime64[us]
2026-10-12 10:00:00.123456789 -> datetime64[ns]
as nanoseconds: OutOfBoundsDatetime
```

A string with nine decimals gets nanosecond resolution, because microseconds lose digits. The year 1500 is a valid timestamp at microsecond resolution, and it is out of bounds at nanosecond resolution.

Missing markers

`read_csv` treats a fixed list of strings as missing: the empty string, `NA`, `N/A`, `NaN`, `null` and a few more. It does not treat `-` as missing:

```
import io

import pandas as pd

text = "a,b\n1,5\n2,N/A\n3,-\n4,7\n"
default = pd.read_csv(io.StringIO(text))
print(default["b"].tolist(), default["b"].dtype)

custom = pd.read_csv(io.StringIO(text), na_values=["-"])
print(custom["b"].tolist(), custom["b"].dtype)
```

```
['5', nan, '-', '7'] str
[5.0, nan, nan, 7.0] float64
```

The dash stays a string, and one string forces the whole column to the text dtype. A column of numbers with one dash in it is a column of strings. Chapter 6 cleans such columns after the read. When you know the marker, `na_values` is the better place to handle it: the column keeps its numeric dtype, as the second line shows.

Read less

Most analyses need a few columns of a wide file. `usecols` does not parse the columns you drop, so the read is faster and the table is smaller:

```
import pandas as pd

wide = pd.read_csv("data/calls.csv")
narrow = pd.read_csv("data/calls.csv", usecols=["call_id", "status"])
print(wide.shape, narrow.shape)
print(int(wide.memory_usage(deep=True).sum()), int(narrow.memory_usage(deep=True).sum()))
```

```
(20000, 9) (20000, 2)
2177698 472354
```

The narrow table needs about a fifth of the memory. With a file that is larger than memory, this is the first thing you try (Chapter 19).

Write without the index

`to_csv` writes the index as a column unless you tell it not to. On the way back, that column has no name and pandas calls it `Unnamed: 0`:

```
import io

import pandas as pd

df = pd.DataFrame({"a": [1, 2]})
print(pd.read_csv(io.StringIO(df.to_csv())).columns.tolist())
print(pd.read_csv(io.StringIO(df.to_csv(index=False))).columns.tolist())
```

```
['Unnamed: 0', 'a']
['a']
```

Write `index=False` unless the index carries information that you want to keep. If it does, name it and read it back with `index_col`.

CSV loses more than the index. It stores no dtype, so a round trip through CSV is a round trip through a guess:

```
import io

import pandas as pd

from pdbook.io import load_calls

calls = load_calls().head(1000)
back = pd.read_csv(io.StringIO(calls.to_csv(index=False)))
print(calls.dtypes.astype(str).to_dict()["callee"], "->", back.dtypes.astype(str).to_dict()
["callee"])
print(calls.dtypes.astype(str).to_dict()["start"], "->", back.dtypes.astype(str).to_dict()
["start"])
```

```
str -> int64
datetime64[us, UTC] -> str
```

Parquet keeps the types

Parquet is a binary column format. It stores the dtype of each column, compresses well and lets you read a subset of columns without reading the rest. pandas uses the `pyarrow` package for it.

```
import pathlib
import tempfile

import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
with tempfile.TemporaryDirectory() as tmp:
    path = pathlib.Path(tmp) / "calls.parquet"
    calls.to_parquet(path)
    back = pd.read_parquet(path)
    only_two = pd.read_parquet(path, columns=["call_id", "status"])
    pd.testing.assert_frame_equal(calls, back)
    print("round trip equal")
    print("csv bytes    :", pathlib.Path("data/calls.csv").stat().st_size)
    print("parquet bytes:", path.stat().st_size)
    print(only_two.shape)
```

```
round trip equal
csv bytes    : 1559678
parquet bytes: 527534
(20000, 2)
```

`assert_frame_equal` compares values, dtypes, index and column names. It raised no error, so every type survived, including the UTC timestamp and the text identifiers. The parquet file is about a third of the size of the CSV file.

The sizes are exact for this data. The read times depend on your machine, and the module `examples/ch02_input_output.py` prints them for each format. Run it to see yours.

JSON needs care

`to_json` and `read_json` work, but JSON has no timestamp type either. In pandas 3.0 the default `date_format` is the old `epoch`, and it warns about its removal:

```
import tempfile
import warnings

import pandas as pd

from pdbook.io import load_calls

calls = load_calls().head(3)
with tempfile.TemporaryDirectory() as tmp:
    with warnings.catch_warnings(record=True) as caught:
        warnings.simplefilter("always")
        calls.to_json(f"{tmp}/a.jsonl", orient="records", lines=True)
    print(caught[0].category.__name__)
    print(open(f"{tmp}/a.jsonl").readline()[:60])
    calls.to_json(f"{tmp}/b.jsonl", orient="records", lines=True, date_format="iso")
    print(open(f"{tmp}/b.jsonl").readline()[:60])
```

```
Pandas4Warning
{"call_id":1,"start":1788214410000,"caller":"S0125","callee"
{"call_id":1,"start":"2026-08-31T22:13:30.000Z","caller":"S0
```

The warning class is `Pandas4Warning`. It tells you that behavior will change in pandas 4. Write `date_format="iso"` and the warning goes away. The result is also readable by people and by other tools.

Excel

`read_excel` and `to_excel` exist. The pandas documentation names `openpyxl` as the package for `.xlsx` files. This book does not install it, so the text does not run Excel code, and the next block checks that:

```
import importlib.util

installed = {name: importlib.util.find_spec(name) is not None
             for name in ["openpyxl", "numexpr", "polars", "duckdb"]}
print(installed)
```

```
{'openpyxl': False, 'numexpr': False, 'polars': False, 'duckdb': False}
```

None of the four optional packages that this book mentions is installed, so every statement about them is *described, not run* (Appendix A). From the documentation, the arguments of `read_excel` that matter are the same as for `read_csv`: `dtype`, `usecols`, `nrows` and `na_values`, plus `sheet_name`. The same identifier rule applies. Appendix A lists what the book describes and does not run.

Build: load_calls and a round-trip test

The loader is two lines of code. Its value is that it is the only place that knows the file format:

```
import io

import pandas as pd

from pdbook.io import load_calls


def round_trip_ok(df: pd.DataFrame, write, read) -> bool:
    """True when write() then read() returns an identical frame, dtypes included."""
    try:
        pd.testing.assert_frame_equal(df, read(write(df)))
    except AssertionError:
        return False
    return True


calls = load_calls()
print(calls.dtypes.astype(str).to_dict()["callee"])
print(round_trip_ok(calls.head(500),
                    write=lambda d: d.to_csv(index=False),
                    read=lambda s: pd.read_csv(io.StringIO(s))))
print(round_trip_ok(calls.head(500),
                    write=lambda d: d.to_csv(index=False),
                    read=lambda s: pd.read_csv(io.StringIO(s),
                                               dtype={"caller": "str", "callee": "str"},
                                               parse_dates=["start"])))
```

```
str
False
True
```

The first test fails, and the second passes. The test tells you which formats you can trust with your data. Run it for each format that you plan to use for storage. Use a sample of your own data. Do it before the first night that the job runs.

How it works

`read_csv` does not know the types of the columns. It reads the characters and then tries a fixed order of types for each column: boolean, integer, float, and last text. A column becomes the first type that every non-missing value fits.

The rule explains every surprise of this chapter. The text `+32434086075` fits an integer, so `callee` is an integer. The text `0477123456` fits an integer too, and the integer has no leading zero.

You can write the rule in ten lines and compare it with `read_csv` on the call records:

```
import pandas as pd


def guess(column: pd.Series) -> str:
    values = column.dropna()
    if values.isin(["True", "False"]).all():
        return "bool"
    if values.str.fullmatch(r"[+-]?\d+").all():
        return "int64"
    if values.str.fullmatch(r"[+-]?(\d+\.?\d*|\.\d+)([eE][+-]?\d+)?").all():
        return "float64"
    return "str"


raw = pd.read_csv("data/calls.csv", dtype=str)
guessed = {name: guess(raw[name]) for name in raw.columns}
actual = pd.read_csv("data/calls.csv").dtypes.astype(str).to_dict()
print(guessed == actual)
print(guessed["callee"], guessed["duration_s"], guessed["start"])
```

```
True
int64 float64 str
```

The rule gives the same nine types as `read_csv`. The guess is the first type in the order that fits, with no knowledge of what the column means. Real `read_csv` has more rules (thousands separators, dates, a `NaN` in an integer column makes a float), but this is the core. The dtype argument of `read_csv` replaces the guess for the columns that you name. The argument `parse_dates` adds one more step after the guess.

- **Invariant.** A column that you did not name gets the first type that fits all its values.
- **Failure.** A type that is good for the sample and wrong for the file. One dash makes a numeric column text.
- **Cost.** One extra pass over the characters. `usecols` skips the columns that you do not need, before the guess.

Reference

```
import pandas as pd

from pdbook.io import load_calls
from pdbook.sigs import show_signature

show_signature(pd.read_csv, ["filepath_or_buffer", "dtype", "parse_dates", "usecols", "nrows",
"na_values", "index_col"])
show_signature(pd.DataFrame.to_csv, ["path_or_buf", "index"])
show_signature(pd.read_parquet, ["path", "columns", "filters"])
show_signature(pd.DataFrame.to_parquet, ["path", "index"])
show_signature(pd.DataFrame.to_json, ["path_or_buf", "orient", "lines", "date_format"])
show_signature(load_calls)
```

```
read_csv(
    filepath_or_buffer: 'FilePath | ReadCsvBuffer[bytes] | ReadCsvBuffer[str]'
    index_col: 'IndexLabel | Literal[False] | None' = None
    usecols: 'UsecolsArgType' = None
    dtype: 'DtypeArg | None' = None
    nrows: 'int | None' = None
    na_values: 'Hashable | Iterable[Hashable] | Mapping[Hashable, Iterable[Hashable]] | None'
= None
    parse_dates: 'bool | Sequence[Hashable] | None' = None
)  # 37 more parameters
NDFrame.to_csv(
    path_or_buf: 'FilePath | WriteBuffer[bytes] | WriteBuffer[str] | None' = None
    index: 'bool' = True
)  # 20 more parameters
read_parquet(
    path: 'FilePath | ReadBuffer[bytes]'
    columns: 'list[str] | None' = None
    filters: 'list[tuple] | list[list[tuple]] | None' = None
)  # 6 more parameters
DataFrame.to_parquet(
    path: 'FilePath | WriteBuffer[bytes] | None' = None
    index: 'bool | None' = None
)  # 7 more parameters
NDFrame.to_json(
    path_or_buf: 'FilePath | WriteBuffer[bytes] | WriteBuffer[str] | None' = None
    orient: "Literal['split', 'records', 'index', 'table', 'columns', 'values'] | None" = None
    date_format: 'str | None' = None
    lines: 'bool' = False
)  # 10 more parameters
load_calls(
    path: 'str | pathlib.Path | None' = None
)
```

Parameter	Default	What it does here	What goes wrong
`read_csv(dtype=...)`	`None`	a dict of column to type; use it for every identifier	without it `callee` is `int64`, and `0477123456` becomes 477123456
`read_csv(parse_dates=...)`	`None`	columns to convert to timestamps	without it `start` stays text; with the `Z` suffix the type is `datetime64[us, UTC]`
`read_csv(usecols=...)`	`None`	read only these columns	— (3 of 9 columns: 472,354 bytes against 2,177,698)
`read_csv(nrows=...)`	`None`	first n rows only	—
`read_csv(na_values=...)`	`None`	more strings that mean missing	a `-` in a numeric column makes the whole column `str`; `na_values=["-"]` keeps `float64`
`read_csv(index_col=...)`	`None`	use a column as the index	—
`to_csv(index=...)`	`True`	write the index as a column	read back as a column `Unnamed: 0`
`read_parquet(columns=...)`	`None`	read only these columns	—
`read_parquet(filters=...)`	`None`	skip rows that fail a test, at the read	— (Chapter 19: 776,300 of 1,000,000 rows, 13 MB against 101 MB)
`to_parquet(index=...)`	`None`	write the index or not	—
`to_json(date_format=...)`	`None`	`"iso"` writes timestamps as ISO text	the default is epoch milliseconds, and it warns with `Pandas4Warning`
`to_json(orient=..., lines=...)`	`None`, `False`	`orient="records", lines=True` writes one JSON object for each row	—
`load_calls(path=...)`	`None`	the file in `data/`, or your own path	—

`read_csv` returns a DataFrame. It raises `FileNotFoundError` for a missing file, and `ParserError` for a file that is not valid CSV.

What you have now

- `callee` is text, and `start` is a UTC timestamp.
- You know four arguments that make `read_csv` safe: `dtype`, `parse_dates`, `usecols`, `na_values`.
- You write CSV with `index=False`, and you do not trust CSV with types.
- You can prove that a file format keeps your types with `assert_frame_equal`.

The next chapter selects rows and columns from the loaded table.

Next: Chapter 3, Selection.

3. Selection

> **Build** — `ops_questions(calls)`: five questions that an operations team asks of the call records, each answered by one selection, with the answers as a dict.
>
> **Theory kept** — `.loc` selects by label, and its slice includes the end. `.iloc` selects by position, and its slice excludes the end. A boolean mask selects the rows where it is `True`. Combine masks with `&`, `|` and `~`, and put each comparison in parentheses.
>
> **Source** — New.
>
> **Run this first** — `uv run python -m examples.ch03_selection`, which is `examples/ch03_selection.py`. Exercises: `exercises/03.md`. Solutions: `exercises/solutions/03.md`.

Four tools, one question each

You have	You want	Use
column names	some columns	`df["a"]` or `df[["a", "b"]]`
row labels	some rows	`df.loc[labels]`
row positions	some rows	`df.iloc[positions]`
a condition	the rows that match	`df[mask]` or `df.query("...")`

Learn `.loc` and `.iloc` first, because they are explicit. The bare `df[...]` does different things for different arguments, and that is a source of errors.

Columns

A string gives a Series. A list of strings gives a DataFrame, in the order of the list:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
print(type(calls["status"]).__name__)
print(calls[["status", "duration_s"]].head(3).to_string())
```

```
Series
     status  duration_s
0  answered        24.0
1      busy         0.0
2  answered       174.0
```

pandas also lets you write `calls.status`. Do not use it in code that you keep. It fails for a column name with a space. It returns a method, not a column, when the name is also a method name. A DataFrame has a method `count`, so

`df.count` is never a column called `count`.

Rows by label and by position

Use a small table with text labels, so that labels and positions are different things:

```
import pandas as pd

d = pd.DataFrame({"v": [10, 20, 30, 40]}, index=["a", "b", "c", "d"])
print(d.loc["a":"c"]["v"].tolist())
print(d.iloc[0:2]["v"].tolist())
print(d.loc["b", "v"], d.iloc[1, 0])
print(d[0:2]["v"].tolist(), d["a":"b"]["v"].tolist())
```

```
[10, 20, 30]
[10, 20]
20 20
[10, 20] [10, 20]
```

The slice `"a":"c"` has three rows, and `0:2` has two. A label slice includes both ends, because the end label has no "next label" that you could stop before. A position slice follows the Python rule and excludes the end.

The last line shows the bare forms. `d[0:2]` is a position slice, and `d["a":"b"]` is a label slice. The same brackets have two meanings. Avoid both forms.

An integer index makes the difference more dangerous. Here the labels are the numbers 3, 2, 1, 0:

```
import pandas as pd

d = pd.DataFrame({"v": [10, 20, 30, 40]}, index=[3, 2, 1, 0])
print(d.loc[0:1]["v"].tolist())
print(d.iloc[0:1]["v"].tolist())
print(d.loc[0, "v"], d.iloc[0, 0])

shuffled = pd.DataFrame({"v": [10, 20, 30, 40]}, index=[2, 0, 3, 1])
print(shuffled.loc[0:1].index.tolist())
```

```
[]
[10]
40 10
[0, 3, 1]
```

The label slice `0:1` is empty. Label 0 is in the last row and label 1 is in the third row. The slice runs from position 3 back to position 2, and that range has no rows.

A label slice on an index that is not sorted is not an error. It returns an empty result or the wrong rows. On the index 2, 0, 3, 1 the slice `0:1` returns the labels 0, 3 and 1, and label 3 is outside the range. Sort the index first with `sort_index()`, or select with a list of labels. Also note that `d.loc[0]` and `d.iloc[0]` are different rows.

Rows by condition

A comparison on a column gives a boolean Series. That Series is a mask. Use it inside the brackets:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
mask = calls["duration_s"] > 600
print(mask.dtype, int(mask.sum()))
print(calls[mask].shape)
print(calls.loc[mask, ["call_id", "duration_s"]].head(3).to_string())
```

```
bool 351
(351, 9)
    call_id  duration_s
75       83       971.0
82       90       713.0
86       94       883.0
```

The sum of a boolean Series is a count, because `True` is 1. The mean is a share. These two idioms replace most loops.

To combine masks, use `&` for and, `|` for or and `~` for not. Put each comparison in parentheses, because `&` binds more tightly than `>` and `==`. Do not use the Python words `and` and `or`:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
try:
    calls[calls["status"] == "answered" and calls["duration_s"] > 600]
except ValueError as error:
    print("ValueError:", str(error)[:42])

try:
    calls[calls["status"] == "answered" & calls["duration_s"] > 600]
except TypeError as error:
    print("TypeError:", str(error)[:42])

both = (calls["status"] == "answered") & (calls["duration_s"] > 600)
either = (calls["status"] == "answered") | (calls["duration_s"] > 600)
print(int(both.sum()), int(either.sum()))
```

```
ValueError: The truth value of a Series is ambiguous.
TypeError: Cannot perform 'rand_' with a dtyped [floa
351 15526
```

Python `and` asks for one truth value for a whole column, and pandas refuses to give one. Without the parentheses you get a different error. In `calls["status"] == "answered" & calls["duration_s"] > 600`, the `&` binds first and combines the text `"answered"` with a number column. The error is a `TypeError`.

Three helpers make masks readable: `isin` for membership, `between` for a range and `isna` for missing values:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
print(int(calls["codec"].isin(["EVS", "AMR-WB"]).sum()))
print(int(calls["duration_s"].between(60, 120).sum()))
print(int(calls["cell_id"].isna().sum()))
```

```
12183
4226
277
```

Missing values in a mask

A comparison with a missing value is not `True`. The rows with a missing value are not selected, and you get no message:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
longer = calls["duration_s"] > 0
shorter_or_equal = calls["duration_s"] <= 0
print(int(longer.sum()), int(shorter_or_equal.sum()), len(calls))
print(int(calls["duration_s"].isna().sum()))
```

```
15526 3441 20000
1033
```

The two masks are opposites, and the counts do not add up to 20,000. The difference is 1,033, the number of nulls. A condition and its opposite do not cover the table when the column has nulls. When you need "not longer than zero", write `~longer` and check what you get.

The text methods in pandas 3.0 behave in a new way here. On the `str` dtype, `calls["cell_id"].str.startswith("BRU")` returns `False` for a missing value. On the `object` dtype, it returns a missing value. The `object` dtype was the default for text in pandas 2.x. A mask with a missing value is an error. Check both:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
bru = calls["cell_id"].str.startswith("BRU")
print(bru.dtype, int(calls["cell_id"].isna().sum()), int(bru.isna().sum()))

old_style = calls["cell_id"].astype(object).str.startswith("BRU")
print(old_style.dtype, int(old_style.isna().sum()))
try:
    calls[old_style]
except ValueError as error:
    print("ValueError:", str(error)[:41])
```

```
bool 277 0
object 277
ValueError: Cannot mask with non-boolean array contai
```

The first result has the dtype `bool`, and no value is missing, although 277 cells are. The second result has the dtype `object` with 277 missing values, and it cannot be a mask. Be aware that, on the `str` dtype, a missing cell counts as "not Brussels". If you need a three-way answer (yes, no, unknown), keep `isna()` in the condition.

query

`query` takes the condition as a string. It reads well when there are many conditions:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
limit = 600
result = calls.query("status == 'answered' and duration_s > @limit and
cell_id.str.startswith('BRU')")
print(result.shape)
```

```
(136, 9)
```

Inside a `query` string, `and` and `or` are correct, and `@name` reads a Python variable. The cost is that an editor does not check the string, and a typo in a column name is a runtime error. Use `query` when it is shorter and clearer. Use masks when you build a condition from parts in a function.

Set values with .loc

To change values, select the rows and the column in one `.loc`. Two steps do not work in pandas 3.0:

```
import warnings

import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
mask = calls["status"] == "noanswer"

with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    calls[mask]["duration_s"] = -1
print(caught[0].category.__name__)
print(int((calls["duration_s"] == -1).sum()))

calls.loc[mask, "duration_s"] = -1
print(int((calls["duration_s"] == -1).sum()))
```

```
ChainedAssignmentError
0
2013
```

The first form selects a copy with `calls[mask]`, then sets a value in the copy. The copy is thrown away, and the table does not change. pandas 3.0 tells you with a `ChainedAssignmentError`, and the table does not change. The second form, with one `.loc`, changes the table. Chapter 15 explains the rule behind this.

Build: ops_questions

An operations team asks five questions. Each one is a mask, a count and a name. A function that returns a dict is easy to test and easy to print:

```
import pandas as pd

from pdbook.io import load_calls


def ops_questions(calls: pd.DataFrame) -> dict[str, int]:
    answered = calls["status"] == "answered"
    return {
        "answered, over 10 min, Brussels cell":
            int((answered & (calls["duration_s"] > 600) &
calls["cell_id"].str.startswith("BRU")).sum()),
        "answered, no cell recorded":
            int((answered & calls["cell_id"].isna()).sum()),
        "failed while roaming":
            int((calls["roaming"] & (calls["status"] == "failed")).sum()),
        "wideband or EVS codec":
            int(calls["codec"].isin(["EVS", "AMR-WB"]).sum()),
        "started on 10 September (UTC)":
            int(calls["start"].between(pd.Timestamp("2026-09-10", tz="UTC"),
                                       pd.Timestamp("2026-09-11", tz="UTC"),
inclusive="left").sum()),
    }


for name, n in ops_questions(load_calls()).items():
    print(f"{n:>7,d}  {name}")
```

```
    136  answered, over 10 min, Brussels cell
    212  answered, no cell recorded
     31  failed while roaming
 12,183  wideband or EVS codec
    782  started on 10 September (UTC)
```

Read the second line. 212 calls were answered, so media flowed, and the record has no cell. This is a data-quality fact, and it needs a decision: Chapter 5 treats it. Read the last line. The range uses `inclusive="left"`, so a call at exactly midnight on 11 September belongs to 11 September.

How it works

Every selection ends as one of two operations on arrays: take the rows at some integer positions, or keep the rows where a boolean array is true. `.loc` with a label first turns the label into a position with the index, and then it takes. `.iloc` skips the first step. A mask is a boolean array of the length of the table, and `df[mask]` takes the positions where it is true.

The label-to-position step is a lookup in a hash table that the `Index` builds the first time that you ask. You can see the step with `get_loc`:

```
import numpy as np
import pandas as pd

from pdbook.io import load_calls

calls = load_calls().set_index("call_id")
position = calls.index.get_loc(7)
print(type(position).__name__, position)
print(calls.loc[7].equals(calls.iloc[position]))

mask = (calls["status"] == "answered") & (calls["duration_s"] > 600)
positions = np.flatnonzero(mask.to_numpy())
print(len(positions), calls[mask].equals(calls.iloc[positions]))

duplicated = pd.Index([5, 7, 7, 9])
unsorted = pd.Index([7, 5, 7, 9])
print(type(duplicated.get_loc(7)).__name__, type(unsorted.get_loc(7)).__name__)
```

```
int 19000
True
351 True
slice ndarray
```

Call 7 is at position 19,000, the late record from Chapter 3, and `loc` and `iloc` give the same row. The mask gives 351 positions, and taking them gives the same table as `df[mask]`. The last line shows three return types of `get_loc`. A unique label gives an integer.

Equal labels next to each other give a slice. Equal labels that are apart give a boolean mask. This is why a non-unique index changes the type of what `loc` returns.

- **Invariant.** A selection is a set of integer positions.
- **Failure.** A label slice on an unsorted index takes everything between the positions of its two end labels.
- **Cost.** The hash table is built once for each index. A boolean mask costs one pass over its column.

Reference

```
import pandas as pd

from examples.ch03_selection import ops_questions
from pdbook.sigs import show_signature

show_signature(pd.DataFrame.query, ["expr", "inplace"])
show_signature(pd.Series.between, ["left", "right", "inclusive"])
show_signature(pd.Series.isin, ["values"])
show_signature(pd.DataFrame.nlargest, ["n", "columns", "keep"])
show_signature(pd.DataFrame.sort_index, ["axis", "ascending", "kind"])
show_signature(ops_questions)
```

```
DataFrame.query(
    expr: 'str'
    inplace: 'bool' = False
)  # 7 more parameters
Series.between(
    left
    right
    inclusive: "Literal['both', 'neither', 'left', 'right']" = 'both'
)  # 1 more parameters
Series.isin(
    values
)  # 1 more parameters
DataFrame.nlargest(
    n: 'int'
    columns: 'IndexLabel'
    keep: 'NsmallestNlargestKeep' = 'first'
)  # 1 more parameters
DataFrame.sort_index(
    axis: 'Axis' = 0
    ascending: 'bool | Sequence[bool]' = True
    kind: 'SortKind' = 'quicksort'
)  # 7 more parameters
ops_questions(
    calls: pandas.DataFrame
)
```

Parameter	Default	What it does here	What goes wrong
`query(expr)`	required	the condition as a string; `and`, `or` and `@name` work inside	a typo in a column name is an error when it runs
`query(inplace=...)`	`False`	change the table instead of returning a new one	—
`between(left, right)`	required	a mask for the values in a range	—
`between(inclusive=...)`	`'both'`	which ends belong to the range	4,226 calls with `between(60, 120)`, 4,179 with `>= 60` and `< 120`
`isin(values)`	required	a mask for the values that are in a list	—
`nlargest(n, columns)`	required	the n rows with the largest values	—
`nlargest(keep=...)`	`'first'`	which rows to keep when values tie	—
`sort_index(kind=...)`	`'quicksort'`	the sort algorithm	—
`sort_index(ascending=...)`	`True`	the direction	`loc[4:10]` on the unsorted index returns 6 rows, not 7
`ops_questions(calls)`	required	five counts for an operations team, as a dict	—

`.loc` and `.iloc` are indexers, not functions, so they have no signature. `.loc[a:b]` includes `b`. `.iloc[a:b]` excludes it.

What you have now

- You select columns with a string or a list, and rows with `.loc`, `.iloc` or a mask.
- You know that a label slice includes its end, and that a label slice on an unsorted index is a trap.
- You combine masks with `&`, `|`, `~` and parentheses.
- You know that missing values fall out of both a condition and its opposite.
- You set values with one `.loc`, never with two brackets.

The next chapter looks at the types of the columns that you selected, and at their memory.

Next: Chapter 4, Types and memory.

4. Types and memory

> **Build** — `shrink(df)`: a function that gives every column the smallest dtype that holds its values, plus a report of the bytes saved and a test that the content did not change.
>
> **Theory kept** — A dtype fixes how many bytes each value takes and what operations mean. `category` stores each distinct text once and each row as a small integer code. An integer dtype that is too small wraps around without an error.
>
> **Source** — New. Library: `pdbook/shrink.py`.
>
> **Run this first** — `uv run python -m examples.ch04_types_memory`, which is `examples/ch04_types_memory.py`. Exercises: `exercises/04.md`. Solutions: `exercises/solutions/04.md`.

The dtypes you meet

dtype	Holds	Missing value	Bytes per value
`int64`, `int32`, `int16`, `int8`	whole numbers	not possible	8, 4, 2, 1
`float64`	decimals	`NaN`	8
`bool`	`True` or `False`	not possible	1
`str`	text	`NaN`	varies
`category`	text or numbers from a fixed set	`NaN`	1, 2 or more
`datetime64[us, UTC]`	a point in time	`NaT`	8
`Int64`, `Int16`, `boolean`	whole numbers or booleans that can be missing	`<NA>`	1 more than the plain type
`object`	any Python object	anything	8 for the pointer, plus the object

The column "missing value" is the reason that types matter beyond memory. An integer column cannot hold a missing value, so one null changes the column to `float64`. Chapter 5 uses this.

Text: str, object and category

In pandas 3.0, a column of text read from a file has the dtype `str`. pandas stores it with the pyarrow library, in a compact form. In pandas 2.x the same column had the dtype `object`, with one Python string for each row. Measure the three representations of three columns:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
print(calls["status"].dtype.storage)
print(f"{'column':8s} {'str':>9s} {'object':>9s} {'category':>9s}")
for col in ["status", "codec", "cell_id", "caller"]:
    s = calls[col]
    print(f"{col:8s} {s.memory_usage(deep=True):>9,d} "
          f"{s.astype(object).memory_usage(deep=True):>9,d} "
          f"{s.astype('category').memory_usage(deep=True):>9,d}")
```

```
pyarrow
column         str    object  category
status     312,354 1,132,354    20,191
codec      241,415   982,857    20,172
cell_id    304,193 1,169,484    20,316
caller     260,132 1,080,132    48,007
```

For `status` the `object` form needs 1,132,354 bytes and the `str` form 312,354 bytes: about a quarter. The `category` form needs 20,191 bytes, because it holds the four words once and a one-byte code for each of 20,000 rows. A column with few distinct values is a very good candidate.

A category is two arrays: the list of categories and an array of integer codes.

```
import pandas as pd

from pdbook.io import load_calls

status = load_calls()["status"].astype("category")
print(status.cat.categories.tolist())
print(status.cat.codes.dtype, status.cat.codes.head(4).tolist())
print(status.head(4).tolist())
```

```
['answered', 'busy', 'failed', 'noanswer']
int8 [0, 1, 0, 0]
['answered', 'busy', 'answered', 'answered']
```

Comparison and grouping work on the codes, so a category column is also faster for these operations (Chapter 18 measures it). Three properties of the category dtype surprise people:

```
import pandas as pd

from pdbook.io import load_calls

status = load_calls()["status"].astype("category")
try:
    status.iloc[0] = "weird"
except TypeError as error:
    print("TypeError:", str(error)[:58])

levels = pd.CategoricalDtype(["answered", "busy", "failed", "noanswer", "other"])
typed = load_calls().assign(status=lambda d: d["status"].astype(levels))
print(typed.groupby("status")["call_id"].count().to_dict())
print(typed.groupby("status", observed=False)["call_id"].count().to_dict())

rank = pd.Series(pd.Categorical(["low", "high", "mid"],
                                categories=["low", "mid", "high"], ordered=True))
print(rank.sort_values().tolist(), (rank > "low").tolist())
```

```
TypeError: Cannot setitem on a Categorical with a new category (weird
{'answered': 15526, 'busy': 1428, 'failed': 1033, 'noanswer': 2013}
{'answered': 15526, 'busy': 1428, 'failed': 1033, 'noanswer': 2013, 'other': 0}
['low', 'mid', 'high'] [False, True, True]
```

First, a new value that is not in the categories raises an error. You must add the category first. Second, `groupby` on a category column returns only the categories that occur in pandas 3.0 (`observed=True` is the new default). Write `observed=False` when you want a row for the empty category, for example in a report that has a fixed layout. Third, an ordered category sorts and compares by the order that you gave, not by the alphabet.

Integers: small types and silent wrap

A column of whole numbers needs only as many bytes as its largest value. `call_id` goes up to 20,000, so `int16` (range ±32,767) holds it in 2 bytes instead of 8. `pd.to_numeric(..., downcast="integer")` picks the smallest type that fits:

```
import numpy as np
import pandas as pd

from pdbook.io import load_calls

ids = load_calls()["call_id"]
small = pd.to_numeric(ids, downcast="integer")
print(ids.dtype, "->", small.dtype)
print(ids.memory_usage(), "->", small.memory_usage())
print(np.iinfo("int16").min, np.iinfo("int16").max)
```

```
int64 -> int16
160132 -> 40132
-32768 32767
```

The risk is that a type that is too small does not fail. The value wraps around, in the same way as a counter in a telephone exchange that goes from 255 back to 0:

```
import pandas as pd

print(pd.Series([300]).astype("int8").tolist())

s = pd.Series([100, 100], dtype="int8")
print((s + s).tolist())
print(s.sum(), s.sum().__class__.__name__)

try:
    s + 1000
except OverflowError as error:
    print("OverflowError:", str(error))
```

```
[44]
[-56, -56]
200 int64
OverflowError: Python integer 1000 out of bounds for int8
```

There are four different results:

- The conversion of 300 to `int8` gives 44, with no error. The value is 300 minus 256.
- The sum of two `int8` columns gives −56, with no error. The value 200 does not fit, and the result wraps.
- The `sum()` method gives 200, because it adds in `int64`.
- A Python scalar that is out of range raises an error.

The silent cases are the dangerous ones. Narrow an integer column only when you know the range of the data. Test the range with `min()` and `max()` before you convert. The function `to_numeric(downcast=...)` does this check for you.

Missing integers: the nullable types

A column of whole numbers with one null becomes `float64`. For a count of seconds, that is the wrong type: 24 becomes `24.0`. The nullable integer dtypes (capital `I`) keep integers and mark missing values as `<NA>`:

```
import pandas as pd

from pdbook.io import load_calls

duration = load_calls()["duration_s"]
whole = duration.astype("Int16")
print(duration.dtype, "->", whole.dtype)
print(duration.memory_usage(), "->", whole.memory_usage())
print(whole.head(3).tolist(), whole.isna().sum())
print(duration.sum() == whole.sum())

values = pd.Series([1, None, 3], dtype="Int64")
print((values == 1).tolist())
try:
    bool(pd.NA)
except TypeError as error:
    print("TypeError:", error)
```

```
float64 -> Int16
160132 -> 60132
[24, 0, 174] 1033
True
[True, <NA>, False]
TypeError: boolean value of NA is ambiguous
```

The `Int16` column keeps the same sum. The comparison `values == 1` gives `<NA>` for the missing value, because "is nothing equal to 1?" has no answer. That is the logic of SQL, and it differs from NumPy, where `NaN == 1` is `False`. A missing value in an `if` is an error, so pandas does not let you take a decision on something you do not know. Chapter 5 uses this.

Build: shrink

`shrink` is a loop over the columns with three rules from this chapter. It does three things that a careful person does by hand, and it does them without the typing errors:

```
import pandas as pd

from pdbook.io import load_calls


def shrink(df: pd.DataFrame, max_unique_share: float = 0.5, float_to_int: bool = False) ->
pd.DataFrame:
    out = df.copy()
    for col in out.columns:
        s = out[col]
        if isinstance(s.dtype, pd.StringDtype) or s.dtype == object:
            if len(s) and s.nunique() / len(s) <= max_unique_share:
                out[col] = s.astype("category")
        elif pd.api.types.is_bool_dtype(s):
            continue
        elif pd.api.types.is_integer_dtype(s) and not isinstance(s.dtype,
pd.api.extensions.ExtensionDtype):
            out[col] = pd.to_numeric(s, downcast="integer")
        elif float_to_int and pd.api.types.is_float_dtype(s):
            whole = s.dropna()
            if len(whole) and (whole == whole.round()).all():
                small = pd.to_numeric(whole.astype("int64"), downcast="integer").dtype
                out[col] = s.astype(str(small).capitalize())
    return out


calls = load_calls()
small = shrink(calls, float_to_int=True)
before = calls.memory_usage(deep=True).sum()
after = small.memory_usage(deep=True).sum()
print(f"{before:,d} -> {after:,d} bytes ({after / before:.0%})")
print(small.dtypes.astype(str).to_dict())
```

```
2,017,698 -> 788,290 bytes (39%)
{'call_id': 'int16', 'start': 'datetime64[us, UTC]', 'caller': 'category', 'callee': 'str',
'duration_s': 'Int16', 'status': 'category', 'cell_id': 'category', 'codec': 'category',
'roaming': 'bool'}
```

The table uses 39% of its memory after the change. The columns `callee` and `start` stay as they were. `callee` has 14,617 distinct values in 20,000 rows, which is more than the limit of 50% in `max_unique_share`. `shrink` leaves it as text.

The next section measures what a category saves here. The module `pdbook/shrink.py` also holds `memory_report` and `same_content`. The second one is the test: it casts the small table back to the original dtypes and compares it with the original.

```
from pdbook.io import load_calls
from pdbook.shrink import memory_report, same_content, shrink

calls = load_calls()
small = shrink(calls, float_to_int=True)
print(same_content(calls, small))
print(memory_report(calls, small)[["before", "after"]].to_string())
```

```
True
                         before                after
call_id                   int64                int16
start       datetime64[us, UTC]  datetime64[us, UTC]
caller                      str             category
callee                      str                  str
duration_s              float64                Int16
status                      str             category
cell_id                     str             category
codec                       str             category
roaming                    bool                 bool
```

`same_content` returns `True`. The two tables say the same, and one needs less than half the memory. Write this test every time you shrink a table.

When not to shrink

Do not shrink a table that is small. The saving is in the order of megabytes, and the cost is a type that surprises the next person. Do not narrow integers that you add or multiply in the same dtype, because the sum wraps.

A category column with nearly as many categories as rows saves little. The column `callee` has 14,617 distinct values in 20,000 rows:

```
import pandas as pd

from pdbook.io import load_calls

callee = load_calls()["callee"]
as_category = callee.astype("category")
print(callee.memory_usage(deep=True), as_category.memory_usage(deep=True))
print(len(as_category.cat.categories))
```

```
400132 334300
14617
```

The saving is 16%, against 94% for `status`. The list of categories holds nearly every string anyway.

Two category columns that have different categories do not stay categories when you combine them:

```
import pandas as pd

a = pd.DataFrame({"k": pd.Categorical(["x", "y"])})
b = pd.DataFrame({"k": pd.Categorical(["y", "z"])})
same = pd.DataFrame({"k": pd.Categorical(["y", "x"])})
print(pd.concat([a, b])["k"].dtype)
print(pd.concat([a, same])["k"].dtype)
print(a.merge(b, on="k")["k"].dtype)
```

```
str
category
str
```

The first line is `str`, and the memory gain is gone. The second line has the same categories on both sides, and the result is still `category`. When you plan to combine tables, give both sides one `CategoricalDtype` object (Chapter 9).

How it works

A dtype fixes how many bytes each value takes and how the bytes are read. An `int16` column is an array of 2-byte numbers: 20,000 values need 40,000 bytes, and a value that does not fit in 16 bits wraps around. A `category` column is two arrays.

The first array holds each distinct value once, the *categories*. The second holds one small integer code for each row, and the code is the position of the value in the first array. Comparison and grouping work on the codes, and the text is only looked up when you display it.

The memory of a category column is the memory of its codes plus the memory of its categories. You can check this against `memory_usage`:

```
import numpy as np
import pandas as pd

from pdbook.io import load_calls

status = load_calls()["status"].astype("category")
codes = status.cat.codes.to_numpy()
categories = status.cat.categories

print(codes.dtype, codes.nbytes)
print(categories.tolist(), int(categories.memory_usage(deep=True)))
print(codes.nbytes + int(categories.memory_usage(deep=True)),
int(status.memory_usage(index=False, deep=True)))

print({name: np.dtype(name).itemsize for name in ["int8", "int16", "int32", "int64",
"float64", "bool"]})
print(np.array_equal(status.cat.categories[codes].to_numpy(dtype=object),
status.to_numpy(dtype=object)))
```

```
int8 20000
['answered', 'busy', 'failed', 'noanswer'] 59
20059 20059
{'int8': 1, 'int16': 2, 'int32': 4, 'int64': 8, 'float64': 8, 'bool': 1}
True
```

The codes are `int8`, one byte for each of 20,000 rows. The four categories cost 59 bytes. The sum is 20,059 bytes, and it equals the memory of the column without its index. (The 20,191 of the first table in this chapter also counts the 132 bytes of the index.) The last line shows that indexing the categories with the codes gives back every original value. The `itemsize` line is the table of the chapter: the bytes of each fixed-size type.

- **Invariant.** The bytes of a column are the rows times the itemsize, plus any variable part (the text, the categories).
- **Failure.** A dtype that is too small wraps around without an error.

- **Cost.** A category needs a lookup of codes when you display or join. A column with nearly as many categories as rows saves little.

Reference

```
import pandas as pd

from pdbook.shrink import memory_report, same_content, shrink
from pdbook.sigs import show_signature

show_signature(pd.to_numeric, ["errors", "downcast"])
show_signature(pd.Series.astype, ["dtype", "errors"])
show_signature(pd.DataFrame.memory_usage, ["index", "deep"])
show_signature(pd.CategoricalDtype, ["categories", "ordered"])
show_signature(shrink)
show_signature(memory_report)
show_signature(same_content)
```

```
to_numeric(
    errors: 'DateTimeErrorChoices' = 'raise'
    downcast: "Literal['integer', 'signed', 'unsigned', 'float'] | None" = None
)  # 2 more parameters
NDFrame.astype(
    dtype
    errors: 'IgnoreRaise' = 'raise'
)  # 2 more parameters
DataFrame.memory_usage(
    index: 'bool' = True
    deep: 'bool' = False
)  # 1 more parameters
CategoricalDtype(
    categories=None
    ordered: 'Ordered' = False
)
shrink(
    df: 'pd.DataFrame'
    max_unique_share: 'float' = 0.5
    float_to_int: 'bool' = False
)
memory_report(
    before: 'pd.DataFrame'
    after: 'pd.DataFrame'
)
same_content(
    before: 'pd.DataFrame'
    after: 'pd.DataFrame'
)
```

Parameter	Default	What it does here	What goes wrong
`to_numeric(errors=...)`	`'raise'`	what to do with text that is not a number: raise, or `'coerce'` to `NaN`	—
`to_numeric(downcast=...)`	`None`	`'integer'` picks the smallest integer type that holds every value	— (`call_id`: 160,132 → 40,132 bytes)
`astype(dtype)`	required	convert to this type	`astype("int8")` on 300 gives 44, with no error
`astype(errors=...)`	`'raise'`	what to do when a value does not convert; the printed type allows `'raise'` and `'ignore'`	—
`memory_usage(deep=...)`	`False`	`True` counts the bytes of the text itself	—
`memory_usage(index=...)`	`True`	include the index	132 bytes for a `RangeIndex` of 20,000 rows
`CategoricalDtype(categories=...)`	`None`	the allowed values, in order	a value outside the list raises `TypeError` on assignment
`CategoricalDtype(ordered=...)`	`False`	`True` makes comparison and sorting follow the order	—
`shrink(df, max_unique_share=...)`	`0.5`	the largest share of distinct values for a text column to become `category`	`callee` (73% distinct) stays `str`
`shrink(df, float_to_int=...)`	`False`	`True` turns whole-number floats into a nullable integer	`duration_s`: 160,132 → 60,132 bytes
`memory_report(before, after)`	required	bytes for each column, before and after	—
`same_content(before, after)`	required	`True` when `after`, cast back, equals `before`	—

`shrink` returns a new DataFrame and does not change its argument.

What you have now

- You know what each dtype costs and what it cannot hold.
- You use `str` for text, `category` for text with few distinct values and the nullable types for integers with nulls.
- You know that a small integer type wraps without an error.
- You have a function that shrinks a table and a test that proves the content is the same.

The next chapter takes the missing values that this chapter only marked, and decides what to do with them.

Next: Chapter 5, Missing data.

5. Missing data

> **Build** — `null_report(df)` and `determinants(df)`: a count of the nulls in each column, and a test that finds the columns that decide whether a value is missing. A null that a column decides is a fact. A null that nothing decides is lost data.
>
> **Theory kept** — A missing value has a cause, and the right treatment depends on the cause. Most aggregations skip missing values. `groupby` drops rows with a missing key. `NaN` is not equal to `NaN`.
>
> **Source** — New. Library: `pdbook/nulls.py`.
>
> **Run this first** — `uv run python -m examples.ch05_missing`, which is `examples/ch05_missing.py`. Exercises: `exercises/05.md`. Solutions: `exercises/solutions/05.md`.

Three kinds of missing

Chapter 1 found 1,033 nulls in `duration_s`, 277 in `cell_id` and 4,474 in `codec`. The three columns look the same in `info()`. They are not the same problem:

- In these data, a call with the status `failed` never started. It has no duration. The value does not exist.
- A call that was not answered has no codec, because no media flowed. The value does not exist.
- A call that was answered over a cell has a cell. If the record has none, the record lost it. The value exists, and we do not know it.

The first two are facts about the call. The third is damage. A good analysis treats them differently. Fill the first two with a value only if the question needs it. Never fill the third with a guess without saying so.

How pandas marks a missing value

dtype	Marker	`x == x`
`float64`	`NaN`	`False`
`str`	`NaN`	`False`
`datetime64`	`NaT`	`False`
`Int64`, `boolean`, `string`	`<NA>`	`<NA>`
`object`	`None` or `NaN`	`False`

The next block checks the table. For each dtype it makes a Series with one missing value, and prints the marker and the result of `x == x`:

```
import numpy as np
import pandas as pd

cases = {
    "float64": pd.Series([np.nan]),
    "str": pd.Series([None], dtype="str"),
    "datetime64": pd.Series([pd.NaT]),
    "Int64": pd.Series([None], dtype="Int64"),
    "boolean": pd.Series([None], dtype="boolean"),
    "string": pd.Series([None], dtype="string"),
    "object": pd.Series([None], dtype=object),
}
for name, s in cases.items():
    print(f"{name:10s} {str(s.iloc[0]):5s} x == x: {str((s == s).iloc[0]):5s} isna:
{s.isna().iloc[0]}")
```

```
float64    nan   x == x: False isna: True
str        nan   x == x: False isna: True
datetime64 NaT   x == x: False isna: True
Int64      <NA>  x == x: <NA>  isna: True
boolean    <NA>  x == x: <NA>  isna: True
string     <NA>  x == x: <NA>  isna: True
object     None  x == x: False isna: True
```

`isna()` and `notna()` work on all of them. Use these two methods and never compare with `== np.nan`, because `NaN` is not equal to itself:

```
import numpy as np
import pandas as pd

print(np.nan == np.nan)
print(pd.Series([np.nan, 1.0]).eq(np.nan).tolist())
print(pd.Series([np.nan, 1.0]).isna().tolist())
```

```
False
[False, False]
[True, False]
```

The comparison with `NaN` is `False` for every row, including the row that holds `NaN`. `isna()` is the reliable test.

Count them, then ask why

The two functions in `pdbook/nulls.py` answer two questions. `null_report` counts. `determinants` tests whether another column decides the null. Run them on the call records:

```
from pdbook.io import load_calls
from pdbook.nulls import determinants, null_report

calls = load_calls()
print(null_report(calls))
print(determinants(calls))
```

```
            nulls   share
duration_s   1033  0.0516
cell_id       277  0.0138
codec        4474  0.2237
{'duration_s': ['status'], 'cell_id': [], 'codec': ['status']}
```

The result for `duration_s` and `codec` is `['status']`. The result for `cell_id` is an empty list. Look at the idea behind the function. A column `by` decides a column `col` when, in each group of `by`, the nulls of `col` are all or none. You can see it by hand:

```
from pdbook.io import load_calls

calls = load_calls()
share = calls.groupby("status")[["duration_s", "codec", "cell_id"]].agg(lambda s:
s.isna().mean())
print(share.round(3).to_string())
```

```
           duration_s  codec  cell_id
status
answered          0.0    0.0    0.014
busy              0.0    1.0    0.014
failed            1.0    1.0    0.015
noanswer          0.0    1.0    0.014
```

In the `duration_s` column the share is 0 for three statuses and 1 for `failed`. In `codec` it is 0 for `answered` and 1 for the other three. The `cell_id` column has the same small share, about 1.4%, in each status.

`determinants` finds only exact relations. A weaker relation is invisible to it, and `groupby` shows it. Look at the share of lost cells for each codec:

```
from pdbook.io import load_calls

calls = load_calls()
lost = calls["cell_id"].isna()
print((100 * lost.groupby(calls["codec"]).mean()).round(2).to_dict())
```

```
{'AMR-NB': 2.09, 'AMR-WB': 1.61, 'EVS': 0.48}
```

The share is 0.48% for `EVS` and 2.09% for `AMR-NB`. A value that is lost "at random" does not do this. The cause is in the process that made the data. The generator chooses the codec from the technology of the cell. A call without a cell gets the choice for a 4G cell, where `EVS` is rare. A real table has the same kind of cause.

A step in the pipeline reads a field, and a missing field takes a default branch. The lesson has two parts. A null that no column decides can still be associated with a column. And the association points at the step that you must read.

Aggregations skip nulls

Most reductions (`mean`, `sum`, `std`, `count`, `min`, `max`) skip missing values. `size` and `len` do not, because they count rows:

```
from pdbook.io import load_calls

duration = load_calls()["duration_s"]
print(len(duration), duration.count())
print(round(duration.mean(), 2))
print(duration.mean(skipna=False))
print(round(duration.fillna(0).mean(), 2))
```

```
20000 18967
108.73
nan
103.12
```

Three different means from one column. The skipping mean is 108.73. It divides by 18,967, the count of non-null values. The mean with `skipna=False` is `NaN`, because one unknown makes the whole mean unknown. The mean after `fillna(0)` is 103.12.

It divides by 20,000, and it treats the 1,033 failed calls as calls of length zero. None of the three is wrong. They answer different questions: "the mean duration of calls that have a duration", "the mean is unknown", and "the mean duration if a failed call counts as zero". Write the question in a comment next to the line.

Fill, drop or keep

`fillna` replaces missing values. `dropna` removes rows. Before you use either, say what you lose.

```
from pdbook.io import load_calls

calls = load_calls()
print(len(calls.dropna()))
print(len(calls.dropna(subset=["cell_id"])))
print(len(calls.dropna(how="all")))
print(len(calls.dropna(thresh=9)))
```

```
15314
19723
20000
15314
```

`dropna()` with no arguments drops every row that has at least one null. Here it removes 4,686 rows, 23% of the table. Of these, 4,474 have a null `codec`, which means that the call was not answered. The other 212 are answered calls with a lost `cell_id`. The table that remains holds only answered calls.

The third call (`how="all"`) drops nothing, because no row is null in every column. The fourth call (`thresh=9`) keeps rows with at least 9 non-null values. The table has 9 columns, so it is the same as the first call. A report on "all calls" computed from it is a report on answered calls. Use `subset=` to name the columns whose nulls matter for your question.

Group keys that are missing

`groupby` drops the rows whose key is missing, without a message:

```
from pdbook.io import load_calls

calls = load_calls()
default = calls.groupby("cell_id")["call_id"].count()
keep = calls.groupby("cell_id", dropna=False)["call_id"].count()
print(int(default.sum()), int(keep.sum()), len(calls))
print(keep[keep.index.isna()].to_dict())
```

```
19723 20000 20000
{nan: 277}
```

The default total is 19,723, and 277 calls are not in the report. With `dropna=False`, the missing key becomes one group and the total is 20,000. A report that adds up to less than the table is one of the most common silent errors in pandas. Check `sum()` of the groups against `len()` of the table.

Time series: gaps

In a time series a missing value is often a missing row. The table `kpi.csv` has one row for each cell and each 5 minutes, and some rows are not in the file. Put the series on a full grid to see the gaps:

```
import pandas as pd

from pdbook.io import load_kpi

kpi = load_kpi()
cell = kpi[kpi["cell_id"] == "BRU-004"].drop_duplicates("ts").set_index("ts")["prb_util_pct"]
grid = pd.date_range(cell.index.min(), cell.index.max(), freq="5min")
full = cell.reindex(grid)
print(len(cell), len(full), int(full.isna().sum()))

gap = full.isna()
run_id = (gap != gap.shift()).cumsum()
longest = gap.groupby(run_id).sum().max()
print("longest gap:", int(longest), "steps =", int(longest) * 5, "minutes")
```

```
822 864 42
longest gap: 24 steps = 120 minutes
```

The cell has 42 missing steps. The longest gap is 24 steps, which is two hours: the outage of the cell. Three tools fill a gap, and each makes a claim:

```
import pandas as pd

from pdbook.io import load_kpi

kpi = load_kpi()
cell = kpi[kpi["cell_id"] == "BRU-004"].drop_duplicates("ts").set_index("ts")["prb_util_pct"]
full = cell.reindex(pd.date_range(cell.index.min(), cell.index.max(), freq="5min"))

print("missing before     :", int(full.isna().sum()))
print("ffill(limit=3)     :", int(full.ffill(limit=3).isna().sum()))
print("interpolate(limit=3):", int(full.interpolate(limit=3).isna().sum()))
print("interpolate()      :", int(full.interpolate().isna().sum()))
```

```
missing before     : 42
ffill(limit=3)     : 21
interpolate(limit=3): 21
interpolate()      : 0
```

`ffill` repeats the last value: "the load did not change". `interpolate` draws a line between the two ends: "the load changed steadily".

Both are claims about a period in which you have no data. Use `limit` to fill short gaps (one lost sample) and to leave long gaps (an outage) visible. The last line fills all 42 steps, including the two-hour outage with a straight line. The result looks like a measurement, and it is not one.

Three-valued logic

The nullable dtypes follow the rules of SQL. An unknown value in a condition gives an unknown result, unless the other side decides the result:

```
import pandas as pd

a = pd.array([True, False, None], dtype="boolean")
print((a | True).tolist())
print((a & False).tolist())
print((a | pd.NA).tolist())
```

```
[True, True, True]
[False, False, False]
[True, <NA>, <NA>]
```

"Unknown or true" is true, because one side is enough. "Unknown and false" is false. "Unknown or unknown" is unknown. A mask that contains `<NA>` cannot select rows, so you must decide with `fillna(False)` or `fillna(True)`, and the choice is part of your question.

How it works

A missing value needs a place in memory. A `float64` array has one: `NaN` is a special bit pattern of the IEEE 754 standard. By the rules of the standard it is not equal to any number, itself included. An integer array has no such pattern,

because every bit pattern of an `int64` is a number.

This is why a column of whole numbers that gets a null becomes `float64`. The nullable types (`Int64`, `boolean`) solve it in another way: they keep the values in an array and add a second array of booleans, the mask, that says which positions are missing.

All of this is visible with a few lines:

```
import numpy as np
import pandas as pd

s = pd.Series([1.0, np.nan, 3.0])
print((s != s).tolist(), s.isna().tolist())
print(np.array([1, 2, 3]).dtype, np.array([1, np.nan, 3]).dtype)

whole = pd.array([1, 2, 3], dtype="int64")
nullable = pd.array([1, None, 3], dtype="Int64")
print(nullable.isna().tolist(), nullable.to_numpy(dtype="float64", na_value=np.nan).tolist())
print(whole.nbytes, nullable.nbytes)
```

```
[False, True, False] [False, True, False]
int64 float64
[False, True, False] [1.0, nan, 3.0]
24 27
```

The first line shows the test that `isna` uses for floats: a value that is not equal to itself is missing. The second line shows the upcast: one `nan` makes the NumPy array `float64`. The last line shows the price of the mask: 3 values need 24 bytes as `int64`, and 27 bytes as `Int64`, which is one more byte for each value.

- **Invariant.** A missing value is a property of a position: a special float, or a bit in a mask.
- **Failure.** A comparison with `==` does not find the missing values. The float is not equal to itself, and the mask gives `<NA>`.
- **Cost.** The nullable type needs one byte more for each value, and each result must combine with the mask.

Reference

```
import pandas as pd

from pdbook.nulls import determinants, null_report
from pdbook.sigs import show_signature

show_signature(pd.DataFrame.isna)
show_signature(pd.Series.fillna, ["value", "limit"])
show_signature(pd.DataFrame.dropna, ["axis", "how", "thresh", "subset"])
show_signature(pd.DataFrame.groupby, ["by", "dropna", "observed"])
show_signature(pd.Series.mean, ["skipna"])
show_signature(pd.Series.interpolate, ["method", "limit"])
show_signature(pd.Series.ffill, ["limit"])
show_signature(null_report)
show_signature(determinants)
```

```
DataFrame.isna(
    self
)
NDFrame.fillna(
    value: 'Hashable | Mapping | Series | DataFrame'
    limit: 'int | None' = None
)  # 3 more parameters
DataFrame.dropna(
    axis: 'Axis' = 0
    how: 'AnyAll | lib.NoDefault' = <no_default>
    thresh: 'int | lib.NoDefault' = <no_default>
    subset: 'IndexLabel | AnyArrayLike | None' = None
)  # 3 more parameters
DataFrame.groupby(
    by=None
    observed: 'bool' = True
    dropna: 'bool' = True
)  # 5 more parameters
Series.mean(
    skipna: 'bool' = True
)  # 4 more parameters
NDFrame.interpolate(
    method: 'InterpolateOptions' = 'linear'
    limit: 'int | None' = None
)  # 6 more parameters
NDFrame.ffill(
    limit: 'None | int' = None
)  # 4 more parameters
null_report(
    df: 'pd.DataFrame'
)
determinants(
    df: 'pd.DataFrame'
    max_levels: 'int' = 12
)
```

Parameter	Default	What it does here	What goes wrong
`isna()`	—	a boolean table: `True` where a value is missing	—
`fillna(value)`	required	the value to put in place of a missing value	`fillna(0)` on `duration_s` moves the mean from 108.73 to 103.12
`fillna(limit=...)`	`None`	the largest number of values to fill in a row	—
`dropna(how=...)`	`any` when not set	`any` drops a row with one null, `all` drops a row with only nulls	`dropna()` removes 4,686 of 20,000 rows
`dropna(subset=...)`	`None`	look only at these columns	`subset=["cell_id"]` removes 277 rows
`dropna(thresh=...)`	not set	keep rows with at least this many values	`thresh=9` on 9 columns is the same as `how="any"`
`groupby(dropna=...)`	`True`	`False` keeps a group for the missing key	`True` loses 277 calls: totals of 19,723 against 20,000
`groupby(observed=...)`	`True`	for category keys, show only the groups that occur	`False` gives a row for an empty category
`mean(skipna=...)`	`True`	`False` gives `NaN` when any value is missing	the mean of `duration_s` is `NaN`
`interpolate(method=..., limit=...)`	`'linear'`, `None`	fill a gap along the index	no `limit`: all 42 missing steps filled, the 2-hour outage included. `limit=3` leaves 21
`ffill(limit=...)`	`None`	repeat the last value	`limit=3` leaves 21 of 42
`null_report(df)`	required	nulls and their share, for each column that has any	—
`determinants(df, max_levels=12)`	`12`	for each column with nulls, the columns that decide it	finds exact relations only, not weaker ones

`null_report` returns a DataFrame. `determinants` returns a dict of column name to a list of column names.

What you have now

- You ask why a value is missing, and you test the answer with `determinants`.
- You know that the mean, the sum and the count skip nulls, and that `fillna(0)` changes the mean.
- You know that `dropna()` without `subset` can remove a quarter of a table.
- You check that group totals add up to the table, and you use `dropna=False` when a missing key matters.
- You fill gaps in time series with a limit.

The next chapter cleans the text and dates of a table that is damaged in many more ways.

Next: Chapter 6, Text and date cleanup.

6. Text and date cleanup

> **Build** — `clean_contacts(raw)`: a function that turns the messy contact export into a clean table and returns a second table with each value that it could not repair, so that nothing is guessed in silence.
>
> **Theory kept** — The `.str` accessor applies a text operation to a whole column and passes missing values through. Clean to a form that you can compare, and keep the raw value. A cleaner must return what it rejected.
>
> **Source** — New. Data: `data/contacts_messy.csv`. Library: `pdbook/clean.py`.
>
> **Run this first** — `uv run python -m examples.ch06_cleanup`, which is `examples/ch06_cleanup.py`. Exercises: `exercises/06.md`. Solutions: `exercises/solutions/06.md`.

The mess

The file `data/contacts_messy.csv` is an export of a customer list that several people typed. It has 271 rows and five columns, and every column has several formats. Read all columns as text, so that pandas guesses nothing:

```
import pandas as pd

raw = pd.read_csv("data/contacts_messy.csv", dtype=str)
print(raw.shape)
print(raw.sample(6, random_state=4).to_string())
print(sorted(raw["plan"].unique())[:8])
```

```
(271, 5)
                  name             phone      plan             signup     balance
210        Rik De Smet  +32 444 36 50 82     BASIC         2023-03-23       24.19
39     Quinten Peeters  +32 492 60 19 31     plus   February 27, 2022      123.59
162     Petra  Jacobs          402725725     plus           23-May-25  127.07 EUR
121       Lambert, Rik     0427 77 34 15    basic          29/01/2023      364.03
103       lotte MARTIN     0466 93 07 36  BUSINESS       May 13, 2025         NaN
133    maarten WILLEMS     0400 55 87 97    basic           16-Jan-24         NaN
['BASIC', 'BUSINESS', 'Basic', 'Business', 'PLUS', 'PREPAID', 'Plus', 'Prepaid']
```

The same plan is written `basic`, `Basic`, `BASIC` and `basic` with a space at the end. The phone numbers have five formats. The dates have four formats. The balance has euro signs, thousands separators, a unit, and markers for "no value". Four things are wrong with the names: extra spaces, upper case, lower case and the order "Last, First".

The method is the same for each column: look at the formats, write one rule for each, count what the rules did not catch, and keep that count visible.

The .str accessor

`.str` gives a Series of text the operations of a Python string, applied to each element:

Method	What it does
`strip()`, `lower()`, `casefold()`	trim, change case
`replace(pat, repl, regex=True)`	substitute by pattern
`contains(pat)`, `startswith(s)`, `match(pat)`	a boolean mask
`extract(pat)`	the groups of a regular expression, as columns
`split(sep, expand=True)`	split into columns
`len()`	length of each text

A missing value stays missing. No method fails on it:

```
import pandas as pd

s = pd.Series(["  Basic ", "PLUS", None, "business"])
print(s.str.strip().str.lower().tolist())
print(s.str.len().tolist())
```

```
['basic', 'plus', nan, 'business']
[8.0, 4.0, nan, 8.0]
```

Plan: trim and lower

One rule, then one check that the rule was enough:

```
import pandas as pd

raw = pd.read_csv("data/contacts_messy.csv", dtype=str)
plan = raw["plan"].str.strip().str.lower()
print(raw["plan"].nunique(), "->", plan.nunique())
print(plan.value_counts().to_dict())
print(sorted(set(plan) - {"prepaid", "basic", "plus", "business"}))
```

```
17 -> 5
{'business': 76, 'plus': 72, 'basic': 63, 'prepaid': 59, 'gold': 1}
['gold']
```

Seventeen spellings became five values. The last line is the check: the plan `gold` is not one of the four plans. No spelling rule repairs it, because it is a different value. The cleaner must reject it, not map it to the nearest plan.

Names: compare on a key, not on a pretty form

Two jobs hide behind "clean a name". One is to display it. The other is to compare it. Do not use one result for both. For comparison, build a key: trim, collapse the spaces, turn "Last, First" round and fold the case:

```
import pandas as pd

from pdbook.clean import name_key

raw = pd.read_csv("data/contacts_messy.csv", dtype=str)
examples = pd.Series(["  hugo PEETERS", "Peeters, Hugo", "Hugo  Peeters ", "Van den Berg,
An"])
print(name_key(examples).tolist())
print(raw["name"].str.strip().nunique(), "->", name_key(raw["name"]).nunique())
print("straße".casefold() == "strasse", "straße".lower() == "strasse")
```

```
['hugo peeters', 'hugo peeters', 'hugo peeters', 'an van den berg']
236 -> 172
True False
```

The first three names become one key. The key for `Van den Berg, An` is `an van den berg`. Use `casefold` and not `lower`, because `casefold` also maps the German `ß` to `ss`. The block prints the proof.

A display form is harder than it looks. The method `title()` capitalizes every word:

```
print("van den berg".title())
print("mcdonald".title())
print("o'neil".title())
```

```
Van Den Berg
Mcdonald
O'Neil
```

All three are wrong for the people who own the names: `Van den Berg`, `McDonald` and `O'Neil` are the usual forms. There is no rule that repairs all names. Keep the raw name for display, and use the key for comparison.

Phones: one canonical form

The canonical form for a telephone number is E.164: a plus, the country code and the number, with no space. Belgian mobile numbers are `+32` and nine digits. The five formats in the file reduce to one in four steps, and each step is a `str.replace` with a pattern:

```
import pandas as pd

raw = pd.read_csv("data/contacts_messy.csv", dtype=str)
phone = raw["phone"]
step1 = phone.str.replace(r"[^\d+]", "", regex=True)       # drop spaces
step2 = step1.str.replace(r"^\+", "00", regex=True)         # + becomes 00
step3 = step2.str.replace(r"^0032", "", regex=True)         # remove the country code
step4 = step3.str.replace(r"^0", "", regex=True)            # remove the trunk zero
for i in [1, 4, 7, 2]:
    print(f"{phone.iloc[i]!r:20} -> {('+32' + step4.iloc[i])!r}")
print(step4.str.len().value_counts().to_dict())
```

```
'+32 478 98 50 35'   -> '+32478985035'
'0489 26 66 69'      -> '+32489266669'
'427792646'          -> '+32427792646'
'0032484095226'      -> '+32484095226'
{9: 269, 4: 1, 13: 1}
```

The last line is the check. After the four steps, 269 numbers have 9 digits, and two do not. Those two are the cases that the rules cannot repair: `12 34` is too short, and the UK number `+44 7700 900123` is a valid number of another country. The function `e164` in `pdbook/clean.py` returns `NaN` for them, and `clean_contacts` lists them in the rejects.

Dates: mixed formats

`to_datetime` looks at the first value to choose a format. A column with four formats fails on the first value that does not fit:

```
import pandas as pd

raw = pd.read_csv("data/contacts_messy.csv", dtype=str)
try:
    pd.to_datetime(raw["signup"])
except ValueError as error:
    print("ValueError:", str(error).splitlines()[0][:60])

parsed = pd.to_datetime(raw["signup"].str.strip(), format="mixed", dayfirst=True,
errors="coerce")
print(int(parsed.isna().sum()), "not parsed")
print(raw.loc[parsed.isna(), "signup"].tolist())
```

```
ValueError: time data "18/07/2025" doesn't match format "%Y-%m-%d". You
2 not parsed
['31/02/2024', 'someday']
```

`format="mixed"` parses each element on its own. `errors="coerce"` turns what it cannot parse into `NaT`, and the last line shows which values those are. `31/02/2024` is not a date and `someday` is not a date. The cleaner must reject both. The argument `coerce` alone hides them as `NaT`.

The argument `dayfirst=True` is a decision about the data. The text `07/06/2025` is 7 June in Belgium and 6 July in the United States:

```
import pandas as pd

s = pd.Series(["07/06/2025"])
print(pd.to_datetime(s, dayfirst=True).iloc[0].date())
print(pd.to_datetime(s, dayfirst=False).iloc[0].date())
```

```
2025-06-07
2025-07-06
```

No code can find out which one is right. A column with the days 13 to 31 proves day first, because there is no month 13. A column without them proves nothing. Ask the source, and write the answer next to the line.

Money

The balance column has a euro sign, thousands separators, a unit `EUR`, negative numbers, and markers for no value. The marker `-` is "no value", and `-3.64` is a number. Handle the markers first, and then strip the decoration, and then convert:

```
import pandas as pd

from pdbook.clean import money

raw = pd.read_csv("data/contacts_messy.csv", dtype=str)
sample = pd.Series(["1,234.50", "€ 12.5", "289.83 EUR", "-", "N/A", "", "-3.64", "twelve
euros"])
print(money(sample).tolist())
print(int(raw["balance"].isna().sum()), int((raw["balance"] == "-").sum()),
int((raw["balance"] == "N/A").sum()))
print(int(money(raw["balance"]).isna().sum()))
```

```
[1234.5, 12.5, 289.83, nan, nan, nan, -3.64, nan]
37 18 0
56
```

The first line shows the function on the cases. `twelve euros` becomes `NaN`, like the markers, and that is why `clean_contacts` compares the result with the markers and lists the difference as a reject.

The second line is a surprise. The file has 37 values that `read_csv` already turned into `NaN`, although you asked for text, and only the 18 dashes stayed as text. The default list of missing markers (`N/A`, `NA`, empty) applies to text columns as well. The third line shows that no `N/A` is left. The last line shows 56 missing values after `money`: the 37, the 18 dashes and the one `twelve euros`.

Duplicates

`duplicated` marks the rows that repeat an earlier row. `drop_duplicates` removes them. The important choice is the key, and it is the `subset`:

```
import pandas as pd

from pdbook.clean import e164, name_key

raw = pd.read_csv("data/contacts_messy.csv", dtype=str)
phone = e164(raw["phone"])
print("raw rows         :", len(raw))
print("distinct raw text:", raw["phone"].nunique())
print("distinct phones  :", phone.nunique())
print("distinct names   :", name_key(raw["name"]).nunique())
print("missing phones   :", int(phone.isna().sum()))
print("duplicated() on a key with two missing values:", int(phone.duplicated().sum()))
print("of which missing :", int((phone.duplicated() & phone.isna()).sum()))
```

```
raw rows         : 271
distinct raw text: 268
distinct phones  : 244
distinct names   : 172
missing phones   : 2
duplicated() on a key with two missing values: 26
of which missing : 1
```

The raw text has more distinct values than the canonical phones, because one number is written in several ways. After the cleaning, 244 phones are left for 271 rows. The name is a bad key: 172 distinct names for 244 phones, because different people have the same name. A name key merges different people.

The last two lines show a trap. `duplicated` treats two missing values as equal, so the second row with a missing phone counts as a duplicate of the first. A row that has no key is not a duplicate of anything. `clean_contacts` excludes missing keys from the test.

Build: clean_contacts

The function applies the rules in this chapter and returns two tables. The second table is the one that makes the first one trustworthy:

```
import pandas as pd

from pdbook.clean import clean_contacts

raw = pd.read_csv("data/contacts_messy.csv", dtype=str)
clean, rejects = clean_contacts(raw)

print(len(raw), "->", len(clean))
print(rejects.to_string(index=False))
print(clean.dtypes.astype(str).to_dict())
print(clean.head(4).to_string())
```

```
271 -> 246
 row  column             raw
  59   phone           12 34
  62   phone +44 7700 900123
 207    plan            gold
  82  signup      31/02/2024
  89  signup         someday
  13 balance    twelve euros
{'name': 'str', 'name_key': 'str', 'phone': 'str', 'plan': 'str', 'signup': 'datetime64[us]',
'balance': 'float64'}
           name      name_key         phone      plan     signup  balance
0   Rik Mertens   rik mertens  +32423333898   prepaid 2023-10-10    32.38
1   Petra Claes   petra claes  +32478985035  business 2025-07-18    84.76
2  hugo PEETERS  hugo peeters  +32484095226   prepaid 2022-05-20   145.25
3   ines JACOBS   ines jacobs  +32426366458      plus 2023-04-22   300.49
```

The input has 271 rows. The output has 246: 25 duplicates are removed, and nothing else. Six values are rejected, one for each kind of damage that no rule repairs. Each reject has the row number in the raw table and the raw text, so a person can open the source and decide.

A cleaner needs tests like any other function. Three properties hold for every input:

```
import pandas as pd

from pdbook.clean import clean_contacts

raw = pd.read_csv("data/contacts_messy.csv", dtype=str)
clean, rejects = clean_contacts(raw)

phones = clean["phone"].dropna()
print(bool(phones.str.match(r"^\+32\d{9}$").all()))
print(bool(phones.is_unique))
print(bool(clean["plan"].dropna().isin(["prepaid", "basic", "plus", "business"]).all()))
print(bool(clean["name"].str.contains(r"^\s|\s$|\s\s").any()))
```

```
True
True
True
False
```

The first three lines are `True`: every phone has the canonical form, no phone repeats, and every plan is one of the four. The last line is `False`: no name has a leading, trailing or double space. Run the same four checks on the real file, every time that it arrives.

How it works

The `.str` accessor takes a function that works on one string and applies it to every element of the column. A missing element goes through unchanged. In pandas 3.0, a `str` column is stored with the Arrow library. The common functions (`strip`, `lower`, `contains`, `replace`) are Arrow compute kernels, so they run in compiled code. A function that Arrow does not have falls back to a Python loop.

You can write the accessor in a list comprehension and get the same result. A literal replace and a pattern replace are two different functions, and the argument `regex` chooses between them:

```
import pandas as pd

plan = pd.Series(["  Basic ", "PLUS", None, "business "])

by_accessor = plan.str.strip().str.lower()
by_loop = pd.Series([None if pd.isna(x) else x.strip().lower() for x in plan])
print(by_accessor.isna().tolist() == by_loop.isna().tolist())
print(by_accessor.dropna().tolist() == by_loop.dropna().tolist())
print(by_accessor.tolist())

phone = pd.Series(["+32 477 12 34 56"])
print(phone.str.replace("+", "00", regex=False).iloc[0])
try:
    phone.str.replace("+", "00", regex=True)
except Exception as error:
    print(type(error).__name__, "-", str(error)[:48])
print(phone.str.replace(r"^\+", "00", regex=True).iloc[0])
print(plan.dtype.storage)
```

```
True
True
['basic', 'plus', nan, 'business']
0032 477 12 34 56
ArrowInvalid - Invalid regular expression: no argument for repe
0032 477 12 34 56
pyarrow
```

The accessor and the loop give the same strings, with `None` kept as missing. A `+` is a plain character for `regex=False` and a repeat operator for `regex=True`, which is why the second call raises an error. With Arrow strings the error comes from the Arrow library (`ArrowInvalid`), and not from Python `re`. The fix is to escape the character or to anchor it, as `r"^\+"` does. The last line shows the storage of the column: `pyarrow`.

- **Invariant.** A `.str` method is an element-wise function, and a missing element stays missing.
- **Failure.** A literal that you wrote as a pattern, or a pattern that you wrote as a literal.
- **Cost.** The common methods are Arrow kernels. A method with no kernel is a Python loop and is as slow as one.

Reference

```
import pandas as pd

from pdbook.clean import clean_contacts, e164, money, name_key
from pdbook.sigs import show_signature

show_signature(pd.Series.str.replace, ["pat", "repl", "regex"])
show_signature(pd.Series.str.strip)
show_signature(pd.Series.str.match, ["pat"])
show_signature(pd.to_datetime, ["format", "dayfirst", "errors", "utc"])
show_signature(pd.DataFrame.drop_duplicates, ["subset", "keep"])
show_signature(pd.DataFrame.duplicated, ["subset", "keep"])
show_signature(e164)
show_signature(name_key)
show_signature(money)
show_signature(clean_contacts)
```

```
StringMethods.replace(
    pat: 'str | re.Pattern | dict'
    repl: 'str | Callable | None' = None
    regex: 'bool' = False
)  # 4 more parameters
StringMethods.strip(
    self
    to_strip=None
)
StringMethods.match(
    pat: 'str | re.Pattern'
)  # 4 more parameters
to_datetime(
    errors: 'DateTimeErrorChoices' = 'raise'
    dayfirst: 'bool' = False
    utc: 'bool' = False
    format: 'str | None' = None
)  # 6 more parameters
DataFrame.drop_duplicates(
    subset: 'Hashable | Iterable[Hashable] | None' = None
    keep: 'DropKeep' = 'first'
)  # 3 more parameters
DataFrame.duplicated(
    subset: 'Hashable | Iterable[Hashable] | None' = None
    keep: 'DropKeep' = 'first'
)  # 1 more parameters
e164(
    phone: 'pd.Series'
)
name_key(
    name: 'pd.Series'
)
money(
    s: 'pd.Series'
)
clean_contacts(
    raw: 'pd.DataFrame'
)
```

Parameter	Default	What it does here	What goes wrong
`str.replace(pat, repl)`	required, `None`	the pattern and what replaces it	—
`str.replace(regex=...)`	`False`	`True` reads `pat` as a regular expression	`"+"` with `regex=True` raises `ArrowInvalid`
`str.strip(to_strip=...)`	`None`	remove white space, or the characters named	—
`str.match(pat)`	required	a mask for strings that match at the start	a pattern with no `^` or `$` accepts longer text
`to_datetime(format=...)`	`None`	`"mixed"` parses each element on its own	without it, the first value decides the format and 18/07/2025 raises `ValueError`
`to_datetime(dayfirst=...)`	`False`	read `07/06/2025` as day first	`False` gives 2025-07-06 and `True` gives 2025-06-07
`to_datetime(errors=...)`	`'raise'`	`'coerce'` turns a value that does not parse into `NaT`	`coerce` hides 31/02/2024 and `someday` as `NaT`
`to_datetime(utc=...)`	`False`	convert to UTC	—
`drop_duplicates(subset=...)`	`None`	the columns that define a duplicate	two missing keys count as equal: 26 duplicates, one of them a missing key
`drop_duplicates(keep=...)`	`'first'`	which copy stays	—
`duplicated(subset=..., keep=...)`	`None`, `'first'`	the mask of the repeats	`keep=False` marks every copy, including the first
`e164(phone)`	required	a Series of phone numbers in the form `+32` and nine digits, `NaN` when it does not fit	—
`name_key(name)`	required	a key for comparing names: trimmed, one space, case-folded, `Last, First` turned round	172 distinct keys for 244 phones
`money(s)`	required	text to float, markers to `NaN`	—
`clean_contacts(raw)`	required	`(clean, rejects)`	271 rows in, 246 out, 6 rejects

`e164`, `name_key` and `money` return a Series. `clean_contacts` returns a tuple of two DataFrames.

What you have now

- You read dirty files as text, and you look at the formats before you write a rule.
- You use `.str` methods for one rule for each format, and you count what the rules missed.
- You compare names on a key and keep the raw text for display.
- You parse dates with `format="mixed"` and a decision about `dayfirst`, and you list the values that did not parse.
- You choose a duplicate key with care, and you know that missing keys are not duplicates.
- You return the rejects with the clean table.

The next chapter adds columns to a clean table, and shows why the way you write the rule matters for speed.

Next: Chapter 7, New columns.

7. New columns

> **Build** — `rate_vector(calls, subs, plans)` : a function that prices each call from three tables, written with whole-column operations, with a proof that a loop and an `apply` give the same bill and are slower by a measured factor.
>
> **Theory kept** — A column operation runs once in compiled code. A function that you pass to `apply` runs once for each row in Python. Write the rule as a column expression, and keep `apply` for the cases that have no column form.
>
> **Source** — New. Library: `pdbook/rating.py` .
>
> **Run this first** — `uv run python -m examples.ch07_new_columns` , which is `examples/ch07_new_columns.py` . Exercises: `exercises/07.md` . Solutions: `exercises/solutions/07.md` .

Add a column

The plain form assigns to a new name. The `assign` method returns a new DataFrame and leaves the original as it was, so it fits in a chain (Chapter 16):

```
import numpy as np

from pdbook.io import load_calls

calls = load_calls()
calls["minutes"] = np.ceil(calls["duration_s"] / 60)
with_hour = calls.assign(hour=lambda d: d["start"].dt.hour)
print(calls.columns.tolist()[-1], with_hour.columns.tolist()[-1])
print(calls.shape, with_hour.shape)
print("hour" in calls.columns)
```

```
minutes hour
(20000, 10) (20000, 11)
False
```

`calls["minutes"] = ...` changed `calls` . `calls.assign(hour=...)` made a new table, and `calls` has no `hour` column. Both work on the whole column at once. The lambda in `assign` receives the table at that point of the chain. A later step can use a column that an earlier step made.

The index decides where values go

When you assign a Series, pandas aligns it to the index of the table. It does not use the order:

```
import pandas as pd

from pdbook.io import load_calls

part = load_calls().head(5).copy()
part["z"] = pd.Series([10, 20, 30], index=[3, 4, 99])
print(part["z"].tolist())

part["w"] = [10, 20, 30, 40, 50]
print(part["w"].tolist())
try:
    part["v"] = [1, 2, 3]
except ValueError as error:
    print("ValueError:", str(error)[:50])
```

```
[nan, nan, nan, 10.0, 20.0]
[10, 20, 30, 40, 50]
ValueError: Length of values (3) does not match length of inde
```

The first assignment puts 10 on index 3 and 20 on index 4. Index 99 does not exist, so its value is dropped, and the other three rows get `NaN`. A list does not align. It goes by position, and it must have the right length. When two tables have the same rows in a different order, a Series goes by label and a list goes by position. The two give different columns, and neither gives an error.

Conditions

Four tools make a column from a condition. Use the first one that fits:

Need	Tool
keep or replace on one condition	`where(cond, other)` or `mask(cond, other)`
several conditions, first match wins	`np.select(conditions, choices, default)`
numbers into labelled ranges	`pd.cut(x, bins, labels)`
a lookup of a value in a table	`Series.map(dict)`

```
import numpy as np
import pandas as pd

from pdbook.io import load_calls

d = load_calls()["duration_s"]
status = load_calls()["status"]

only_positive = d.where(d > 0)
band = np.select(
    [d.isna(), d.eq(0), d.lt(30), d.lt(300)],
    ["none", "unanswered", "short", "normal"],
    default="long",
)
cut = pd.cut(d, [-0.5, 0.5, 29.5, 299.5, np.inf], labels=["unanswered", "short", "normal",
"long"])
sign = status.map({"answered": 1, "busy": 0, "noanswer": 0})

print(int(only_positive.isna().sum()))
print(pd.Series(band).value_counts().to_dict())
print(cut.value_counts(dropna=False).to_dict())
print(int(sign.isna().sum()))
```

```
4474
{'normal': 11578, 'unanswered': 3441, 'short': 2487, 'long': 1461, 'none': 1033}
{'normal': 11578, 'unanswered': 3441, 'short': 2487, 'long': 1461, nan: 1033}
1033
```

Read the results. `where(d > 0)` keeps the positive durations and makes 4,474 values `NaN`: the 3,441 zeros and the 1,033 nulls. `np.select` takes the first condition that is true, so the order of the conditions is part of the rule. Put `isna` first, because `NaN < 30` is `False` and the null falls to the default `long`.

`pd.cut` gives a category and leaves the nulls as `NaN`, and `np.select` gives text with a label of your choice for them. `map` with a dict gives `NaN` for a key that is not in the dict: `failed` is missing from the dict above, and 1,033 values are `NaN`. This is the correct behavior and a trap, so check `isna().sum()` after a `map`.

The three speeds

A bill needs a rule with a condition, a rounding, and a lookup in two tables. Only answered calls are billed. The billed time is the duration rounded up to whole minutes, with a minimum of one minute. The price per minute depends on the plan of the caller. Here are three ways to write it. All three are in `pdbook/rating.py`.

The loop is the way that a programmer writes it first:

```
import math


def rate_loop(calls, subs, plans):
    plan_of = dict(zip(subs["subscriber_id"], subs["plan"]))
    rate_of = dict(zip(plans["plan"], plans["rate_per_min_eur"]))
    out = []
    for _, row in calls.iterrows():
        if row["status"] != "answered":
            out.append(0.0)
            continue
        minutes = max(1, math.ceil(row["duration_s"] / 60))
        out.append(round(minutes * rate_of[plan_of[row["caller"]]], 2))
    return out
```

The `apply` form is the same function body, handed to pandas. Pandas still calls it once for each row. The vectorized form states the rule for whole columns:

```
import numpy as np


def rate_vector(calls, subs, plans):
    plan_of = dict(zip(subs["subscriber_id"], subs["plan"]))
    rate_of = dict(zip(plans["plan"], plans["rate_per_min_eur"]))
    rate = calls["caller"].map(plan_of).map(rate_of)
    minutes = np.maximum(1, np.ceil(calls["duration_s"] / 60))
    cost = (minutes * rate).where(calls["status"] == "answered", 0.0)
    return cost.round(2)
```

Now run all three, check that they agree, and time them. The assertion at the end is part of the chapter: it fails if the vectorized form is not at least ten times faster than the loop on your machine.

```
import time

import pandas as pd

from pdbook.io import load_calls, load_plans, load_subscribers
from pdbook.rating import rate_apply, rate_loop, rate_vector

calls, subs, plans = load_calls(), load_subscribers(), load_plans()


def best_of(fn, repeat):
    best, result = float("inf"), None
    for _ in range(repeat):
        t0 = time.perf_counter()
        result = fn(calls, subs, plans)
        best = min(best, time.perf_counter() - t0)
    return best, result


t_loop, r_loop = best_of(rate_loop, 1)
t_apply, r_apply = best_of(rate_apply, 1)
t_vec, r_vec = best_of(rate_vector, 5)

pd.testing.assert_series_equal(r_loop, r_vec)
pd.testing.assert_series_equal(r_apply, r_vec)
print(f"loop {t_loop:.3f}s  apply {t_apply:.3f}s  vector {t_vec:.4f}s")
print(f"loop/vector {t_loop / t_vec:.0f}x  apply/vector {t_apply / t_vec:.0f}x")
print(f"total bill {r_vec.sum():,.2f}")
assert t_loop / t_vec > 10 and t_apply / t_vec > 3
```

```
# output varies
loop 0.583s  apply 0.169s  vector 0.0089s
loop/vector 65x  apply/vector 19x
total bill 4,245.09
```

The block prints the times of one run. The assertion at the end of the block checks the order of magnitude: the loop must be more than 10 times slower than the column form, and `apply` more than 3 times slower. The ratios depend on the machine and on the load at that moment, and they depend on the number of rows. The first exercise measures how.

The bill is the same in all three: 4,245.09 euros over 15,526 billed calls. The `assert_series_equal` lines prove it, and so you can use the fast form with the same trust.

Why the loop is slow

`iterrows` builds a Series for every row. The block in the section on how it works prints its type. The dtype of such a row is the dtype that holds all the columns, which is `object` for a table with text. That Series is a new Python object.

The loop body then runs bytecode for each of 20,000 rows. `apply(axis=1)` does the same without the loop in your code. The vectorized form calls NumPy once, and NumPy loops in C over a contiguous array.

You can tell the difference by its shape. A vectorized operation names columns and never names a row. If you write `for` or `row[...]` over a table, ask first whether a column expression says the same.

When apply is the right tool

Some work has no column form: a function from a library that takes one value, or a parser that reads a nested record. Then `apply` or `map` on one column is fine. It is fast enough when the column is small or when the function is the slow part:

```
import pandas as pd

from pdbook.io import load_calls

caller = load_calls()["caller"]
number = caller.map(lambda s: int(s[1:]))
print(number.head(3).tolist(), number.dtype)
print(caller.str[1:].astype(int).equals(number))
```

```
[125, 108, 414] int64
True
```

The second line shows that `.str[1:].astype(int)` gives the same result without a Python function. Try the column form first. When you must use `apply`, prefer `Series.map` or `Series.apply` over `DataFrame.apply(axis=1)`, because a column holds one type, and a row holds a mix.

Build: the bill

`rate_vector` is the function to keep. The test that makes it trustworthy is the comparison with the loop on a sample. A loop is slow, so use a small sample, and use a loop that is easy to read:

```
import pandas as pd

from pdbook.io import load_calls, load_plans, load_subscribers
from pdbook.rating import rate_loop, rate_vector

calls, subs, plans = load_calls(), load_subscribers(), load_plans()
sample = calls.sample(1000, random_state=0)
pd.testing.assert_series_equal(rate_loop(sample, subs, plans), rate_vector(sample, subs,
plans))

bill = rate_vector(calls, subs, plans)
by_plan = bill.groupby(calls["caller"].map(dict(zip(subs["subscriber_id"],
subs["plan"])))).sum().round(2)
print(by_plan.to_dict())
print(round(float(bill.sum()), 2))
```

```
{'basic': 1399.7, 'business': 193.72, 'plus': 563.22, 'prepaid': 2088.45}
4245.09
```

The sample test uses the real rule on 1,000 random rows, and it passes. The bill by plan adds up to the total, which is the check that every call has a plan. A call whose caller is not in `subs` has a missing plan, and the group total is less than the full total.

How it works

A column operation is one call into compiled code. NumPy takes the array, loops over it in C, and returns a new array. A function that you give to `apply(axis=1)` is different: pandas builds a Series for each row, calls your Python function once for each row, and collects the results. The cost of a Python call (a few hundred nanoseconds, plus the Series) is paid 20,000 times. The speed ratios of this chapter are the price of that call, repeated.

You can count the calls:

```
import numpy as np

from pdbook.io import load_calls

calls = load_calls()
seen = {"row": 0, "column": 0}


def per_row(row):
    seen["row"] += 1
    seen["type"] = f"{type(row).__name__}, dtype {row.dtype}"
    return float(np.ceil(row["duration_s"] / 60)) if row["status"] == "answered" else 0.0


def per_column(duration_s):
    seen["column"] += 1
    return np.ceil(duration_s / 60)


by_row = calls.apply(per_row, axis=1)
by_column = per_column(calls["duration_s"])

print(len(calls), seen)
print(calls["duration_s"].to_numpy().flags["C_CONTIGUOUS"],
calls["duration_s"].to_numpy().dtype)
answered = calls["status"] == "answered"
print(bool(np.array_equal(by_row[answered].to_numpy(), by_column[answered].to_numpy())))
```

```
20000 {'row': 20000, 'column': 1, 'type': 'Series, dtype object'}
True float64
True
```

The row version called the function 20,000 times, one call for each row. The column version called it once, and the loop ran inside NumPy over a contiguous array of `float64`. The third line shows that the two give the same values for the answered calls. (The rows that are not answered differ on purpose in this example, because the row function returns 0 for them.)

- **Invariant.** A column operation calls the function once and loops in compiled code. `apply(axis=1)` calls it once for each row.
- **Failure.** A rule with `if` and `for` over rows. Rewrite it with `where`, `np.select` or `map`.
- **Cost.** The call count. 20,000 calls here, and the speed ratio of this chapter is the price of those calls.

Reference

```
import numpy as np
import pandas as pd

from pdbook.rating import rate_apply, rate_loop, rate_vector
from pdbook.sigs import show_signature

show_signature(pd.DataFrame.assign)
show_signature(pd.Series.where, ["cond", "other"])
show_signature(pd.Series.mask, ["cond", "other"])
show_signature(np.select, ["condlist", "choicelist", "default"])
show_signature(pd.cut, ["x", "bins", "labels", "right"])
show_signature(pd.Series.map, ["func", "na_action"])
show_signature(pd.DataFrame.apply, ["func", "axis"])
show_signature(rate_vector)
show_signature(rate_apply)
show_signature(rate_loop)
```

```
DataFrame.assign(
    self
    **kwargs
)
NDFrame.where(
    cond
    other=<no_default>
)  # 4 more parameters
NDFrame.mask(
    cond
    other=<no_default>
)  # 4 more parameters
select(
    condlist
    choicelist
    default=0
)
cut(
    x
    bins
    right: 'bool' = True
    labels=None
)  # 5 more parameters
Series.map(
    func: 'Callable | Mapping | Series | None' = None
    na_action: "Literal['ignore'] | None" = None
)  # 3 more parameters
DataFrame.apply(
    func: 'AggFuncType'
    axis: 'Axis' = 0
)  # 8 more parameters
rate_vector(
    calls: 'pd.DataFrame'
    subs: 'pd.DataFrame'
    plans: 'pd.DataFrame'
)
rate_apply(
    calls: 'pd.DataFrame'
    subs: 'pd.DataFrame'
    plans: 'pd.DataFrame'
)
rate_loop(
    calls: 'pd.DataFrame'
    subs: 'pd.DataFrame'
    plans: 'pd.DataFrame'
)
```

Parameter	Default	What it does here	What goes wrong
`assign(**kwargs)`	—	each keyword is a new column; a function gets the table at that point	—
`where(cond, other=...)`	not set	keep the values where `cond` is `True`; elsewhere put `other` or `NaN`	`where(d > 0)` makes 4,474 values `NaN`
`mask(cond, other=...)`	not set	the opposite of `where`	—
`np.select(condlist, choicelist, default=...)`	`0`	the first true condition decides	`isna` must come first, because `NaN < 30` is `False`
`pd.cut(x, bins, right=..., labels=...)`	`True`, `None`	numbers into labelled ranges, `(a, b]` for `right=True`	nulls stay `NaN`
`map(func, na_action=...)`	`None`, `None`	a dict or a function for each value	a key not in the dict gives `NaN`: 1,033 for `failed`
`apply(func, axis=...)`	required, `0`	`axis=1` calls `func` once for each row	20,000 calls, and more than 10 times slower than the column form (asserted)
`rate_vector(calls, subs, plans)`	required	the bill for each call, from whole columns	—
`rate_apply(calls, subs, plans)`	required	the same bill with `apply(axis=1)`	—
`rate_loop(calls, subs, plans)`	required	the same bill with `iterrows`	—

The three `rate_` functions return a Series named `cost_eur` with the index of `calls`, and they give equal results.

What you have now

- You add columns with `[...]` and `assign`, and you know that a Series assigns by index.
- You make conditional columns with `where`, `np.select`, `pd.cut` and `map`, and you check the nulls after each one.
- You write a rule as a column expression, and you can show that a loop gives the same answer.
- You know the measured cost of `iterrows` and `apply`, and when `apply` is the right tool.

The next chapter turns the bill and the calls into a report with one row per subscriber.

Next: Chapter 8, Group by.

8. Group by

> **Build** — `subscriber_summary(calls, subs)` : a table with one row for each subscriber (calls, answered, failed, talk time, cells used, first and last call), and four checks that its totals match the call table.
>
> **Theory kept** — `groupby` splits the rows by a key, applies a function to each group and combines the results. `agg` returns one row for each group. `transform` returns one value for each original row. `filter` keeps or drops whole groups. The totals of the groups must add up to the table.
>
> **Source** — New. Library: `pdbook/summary.py` .
>
> **Run this first** — `uv run python -m examples.ch08_group_by` , which is `examples/ch08_group_by.py` . Exercises: `exercises/08.md` . Solutions: `exercises/solutions/08.md` .

Split, apply, combine

A `groupby` does three things. It splits the table into groups, one for each distinct key. It applies a function to each group. It combines the results into one object. Take six rows:

```
import pandas as pd

mini = pd.DataFrame({
    "caller": ["A", "B", "A", "C", "B", "A"],
    "status": ["answered", "failed", "answered", "busy", "answered", "failed"],
    "duration_s": [60, None, 120, 0, 30, None],
})
g = mini.groupby("caller")
print(g.size().to_dict())
print(g["duration_s"].sum().to_dict())
print(g["duration_s"].transform("sum").tolist())
```

```
{'A': 3, 'B': 2, 'C': 1}
{'A': 180.0, 'B': 30.0, 'C': 0.0}
[180.0, 30.0, 180.0, 0.0, 30.0, 180.0]
```

`size` and `sum` have one value for each caller: A, B and C. `transform("sum")` has one value for each of the six rows, and each row carries the sum of its own group. The index of the `transform` result is the index of `mini` , so it fits back into the table as a column.

The three operations answer different questions:

Operation	Rows in the result	Question
`agg`	one for each group	"How many calls did each subscriber make?"
`transform`	one for each original row	"What share of the subscriber's calls is this call?"
`filter`	the rows of the groups that pass	"Show the calls of subscribers with more than 60 calls."

agg and named aggregation

With `agg` you give names to the outputs. The form `name=("column", "function")` is the clearest, and it uses the fast built-in path for functions given as strings:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
flagged = calls.assign(answered=calls["status"].eq("answered"))
by_status = flagged.groupby("status").agg(
    rows=("call_id", "size"),
    with_duration=("duration_s", "count"),
    mean_s=("duration_s", "mean"),
    longest_s=("duration_s", "max"),
)
print(by_status.round(1).to_string())
```

```
           rows  with_duration  mean_s  longest_s
status
answered  15526          15526   132.8     3836.0
busy       1428           1428     0.0        0.0
failed     1033              0     NaN        NaN
noanswer   2013           2013     0.0        0.0
```

The second column is `count`, the number of non-null values. The first is `size`, the number of rows. They differ for `failed`: 1,033 rows and no duration.

`mean` and `max` of an empty group are `NaN`. When you want "how many rows", use `size`. When you want "how many have a value", use `count`.

A boolean column is a counter. `sum` counts the `True` values and `mean` gives the share:

```
import pandas as pd

from pdbook.io import load_calls, load_subscribers

calls, subs = load_calls(), load_subscribers()
plan = calls["caller"].map(dict(zip(subs["subscriber_id"], subs["plan"]))).rename("plan")
answered = calls["status"].eq("answered")
rate = answered.groupby(plan).agg(["sum", "size", "mean"]).round(3)
print(rate.to_string())
```

```
           sum  size   mean
plan
basic     5153  6664  0.773
business  1763  2290  0.770
plus      3497  4475  0.781
prepaid   5113  6571  0.778
```

This is a Series grouped by another Series. The key `plan` is not a column of `calls`. A key can be any Series that has the same index, or an array of the same length. The four rates are between 0.77 and 0.79: the plan does not change the chance that a call is answered.

Several keys

Give a list of keys. The result has one index level for each key, which Chapter 11 explains. Use `as_index=False` when you want the keys as ordinary columns:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
table = calls.groupby(["status", "roaming"], as_index=False).agg(rows=("call_id", "size"))
print(table.to_string(index=False))
```

```
  status  roaming  rows
answered    False 15081
answered     True   445
    busy    False  1382
    busy     True    46
  failed    False  1002
  failed     True    31
noanswer    False  1972
noanswer     True    41
```

transform: values that fit back into the table

A statistic of the group, next to each row, lets you compare a row with its group. Here is the share of each subscriber's talk time that each call makes up, and a flag for calls much longer than the subscriber's average:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
answered = calls[calls["status"] == "answered"].copy()
by_caller = answered.groupby("caller")["duration_s"]
answered["share"] = answered["duration_s"] / by_caller.transform("sum")
answered["vs_mean"] = answered["duration_s"] / by_caller.transform("mean")

print(round(float(answered.groupby("caller")["share"].sum().min()), 6),
      round(float(answered.groupby("caller")["share"].sum().max()), 6))
print(int((answered["vs_mean"] > 3).sum()), "calls are more than 3 times the mean of their
caller")
```

```
1.0 1.0
867 calls are more than 3 times the mean of their caller
```

The first line is a check. The shares of each subscriber add up to 1, so the minimum and the maximum of the sums are both 1. The same result without `transform` needs an `agg`, then a `merge` back to the table, and a risk of a wrong key. `transform` does it in one line, with the index preserved.

filter: keep or drop whole groups

`filter` takes a function that returns one boolean for each group. The rows of the groups that return `True` stay:

```
import time

import pandas as pd

from pdbook.io import load_calls

calls = load_calls()


def best_of(fn, repeat=5):
    best = float("inf")
    for _ in range(repeat):
        t0 = time.perf_counter()
        result = fn()
        best = min(best, time.perf_counter() - t0)
    return best, result


t_filter, heavy = best_of(lambda: calls.groupby("caller").filter(lambda g: len(g) >= 60))
size = calls.groupby("caller")["call_id"].transform("size")
t_mask, same = best_of(lambda: calls[calls.groupby("caller")["call_id"].transform("size") >=
60])

print(len(heavy), heavy["caller"].nunique())
print(heavy.index.equals(same.index))
print(f"filter {t_filter * 1000:.1f} ms, mask {t_mask * 1000:.1f} ms, ratio {t_filter /
t_mask:.0f}x")
assert t_filter / t_mask > 3
```

```
# output varies
7058 56
True
filter 50.5 ms, mask 2.8 ms, ratio 18x
```

56 subscribers have 60 calls or more, and they make 7,058 of the 20,000 calls. The second form uses `transform("size")` and a mask. It gives the same rows. It is faster, because the lambda of `filter` runs once for each of the 600 groups in Python, and `transform("size")` runs once in compiled code. The block prints the times of one run. The assertion fails if the mask form is not at least 3 times faster on your machine.

Keys that are not in the table

Group by a day or an hour of a timestamp column with `dt` or with `pd.Grouper`. The zone of the timestamp decides which calls belong to a day:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
utc_day = calls.groupby(calls["start"].dt.date).size()
local = calls["start"].dt.tz_convert("Europe/Brussels")
local_day = calls.groupby(local.dt.date).size()

print(len(utc_day), len(local_day))
print(utc_day.index[0], int(utc_day.iloc[0]), "|", local_day.index[0], int(local_day.iloc[0]))
print(int((utc_day.reindex(local_day.index).fillna(0).astype(int) != local_day).sum()), "days
differ")
print(local_day.index.min(), local_day.index.max())
```

```
31 30
2026-08-31 7 | 2026-09-01 728
26 days differ
2026-09-01 2026-09-30
```

The call records are in UTC. In UTC the month has 31 days, because the first calls of 1 September (local time) fall on 31 August in UTC. In local time it has the 30 days of September. The counts of the days that exist in both are different for most of the days, because a local day starts at 22:00 UTC. A daily report must say which zone it uses. Chapter 12 treats time zones.

Totals must add up

Every report from a `groupby` has one check: the groups must add up to the table. You saw the ways that it fails. A missing key drops the group (Chapter 5).

A `category` key with `observed=True` hides the empty groups (Chapter 4). A `filter` or a `merge` before the `groupby` drops rows. Add the check as a line of code, so that it runs every time:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
by_cell = calls.groupby("cell_id").size()
print(int(by_cell.sum()), len(calls), int(by_cell.sum()) == len(calls))

by_cell = calls.groupby("cell_id", dropna=False).size()
print(int(by_cell.sum()), len(calls), int(by_cell.sum()) == len(calls))
```

```
19723 20000 False
20000 20000 True
```

Build: subscriber_summary

The summary has one row for each subscriber, and subscribers without calls stay in it. A `groupby` alone drops them. The function starts from the subscriber table and joins the aggregates on, so every subscriber is a row:

```
import pandas as pd

from pdbook.io import load_calls, load_subscribers
from pdbook.summary import subscriber_summary

calls, subs = load_calls(), load_subscribers()
summary = subscriber_summary(calls, subs)
print(summary.shape)
print(summary.drop(columns=["first_call", "last_call"]).head(4).to_string(index=False))
print(summary["calls"].describe().round(1).to_dict())
```

```
(600, 11)
subscriber_id    plan     city  calls  answered  failed  talk_min  cells  answer_rate
        S0001 prepaid Brussels     11         8       1      10.0      5        0.727
        S0002   basic   Leuven     14         9       2      16.1      2        0.643
        S0003 prepaid  Antwerp     27        18       3      33.8      5        0.667
        S0004 prepaid  Antwerp     11         9       1      14.5      5        0.818
{'count': 600.0, 'mean': 33.3, 'std': 44.2, 'min': 6.0, '25%': 17.0, '50%': 22.0, '75%': 33.0,
'max': 628.0}
```

The distribution of calls is very uneven. The median subscriber makes 22 calls and the maximum is 628. A mean of 33.3 calls for each subscriber describes nobody. Report the median and the percentiles for such a column.

Now the four checks. They run in the example module as well:

```
import pandas as pd

from pdbook.io import load_calls, load_subscribers
from pdbook.summary import subscriber_summary

calls, subs = load_calls(), load_subscribers()
summary = subscriber_summary(calls, subs)
print(len(summary) == len(subs), bool(summary["subscriber_id"].is_unique))
print(int(summary["calls"].sum()) == len(calls))
print(int(summary["answered"].sum()) == int((calls["status"] == "answered").sum()))
print(round(float(summary["talk_min"].sum()), 1),
round(float(calls["duration_s"].fillna(0).sum() / 60), 1))
```

```
True True
True
True
34372.0 34372.5
```

The talk time differs by 0.5 minutes, from 34,372.0 to 34,372.5. The function rounds the minutes of each subscriber to one decimal, and 600 roundings add up to a small difference. A check must allow for the rounding that you put in. The test in the example module allows 0.05 minutes for each subscriber. Round for display, not in the table that you add up.

How it works

A `groupby` does not move rows into groups. It turns the key column into an array of small integer codes, one for each row, and a list of the distinct keys. A reduction then runs once over the data with the codes as the group number. This is the `factorize` step, and `pd.factorize` shows it. The reduction `sum` for each group is what `np.bincount` does with weights.

You can rebuild a grouped sum from the two parts, and see that `transform` is the same reduction followed by a take:

```
import numpy as np
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
duration = calls["duration_s"].fillna(0).to_numpy()

codes, keys = pd.factorize(calls["caller"])
print(codes[:5].tolist(), len(keys), codes.dtype)

sums = np.bincount(codes, weights=duration)
by_hand = pd.Series(sums, index=keys).sort_index()
by_pandas = calls.groupby("caller")["duration_s"].sum().sort_index()
print(bool(np.allclose(by_hand.to_numpy(), by_pandas.to_numpy())))

as_column = sums[codes]
print(bool(np.allclose(as_column, calls.groupby("caller")
["duration_s"].transform("sum").fillna(0).to_numpy())))
```

```
[0, 1, 2, 3, 4] 600 int64
True
True
```

The codes are small integers (`int64` here, from 0 to 599), and there are 600 keys. The hand-made sum equals the sum of `groupby`. The `transform` is `sums[codes]`. The reduced array, indexed by the codes, puts each group value back on its rows. This is why the result has the index of the original table.

- **Invariant.** The groups are the distinct keys. A missing key has no code, so it is dropped unless `dropna=False`.
- **Failure.** The missing key of Chapter 5: the rows with no code are in no group.
- **Cost.** One pass to factorize the key, and one pass for each reduction. A key that is already a `category` has its codes, so the first pass is free.

Reference

```
import pandas as pd

from pdbook.sigs import show_signature
from pdbook.summary import subscriber_summary

show_signature(pd.DataFrame.groupby, ["by", "as_index", "sort", "dropna", "observed"])
show_signature(pd.core.groupby.DataFrameGroupBy.agg, ["func"])
show_signature(pd.core.groupby.DataFrameGroupBy.transform, ["func"])
show_signature(pd.core.groupby.DataFrameGroupBy.filter, ["func", "dropna"])
show_signature(pd.Grouper)
show_signature(subscriber_summary)
```

```
DataFrame.groupby(
    by=None
    as_index: 'bool' = True
    sort: 'bool' = True
    observed: 'bool' = True
    dropna: 'bool' = True
)  # 3 more parameters
DataFrameGroupBy.aggregate(
    func=None
)  # 5 more parameters
DataFrameGroupBy.transform(
    func
)  # 5 more parameters
DataFrameGroupBy.filter(
    func
    dropna: 'bool' = True
)  # 3 more parameters
Grouper(
    *args
    **kwargs
)
subscriber_summary(
    calls: 'pd.DataFrame'
    subs: 'pd.DataFrame'
)
```

Parameter	Default	What it does here	What goes wrong
`groupby(by)`	`None`	the column, list of columns or Series that defines the groups	a key that is a Series must have the index of the table
`groupby(as_index=...)`	`True`	`False` keeps the keys as columns	—
`groupby(sort=...)`	`True`	sort the groups by key	—
`groupby(observed=...)`	`True`	category keys: only the groups that occur	—
`groupby(dropna=...)`	`True`	`False` keeps the missing key as a group	277 calls missing from the report when `True`
`agg(func)`	`None`	a name or function, or `name=("column", "function")`	a lambda is slower than a built-in name
`transform(func)`	required	a result with one value for each original row	—
`filter(func, dropna=...)`	required, `True`	keep the groups for which `func` is `True`	more than 3 times slower than a mask from `transform("size")` (asserted)
`Grouper(key=..., freq=...)`	—	a time key, for example `key="start", freq="D"`	the zone of the column decides the day
`subscriber_summary(calls, subs)`	required	one row for each subscriber, zeros for no calls	—

`agg` returns one row for each group. `transform` returns the index of the table. `subscriber_summary` returns a DataFrame with 600 rows.

What you have now

- You pick `agg`, `transform` or `filter` from the shape of the result you want.
- You write named aggregations, and you use boolean columns as counters.
- You check that the groups add up to the table.
- You know that a group key can be a Series that is not a column, and that the zone of a timestamp decides the day.
- You have a summary table with one row for each subscriber.

The next chapter joins this summary with other tables, and shows how a join can silently multiply rows.

Next: Chapter 9, Joins.

9. Joins

> **Build** — `enrich(calls, subs, cells, plans)` : a function that adds the plan, the cities, the technology, the price per minute and an on-net flag to each call. It refuses to run when a lookup table has a duplicate key, and it reports every key that it could not find.
>
> **Theory kept** — A join matches rows on a key. The number of result rows depends on how many rows match each key. A lookup table must have a unique key, and `validate` states and checks that.
>
> **Source** — New. Library: `pdbook/enrich.py` .
>
> **Run this first** — `uv run python -m examples.ch09_joins` , which is `examples/ch09_joins.py` . Exercises: `exercises/09.md` . Solutions: `exercises/solutions/09.md` .

Four joins on two small tables

```
import pandas as pd

left = pd.DataFrame({"k": ["a", "b", "c"], "x": [1, 2, 3]})
right = pd.DataFrame({"k": ["b", "c", "d"], "y": [20, 30, 40]})
for how in ["inner", "left", "right", "outer"]:
    m = left.merge(right, on="k", how=how)
    print(f"{how:6s}", m["k"].tolist(), m["x"].tolist(), m["y"].tolist())
```

```
inner  ['b', 'c'] [2, 3] [20, 30]
left   ['a', 'b', 'c'] [1, 2, 3] [nan, 20.0, 30.0]
right  ['b', 'c', 'd'] [2.0, 3.0, nan] [20, 30, 40]
outer  ['a', 'b', 'c', 'd'] [1.0, 2.0, 3.0, nan] [nan, 20.0, 30.0, 40.0]
```

`how`	Rows in the result
`inner`	the keys that are in both tables
`left`	all keys of the left table, with `NaN` where the right table has no match
`right`	all keys of the right table
`outer`	all keys of both tables

For enrichment, use `left` : the table that you enrich keeps all its rows, and a missing match shows as `NaN` . An `inner` join drops the rows without a match, and you lose them in silence.

The number of rows

A join of one row on the left with `n` rows on the right gives `n` rows. The result has as many rows as there are pairs of matching rows. Three relationships matter:

Relationship	Left key	Right key	Rows in a left join
one-to-one (`1:1`)	unique	unique	as many as the left table
many-to-one (`m:1`)	repeats	unique	as many as the left table
many-to-many (`m:m`)	repeats	repeats	more than the left table

The call records to a lookup table is `m:1`: many calls for each subscriber, one row for each subscriber. This is the join that you want. The `m:m` join is the one that multiplies:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls().head(1000)
pairs = calls.merge(calls[["caller", "call_id"]], on="caller", suffixes=("", "_other"))
print(len(calls), len(pairs), round(len(pairs) / len(calls), 2))
```

```
1000 5356 5.36
```

The first 1,000 calls have 5,356 pairs of calls with the same caller. Nobody wanted that, and no error was raised. A join of the calls with themselves on the caller is the pattern behind "all pairs of calls of one subscriber", and it grows with the square of the group size.

A duplicate in the lookup table

The same effect appears by accident. A subscriber table that has the same `subscriber_id` twice gives each call of that subscriber twice:

```
import pandas as pd
from pandas.errors import MergeError

from pdbook.io import load_calls, load_subscribers

calls, subs = load_calls(), load_subscribers()
dirty = pd.concat([subs, subs.head(10)], ignore_index=True)

joined = calls.merge(dirty[["subscriber_id", "plan"]], left_on="caller",
right_on="subscriber_id", how="left")
print(len(calls), "->", len(joined), "rows;", len(joined) - len(calls), "extra")
print(round(float(calls["duration_s"].sum()), 0), "->",
round(float(joined["duration_s"].sum()), 0))

try:
    calls.merge(dirty[["subscriber_id", "plan"]], left_on="caller", right_on="subscriber_id",
                how="left", validate="m:1")
except MergeError as error:
    print("MergeError:", str(error).splitlines()[0])
```

```
20000 -> 20284 rows; 284 extra
2062348.0 -> 2088598.0
MergeError: Merge keys are not unique in right dataset; not a many-to-one merge
```

Ten duplicate subscribers added 284 rows, and the total talk time grew from 2,062,348 to 2,088,598 seconds. Every later sum, count and mean is wrong, and the table looks normal. The argument `validate="m:1"` makes pandas check the right table and raise `MergeError`. Write it on every join that is a lookup. The check is cheap, because pandas only tests that the key is unique.

The values of `validate` are `"1:1"`, `"1:m"`, `"m:1"` and `"m:m"`. The check is on uniqueness of the key on the side that you mark with `1`.

Keys must have the same type

A key of integers does not join a key of text:

```
import pandas as pd

from pdbook.io import load_calls, load_subscribers

calls, subs = load_calls(), load_subscribers()
as_int = subs.assign(msisdn=subs["msisdn"].str[1:].astype("int64"))
try:
    calls.merge(as_int, left_on="callee", right_on="msisdn")
except ValueError as error:
    print("ValueError:", str(error)[:60])
```

```
ValueError: You are trying to merge on str and int64 columns for key 'ca
```

That is the good case: pandas stops. The dangerous case is two keys of the same type and different form. `+32477123456` and `0477123456` are both text, and they never match.

The join runs, and every row is unmatched. Chapter 6 gave one form for phone numbers for this reason: clean both sides first.

Missing keys match each other

A missing key matches a missing key. This differs from SQL, where `NULL` never equals `NULL`:

```
import numpy as np
import pandas as pd

left = pd.DataFrame({"k": [1.0, np.nan], "a": [1, 2]})
right = pd.DataFrame({"k": [1.0, np.nan], "b": [10, 20]})
print(left.merge(right, on="k").to_dict("records"))
```

```
[{'k': 1.0, 'a': 1, 'b': 10}, {'k': nan, 'a': 2, 'b': 20}]
```

The row with the missing key found a partner. On the call records this is invisible, because the cell table has no missing key. With a lookup table that has a row with a missing key, every call without a cell gets the values of that row. Drop rows with a missing key from a lookup table before the join.

Which keys did not match?

`indicator=True` adds a column `_merge` with `both`, `left_only` or `right_only`. Use it to find the unmatched keys, and separate them from the missing keys:

```
import pandas as pd

from pdbook.io import load_calls, load_cells

calls, cells = load_calls(), load_cells()
m = calls.merge(cells[["cell_id", "tech"]], on="cell_id", how="left", indicator=True)
print(m["_merge"].value_counts().to_dict())
print(int(m["cell_id"].isna().sum()))
print(int(((m["_merge"] == "left_only") & m["cell_id"].notna()).sum()))
```

```
{'both': 19723, 'left_only': 277, 'right_only': 0}
277
0
```

277 rows are `left_only`, and all 277 have a missing `cell_id`. They did not fail to match. They had nothing to match. The third line counts the real misses, a `cell_id` that is present and not in the cell table, and it is 0. The same split is what `enrich` reports.

To find the keys of one table that are missing from another without a join, use `isin`:

```
from pdbook.io import load_calls, load_cells

calls, cells = load_calls(), load_cells()
unknown = ~calls["cell_id"].dropna().isin(cells["cell_id"])
unused = ~cells["cell_id"].isin(calls["cell_id"])
print(int(unknown.sum()), int(unused.sum()))
```

```
0 0
```

concat: stacking tables

`concat` puts tables under each other (`axis=0`) or next to each other (`axis=1`). It aligns on column names for the first and on the index for the second:

```
import pandas as pd

a = pd.DataFrame({"call_id": [1, 2], "status": ["answered", "busy"]})
b = pd.DataFrame({"call_id": [3], "status": ["failed"], "codec": ["EVS"]})
both = pd.concat([a, b], ignore_index=True)
print(both.to_dict("list"))
print(both.index.tolist())
```

```
{'call_id': [1, 2, 3], 'status': ['answered', 'busy', 'failed'], 'codec': [nan, nan, 'EVS']}
[0, 1, 2]
```

A column that one table lacks becomes `NaN`. Use `ignore_index=True` to get a new `RangeIndex`. Without it, the index repeats labels (0, 1, 0), and a later `.loc[0]` returns two rows. Build a list of tables and call `concat` once. A `concat` inside a loop copies the growing table at each step (Chapter 18).

Build: enrich

`enrich` chains four many-to-one lookups. Each one is a `left` join with `validate="m:1"` and `indicator=True`. After each join it checks that the row count did not change, and it counts the keys that were present and not found:

```
import pandas as pd

from pdbook.enrich import enrich
from pdbook.io import load_calls, load_cells, load_plans, load_subscribers

calls, subs, cells, plans = load_calls(), load_subscribers(), load_cells(), load_plans()
enriched, report = enrich(calls, subs, cells, plans)
print(len(calls), "->", len(enriched), "rows,", enriched.shape[1], "columns")
print(report)
print(int(enriched["on_net"].sum()), round(float(enriched["on_net"].mean()), 3))
print(enriched.groupby("tech", dropna=False).size().to_dict())
```

```
20000 -> 20000 rows, 16 columns
{'caller -> subscribers': 0, 'cell_id -> cells': 0, 'plan -> plans': 0}
5982 0.299
{'4G': 11383, '5G': 8340, nan: 277}
```

The table keeps its 20,000 rows and gets seven new columns. No key is unmatched. 5,982 calls (29.9%) go to a number that belongs to one of the 600 subscribers. The last line has three groups. The 277 calls without a cell have no technology, and that group exists only because of `dropna=False` (Chapter 5).

The function must also refuse bad input. Plant a fault and see:

```
import pandas as pd
from pandas.errors import MergeError

from pdbook.enrich import enrich
from pdbook.io import load_calls, load_cells, load_plans, load_subscribers

calls, subs, cells, plans = load_calls(), load_subscribers(), load_cells(), load_plans()
dirty_subs = pd.concat([subs, subs.head(10)], ignore_index=True)
dirty_plans = pd.concat([plans, plans.head(1)], ignore_index=True)

for label, args in [("duplicate subscribers", (dirty_subs, cells, plans)),
                    ("duplicate plans", (subs, cells, dirty_plans))]:
    try:
        enrich(calls, *args)
        print(label, "-> accepted")
    except MergeError:
        print(label, "-> MergeError")
```

```
duplicate subscribers -> MergeError
duplicate plans -> MergeError
```

Both faults stop the job. A fault that stops a nightly job is a good fault, because someone looks at it the next morning.

How it works

A join is a lookup of keys. For a many-to-one left join, pandas builds a hash table from the keys of the right table. It looks up each key of the left table in it. The result is one position in the right table, or the value `-1` for a key that is not there. It then takes the rows of the right table at those positions and puts them next to the left rows. The number of result rows is the number of left rows, because each key has at most one position.

The lookup step is `Index.get_indexer`, and you can use it as a join:

```
import pandas as pd

from pdbook.io import load_calls, load_subscribers

calls, subs = load_calls(), load_subscribers()

lookup = subs.set_index("subscriber_id")
positions = lookup.index.get_indexer(calls["caller"])
print(positions[:5].tolist(), int((positions == -1).sum()))

by_hand = lookup["plan"].to_numpy()[positions]
merged = calls.merge(subs[["subscriber_id", "plan"]], left_on="caller",
right_on="subscriber_id", how="left")
print(bool((merged["plan"].to_numpy() == by_hand).all()), len(merged))

few = calls.head(1000)
left_counts = few["caller"].value_counts()
pairs = int((left_counts * left_counts).sum())
print(pairs, len(few.merge(few[["caller"]], on="caller")))
```

```
[124, 107, 413, 261, 502] 0
True 20000
5356 5356
```

The positions are integers into the 600 subscriber rows, and none is `-1`, because every caller is known. Taking the plans at those positions gives the same column as `merge`. The last line is the many-to-many rule: for each key, the result has the left count times the right count, so the total is the sum of the products. The 1,000 calls have 5,356 pairs, which is the number that the section on row counts printed.

- **Invariant.** A left join on a unique right key keeps the number of left rows and the order of the left table.
- **Failure.** A repeated right key makes one position into several, and the row count goes up by the sum of the repeats.
- **Cost.** One hash table of the right keys and one lookup for each left row. A smaller right table is cheaper to build.

Reference

```
import pandas as pd

from pdbook.enrich import enrich
from pdbook.sigs import show_signature

show_signature(pd.DataFrame.merge, ["right", "how", "on", "left_on", "right_on", "suffixes",
"indicator", "validate"])
show_signature(pd.concat, ["objs", "axis", "ignore_index"])
show_signature(pd.Index.get_indexer, ["target"])
show_signature(enrich)
```

```
DataFrame.merge(
    right: 'DataFrame | Series'
    how: 'MergeHow' = 'inner'
    on: 'IndexLabel | AnyArrayLike | None' = None
    left_on: 'IndexLabel | AnyArrayLike | None' = None
    right_on: 'IndexLabel | AnyArrayLike | None' = None
    suffixes: 'Suffixes' = ('_x', '_y')
    indicator: 'str | bool' = False
    validate: 'MergeValidate | None' = None
)  # 5 more parameters
concat(
    objs: 'Iterable[Series | DataFrame] | Mapping[HashableT, Series | DataFrame]'
    axis: 'Axis' = 0
    ignore_index: 'bool' = False
)  # 7 more parameters
Index.get_indexer(
    target
)  # 4 more parameters
enrich(
    calls: 'pd.DataFrame'
    subs: 'pd.DataFrame'
    cells: 'pd.DataFrame'
    plans: 'pd.DataFrame'
)
```

Parameter	Default	What it does here	What goes wrong
`merge(right)`	required	the table to join	—
`merge(how=...)`	`'inner'`	`left` keeps all left rows	`inner` loses 144 calls when 5 subscribers are not in the lookup: 19,856 against 20,000
`merge(on=...)`, `left_on`, `right_on`	`None`	the key columns	a text key and an integer key raise `ValueError`; two keys in different forms match nothing
`merge(suffixes=...)`	`('_x', '_y')`	names for columns that exist on both sides	—
`merge(indicator=...)`	`False`	`True` adds `_merge`: `both`, `left_only`, `right_only`	— (277 `left_only`, all with a missing key)
`merge(validate=...)`	`None`	`"m:1"` checks that the right key is unique	without it, 10 duplicate subscribers add 284 rows and no error
`concat(objs)`	required	a list of tables to stack	—
`concat(axis=...)`	`0`	`0` stacks rows, `1` puts tables side by side	—
`concat(ignore_index=...)`	`False`	`True` makes a new `RangeIndex`	`False` repeats labels: `loc[0]` returns two rows
`get_indexer(target)`	required	the position of each label, `-1` when not found	—
`enrich(calls, subs, cells, plans)`	required	`(table, report)`: the calls with plan, cities, technology, rate and on-net flag	a duplicate key in a lookup raises `MergeError`

`merge` returns a DataFrame. With `validate`, it raises `MergeError` when the check fails.

What you have now

- You use `left` joins for enrichment, and `validate="m:1"` on every lookup.
- You know the three relationships, and that `m:m` multiplies.
- You separate unmatched keys from missing keys with `indicator=True`.
- You know that keys must have the same type and the same form, and that missing keys match each other.
- You stack with one call to `concat` and `ignore_index=True`.

The next chapter reshapes a table from long to wide and back.

Next: Chapter 10, Pivot and melt.

10. Pivot and melt

> **Build** — `traffic_grid(calls)` : a 7 by 24 table of the calls in each weekday and local hour, with `to_long` and `to_wide` that convert it to one row for each cell and back, and a test that the round trip changes nothing.
>
> **Theory kept** — A long table has one row for each observation. A wide table has one row for each entity and one column for each variable. Long is the form for `groupby` , for plots and for models. Wide is the form for the eye. `pivot_table` aggregates, `pivot` does not, and `melt` reverses.
>
> **Source** — New. Library: `pdbook/traffic.py` .
>
> **Run this first** — `uv run python -m examples.ch10_pivot` , which is `examples/ch10_pivot.py` . Exercises: `exercises/10.md` . Solutions: `exercises/solutions/10.md` .

Long and wide

The call records are long: one row for each call. A traffic report is wide: one row for each weekday, one column for each hour. The same numbers are in both. Which form you need depends on the next step:

You want to	Form
group, filter, join, fit a model	long
plot several series against one axis	long (or wide for `df.plot`)
read the numbers, compare columns	wide
write a report for a person	wide

Most work is in long form, and a report ends in wide form. `pivot_table` goes from long to wide. `melt` goes from wide to long.

pivot_table: aggregate and spread

A `pivot_table` needs three things: which column gives the rows, which gives the columns, and what to put in the cells.

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
local = calls["start"].dt.tz_convert("Europe/Brussels")
frame = pd.DataFrame({"weekday": local.dt.dayofweek, "hour": local.dt.hour, "status":
calls["status"],
                      "duration_s": calls["duration_s"]})

counts = frame.assign(n=1).pivot_table(index="weekday", columns="hour", values="n",
                                       aggfunc="sum", fill_value=0)
print(counts.shape, int(counts.to_numpy().sum()))

mean_duration = frame.pivot_table(index="weekday", columns="status", values="duration_s")
print(mean_duration.round(1).loc[0].to_dict())
```

```
(7, 24) 20000
{'answered': 143.0, 'busy': 0.0, 'noanswer': 0.0}
```

The first table has 7 rows and 24 columns, and its cells add up to 20,000. The default `aggfunc` is `mean`, and the second table uses it: the mean duration for each weekday and status. The printed row is Monday. The `NaN` for `failed` is not in the printed dict, because `failed` has no durations, so pandas dropped the column `failed` from the result. With `dropna=True`, the default, pandas drops a column that is all `NaN`.

Three arguments change the table:

- `aggfunc` takes a string, a function or a list. A list gives one block of columns for each function.
- `fill_value` replaces `NaN` for a combination that does not exist. Use `0` for counts.
- `margins=True` adds a row and a column `All` with the totals.

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
local = calls["start"].dt.tz_convert("Europe/Brussels")
frame = pd.DataFrame({"weekday": local.dt.dayofweek, "status": calls["status"], "call_id":
calls["call_id"]})

t = frame.pivot_table(index="weekday", columns="status", values="call_id", aggfunc="count",
margins=True)
print(t.loc["All"].to_dict())
print(t.loc[5].to_dict())
```

```
{'answered': 15526, 'busy': 1428, 'failed': 1033, 'noanswer': 2013, 'All': 20000}
{'answered': 1360, 'busy': 129, 'failed': 81, 'noanswer': 175, 'All': 1745}
```

The row `All` gives the total of each status and the total of the table. A check: the last value is 20,000.

crosstab: counts and shares

`pd.crosstab` is a `pivot_table` for counts of two columns. It has `normalize`, which turns counts into shares:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
share = pd.crosstab(calls["status"], calls["roaming"], normalize="columns").round(3)
print(share.to_dict())

n = int(calls["roaming"].sum())
p = float(share.loc["answered", True])
print(n, round(100 * (p * (1 - p) / n) ** 0.5, 1))
```

```
{False: {'answered': 0.776, 'busy': 0.071, 'failed': 0.052, 'noanswer': 0.101}, True:
{'answered': 0.79, 'busy': 0.082, 'failed': 0.055, 'noanswer': 0.073}}
563 1.7
```

Each column adds up to 1. Of the roaming calls, 79.0% are answered, against 77.6% of the other calls. The last line gives the number of roaming calls, 563, and the standard error of the share of 0.790 for that many calls: 1.7 points. The difference of 1.4 points is smaller than one standard error. This table does not show that roaming changes the chance of an answer.

pivot: reshape without aggregating

`pivot` has no `aggfunc`. It assumes that each pair of row key and column key occurs once. When a pair occurs twice, there is no rule to choose, and it raises an error:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
local = calls["start"].dt.tz_convert("Europe/Brussels")
frame = pd.DataFrame({"weekday": local.dt.dayofweek, "hour": local.dt.hour, "call_id":
calls["call_id"]})
try:
    frame.pivot(index="weekday", columns="hour", values="call_id")
except ValueError as error:
    print("ValueError:", error)
```

```
ValueError: Index contains duplicate entries, cannot reshape
```

That error is useful. It says that the data are not in the shape that you assumed. Use `pivot` when you know the key is unique, so that a duplicate is a surprise, and use `pivot_table` when you want an aggregate.

melt: wide to long

`melt` takes the columns of a wide table and stacks them into one column of values, with a second column for the old column names:

```
import pandas as pd

wide = pd.DataFrame({"cell": ["A", "B"], "mon": [5, 7], "tue": [6, 9]})
long = wide.melt(id_vars="cell", var_name="day", value_name="calls")
print(long.to_dict("list"))
print(long.groupby("day")["calls"].sum().to_dict())
```

```
{'cell': ['A', 'B', 'A', 'B'], 'day': ['mon', 'mon', 'tue', 'tue'], 'calls': [5, 7, 6, 9]}
{'mon': 12, 'tue': 15}
```

`id_vars` are the columns that stay. The other columns go into `day` and `calls`. After the `melt` the data work with `groupby`, and with a plotting library that wants one column of values and one column of groups.

There is a trap in the column names that come from a wide table of numbers. The new column holds the old labels, and the dtype can be `object`, even when the labels are integers:

```
import pandas as pd

from pdbook.io import load_calls
from pdbook.traffic import traffic_grid

grid = traffic_grid(load_calls())
long = grid.reset_index().melt(id_vars="weekday", var_name="hour", value_name="calls")
print(long["hour"].dtype)
print(long["hour"].astype("int64").dtype)
```

```
object
int64
```

The `reset_index` put the text `weekday` and the integer hours in one set of column labels, so the labels became `object`. A later join with a table that has integer hours finds no match. Convert the column to the type that you want, immediately after the `melt`.

stack and unstack

`unstack` moves an index level into the columns, and `stack` does the reverse. They work on an index, and they are the tool after a `groupby` with two keys:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
by = calls.groupby(["roaming", "status"]).size()
print(type(by.index).__name__, by.shape)
print(by.unstack().to_string())
print(by.unstack(level="roaming").shape)
```

```
MultiIndex (8,)
status   answered  busy  failed  noanswer
roaming
False       15081  1382    1002      1972
True          445    46      31        41
(4, 2)
```

A `groupby` with two keys gives a Series with a two-level index. `unstack` moves the last level to the columns, and `unstack(level="roaming")` moves the other one. Chapter 11 explains the index.

Build: the traffic grid

`traffic_grid` makes the 7 by 24 table. It converts the zone first, because a call at 23:30 UTC is already the next day in Belgium. It reindexes to all 7 weekdays and 24 hours with zeros, so a missing combination is a 0 and the shape never changes:

```
import pandas as pd

from pdbook.io import load_calls
from pdbook.traffic import to_long, to_wide, traffic_grid

calls = load_calls()
grid = traffic_grid(calls)
long = to_long(grid)
print(grid.shape, int(grid.to_numpy().sum()) == len(calls))
print(long.shape, long.dtypes.astype(str).to_dict())

pd.testing.assert_frame_equal(to_wide(long), grid)
print("round trip equal")

busy = long.sort_values("calls", ascending=False).head(3)
print(busy.to_dict("records"))
print(grid.sum(axis=1).to_dict())
```

```
(7, 24) True
(168, 3) {'weekday': 'str', 'hour': 'int64', 'calls': 'int64'}
round trip equal
[{'weekday': 'Tue', 'hour': 9, 'calls': 300}, {'weekday': 'Wed', 'hour': 19, 'calls': 288},
{'weekday': 'Wed', 'hour': 10, 'calls': 278}]
{'Mon': 2925, 'Tue': 3698, 'Wed': 3836, 'Thu': 3021, 'Fri': 3057, 'Sat': 1745, 'Sun': 1718}
```

The busiest cell is Tuesday at 09:00 with 300 calls. Wednesday at 19:00 follows with 288, and Wednesday at 10:00 with 278. Wednesday is the busiest day with 3,836 calls, and the weekend days have 1,745 and 1,718, which is about half. A person reads the grid with the eye, and a program reads the long form. The functions convert between them without loss, and the `assert_frame_equal` shows it.

How it works

A `pivot_table` is a `groupby` followed by an `unstack`. The `groupby` reduces the table to one row for each pair of row key and column key. The `unstack` then moves the last index level into the columns. `melt` is the reverse: it moves the columns back into rows. The two directions are `unstack` and `stack` on the index, and `pivot_table` and `melt` are the names that work on columns.

You can build the traffic grid of this chapter in two lines and compare it with `traffic_grid`:

```
import pandas as pd

from pdbook.io import load_calls
from pdbook.traffic import to_long, traffic_grid

calls = load_calls()
local = calls["start"].dt.tz_convert("Europe/Brussels")
by_pair = calls.groupby([local.dt.dayofweek, local.dt.hour]).size()
print(type(by_pair.index).__name__, by_pair.shape)

grid = by_pair.unstack(fill_value=0)
reference = traffic_grid(calls)
print(grid.shape, bool((grid.to_numpy() == reference.to_numpy()).all()))

back = reference.stack()
print(back.shape, int(back.sum()), len(to_long(reference)))
```

```
MultiIndex (168,)
(7, 24) True
(168,) 20000 168
```

The `groupby` gives a Series with a two-level index, one entry for each pair that occurs. The `unstack` makes the 7 by 24 grid, and the values are the same as `traffic_grid`. The `stack` of the grid gives 168 values, one for each cell, that add up to 20,000, and `to_long` has the same 168 rows.

- **Invariant.** A pivot table has one cell for each pair of keys, and it holds an aggregate of every row with that pair.
- **Failure.** A pair that occurs twice in `pivot` has no rule to follow, and it raises an error. In `pivot_table`, it is silently aggregated.
- **Cost.** The `groupby` pass plus a copy into the 2-D array. A wide result with many empty cells holds `NaN` or 0 for each one.

Reference

```
import pandas as pd

from pdbook.sigs import show_signature
from pdbook.traffic import to_long, to_wide, traffic_grid

show_signature(pd.DataFrame.pivot_table, ["values", "index", "columns", "aggfunc",
"fill_value", "margins", "dropna"])
show_signature(pd.DataFrame.pivot, ["columns", "index", "values"])
show_signature(pd.crosstab, ["index", "columns", "normalize"])
show_signature(pd.DataFrame.melt, ["id_vars", "value_vars", "var_name", "value_name"])
show_signature(pd.Series.unstack, ["level", "fill_value"])
show_signature(traffic_grid)
show_signature(to_long)
show_signature(to_wide)
```

```
DataFrame.pivot_table(
    values=None
    index=None
    columns=None
    aggfunc: 'AggFuncType' = 'mean'
    fill_value=None
    margins: 'bool' = False
    dropna: 'bool' = True
)  # 5 more parameters
DataFrame.pivot(
    columns
    index=<no_default>
    values=<no_default>
)  # 1 more parameters
crosstab(
    index
    columns
    normalize: "bool | Literal[0, 1, 'all', 'index', 'columns']" = False
)  # 7 more parameters
DataFrame.melt(
    id_vars=None
    value_vars=None
    var_name=None
    value_name: 'Hashable' = 'value'
)  # 3 more parameters
Series.unstack(
    level: 'IndexLabel' = -1
    fill_value: 'Hashable | None' = None
)  # 2 more parameters
traffic_grid(
    calls: 'pd.DataFrame'
    tz: 'str' = 'Europe/Brussels'
)
to_long(
    grid: 'pd.DataFrame'
)
to_wide(
    long: 'pd.DataFrame'
)
```

Parameter	Default	What it does here	What goes wrong
`pivot_table(values, index, columns)`	`None`	the cell, the row key and the column key	—
`pivot_table(aggfunc=...)`	`'mean'`	how to combine the rows of one cell	a duplicate row is added in silence: total 20,016 and not 20,000
`pivot_table(fill_value=...)`	`None`	the value for a cell with no rows	—
`pivot_table(margins=...)`	`False`	`True` adds a row and a column `All`	—
`pivot_table(dropna=...)`	`True`	drop a column that is all `NaN`	the status `failed` disappears from a table of mean durations
`pivot(columns, index, values)`	`index`, `values`: not set	reshape with no aggregation	a pair that occurs twice raises `ValueError`
`crosstab(index, columns)`	required	counts of two columns	—
`crosstab(normalize=...)`	`False`	`"columns"` makes each column add up to 1	—
`melt(id_vars, value_vars)`	`None`	the columns to keep and the columns to stack	—
`melt(var_name, value_name)`	`None`, `'value'`	names of the two new columns	`hour` comes back as `object`; convert it
`unstack(level, fill_value)`	`-1`, `None`	move an index level to the columns	a pair that does not occur is `NaN`
`traffic_grid(calls, tz=...)`	required, `'Europe/Brussels'`	the 7 by 24 table of calls	the grid in UTC differs in 164 of 168 cells
`to_wide(long)`	required	back to the grid	a repeated row raises `ValueError`

`traffic_grid` returns a DataFrame indexed by weekday names with the columns 0 to 23. `to_long` returns 168 rows.

What you have now

- You choose long or wide from the next step.
- You use `pivot_table` to aggregate, `pivot` to reshape a unique key, `crosstab` for counts and shares.
- You use `melt` to go back, and you fix the type of the new column at once.
- You use `stack` and `unstack` after a `groupby` with two keys.
- You have a traffic grid and a test that the round trip is exact.

The next chapter looks at the object that `unstack` and `groupby` left on the rows: the index.

Next: Chapter 11, The index and MultiIndex.

11. The index and MultiIndex

> **Build** — `index_audit(df)`: a function that states what the index of a table promises (unique, sorted, named, typed, no missing labels), so that you check it before you rely on it.
>
> **Theory kept** — The index is a set of row labels. It gives fast lookup, it drives alignment, and it is the key of a join on the index. It does not have to be unique, and it does not have to be sorted, and a lot of code assumes both. A MultiIndex is an index with several levels.
>
> **Source** — New. Library: `pdbook/indexcheck.py`.
>
> **Run this first** — `uv run python -m examples.ch11_index`, which is `examples/ch11_index.py`. Exercises: `exercises/11.md`. Solutions: `exercises/solutions/11.md`.

What the index is for

Three jobs belong to the index:

1. **Lookup.** `df.loc[label]` finds a row by a hash of the label, without a scan of the column.
2. **Alignment.** An operation between two objects matches the rows by label (Chapter 1).
3. **Time.** A `DatetimeIndex` gives time-based slicing and resampling (Chapter 12).

If your code does none of the three, the default `RangeIndex` is the right index. Do not call `set_index` out of habit. A column is easier to filter, to join on and to write to a file.

Set it, reset it, sort it

`set_index` moves a column into the index. `reset_index` moves it back. `sort_index` sorts by the labels:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
by_id = calls.set_index("call_id")
print(by_id.index.name, by_id.index.is_unique, by_id.index.is_monotonic_increasing)
print(by_id.loc[7, "status"], by_id.loc[[7, 5]].index.tolist())

ordered = by_id.sort_index()
print(ordered.index.is_monotonic_increasing)
print(ordered.reset_index().columns.tolist()[:3])
```

```
call_id True False
answered [7, 5]
True
['call_id', 'start', 'caller']
```

The index of `by_id` is unique and not sorted. The file has 1,000 late-arriving records at its end (Chapter 3). A list of labels returns the rows in the order of the list, so `[7, 5]` gives call 7 first. The index is not sorted, and `sort_index` fixes that.

Lookup by label is fast. A boolean mask scans the whole column for each lookup:

```
import time

from pdbook.io import load_calls

calls = load_calls()
by_id = calls.set_index("call_id")
ids = calls["call_id"].sample(500, random_state=1).tolist()

t0 = time.perf_counter()
a = [by_id.loc[i, "status"] for i in ids]
t_index = time.perf_counter() - t0

t0 = time.perf_counter()
b = [calls.loc[calls["call_id"] == i, "status"].iloc[0] for i in ids]
t_mask = time.perf_counter() - t0

print(a == b)
print(f"index {t_index * 1000:.1f} ms, mask {t_mask * 1000:.1f} ms, ratio {t_mask /
t_index:.0f}x")
assert a == b and t_mask / t_index > 3
```

```
# output varies
True
index 18.1 ms, mask 163.1 ms, ratio 9x
```

The block prints the time of 500 single lookups by the index and by mask. The numbers depend on your hardware. The assertion fails if the index is not at least 3 times faster on yours. If you look up many labels at once, use one `loc` with a list of labels: `by_id.loc[ids]` is a single operation.

Duplicates in the index

A pandas index can repeat labels. Then `loc` returns a DataFrame where you expected a scalar or a row:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls().head(2)
stacked = pd.concat([calls, calls])
print(stacked.index.tolist())
print(stacked.loc[0].shape, calls.loc[0].shape)
print(stacked.reset_index(drop=True).index.tolist())
```

```
[0, 1, 0, 1]
(2, 9) (9,)
[0, 1, 2, 3]
```

`calls.loc[0]` is one row and has the shape `(9,)`. `stacked.loc[0]` is two rows and has the shape `(2, 9)`. Code that worked on the first table breaks, or gives a wrong answer, on the second. `concat(..., ignore_index=True)` and `reset_index(drop=True)` give a clean index. Chapter 9 made the same point about `concat`.

reindex: choose the labels you want

`reindex` returns the rows for exactly the labels that you give, in that order. A label that does not exist gives a row of `NaN`:

```
import pandas as pd

from pdbook.io import load_calls

by_id = load_calls().set_index("call_id")
print(by_id["duration_s"].reindex([1, 2, 99999]).tolist())

hours = pd.Series({9: 300, 10: 270, 14: 254})
print(hours.reindex(range(8, 16), fill_value=0).to_dict())
```

```
[24.0, 0.0, nan]
{8: 0, 9: 300, 10: 270, 11: 0, 12: 0, 13: 0, 14: 254, 15: 0}
```

`loc` with a missing label raises `KeyError`. `reindex` does not. This is the tool for "I want all 24 hours, even the hours without calls", which is what `traffic_grid` did in Chapter 10.

Alignment in arithmetic

An operator aligns on the index. A missing label gives `NaN`. The method form has `fill_value`:

```
import pandas as pd

a = pd.Series([1, 2, 3], index=[1, 2, 3])
b = pd.Series([10, 20], index=[2, 3])
print((a + b).tolist())
print(a.add(b, fill_value=0).tolist())
```

```
[nan, 12.0, 23.0]
[1.0, 12.0, 23.0]
```

The sum has a `NaN` at the label that only `a` has. `add(b, fill_value=0)` treats the missing label as 0 and keeps the value 1. This is correct for counts, where a missing label means "none". It is wrong for a price, where a missing label means "unknown".

MultiIndex: several levels

A `groupby` with two keys gives a MultiIndex. So does `set_index` with two columns. It is an index whose labels are tuples:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
counts = calls.groupby(["caller", "status"]).size()
print(counts.index.names, counts.index.nlevels)
print(counts.loc["S0001"].to_dict())
print(int(counts.loc[("S0001", "answered")]))
print(counts.xs("answered", level="status").shape)
```

```
['caller', 'status'] 2
{'answered': 8, 'failed': 1, 'noanswer': 2}
8
(600,)
```

`loc` with one label selects the first level, and `loc` with a tuple selects both. `xs` selects one value of a level that is not the first. `unstack` and `reset_index` leave the MultiIndex:

```
import pandas as pd

from pdbook.io import load_calls

counts = load_calls().groupby(["caller", "status"]).size()
wide = counts.unstack(fill_value=0)
flat = counts.reset_index(name="n")
print(wide.shape, wide.loc["S0001"].to_dict())
print(flat.head(2).to_dict("records"))
```

```
(600, 4) {'answered': 8, 'busy': 0, 'failed': 1, 'noanswer': 2}
[{'caller': 'S0001', 'status': 'answered', 'n': 8}, {'caller': 'S0001', 'status': 'failed',
'n': 1}]
```

For most work, prefer `reset_index` and ordinary columns after a `groupby`. A column is easy to filter, to join and to write. Keep the MultiIndex for a final table that a person reads, or for repeated `loc` lookups on a table.

Sorted or not

Slicing on a MultiIndex needs a sorted index. On an unsorted index, a slice on the second level raises an error:

```
import pandas as pd
from pandas.errors import UnsortedIndexError

from pdbook.io import load_calls

calls = load_calls()
multi = calls.set_index(["caller", "start"])
window = ("S0001", slice("2026-09-01", "2026-09-10"))
try:
    multi.loc[window, :]
except UnsortedIndexError as error:
    print("UnsortedIndexError:", str(error)[:46])

ordered = multi.sort_index()
print(ordered.index.is_monotonic_increasing)
print(ordered.loc[window, :].shape)
```

```
UnsortedIndexError: 'MultiIndex slicing requires the index to be l
True
(7, 7)
```

The error is a good one. pandas stops, and it does not guess the order of the rows. A habit that avoids the error: sort the index right after you set it.

Build: index_audit

`index_audit` is a function that reads the properties of an index, so that the assumptions are explicit. The audit of a table is five facts:

```
import pandas as pd

from pdbook.indexcheck import index_audit
from pdbook.io import load_calls

calls = load_calls()
for name, table in [
    ("default", calls),
    ("by call_id", calls.set_index("call_id")),
    ("two copies", pd.concat([calls.head(3), calls.head(3)])),
    ("caller, start", calls.set_index(["caller", "start"])),
]:
    a = index_audit(table)
    print(f"{name:14s} levels={a['levels']} unique={a['unique']!s:5} sorted={a['sorted']!s:5}
duplicates={a['duplicate_labels']}")
```

```
default        levels=1 unique=True  sorted=True  duplicates=0
by call_id     levels=1 unique=True  sorted=False duplicates=0
two copies     levels=1 unique=False sorted=False duplicates=3
caller, start  levels=2 unique=False sorted=False duplicates=1
```

The fourth line is a surprise. The index `(caller, start)` has one duplicate label, so even a pair of subscriber and second is not a key: one subscriber started two calls in the same second. Test a key that you assumed.

Put `index_audit` in the first lines of a function that assumes a sorted or unique index. An `assert audit["unique"]` is one line, and it turns a wrong answer into a stop.

How it works

An `Index` is an array of labels with a lookup engine. The first call to `get_loc` builds the engine: a hash table for any index, and a binary search for an index that is unique and sorted. Later calls reuse it.

A `MultiIndex` does not store tuples. It stores *levels*, the distinct values of each key, and *codes*, small integers that say which level value each row has. This is the same idea as a `category` (Chapter 4).

You can see both:

```
import time

import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
index = pd.Index(calls["call_id"].sort_values().to_numpy())
print(index.is_unique, index.is_monotonic_increasing)
print(index.get_loc(12345), int(index.searchsorted(12345)))

fresh = pd.Index(calls["call_id"].to_numpy())
t0 = time.perf_counter()
fresh.get_loc(12345)
t_first = time.perf_counter() - t0
t0 = time.perf_counter()
for _ in range(1000):
    fresh.get_loc(12345)
t_next = (time.perf_counter() - t0) / 1000
print(f"first lookup {t_first * 1e6:.0f} us, later lookups {t_next * 1e6:.1f} us")
assert t_first > 5 * t_next

multi = calls.groupby(["roaming", "status"]).size().index
print([level.tolist() for level in multi.levels])
print([code.tolist() for code in multi.codes])
rebuilt = [(multi.levels[0][a], multi.levels[1][b]) for a, b in zip(*multi.codes)]
print(rebuilt == multi.tolist())
```

```
# output varies
True True
12344 12344
first lookup 310 us, later lookups 0.4 us
[[False, True], ['answered', 'busy', 'failed', 'noanswer']]
[[0, 0, 0, 0, 1, 1, 1, 1], [0, 1, 2, 3, 0, 1, 2, 3]]
True
```

The sorted unique index answers with `get_loc` and with `searchsorted`, and both give 12,344 (the position of the call number 12,345). The block prints the time of the first lookup on a new index and of the later ones. The first builds the hash table, and the assertion fails if it is not at least five times slower than the later ones. The `MultiIndex` of a `groupby` on `roaming` and `status` has two levels of 2 and 4 values, and 8 rows of codes. Rebuilding the tuples from the levels and the codes gives the same tuples.

- **Invariant.** A lookup in a unique index is one hash probe, after a build that happens once.
- **Failure.** A non-unique index returns a slice or a mask, and a code that expects one row gets several.
- **Cost.** The build of the engine, on the first lookup of each new index object. A new index object, for example after `reset_index`, pays it again.

Reference

```
import pandas as pd

from pdbook.indexcheck import index_audit
from pdbook.sigs import show_signature

show_signature(pd.DataFrame.set_index, ["keys", "drop", "verify_integrity"])
show_signature(pd.DataFrame.reset_index, ["drop", "level", "names"])
show_signature(pd.DataFrame.reindex, ["labels", "fill_value"])
show_signature(pd.DataFrame.sort_index, ["level", "ascending"])
show_signature(pd.Series.add, ["other", "fill_value"])
show_signature(pd.DataFrame.xs, ["key", "axis", "level"])
show_signature(pd.Index.get_loc, ["key"])
show_signature(index_audit)
```

```
DataFrame.set_index(
    keys
    drop: 'bool' = True
    verify_integrity: 'bool | lib.NoDefault' = <no_default>
)  # 3 more parameters
DataFrame.reset_index(
    level: 'IndexLabel | None' = None
    drop: 'bool' = False
    names: 'Hashable | Sequence[Hashable] | None' = None
)  # 5 more parameters
DataFrame.reindex(
    labels=None
    fill_value: 'Scalar | None' = nan
)  # 9 more parameters
DataFrame.sort_index(
    level: 'IndexLabel | None' = None
    ascending: 'bool | Sequence[bool]' = True
)  # 8 more parameters
Series.add(
    other
    fill_value=None
)  # 3 more parameters
NDFrame.xs(
    key: 'IndexLabel'
    axis: 'Axis' = 0
    level: 'IndexLabel | None' = None
)  # 2 more parameters
Index.get_loc(
    key
)  # 1 more parameters
index_audit(
    df: 'pd.DataFrame'
)
```

Parameter	Default	What it does here	What goes wrong
`set_index(keys)`	required	a column or list of columns to use as the index	the pair `(caller, start)` has one repeated label
`set_index(drop=...)`	`True`	remove the column after it moves	—
`reset_index(drop=...)`	`False`	`True` throws the old index away	`False` adds it back as a column
`reindex(labels, fill_value=...)`	`None`, `nan`	the rows for exactly these labels	a label that is not there gives a row of `NaN`, and `loc` raises `KeyError`
`sort_index(level, ascending)`	`None`, `True`	sort by the labels	slicing a MultiIndex that is not sorted raises `UnsortedIndexError`
`add(other, fill_value=...)`	required, `None`	add by label; treat a missing label as this value	`fill_value=0` is right for counts and wrong for prices
`xs(key, level=...)`	required, `None`	one value of one level	—
`get_loc(key)`	required	an integer, a slice or a mask	a repeated label returns a slice or a mask
`index_audit(df)`	required	a dict: levels, names, dtype, unique, sorted, duplicate labels, missing	—

`index_audit` returns a dict. On the call records indexed by `(caller, start)`: `unique=False`, `duplicate_labels=1`.

What you have now

- You use the index for lookup, alignment and time, and you keep the default index otherwise.
- You know that an index has duplicates or is unsorted in some tables, and what that does to `loc`.
- You use `reindex` to ask for exact labels, and `add(fill_value=)` when a missing label means none.
- You use a MultiIndex for the output of a `groupby`, and `reset_index` when you continue to work.
- You can audit an index in one call.

The next chapter takes the index that matters most for operations data: time.

Next: Chapter 12, Dates and resampling.

12. Dates and resampling

> **Build** — `hourly_kpi(kpi)` : a function that turns 5-minute cell measurements into hourly values in local time, removes repeated rows, and keeps the number of samples behind each hour, so that you can see which hours are incomplete. It handles the day with 25 hours.
>
> **Theory kept** — Store time in UTC and convert to a zone to display and to group by day. A timestamp without a zone is a guess. `resample` groups by time bins, and it keeps the empty bins. A local day is not always 24 hours.
>
> **Source** — New. Data: `data/kpi.csv` . Library: `pdbook/kpi.py` .
>
> **Run this first** — `uv run python -m examples.ch12_dates` , which is `examples/ch12_dates.py` . Exercises: `exercises/12.md` . Solutions: `exercises/solutions/12.md` .

Timestamps with a zone

A timestamp can be naive (no zone) or aware (with a zone). A naive time is a wall-clock reading, and it does not say which clock. An aware time is a point on the time line. The call records are aware and in UTC. Two methods change the zone, and they do different things:

Method	What it does
`tz_localize(zone)`	attach a zone to a naive time. The wall-clock reading stays and the instant changes.
`tz_convert(zone)`	express an aware time in another zone. The instant stays and the reading changes.

```
import pandas as pd

utc = pd.Timestamp("2026-09-10 16:30", tz="UTC")
print(utc.tz_convert("Europe/Brussels"))
print(utc.tz_convert("America/New_York"))

naive = pd.Timestamp("2026-09-10 16:30")
print(naive.tz_localize("Europe/Brussels"))
print(naive.tz_localize("Europe/Brussels") == utc)
```

```
2026-09-10 18:30:00+02:00
2026-09-10 12:30:00-04:00
2026-09-10 16:30:00+02:00
False
```

The same reading, 16:30, is a different instant in each zone. Belgium is two hours ahead of UTC in September, so 16:30 in Brussels is 14:30 in UTC, and the last line is `False` . Store timestamps as UTC, and convert at the edge, when you show them or group by day.

The accessor `dt` gives the parts: `dt.hour` , `dt.dayofweek` , `dt.date` , `dt.floor("h")` , `dt.strftime(...)` . The parts depend on the zone of the column. Convert first.

Summer time

The zone database knows the rules of summer time, and it makes two kinds of local readings that a naive time cannot represent:

```
import pandas as pd

zone = "Europe/Brussels"
try:
    pd.Timestamp("2026-10-25 02:30").tz_localize(zone)
except ValueError as error:
    print("ambiguous:", type(error).__name__)

try:
    pd.Timestamp("2027-03-28 02:30").tz_localize(zone)
except ValueError as error:
    print("missing  :", type(error).__name__)

first = pd.Timestamp("2026-10-25 02:30").tz_localize(zone, ambiguous=True)
second = pd.Timestamp("2026-10-25 02:30").tz_localize(zone, ambiguous=False)
print(first, second)
```

```
ambiguous: ValueError
missing  : ValueError
2026-10-25 02:30:00+02:00 2026-10-25 02:30:00+01:00
```

On 25 October 2026 the clocks go from 03:00 back to 02:00, so the reading 02:30 happens twice. On 28 March 2027 they go from 02:00 to 03:00, so 02:30 never happens. pandas refuses both and does not guess. You choose with `ambiguous=` and `nonexistent=`. A system that logs local time without the offset cannot tell the two readings apart. A log of this kind is ambiguous for one hour in each year.

A day with 25 hours

The KPI file covers 24 to 26 October 2026 in UTC, and the clocks change on the 25th. Count the hours of each local day with `date_range`:

```
import pandas as pd

hours = pd.date_range("2026-10-24", "2026-10-27", freq="h", tz="Europe/Brussels",
inclusive="left")
per_day = pd.Series(1, index=hours).groupby(hours.date).sum()
print(per_day.to_dict())
```

```
{datetime.date(2026, 10, 24): 24, datetime.date(2026, 10, 25): 25, datetime.date(2026, 10,
26): 24}
```

The 25th has 25 hours. A daily report that assumes 24 hours is wrong for one day of the year: a "calls per hour" figure divides by 24 and misses by 4%, and a rolling window of 24 rows is not one day on that date. Use the time itself, not the number of rows, to define a day.

A DatetimeIndex

When a column of timestamps becomes the index, you can slice by strings and you get `resample`:

```
import pandas as pd

from pdbook.io import load_kpi

kpi = load_kpi().drop_duplicates(["ts", "cell_id"])
one = kpi[kpi["cell_id"] == "ANT-001"].set_index("ts")["prb_util_pct"]
print(one.index.is_monotonic_increasing, len(one))
print(len(one["2026-10-25"]))
print(len(one["2026-10-25 08:00":"2026-10-25 08:59"]))
print(len(one.between_time("08:00", "08:59")))
```

```
True 851
284
12
36
```

The string `"2026-10-25"` selects the whole UTC day. The slice with two times includes both ends, as `.loc` does (Chapter 3). The hour from 08:00 to 08:59 has 12 samples. The method `between_time` takes the same minutes of the three days: 36 samples.

The partial-string slice needs a sorted index. The call records are not sorted, and then you get an error:

```
import pandas as pd

from pdbook.io import load_calls

by_time = load_calls().set_index("start")
print(by_time.index.is_monotonic_increasing)
try:
    by_time.loc["2026-09-10":"2026-09-10"]
except KeyError as error:
    print("KeyError:", str(error)[:50])

ordered = by_time.sort_index()
print(len(ordered.loc["2026-09-10"]))
```

```
False
KeyError: 'Value based partial slicing on non-monotonic Date
782
```

The 1,000 late records at the end of the file make the index non-monotonic. pandas stops, because a partial-string slice with bounds that are not in the index is not allowed on such an index. After `sort_index`, the slice returns the 782 calls of that UTC day. `resample` itself does not need a sorted index.

resample

`resample` groups by time bins and aggregates. It is `groupby` with a time key. The frequency is a string: `"5min"`, `"h"`, `"D"`, `"W"`, `"MS"` for the start of a month:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls().set_index("start")
per_hour = calls.resample("h").size()
print(len(per_hour), int(per_hour.sum()))
print(per_hour.idxmax(), int(per_hour.max()))
print(int((per_hour == 0).sum()), "hours without a call")

local_day = calls.tz_convert("Europe/Brussels").resample("D").size()
print(len(local_day), local_day.idxmax().date(), int(local_day.max()))
```

```
720 20000
2026-09-10 16:00:00+00:00 73
4 hours without a call
30 2026-09-30 793
```

The month has 720 hours, 30 days times 24, and the counts add up to 20,000. The busiest hour is 16:00 UTC on 10 September with 73 calls. Hours without a call stay in the result with a count of 0, because `resample` makes every bin between the first and the last time. The 30 local days are all in September, which has no clock change. The build below uses three days that have one.

For a mean, an empty bin is `NaN` and not 0, because the mean of nothing is not zero. That is the second reason to keep `n` next to a mean.

Build: hourly_kpi

`hourly_kpi` does the steps that a person does by hand, and it keeps the evidence of each step:

```
import pandas as pd

from pdbook.io import load_kpi
from pdbook.kpi import hourly_kpi

kpi = load_kpi()
hourly = hourly_kpi(kpi)
print(len(kpi), len(kpi.drop_duplicates(["ts", "cell_id"])), len(hourly))
print(hourly["n"].value_counts().sort_index().to_dict())
print(hourly.groupby("cell_id").size().to_dict())

day = hourly[hourly["ts"].dt.strftime("%Y-%m-%d") == "2026-10-25"]
print(day.groupby("cell_id").size().unique().tolist())
print(int(hourly["n"].sum()), int(hourly["complete"].sum()))
```

```
5104 5079 432
{0: 2, 10: 5, 11: 71, 12: 354}
{'ANT-001': 72, 'ANT-002': 72, 'BRU-001': 72, 'BRU-002': 72, 'BRU-003': 72, 'BRU-004': 72}
[25]
5079 354
```

The raw file has 5,104 rows. 25 of them are repeats of another row, which leaves 5,079. The result has 432 rows: 6 cells and 72 hours. The hours have 12 samples when they are full.

354 hours are full, 71 have 11 samples, 5 have 10, and 2 have none. The two empty hours are the outage of the cell `BRU-004`, and they show `NaN` in the mean and 0 in `n`. The samples add up to 5,079, so no sample was lost or counted twice. Each cell has 25 hours on 25 October.

A mean of 11 samples is not the same quality as a mean of 12 samples. A mean of 0 samples is not a value. A dashboard that draws only the mean shows all hours with the same confidence. Draw `n` or `complete` with it.

How it works

A timestamp is stored as one 64-bit integer: the number of time units (here microseconds) since 1 January 1970, 00:00 UTC. A time zone is not part of the number. It is a label on the column that tells pandas how to *display* the integer and how to cut it into days. This is why `tz_convert` does not change a stored value. A day boundary in another zone is a different cut of the same numbers.

`resample` is a `floor` and a `groupby`: each timestamp is rounded down to the start of its bin, and the bins are grouped. You can check both claims:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
utc = calls["start"]
local = utc.dt.tz_convert("Europe/Brussels")
print(utc.dtype, local.dtype)
print(utc.astype("int64").equals(local.astype("int64")))
print(int(utc.astype("int64").iloc[0]), utc.iloc[0].isoformat(), local.iloc[0].isoformat())

by_floor = calls.groupby(utc.dt.floor("h")).size()
by_resample = calls.set_index("start").resample("h").size()
print(len(by_floor), len(by_resample))
print(bool((by_floor.reindex(by_resample.index, fill_value=0) == by_resample).all()))
```

```
datetime64[us, UTC] datetime64[us, Europe/Brussels]
True
1788214410000000 2026-08-31T22:13:30+00:00 2026-09-01T00:13:30+02:00
716 720
True
```

The two columns have different dtypes and hold the same integers. The first stored value is a number of microseconds, and its UTC and local displays differ by two hours. The `floor` and `groupby` give 716 hours, because they have only

the hours that occur. `resample` gives 720, because it keeps the empty bins. After a `reindex` with zeros the two agree in every hour.

- **Invariant.** The stored integer is a point in time, in UTC. The zone changes only the display and the cut into days.
- **Failure.** A naive timestamp has no zone, so the same reading is different points in time for two readers.
- **Cost.** A zone conversion is one pass of arithmetic with the zone rules, and no copy of the text. A `resample` makes every bin, empty ones included.

Reference

```
import pandas as pd

from pdbook.kpi import hourly_kpi
from pdbook.sigs import show_signature

show_signature(pd.Series.dt.tz_convert, ["tz"])
show_signature(pd.Series.dt.tz_localize, ["tz", "ambiguous", "nonexistent"])
show_signature(pd.DataFrame.resample, ["rule", "closed", "label"])
show_signature(pd.date_range, ["start", "end", "periods", "freq", "tz", "inclusive"])
show_signature(pd.DataFrame.between_time, ["start_time", "end_time", "inclusive"])
show_signature(hourly_kpi)
```

```
DatetimeArray.tz_convert(
    tz
)  # 1 more parameters
DatetimeArray.tz_localize(
    tz
    ambiguous: 'TimeAmbiguous' = 'raise'
    nonexistent: 'TimeNonexistent' = 'raise'
)  # 1 more parameters
NDFrame.resample(
    rule
    closed: "Literal['right', 'left'] | None" = None
    label: "Literal['right', 'left'] | None" = None
)  # 7 more parameters
date_range(
    start=None
    end=None
    periods=None
    freq=None
    tz=None
    inclusive: 'IntervalClosedType' = 'both'
)  # 4 more parameters
NDFrame.between_time(
    start_time
    end_time
    inclusive: 'IntervalClosedType' = 'both'
)  # 2 more parameters
hourly_kpi(
    kpi: 'pd.DataFrame'
    tz: 'str' = 'Europe/Brussels'
    full_hour: 'int' = 12
)
```

Parameter	Default	What it does here	What goes wrong
`tz_convert(tz)`	required	show an aware timestamp in another zone	— (the stored integers do not change)
`tz_localize(tz)`	required	attach a zone to a naive timestamp	the reading is a different instant in each zone
`tz_localize(ambiguous=...)`	`'raise'`	what to do with a reading that happens twice	`2026-10-25 02:30` raises; `True` and `False` choose the first and the second
`tz_localize(nonexistent=...)`	`'raise'`	what to do with a reading that never happens	`2027-03-28 02:30` raises
`resample(rule)`	required	`"h"`, `"D"`, `"5min"` and so on	empty bins stay: 720 hours, 4 with no call
`resample(closed=..., label=...)`	`None`, `None`	which end of a bin is inside, and which end names it	—
`date_range(start, end, freq, tz)`	`None`	a regular grid of timestamps	with `tz="Europe/Brussels"` the day of 25 October has 25 hours
`date_range(inclusive=...)`	`'both'`	which ends are in the range	—
`between_time(start_time, end_time)`	required	rows between two clock times, any date	—
`hourly_kpi(kpi, tz, full_hour)`	required, `'Europe/Brussels'`, `12`	hourly mean for each cell, with `n` and `complete`	5,104 rows, 25 repeats, 432 hours, 354 complete

`resample` returns a resampler object. Call `size`, `mean` or `agg` on it.

What you have now

- You store time as UTC and convert to a zone to show or group.
- You know `tz_localize` from `tz_convert`, and that summer time makes ambiguous and missing readings.
- You count hours, not rows, in a local day, and you know the day with 25 hours.
- You slice a sorted `DatetimeIndex` by strings, and you sort first.
- You resample, and you keep the sample count with every mean.

The next chapter looks at the series inside each hour: rolling windows, shifts and gaps.

Next: Chapter 13, Windows, shifts and gaps.

13. Windows, shifts and gaps

> **Build** — `flag_spikes(kpi)` : a function that flags a measurement that is far above what the same cell did in the previous day, with a score of the flags against the one spike that is known to be in the data.
>
> **Theory kept** — `shift` and `rolling` work on rows. Time series have gaps, so a row is not a unit of time. A window by time (`"1h"`) has a fixed length and a varying count. A baseline that includes the sample that it judges is contaminated.
>
> **Source** — New. Library: `pdbook/spikes.py` .
>
> **Run this first** — `uv run python -m examples.ch13_windows` , which is `examples/ch13_windows.py` . Exercises: `exercises/13.md` . Solutions: `exercises/solutions/13.md` .

Shift and diff

`shift(n)` moves the values down by `n` rows. `diff()` is the value minus the previous value. `pct_change()` is the relative change:

```
import pandas as pd

s = pd.Series([10.0, 12.0, 9.0, 9.0], index=pd.date_range("2026-10-24 10:00", periods=4,
freq="5min"))
print(s.shift(1).tolist())
print(s.diff().tolist())
print(s.pct_change().round(3).tolist())
```

```
[nan, 10.0, 12.0, 9.0]
[nan, 2.0, -3.0, 0.0]
[nan, 0.2, -0.25, 0.0]
```

The first row has no previous value, so it is `NaN` . These methods work on the position, one row up. That is only "five minutes ago" when no row is missing.

Rows are not time

The KPI table has gaps. Look at the cell `BRU-004` and the time between each row and the row before it:

```
import pandas as pd

from pdbook.io import load_kpi

kpi = load_kpi().drop_duplicates(["ts", "cell_id"])
cell = kpi[kpi["cell_id"] == "BRU-004"].set_index("ts")["prb_util_pct"].sort_index()
step = cell.index.to_series().diff()
print(step.value_counts().head(4).to_dict())
print(step.max(), step.idxmax())

wrong = cell.diff()
right = cell.diff().where(step == pd.Timedelta("5min"))
print(int(wrong.notna().sum()), int(right.notna().sum()))
```

```
{Timedelta('0 days 00:05:00'): 803, Timedelta('0 days 00:10:00'): 16, Timedelta('0 days
00:15:00'): 1, Timedelta('0 days 02:05:00'): 1}
0 days 02:05:00 2026-10-25 12:00:00+00:00
821 803
```

The first line counts the steps. 803 are 5 minutes, 16 are 10 minutes and one is 15 minutes. One step is 2 hours and 5 minutes: the outage.

At the moment after the outage, `diff()` subtracts a value from two hours earlier, and the number looks like a change of five minutes. The second form keeps only the differences between rows that are 5 minutes apart. The count falls from 821 to 803 rows with a value.

There are two ways to make rows into time. Put the series on a regular grid with `reindex` or `asfreq`, so that a missing sample is an explicit `NaN` and `shift(1)` is five minutes. Or use time-based methods, such as `shift(freq="5min")`, and windows given as a string.

Rolling windows

A rolling window gives each row a statistic over the rows near it. The window can be a number of rows or a time span:

```
import pandas as pd

from pdbook.io import load_kpi

kpi = load_kpi().drop_duplicates(["ts", "cell_id"])
cell = kpi[kpi["cell_id"] == "BRU-004"].set_index("ts")["prb_util_pct"].sort_index()
moment = pd.Timestamp("2026-10-25 12:30", tz="UTC")

by_rows = cell.rolling(12).mean().loc[moment]
by_time = cell.rolling("1h").mean().loc[moment]
count = cell.rolling("1h").count().loc[moment]
span = moment - cell.index[cell.index.get_loc(moment) - 11]
print(round(float(by_rows), 2), round(float(by_time), 2), int(count), span)
```

```
54.26 61.67 7 0 days 02:55:00
```

The window of 12 rows ends at 12:30. It begins 2 hours and 55 minutes earlier, because it counts rows and the outage has none. It holds the 7 samples after the outage and 5 samples from before it. The window of one hour holds only the 7 samples after the outage, and the mean is 61.67 against 54.26. Use the time window when you mean a time span. Use the row window only after you put the series on a regular grid.

The argument `min_periods` says how many samples a window needs before it gives a value. With a time window, the default is 1, so a window with one sample gives a mean of one sample. Set it:

```
import pandas as pd

from pdbook.io import load_kpi

kpi = load_kpi().drop_duplicates(["ts", "cell_id"])
cell = kpi[kpi["cell_id"] == "BRU-004"].set_index("ts")["prb_util_pct"].sort_index()

loose = cell.rolling("1h").mean()
strict = cell.rolling("1h", min_periods=9).mean()
print(int(loose.isna().sum()), int(strict.isna().sum()))
print(int(cell.rolling("1h").count().eq(1).sum()))
```

```
0 16
2
```

With the default, no value is `NaN`, and 2 windows hold only one sample. With `min_periods=9`, which is three quarters of a full hour, 16 values are `NaN`: the windows with fewer than 9 samples. This is the same idea as the column `complete` of Chapter 12: a statistic needs enough data to mean something.

Expanding and exponential windows

`expanding()` uses all rows from the start to the current row. `ewm` weights recent rows more:

```
import pandas as pd

s = pd.Series([1.0, 2.0, 3.0, 10.0])
print(s.expanding().mean().tolist())
print(s.ewm(span=3).mean().round(2).tolist())
```

```
[1.0, 1.5, 2.0, 4.0]
[1.0, 1.67, 2.43, 6.47]
```

The expanding mean is a running mean, and the exponential mean follows the last value more quickly. The first is the right tool for a cumulative figure, and the second for a signal with a drift.

The baseline must not contain the sample

An anomaly score compares a sample with a baseline. If the baseline includes the sample, a large sample raises its own baseline and hides itself. `closed="left"` excludes the current row from the window:

```
import pandas as pd

from pdbook.io import load_kpi

kpi = load_kpi().drop_duplicates(["ts", "cell_id"])
s = kpi[kpi["cell_id"] == "BRU-002"].set_index("ts")["drop_rate_pct"].sort_index()
at = pd.Timestamp("2026-10-25 14:00", tz="UTC")

print(f"sample at 14:00 = {s.loc[at]:.2f}%, median of the cell = {s.median():.2f}%")
for closed in ("right", "left"):
    roll = s.rolling("1D", min_periods=50, closed=closed)
    z = ((s - roll.mean()) / roll.std()).loc[at]
    print(f"closed={closed:5s} score at 14:00 = {z:.1f}")
```

```
sample at 14:00 = 11.34%, median of the cell = 0.58%
closed=right score at 14:00 = 16.5
closed=left  score at 14:00 = 94.2
```

The sample at 14:00 is 11.34%, and the median of the cell is 0.58%. With `closed="right"` (the default) the baseline contains the spike and the score is lower. With `closed="left"` the score is the score of a sample against the past. Both scores are far above any usual threshold, so the choice does not change the result in this case. It changes the score by a factor of 5.7.

Build: flag_spikes

`flag_spikes` computes the score for each cell, with a trailing baseline of one day, and flags the samples above a threshold `z`. The data have one injected spike: 7 samples of `BRU-002` from 14:00 to 14:30 UTC on 25 October. That gives a way to score the detector, which is rare with real data:

```
import pandas as pd

from pdbook.io import load_kpi
from pdbook.spikes import flag_spikes

kpi = load_kpi().drop_duplicates(["ts", "cell_id"]).reset_index(drop=True)
truth = (kpi["cell_id"] == "BRU-002") & kpi["ts"].between("2026-10-25 14:00", "2026-10-25
14:30")
print("injected samples:", int(truth.sum()))
for z in (2, 3, 4, 5, 8):
    flags = flag_spikes(kpi, z=z)
    print(f"z={z}: found {int((flags & truth).sum())}, false {int((flags & ~truth).sum())}")

late = flag_spikes(kpi, z=8) & truth
print("found at z=8:", kpi.loc[late, "ts"].dt.strftime("%H:%M").tolist())
```

```
injected samples: 7
z=2: found 7, false 174
z=3: found 7, false 10
z=4: found 7, false 0
z=5: found 7, false 0
z=8: found 5, false 0
found at z=8: ['14:00', '14:05', '14:10', '14:15', '14:20']
```

Read the table from the top. At `z=2`, all 7 samples are found, and 174 other samples are flagged: the detector is noisy. At `z=3`, 10 other samples are flagged. At `z=4` and `z=5`, the result is exact:

7 found and none false. At `z=8`, only 5 of the 7 are found, and the last line shows which: the first five, from 14:00 to 14:20. The first spike samples enter the baseline of the later ones. They raise its mean and its standard deviation, and the last two samples no longer pass.

This is the cost of any threshold. A low threshold floods a person with alerts. A high one misses a long event, and it misses the end of an event first. Choose the threshold with a score like this one, on data where you know the answer. Do not choose it by looking at the alerts of one week.

Two properties of the function are worth a test:

```
import pandas as pd

from pdbook.io import load_kpi
from pdbook.spikes import flag_spikes

kpi = load_kpi().drop_duplicates(["ts", "cell_id"]).reset_index(drop=True)
shuffled = kpi.sample(frac=1, random_state=0)
a = flag_spikes(kpi, z=4)
b = flag_spikes(shuffled, z=4).reindex(kpi.index)
print(bool((a == b).all()))

first_day = kpi["ts"] < pd.Timestamp("2026-10-24 04:10", tz="UTC")
print(int(flag_spikes(kpi, z=0)[first_day].sum()))
```

```
True
0
```

The first line is `True`: the result does not depend on the order of the rows, because the function sorts by time inside each cell. The second line is 0. With the threshold `z=0`, every sample above the mean is flagged. Still, no sample in the first 4 hours is flagged, because they have fewer than 50 samples of history. A detector has no opinion before it has data.

How it works

A rolling window by rows is a fixed number of neighbours. A rolling window by time is different. For each timestamp, pandas finds where the window begins in the sorted index, with a binary search. Then it computes the statistic over the rows from that position to the current one.

The window has a fixed length in time and a count that changes. The argument `closed` says which ends belong to the window: `"right"` is the default and includes the current row, and `"left"` leaves it out.

You can compute a few windows by hand with a boolean mask on the time, and compare:

```
import pandas as pd

from pdbook.io import load_kpi

kpi = load_kpi().drop_duplicates(["ts", "cell_id"])
cell = kpi[kpi["cell_id"] == "BRU-004"].set_index("ts")["prb_util_pct"].sort_index()

rolled = cell.rolling("1h").mean()
rolled_left = cell.rolling("1h", closed="left").mean()
one_hour = pd.Timedelta("1h")

ok = True
for t in cell.index[[20, 300, 400, 600]]:
    right = cell[(cell.index > t - one_hour) & (cell.index <= t)].mean()
    left = cell[(cell.index >= t - one_hour) & (cell.index < t)].mean()
    ok &= abs(right - rolled.loc[t]) < 1e-9 and abs(left - rolled_left.loc[t]) < 1e-9
print(ok)
print(int(cell.rolling("1h").count().max()), int(cell.rolling("1h",
closed="left").count().max()))
print(cell.index.is_monotonic_increasing)
```

```
True
12 12
True
```

The hand computation agrees with `rolling` for four timestamps, for both settings of `closed`. A one-hour window holds at most 12 samples of 5 minutes with the default. The setting `"left"` gives the same maximum, because the window then begins at the sample one hour before and leaves the current one out. The index is sorted, which `rolling` with a time window requires.

- **Invariant.** A time window covers a fixed span of time, and its count of samples varies.
- **Failure.** A window by rows spans a longer time when rows are missing (the 2 hours 55 minutes of this chapter).
- **Cost.** A binary search for each row, then an incremental update of the statistic, so one pass over the data for the mean.

Reference

```
import pandas as pd

from pdbook.sigs import show_signature
from pdbook.spikes import flag_spikes

show_signature(pd.Series.rolling, ["window", "min_periods", "center", "closed"])
show_signature(pd.Series.expanding, ["min_periods"])
show_signature(pd.Series.ewm, ["span", "alpha", "adjust"])
show_signature(pd.Series.shift, ["periods", "freq"])
show_signature(pd.Series.diff, ["periods"])
show_signature(pd.Series.pct_change, ["periods"])
show_signature(flag_spikes)
```

```
NDFrame.rolling(
    window: 'int | dt.timedelta | str | BaseOffset | BaseIndexer'
    min_periods: 'int | None' = None
    center: 'bool' = False
    closed: 'IntervalClosedType | None' = None
)  # 5 more parameters
NDFrame.expanding(
    min_periods: 'int' = 1
)  # 2 more parameters
NDFrame.ewm(
    span: 'float | None' = None
    alpha: 'float | None' = None
    adjust: 'bool' = True
)  # 7 more parameters
NDFrame.shift(
    periods: 'int | Sequence[int]' = 1
    freq=None
)  # 4 more parameters
Series.diff(
    periods: 'int' = 1
)  # 1 more parameters
NDFrame.pct_change(
    periods: 'int' = 1
)  # 4 more parameters
flag_spikes(
    kpi: 'pd.DataFrame'
    column: 'str' = 'drop_rate_pct'
    window: 'str' = '1D'
    z: 'float' = 4.0
    min_periods: 'int' = 50
)
```

Parameter	Default	What it does here	What goes wrong
`rolling(window)`	required	rows (`12`) or time (`"1h"`)	a window of 12 rows reached back 2 hours 55 minutes across the outage
`rolling(min_periods=...)`	`None`	the samples a window needs before it gives a value	with a time window the default is 1 sample: 0 `NaN` values, against 16 with `min_periods=9`
`rolling(center=...)`	`False`	put the value in the middle of the window	—
`rolling(closed=...)`	`None`	which ends of the window are inside; `"left"` leaves the current row out	`closed="right"` scored the spike at 16.5, `"left"` at 94.2
`expanding(min_periods=...)`	`1`	all rows from the start to now	—
`ewm(span=..., alpha=...)`	`None` , `None`	a mean that weights recent rows more	—
`shift(periods, freq=...)`	`1` , `None`	move values by rows, or by a time with `freq`	across a gap, `shift(1)` is not 5 minutes
`diff(periods=...)`	`1`	value minus the value `periods` rows before	across the outage, a difference of two hours looks like one of five minutes
`pct_change(periods=...)`	`1`	relative change	—
`flag_spikes(kpi, column, window, z, min_periods)`	`'drop_rate_pct'` , `'1D'` , `4.0` , `50`	`True` where the value is more than `z` deviations above the trailing mean	`z=2` flags 174 other samples; `z=8` finds 5 of 7

`flag_spikes` returns a boolean Series with the index of `kpi` . The input must be unique on `ts` and `cell_id` .

What you have now

- You know that `shift` , `diff` and `rolling(n)` work on rows, and that rows are not time when there are gaps.
- You use windows given as time, and you set `min_periods` .
- You exclude the current sample from its own baseline.
- You score a detector against a known event, and you see the cost of the threshold.

The next chapter ranks, accumulates and splits a series of events into sessions.

Next: Chapter 14, Ranks, running totals and sessions.

14. Ranks, running totals and sessions

> **Build** — `sessionize(calls, gap)`: a function that groups the calls of each subscriber into sessions, where a quiet period longer than `gap` starts a new session, with tests that the sessions cover every call and do not depend on the row order.
>
> **Theory kept** — A rank, a running total and a "time since the previous event" are window functions: each row gets a value from its neighbours in a group. The idiom for sessions is three steps: sort, flag the rows that start a session, take the running sum of the flags.
>
> **Source** — New. Library: `pdbook/sessions.py`.
>
> **Run this first** — `uv run python -m examples.ch14_sessions`, which is `examples/ch14_sessions.py`. Exercises: `exercises/14.md`. Solutions: `exercises/solutions/14.md`.

Rank

`rank` gives each value its position in the sorted order. The argument `method` decides what happens with ties:

```
import pandas as pd

x = pd.Series([10, 20, 20, 30])
for method in ["average", "min", "max", "dense", "first"]:
    print(f"{method:8s}", x.rank(method=method).tolist())
```

```
average  [1.0, 2.5, 2.5, 4.0]
min      [1.0, 2.0, 2.0, 4.0]
max      [1.0, 3.0, 3.0, 4.0]
dense    [1.0, 2.0, 2.0, 3.0]
first    [1.0, 2.0, 3.0, 4.0]
```

The two values of 20 share the ranks 2 and 3. `average` gives both 2.5. `min` gives both 2, and `max` gives both 3. `dense` gives both 2 and the next value 3, with no gap.

`first` breaks the tie by the order of the rows. Choose by the question: "how many are strictly higher" is `min`, and "a number from 1 to n with no ties" is `first`.

Top n in each group

The tool is `rank` inside a `groupby`, or `sort_values` and `head`:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
answered = calls[calls["status"] == "answered"].copy()

answered["rank"] = answered.groupby("caller")["duration_s"].rank(method="first",
ascending=False)
top3 = answered[answered["rank"] <= 3]
print(len(top3), top3["caller"].nunique())

same = (answered.sort_values(["caller", "duration_s"], ascending=[True, False])
        .groupby("caller").head(3))
print(len(same), set(same.index) == set(top3.index))
print(top3.groupby("caller").size().value_counts().to_dict())
```

```
1800 600
1800 True
{3: 600}
```

Both ways give 1,800 rows: 3 calls for each of the 600 subscribers. The sets of rows are the same. The `rank` form keeps the rank as a column, and the sort form is shorter.

In both forms, the order of the rows breaks a tie at the third place: `method="first"` in the first form, and a stable sort in the second. Pass a full sort key when ties matter.

The last line shows that every subscriber has at least 3 answered calls: all 600 groups have size 3. A group with fewer than 3 rows shows a smaller size.

Running totals

`cumsum`, `cumcount`, `cummax` and `cummin` give a value from the start of the group to the current row. They need an order, so sort first:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls().sort_values(["caller", "start"])
calls["talk_min"] = calls["duration_s"].fillna(0) / 60
calls["n_so_far"] = calls.groupby("caller").cumcount() + 1
calls["talk_so_far"] = calls.groupby("caller")["talk_min"].cumsum()

last = calls.groupby("caller").tail(1)
print(int(last["n_so_far"].sum()), round(float(last["talk_so_far"].sum()), 1))
print(len(calls), round(float(calls["talk_min"].sum()), 1))
```

```
20000 34372.5
20000 34372.5
```

The last row of each group carries the total of the group, so the sum of the last rows is the sum of the table. That is the check, and it is the same as the totals check of Chapter 8. Without the `sort_values`, `cumsum` follows the row order of the file, and the "so far" is not "so far in time".

Time since the previous event

On a timestamp column, `groupby(...).diff()` gives the time since the previous row of the same group. It is `NaT` for the first row of each group:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls().sort_values(["caller", "start"])
gap = calls.groupby("caller")["start"].diff()
print(int(gap.isna().sum()), calls["caller"].nunique())
print(gap.dropna().median().round("h"))
print(int((gap.dropna() < pd.Timedelta("1min")).sum()))
```

```
600 600
0 days 10:00:00
49
```

There is one `NaT` for each of the 600 subscribers: the first call. The median gap between two calls of one subscriber is 10 hours, and 49 gaps are under a minute.

Sessions

A session is a run of events with no gap larger than a threshold. The idiom has three steps:

1. Sort by group and time.
2. Mark the rows that start a session: the first row of a group, or a gap larger than the threshold.
3. Take the running sum of the marks inside the group. The sum is the session number.

```
import pandas as pd

t = pd.to_datetime(["10:00", "10:05", "10:50", "10:52", "11:30"], format="%H:%M")
df = pd.DataFrame({"caller": "A", "start": t})
gap = df.groupby("caller")["start"].diff()
new = gap.isna() | (gap > pd.Timedelta("30min"))
df["session"] = new.groupby(df["caller"]).cumsum()
print(df.assign(gap=gap.dt.total_seconds() / 60, new=new)[["gap", "new",
"session"]].to_string())
```

```
    gap    new  session
0   NaN   True        1
1   5.0  False        1
2  45.0   True        2
3   2.0  False        2
4  38.0   True        3
```

The gaps are 5, 45, 2 and 38 minutes. Two of them are larger than 30, so there are three sessions: the first two calls, the next two, and the last one. The marks are 1 at the start of each session, and `cumsum` turns them into 1, 1, 2, 2, 3.

Build: sessionize

`sessionize` applies the three steps to the call records. It sorts inside the function, so the result does not depend on the order of the rows that you pass. It uses `>`, so a gap of exactly `gap` stays in the session. Run it for several gaps:

```
import pandas as pd

from pdbook.io import load_calls
from pdbook.sessions import session_table, sessionize

calls = load_calls()
print(f"{'gap':>6s} {'sessions':>9s} {'longest':>8s} {'covers':>7s}")
for gap in ("1min", "5min", "30min", "2h", "1D"):
    table = session_table(sessionize(calls, gap))
    print(f"{gap:>6s} {len(table):>9,d} {int(table['calls'].max()):>8d}
{int(table['calls'].sum()):>7,d}")
```

```
   gap  sessions  longest  covers
  1min    19,951        3  20,000
  5min    19,691        4  20,000
 30min    18,420        8  20,000
    2h    15,450       30  20,000
    1D     5,866      628  20,000
```

The last column is the check: the sessions cover all 20,000 calls for every gap. The number of sessions falls as the gap grows, from 19,951 for one minute to 5,866 for one day, and it never rises. In this data the calls spread over a month, and 17,196 of the 18,420 sessions of 30 minutes have a single call. The generator of the data does not model sessions, so the numbers show no real structure. The code is the same for call records that have bursts.

Two more tests, which are properties of the idea of a session:

```
import pandas as pd

from pdbook.io import load_calls
from pdbook.sessions import session_table, sessionize

calls = load_calls()
a = sessionize(calls)
b = sessionize(calls.sample(frac=1, random_state=3))
print(a.equals(b))

counts = [len(session_table(sessionize(calls, g))) for g in ("1min", "5min", "30min", "2h",
"1D")]
print(counts == sorted(counts, reverse=True))

table = session_table(a)
print(table.sort_values("calls", ascending=False).head(2)[["session_id", "calls",
"first_start", "last_start"]]
      .to_string(index=False))
```

```
True
True
session_id  calls               first_start                last_start
 S0437-093      8 2026-09-08 15:38:24+00:00 2026-09-08 16:51:00+00:00
 S0437-128      8 2026-09-11 16:22:08+00:00 2026-09-11 18:12:12+00:00
```

The first line is `True`: a shuffled input gives the same table. The second line is `True`: more gap gives fewer or equal sessions. The last block shows the two longest sessions. Both belong to subscriber S0437 and have 8 calls each, the first between 15:38 and 16:51 UTC on 8 September.

How it works

A session number is a running sum of boundary marks. First sort by caller and time. A row then starts a new session when its gap to the previous row is large. The first row of a caller also starts a session.

Mark those rows with 1 and the others with 0, and the running sum inside each caller is the session number. This is a vectorized form of a small state machine: walk the rows in order, keep the caller and the time of the previous row, and add 1 to the counter at each boundary.

You can write the state machine with a plain loop and compare it with `sessionize`:

```
import pandas as pd

from pdbook.io import load_calls
from pdbook.sessions import sessionize

calls = load_calls()
ordered = calls.sort_values(["caller", "start", "call_id"],
kind="stable").reset_index(drop=True)
limit = pd.Timedelta("30min")

session, previous_caller, previous_time, counter = [], None, None, 0
for caller, start in zip(ordered["caller"], ordered["start"]):
    if caller != previous_caller:
        counter = 1
    elif start - previous_time > limit:
        counter += 1
    session.append(counter)
    previous_caller, previous_time = caller, start

by_pandas = sessionize(calls, "30min")["session"].tolist()
print(len(session), session == by_pandas)
print(max(session), int(ordered.groupby("caller").cumcount().max()) + 1)
```

```
20000 True
347 628
```

The loop and `sessionize` give the same session number for each of the 20,000 rows. The loop is easy to read and slow, and the vectorized form is the same logic as one running sum. The last line shows the longest session number of any caller (the most sessions of one subscriber), and the largest number of calls of one caller, 628, which bounds it.

- **Invariant.** Session numbers never decrease inside a caller, and they go up by 1 at a boundary.
- **Failure.** Rows in the wrong order: the gap to the previous row is then a gap to a different call. Always sort inside the function.
- **Cost.** One sort, one grouped `diff`, one grouped `cumsum`. The sort is the largest part.

Reference

```
import pandas as pd

from pdbook.sessions import session_table, sessionize
from pdbook.sigs import show_signature

show_signature(pd.Series.rank, ["method", "ascending", "pct"])
show_signature(pd.core.groupby.DataFrameGroupBy.cumcount, ["ascending"])
show_signature(pd.core.groupby.DataFrameGroupBy.head, ["n"])
show_signature(pd.DataFrame.nlargest, ["n", "columns", "keep"])
show_signature(sessionize)
show_signature(session_table)
```

```
NDFrame.rank(
    method: "Literal['average', 'min', 'max', 'first', 'dense']" = 'average'
    ascending: 'bool' = True
    pct: 'bool' = False
)  # 4 more parameters
GroupBy.cumcount(
    ascending: 'bool' = True
)  # 1 more parameters
GroupBy.head(
    n: 'int' = 5
)  # 1 more parameters
DataFrame.nlargest(
    n: 'int'
    columns: 'IndexLabel'
    keep: 'NsmallestNlargestKeep' = 'first'
)  # 1 more parameters
sessionize(
    calls: 'pd.DataFrame'
    gap: 'str' = '30min'
)
session_table(
    sessions: 'pd.DataFrame'
)
```

Parameter	Default	What it does here	What goes wrong
`rank(method=...)`	`'average'`	the rule for ties	for the values 10, 20, 20, 30 the two 20s get 2.5 (`average`), 2 and 2 (`min`), 3 and 3 (`max`), 2 and 2 (`dense`), 2 and 3 (`first`)
`rank(ascending=...)`	`True`	`False` gives rank 1 to the largest	—
`rank(pct=...)`	`False`	ranks as a share of 1	—
`cumcount(ascending=...)`	`True`	the position of each row inside its group	needs a sorted table to mean "so far in time"
`head(n)`	`5`	the first n rows of each group	—
`nlargest(n, columns, keep=...)`	required, `'first'`	the n rows with the largest values	—
`sessionize(calls, gap=...)`	required, `'30min'`	adds `session` and `session_id`	the same code without a sort gives 18,151 sessions against 18,420
`session_table(sessions)`	required	one row for each session	—

`sessionize` returns the table sorted by caller and start, with two more columns. A gap of exactly `gap` stays in the session.

What you have now

- You rank with a chosen tie rule, and you take the top n of each group in two ways.

- You make running totals after a sort, and you check them against the table.
- You get the time since the previous event with `groupby(...).diff()`.
- You build sessions with sort, mark, running sum, and you test them.

The next chapter looks at the traps that make a pandas result wrong without an error, in one place.

Next: Chapter 15, Copy-on-Write and traps.

15. Copy-on-Write and traps

> **Build** — A catalog of ten traps, each as a pair of functions (the code that people write and the code that they meant), with a test that all ten still give a wrong answer without an error in the installed pandas.
>
> **Theory kept** — In pandas 3.0, every object that you derive from another object behaves as a copy. A change to one never changes another. Chained assignment does nothing and warns. A function that assigns a column to its argument still changes the caller's table, because there is only one object.
>
> **Source** — New. Library: `pdbook/traps.py`.
>
> **Run this first** — `uv run python -m examples.ch15_traps`, which is `examples/ch15_traps.py`. Exercises: `exercises/15.md`. Solutions: `exercises/solutions/15.md`.

The rule in pandas 3.0

Before 3.0, a selection from a DataFrame was sometimes a view of the original data and sometimes a copy. The rule depended on internals. A write could change the original, or not, and `SettingWithCopyWarning` guessed which. pandas 3.0 replaces this with one rule, copy-on-write:

> Any DataFrame or Series that you get from another one behaves as a separate copy. A write to it never changes the other.

pandas does not copy the data at once. It shares the memory and copies at the first write. You do not see the mechanism. You see the rule:

```
import pandas as pd

a = pd.DataFrame({"x": [10, 20, 30], "y": [1, 2, 3]})

col = a["x"]
col.iloc[0] = -1
sub = a[["x"]]
sub.iloc[0, 0] = -2
print("after writes to derived objects:", a["x"].tolist())

same = a
same["x"] = 0
print("after a write to a second name :", a["x"].tolist())
```

```
after writes to derived objects: [10, 20, 30]
after a write to a second name : [0, 0, 0]
```

The writes to `col` and `sub` did not change `a`. The write through `same` did, because `same = a` does not make a new object. It gives a second name to the same object. Assignment of a name never copies. Use `a.copy()` when you want a separate object.

Chained assignment

`df[mask]["x"] = value` has two steps. The first, `df[mask]`, makes a derived object. The second writes to it. Under the rule, the write goes to a copy that is thrown away:

```
import warnings

import pandas as pd

df = pd.DataFrame({"x": [1, 2, 3]})
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    df[df["x"] > 1]["x"] = 0
    df["x"][0] = 99
print(df["x"].tolist(), sorted({w.category.__name__ for w in caught}))

df.loc[df["x"] > 1, "x"] = 0
print(df["x"].tolist())
```

```
[1, 2, 3] ['ChainedAssignmentError']
[1, 0, 0]
```

Both chained forms warn with `ChainedAssignmentError` and change nothing. The `.loc` form with a mask and a column in one call changes the table. The same rule makes `inplace=True` on a column useless:

```
import warnings

import pandas as pd

df = pd.DataFrame({"a": [1.0, None, 3.0]})
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    df["a"].fillna(0, inplace=True)
print(df["a"].tolist(), caught[0].category.__name__)

df["a"] = df["a"].fillna(0)
print(df["a"].tolist())
```

```
[1.0, nan, 3.0] ChainedAssignmentError
[1.0, 0.0, 3.0]
```

`df["a"]` is a derived object. `fillna(inplace=True)` fills that object and not the table. The form `df["a"] = df["a"].fillna(0)` is clear and changes the table. Prefer it, and write `inplace=True` nowhere.

A function that changes its argument

Copy-on-write protects against a hidden link between two objects. It does not protect against an assignment to the object itself. A function that assigns a column to its argument changes the table of the caller:

```
import pandas as pd


def add_minutes_bad(df):
    df["minutes"] = df["duration_s"] / 60
    return df


def add_minutes_good(df):
    return df.assign(minutes=df["duration_s"] / 60)


calls = pd.DataFrame({"duration_s": [60, 90]})
add_minutes_good(calls)
print(calls.columns.tolist())
add_minutes_bad(calls)
print(calls.columns.tolist())
```

```
['duration_s']
['duration_s', 'minutes']
```

The good function leaves the table of the caller as it was. The bad one adds a column to it. The caller has no sign of this in the code. A second call to a function that expects the column to be absent can fail or give a different result. Write functions that return a new table. Use `assign`, or begin with `df = df.copy()`.

Arrays from pandas

`to_numpy()` gives the NumPy array of a column. Under copy-on-write that array is read-only, because a write to it changes the table without going through pandas:

```
import pandas as pd

s = pd.Series([1, 2, 3])
arr = s.to_numpy()
print(arr.flags.writeable)
try:
    arr[0] = 5
except ValueError as error:
    print("ValueError:", str(error))

own = s.to_numpy(copy=True)
own[0] = 5
print(own.tolist(), s.tolist())
```

```
False
ValueError: assignment destination is read-only
[5, 2, 3] [1, 2, 3]
```

Ask for `copy=True` when you need an array that you can write to.

The other traps

Copy-on-write fixes one class of surprise. The others are in the data, and the earlier chapters showed each one. The module `pdbook/traps.py` holds ten of them as pairs of functions, on small data, so that you can run them and see them:

```
from pdbook.traps import TRAPS, live

for i, trap in enumerate(TRAPS, 1):
    print(f"{i:>2d} ch{trap.chapter:<3d} {str(trap.naive()):<14.14s} {str(trap.correct()):
<14.14s} {trap.name}")
print(all(live(t) for t in TRAPS))
```

```
 1 ch1   [4, 4, 4]      [2, 4, 6]      a Series adds by label, not by position
 2 ch2   [477123456]    ['0477123456'] read_csv turns a phone number into an integer
 3 ch5   4              6              groupby drops rows with a missing key
 4 ch3   2              3              a condition and its opposite miss the nulls
 5 ch3   [1, 2, 3]      [1, 0, 0]      chained assignment changes nothing
 6 ch3   [10, 20, 30]   [10, 20]       a label slice includes its end
 7 ch9   150            90             a duplicate in a lookup table multiplies rows
 8 ch4   [-56, -56]     [200, 200]     a small integer type wraps around
 9 ch15  False          True           0.1 + 0.2 is not 0.3
10 ch5   3.0            2.25           mean skips nulls
True
```

Each row is a result that looks right and is not. The first value is what the usual code gives, and the second is what a person meant. The list is also a test: `live` is `True` when the two differ. If a later version of pandas fixes a trap, the example module stops, and the book needs a change. The traps in the list are:

1. Arithmetic matches by label, so two Series in different orders give a different sum from two lists (Chapter 1).
2. `read_csv` reads a phone number as an integer and loses the leading zero (Chapter 2).
3. `groupby` drops rows with a missing key (Chapter 5).
4. A condition and its opposite both skip the nulls (Chapter 3).
5. Chained assignment changes nothing (Chapter 3).
6. A label slice includes its end (Chapter 3).
7. A duplicate key in a lookup table multiplies rows (Chapter 9).
8. A small integer type wraps around (Chapter 4).
9. `0.1 + 0.2 == 0.3` is `False`, in pandas as in any language with binary floating point numbers.
10. `mean` skips nulls, so "the mean" is the mean of the known values (Chapter 5).

The ninth is new in this chapter. A float comparison with `==` is a trap in every column of `float64`. Use `np.isclose` or `abs(a - b) < tolerance`, and use integers (cents, not euros) for money.

Build: how to use the catalog

A catalog of known traps is also a review checklist. Before a pipeline goes to production, go through the ten lines and for each ask: does my code have this shape? Examples:

a `merge` without `validate`, a `groupby` without a total check, a `read_csv` without `dtype`. Another example is a `.loc` slice on an index that you did not sort. Each answer is a line of code that you add or a test that you write. The chapters named in the list give the form of the test.

You can also make a trap that is yours. When a wrong result reaches a report, write the small pair of functions before you fix the pipeline. The pair is a test that the fix works, and it is a line in the catalog of the team.

How it works

Copy-on-write does not copy at once. A derived object, such as `df[["x"]]`, points at the same memory as its parent. Only when you write to one of the two does pandas make a private copy for the object that you write to. Until that moment the two share one array, and this is why a selection is cheap. The rule that a write never reaches another object is kept by the copy that happens at the write.

NumPy can tell whether two arrays share memory:

```
import warnings

import numpy as np
import pandas as pd

df = pd.DataFrame({"x": np.arange(5), "y": np.arange(5) * 10})
sub = df[["x"]]
print(np.shares_memory(df["x"].to_numpy(), sub["x"].to_numpy()))

sub.iloc[0, 0] = -1
print(np.shares_memory(df["x"].to_numpy(), sub["x"].to_numpy()))
print(df["x"].tolist(), sub["x"].tolist())

view = df["y"].to_numpy()
print(view.flags.writeable)
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    value = pd.options.mode.copy_on_write
print(value, caught[0].category.__name__)
```

```
True
False
[0, 1, 2, 3, 4] [-1, 1, 2, 3, 4]
False
True Pandas4Warning
```

Before the write, the two objects share memory. After `sub.iloc[0, 0] = -1`, they do not, `sub` has its own array with the change, and `df` is as it was. The array from `to_numpy()` is read-only, so a write through it cannot change the table in a way that skips the rule. The last line reads the old option `mode.copy_on_write`. It is `True`, and reading it warns with `Pandas4Warning`: in pandas 3.0 copy-on-write is always on and the option has no effect.

- **Invariant.** A write to one object never changes another object. A read of one object never sees a write to the other.
- **Failure.** A name is not an object: `b = a` makes a second name. A function that assigns to its argument changes the caller's object.

- **Cost.** A selection costs no copy. The first write costs one copy of the array that you write to, not of the whole table.

Reference

```
import pandas as pd

from pdbook.sigs import show_signature
from pdbook.traps import Trap, live

show_signature(pd.DataFrame.copy, ["deep"])
show_signature(pd.Series.to_numpy, ["dtype", "copy", "na_value"])
show_signature(pd.DataFrame.assign)
show_signature(pd.DataFrame.fillna, ["value", "inplace"])
show_signature(Trap)
show_signature(live)
```

```
NDFrame.copy(
    deep: 'bool' = True
)  # 1 more parameters
IndexOpsMixin.to_numpy(
    dtype: 'npt.DTypeLike | None' = None
    copy: 'bool' = False
    na_value: 'object' = <no_default>
)  # 2 more parameters
DataFrame.assign(
    self
    **kwargs
)
NDFrame.fillna(
    value: 'Hashable | Mapping | Series | DataFrame'
    inplace: 'bool' = False
)  # 3 more parameters
Trap(
    name: 'str'
    chapter: 'int'
    naive: 'Callable[[], Any]'
    correct: 'Callable[[], Any]'
)
live(
    trap: 'Trap'
)
```

Parameter	Default	What it does here	What goes wrong
`copy(deep=...)`	`True`	a separate object	`b = a` is not a copy: a write through `b` changes `a`
`to_numpy(copy=...)`	`False`	the NumPy array of a column	the array is read-only: a write raises `ValueError`; ask for `copy=True` to write
`to_numpy(dtype=..., na_value=...)`	`None`, not set	the type of the array, and what a missing value becomes	—
`assign(**kwargs)`	—	a new table with the columns added	—
`fillna(value, inplace=...)`	required, `False`	fill missing values	`df["a"].fillna(0, inplace=True)` warns with `ChainedAssignmentError` and changes nothing
`Trap(name, chapter, naive, correct)`	required	a trap as two functions of no arguments	—
`live(trap)`	required	`True` when the two functions give different results	all 10 traps are live in this version

`copy` returns a new object that has its own data.

What you have now

- You know the copy-on-write rule and what it does not cover.
- You write `df.loc[mask, column] = value`, and `df[column] = df[column].fillna(...)`, and never `inplace=True`.
- You write functions that return a new table.
- You have ten named traps, each with a test, and a method to add one.

The next chapter puts the steps of a pipeline into one readable chain.

Next: Chapter 16, Method chains and pipelines.

16. Method chains and pipelines

> **Build** — `monthly_report(calls, subs, cells, plans)` : the report of calls, answer rate, billed minutes and revenue for each plan and city, written as one chain with two checks inside it, and three tests that prove it: the total agrees with an independent computation, the result does not depend on the row order, and a planted fault stops it.
>
> **Theory kept** — A method chain is a pipeline of steps that each take a table and return a table, with no mutation. `assign` makes columns, `pipe` runs your own function, and a check is a step that returns its input or raises an error.
>
> **Source** — New. Library: `pdbook/report.py` .
>
> **Run this first** — `uv run python -m examples.ch16_chain` , which is `examples/ch16_chain.py` . Exercises: `exercises/16.md` . Solutions: `exercises/solutions/16.md` .

Why a chain

Compare two ways to write the same three steps:

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()

# step by step, with a name for each state
df = calls[calls["status"] == "answered"]
df = df.assign(minutes=df["duration_s"] / 60)
by_codec = df.groupby("codec").agg(calls=("call_id", "size"), minutes=("minutes", "sum"))
print(by_codec.round(1).to_dict("index")["EVS"])

# the same, as one chain
by_codec2 = (
    calls
    .query("status == 'answered'")
    .assign(minutes=lambda d: d["duration_s"] / 60)
    .groupby("codec")
    .agg(calls=("call_id", "size"), minutes=("minutes", "sum"))
)
pd.testing.assert_frame_equal(by_codec, by_codec2)
print("equal")
```

```
{'calls': 4791, 'minutes': 10564.6}
equal
```

The results are equal. The chain has three properties that the first form does not have:

- **Nothing is assigned twice.** The variable `df` in the first form means two different things. A cell that you run twice in a notebook gives a different result the second time. The chain has one name at the start and one at the

end.

- **Each step reads as a verb.** A person who reads from the top knows the order of the work without searching for the assignments.
- **There is no state to leave behind.** A step cannot change the input table, and the input is as it was after the chain.

The pieces

The methods that give you a chain are the ones that return a new object:

Step	Method
keep rows	`query`, `loc[mask]`, `drop_duplicates`, `head`
make columns	`assign`, `rename`, `astype`, `where`
change shape	`groupby(...).agg`, `pivot_table`, `melt`, `merge`
order	`sort_values`, `reset_index`, `set_axis`
your own step	`pipe`

`assign` takes a function when the column depends on the table as it is at that point of the chain. The lambda receives that table. The arguments of one `assign` are evaluated from left to right, so a later one can use an earlier one:

```
import pandas as pd

from pdbook.io import load_calls

result = (
    load_calls()
    .assign(
        minutes=lambda d: d["duration_s"] / 60,
        long=lambda d: d["minutes"] > 10,
    )
    .query("long")
    .shape
)
print(result)
```

```
(351, 11)
```

pipe: your own steps

`pipe` calls a function with the table as the first argument, and passes on what it returns. This puts a function of yours in the chain, and lets you put a check or a print in it:

```
import pandas as pd

from pdbook.io import load_calls


def tap(df, label):
    print(f"{label:12s} {df.shape}")
    return df


def require(df, test, message):
    if not test(df):
        raise ValueError(message)
    return df


out = (
    load_calls()
    .pipe(tap, "loaded")
    .query("status == 'answered'")
    .pipe(tap, "answered")
    .pipe(require, lambda d: d["duration_s"].notna().all(), "an answered call has no
duration")
    .groupby("caller")
    .size()
    .pipe(tap, "per caller")
)
print(int(out.sum()))
```

```
loaded       (20000, 9)
answered     (15526, 9)
per caller   (600,)
15526
```

`tap` prints the shape and returns the table, so it does not change the result. `require` is a check inside the chain: when the test is false the chain stops with the message, and when it is true the table goes on. The shapes show where rows were lost:

20,000 loaded, 15,526 answered, and 600 callers. The check says that an answered call always has a duration. That is a fact about the data, and you want to be told when it is false.

The same chain with a planted fault stops at the check:

```
import numpy as np
import pandas as pd

from pdbook.io import load_calls
from pdbook.report import require

calls = load_calls()
broken = calls.copy()
broken.loc[broken["status"] == "answered", "duration_s"] = np.nan

try:
    broken.query("status == 'answered'").pipe(
        require, lambda d: d["duration_s"].notna().all(), "an answered call has no duration")
except ValueError as error:
    print("ValueError:", error)
```

```
ValueError: an answered call has no duration
```

When a chain is too long

A chain of 15 steps is hard to debug: you cannot print the middle. Three habits keep a chain useful:

1. **Cut it at a meaning.** Give a name to the table at the point where it means something, for example `enriched`. Start a second chain from it.
2. **Put the cut in a function.** A function with a name and a docstring is a unit that you can test. A chain inside a function is easy to read.
3. **Debug by cutting from the end.** Put `.pipe(tap, "here")` after a step, or delete steps from the end until the output is what you expect.

Build: monthly_report

The report is one function with one chain. It takes the four tables and enriches the calls (Chapter 9). It adds the answered flag and the billed minutes (Chapter 7). It groups by plan and home city (Chapter 8). Last, it checks that the groups add up to the calls:

```
import pandas as pd

from pdbook.io import load_calls, load_cells, load_plans, load_subscribers
from pdbook.report import monthly_report

calls, subs, cells, plans = load_calls(), load_subscribers(), load_cells(), load_plans()
report = monthly_report(calls, subs, cells, plans)
print(report.shape)
print(report.head(5).to_string(index=False))
print(report.groupby("plan")[["calls", "revenue_eur"]].sum().round(2).to_dict("index"))
print(plans.set_index("plan")["rate_per_min_eur"].to_dict())
```

```
(20, 7)
 plan home_city  calls  answered  billed_min  revenue_eur  answer_rate
basic   Antwerp   1083       854        2463        246.3        0.789
basic  Brussels   3073      2353        6156        615.6        0.766
basic     Ghent   1306      1007        2754        275.4        0.771
basic    Leuven    276       212         567         56.7        0.768
basic     Liège    926       727        2057        205.7        0.785
{'basic': {'calls': 6664, 'revenue_eur': 1399.7}, 'business': {'calls': 2290, 'revenue_eur':
193.72}, 'plus': {'calls': 4475, 'revenue_eur': 563.22}, 'prepaid': {'calls': 6571,
'revenue_eur': 2088.45}}
{'prepaid': 0.15, 'basic': 0.1, 'plus': 0.06, 'business': 0.04}
```

The report has 20 rows: 4 plans and 5 cities. The `prepaid` plan makes the most revenue, 2,088.45 euros, although it is not the largest in calls. Its price per minute is the highest, 0.15 euros. The `business` plan has 2,290 calls and makes 193.72 euros.

Three tests prove the function. They are independent of each other:

```
import numpy as np
import pandas as pd

from pdbook.io import load_calls, load_cells, load_plans, load_subscribers
from pdbook.rating import rate_vector
from pdbook.report import monthly_report

calls, subs, cells, plans = load_calls(), load_subscribers(), load_cells(), load_plans()
report = monthly_report(calls, subs, cells, plans)

independent = float(rate_vector(calls, subs, plans).sum())
print(round(float(report["revenue_eur"].sum()), 2), round(independent, 2))
print(bool(np.isclose(report["revenue_eur"].sum(), independent)))

shuffled = monthly_report(calls.sample(frac=1, random_state=1), subs, cells, plans)
pd.testing.assert_frame_equal(report, shuffled)
print("order independent")

duplicated = pd.concat([calls, calls.head(1)], ignore_index=True)
try:
    monthly_report(duplicated, subs, cells, plans)
except ValueError as error:
    print("ValueError:", error)
```

```
4245.09 4245.09
True
order independent
ValueError: call_id is not unique
```

The first test compares the revenue of the chain with the revenue of `rate_vector` from Chapter 7. They are two different computations of the same number, and they agree. The second test shuffles the rows and gets the same report. The third plants a repeated call, and the check inside the chain stops it.

How it works

Each method in a chain is a function from a table to a new table. The step `pipe(f, *args)` is only `f(df, *args)` written as a method, so that your function can stand in the chain with the others. A chain is therefore a composition of functions, read from the top. Nothing in it assigns to a name, and the input table is not changed.

You can check the three claims:

```
import pandas as pd

from pdbook.io import load_calls
from pdbook.report import require

calls = load_calls()
snapshot = calls.copy()


def add_minutes(df, scale):
    return df.assign(minutes=df["duration_s"] / scale)


by_pipe = calls.pipe(add_minutes, 60)
by_call = add_minutes(calls, 60)
print(by_pipe.equals(by_call))

result = (
    calls
    .query("status == 'answered'")
    .assign(minutes=lambda d: d["duration_s"] / 60)
    .pipe(require, lambda d: d["minutes"].notna().all(), "a minute is missing")
)
print(result is calls, result.shape, calls.shape)
print(calls.equals(snapshot), "minutes" in calls.columns)
```

```
True
False (15526, 10) (20000, 9)
True False
```

`pipe` and the plain call give the same table. The chain returns a new object of a smaller shape. `calls` is still the same 20,000 rows, and it is equal to the copy that was made before. The column `minutes` exists only in the result.

- **Invariant.** A step takes a table and returns a table, and it leaves its input as it was.
- **Failure.** A step that assigns to its argument or returns `None`, such as a method with `inplace=True`, breaks the chain.
- **Cost.** A copy for each step that changes data. Copy-on-write keeps this cheap: columns that a step does not touch are shared, not copied.

Reference

```
import pandas as pd

from pdbook.report import monthly_report, require
from pdbook.sigs import show_signature

show_signature(pd.DataFrame.pipe, ["func"])
show_signature(pd.DataFrame.assign)
show_signature(pd.DataFrame.query, ["expr"])
show_signature(pd.DataFrame.sort_values, ["by", "ascending", "kind"])
show_signature(require)
show_signature(monthly_report)
```

```
NDFrame.pipe(
    func: 'Callable[Concatenate[Self, P], T] | tuple[Callable[..., T], str]'
)  # 3 more parameters
DataFrame.assign(
    self
    **kwargs
)
DataFrame.query(
    expr: 'str'
)  # 8 more parameters
DataFrame.sort_values(
    by: 'IndexLabel'
    ascending: 'bool | list[bool] | tuple[bool, ...]' = True
    kind: 'SortKind' = 'quicksort'
)  # 6 more parameters
require(
    df: 'pd.DataFrame'
    test: 'Callable[[pd.DataFrame], bool]'
    message: 'str'
)
monthly_report(
    calls: 'pd.DataFrame'
    subs: 'pd.DataFrame'
    cells: 'pd.DataFrame'
    plans: 'pd.DataFrame'
)
```

Parameter	Default	What it does here	What goes wrong
`pipe(func, *args, **kwargs)`	required	calls `func(df, *args, **kwargs)` and passes on the result	a `func` that returns `None` ends the chain
`assign(**kwargs)`	—	columns from values or from functions of the table	—
`query(expr)`	required	filter by a string	a typo in a name is an error when it runs
`sort_values(by, ascending, kind)`	required, `True`, `'quicksort'`	order the rows	—
`require(df, test, message)`	required	returns `df`, or raises `ValueError(message)`	—
`monthly_report(calls, subs, cells, plans)`	required	20 rows: plan and city	a plan missing from `plans` gives revenue 3,681.87 against 4,245.09, with no error unless you add a check

`monthly_report` returns a DataFrame sorted by plan and city. It raises `ValueError` for a repeated `call_id`.

What you have now

- You write a transformation as a chain with one name at the start and one at the end.
- You use `assign` with lambdas, `pipe` for your own steps, and `require` for checks.
- You cut a long chain at a meaning, and put the cut in a named function.
- You test a report against an independent computation, against a shuffled input and against a planted fault.

The next chapter takes the checks out of the chain and makes them a validation layer for the data.

Next: Chapter 17, Data validation.

17. Data validation

> **Build** — A small validation layer for the call records: a list of rules, a report of how many rows break each one, a schema check and a way to stop a job. The layer is measured. It must find exactly the faults that a function plants in a copy of the table.
>
> **Theory kept** — A cleaner repairs what it can. A validator states what must be true and reports the rows where it is not. Run it where data enter the job, before the first join and before the report. Test the validator itself with faults that you plant.
>
> **Source** — New. Library: `pdbook/validate.py`.
>
> **Run this first** — `uv run python -m examples.ch17_validation`, which is `examples/ch17_validation.py`. Exercises: `exercises/17.md`. Solutions: `exercises/solutions/17.md`.

What to check

Most faults in a table fall in six classes. A rule in each class costs one line:

Class	Question	Example in the call records
schema	right columns, right dtypes?	`callee` is text, not an integer
domain	values in the allowed set or range?	`status` is one of four words; duration is not negative
uniqueness	is the key unique?	`call_id`
completeness	are required values present?	a call has a cell
relational	does the key exist in the other table?	the caller is a known subscriber
cross-column	do two columns agree?	an answered call has a duration above 0

Cleaning (Chapter 6) and validation do different work. A cleaner changes data. A validator never changes data. It says which rows break a rule, and it lets the job decide: stop, drop, or send to a person.

The schema

The first check is cheap and finds the largest class of faults, a file that changed format:

```
import pandas as pd

from pdbook.io import load_calls
from pdbook.validate import check_schema

calls = load_calls()
print(check_schema(calls))

naive = pd.read_csv("data/calls.csv")
for problem in check_schema(naive):
    print(problem)
```

```
[]
start: dtype str, expected datetime64[us, UTC]
callee: dtype int64, expected str
```

The loader gives a table that passes. The plain `read_csv` gives two problems: `start` is text and `callee` is an integer. This check, on the first line of a job, finds a changed file before the job spends an hour on it.

Rules as data

A rule is a name, a function that returns `True` for each row that breaks it, and a severity. Keep the rules in a list, so that a report can go through them:

```
import pandas as pd

from pdbook.io import load_calls, load_cells, load_subscribers
from pdbook.validate import call_rules, validate

calls, subs, cells = load_calls(), load_subscribers(), load_cells()
rules = call_rules(subs, cells)
print(len(rules), rules[3].name)

report = validate(calls, rules)
print(report[["rule", "severity", "violations"]].to_string(index=False))
```

```
10 an answered call has a duration above 0
                                        rule severity  violations
                            call_id is unique    error           0
                      status is one of the four    error           0
                       duration is not negative    error           0
        an answered call has a duration above 0    error           0
a call that is not answered has a codec of none    error           0
      callee is in the form +32 and nine digits    error           0
                  caller is a known subscriber    error           0
                     start is inside the month    error           0
               cell is known when it is present    error           0
                            a call has a cell     warn         277
```

On the real data, nine rules have no violation. The tenth, "a call has a cell", has 277 with the severity `warn`. Chapter 5 showed those 277 calls: the loss of cell identifiers is a known property of the data. A `warn` rule is reported and does not

stop the job. An `error` rule stops it.

Measure the validator

A validator that finds nothing on the real data is right, or it is blind. The way to know is to break the data in a known way and count. The function `plant_faults` makes six kinds of fault on separate rows and returns the number of violations that each rule must find:

```
import pandas as pd

from pdbook.io import load_calls, load_cells, load_subscribers
from pdbook.validate import call_rules, plant_faults, validate

calls, subs, cells = load_calls(), load_subscribers(), load_cells()
broken, expected = plant_faults(calls)
report = validate(broken, call_rules(subs, cells))
found = dict(zip(report["rule"], report["violations"]))

for rule, n in expected.items():
    print(f"planted {n:>2d} found {found[rule]:>2d}  {rule}")
print(all(found[r] == n for r, n in expected.items()))
print({r: v for r, v in found.items() if r not in expected})
```

```
planted  7 found  7  call_id is unique
planted  5 found  5  duration is not negative
planted 11 found 11  an answered call has a duration above 0
planted  4 found  4  a call that is not answered has a codec of none
planted  3 found  3  callee is in the form +32 and nine digits
planted  4 found  4  caller is a known subscriber
planted  2 found  2  start is inside the month
True
{'status is one of the four': 0, 'cell is known when it is present': 0, 'a call has a cell':
277}
```

The rule for answered calls finds 11, not 6 or 5. The function broke 5 durations to a negative value and 6 to a missing value. A negative value is not "above 0" either, so both kinds break that rule. The expected number includes it. A rule can have two causes, and the report says only how many rows break it.

The last line shows the rules that no fault was planted for: `status`, `cell is known` and the warning. They find 0, 0 and 277, the same as on the clean data. A validator that is exact on 7 rules, and gives the clean-data result on the other 3, is what you want.

Stop the job

`assert_valid` raises an error when a rule with the severity `error` has any violation, and it lists the rules and the counts:

```
import pandas as pd

from pdbook.io import load_calls, load_cells, load_subscribers
from pdbook.validate import assert_valid, call_rules, plant_faults, validate

calls, subs, cells = load_calls(), load_subscribers(), load_cells()
rules = call_rules(subs, cells)

assert_valid(validate(calls, rules))
print("clean data pass")

broken, _ = plant_faults(calls)
try:
    assert_valid(validate(broken, rules))
except ValueError as error:
    print(error)
```

```
clean data pass
data validation failed:
  call_id is unique: 7
  duration is not negative: 5
  an answered call has a duration above 0: 11
  a call that is not answered has a codec of none: 4
  callee is in the form +32 and nine digits: 3
  caller is a known subscriber: 4
  start is inside the month: 2
```

A job that stops on a violation is a design choice. The alternative is to remove the rows and to continue, and to send the removed rows to a person. Both are better than a report that has the faults inside it. Which one you choose depends on the cost of a missing row against the cost of a wrong row. A billing job prefers to stop. A dashboard prefers to drop and to show the count.

Compare tables in tests

The validator checks a table against rules. A test of a function checks a result against an expected table. The tool is `pd.testing.assert_frame_equal`, and its arguments say what "equal" means:

```
import pandas as pd

a = pd.DataFrame({"x": [1.0, 2.0], "y": ["u", "v"]}, index=[0, 1])
b = pd.DataFrame({"y": ["v", "u"], "x": [2.0, 1.0 + 1e-9]}, index=[1, 0])

for label, kwargs in [("strict", {}), ("like", {"check_like": True}),
                      ("rtol=0", {"check_like": True, "rtol": 0}),
                      ("exact", {"check_like": True, "check_exact": True})]:
    try:
        pd.testing.assert_frame_equal(a, b, **kwargs)
        print(f"{label:12s} equal")
    except AssertionError as error:
        print(f"{label:12s} different: {str(error).splitlines()[0]}")
```

```
strict       different: DataFrame.index are different
like         equal
rtol=0       equal
exact        different: DataFrame.iloc[:, 1] (column name="x") are different
```

The strict comparison fails on the order of the rows and the columns. With `check_like=True` the order does not matter, and the two tables are equal. The difference of 1e-9 is inside the default tolerances, which are `rtol=1e-5` and `atol=1e-8`. Setting `rtol=0` alone is not enough, because the absolute tolerance still accepts the difference.

Only `check_exact=True` makes the same two tables different. Set the tolerance on purpose. A test with the defaults hides a relative difference of 0.001%, and a test for money must be exact.

Where to put it

Run the validator at three places:

1. **At the input.** Right after the load, before you spend time on the data.
2. **Before a join.** The lookup table must be unique on its key (`validate="m:1"` in Chapter 9).
3. **Before the output.** The report must add up to the input (Chapter 8).

At each place, write the rule as a line of code and not as a comment. A comment that says "the ids are unique" is true on the day that you write it.

How it works

A rule is a function from a table to a boolean Series: `True` on the rows that break the rule. The report counts the `True` values. The validator has no other part.

The consequence is the one of Chapter 3: a comparison with a missing value is `False`, so a rule of the form "value below 0" does not fire on a null. A null needs its own rule.

You can check that the report is the sum of the masks, and see what a null does to a range rule:

```
import pandas as pd

from pdbook.io import load_calls, load_cells, load_subscribers
from pdbook.validate import Rule, call_rules, validate

calls = load_calls()
broken = calls.copy()
broken.loc[broken.index[:3], "duration_s"] = -1.0
broken.loc[broken.index[3:5], "duration_s"] = float("nan")

negative = Rule("duration is not negative", lambda d: d["duration_s"] < 0)
mask = negative.violated(broken)
report = validate(broken, [negative])
print(int(mask.sum()), int(report["violations"].iloc[0]), report["first_rows"].iloc[0])

s = pd.Series([1.0, None, -1.0])
print((s < 0).tolist(), (~s.isin([1.0, 2.0])).tolist())

rules = call_rules(load_subscribers(), load_cells())
print(len(rules), sum(1 for r in rules if r.severity == "error"))
```

```
3 3 [0, 1, 2]
[False, False, True] [False, True, True]
10 9
```

The mask has 3 `True` values, and the report says 3 with the labels 0, 1 and 2. The two rows that were set to `NaN` are not in the count, because `NaN < 0` is `False`. The second line shows the same for `~isin`: the missing value is "not in the list", so it is `True`, and a rule of that form fires on nulls. The two forms differ, and a rule needs to be written with its behavior on nulls in mind.

- **Invariant.** The number in the report is the number of `True` values in the mask of the rule.
- **Failure.** A range rule does not see a null. A set rule (`~isin`) does.
- **Cost.** One pass over the table for each rule. Ten rules on 20,000 rows are ten column operations, not ten loops.

Reference

```
import pandas as pd

from pdbook.sigs import show_signature
from pdbook.validate import Rule, assert_valid, call_rules, check_schema, plant_faults,
validate

show_signature(pd.testing.assert_frame_equal, ["check_dtype", "check_like", "check_exact",
"rtol", "atol"])
show_signature(pd.Series.isin, ["values"])
show_signature(Rule)
show_signature(check_schema)
show_signature(call_rules)
show_signature(validate)
show_signature(assert_valid)
show_signature(plant_faults)
```

```
assert_frame_equal(
    check_dtype: "bool | Literal['equiv']" = True
    check_exact: 'bool | lib.NoDefault' = <no_default>
    check_like: 'bool' = False
    rtol: 'float | lib.NoDefault' = <no_default>
    atol: 'float | lib.NoDefault' = <no_default>
)  # 12 more parameters
Series.isin(
    values
)  # 1 more parameters
Rule(
    name: 'str'
    violated: 'Callable[[pd.DataFrame], pd.Series]'
    severity: 'str' = 'error'
)
check_schema(
    df: 'pd.DataFrame'
    expected: 'dict[str, str]' = {'call_id': 'int64', 'start': 'datetime64[us, UTC]',
'caller': 'str', 'callee': 'str', 'duration_s': 'float64', 'status': 'str', 'cell_id': 'str',
'codec': 'str', 'roaming': 'bool'}
)
call_rules(
    subs: 'pd.DataFrame'
    cells: 'pd.DataFrame'
    first: 'str' = '2026-08-31 22:00'
    last: 'str' = '2026-09-30 22:00'
)
validate(
    df: 'pd.DataFrame'
    rules: 'list[Rule]'
)
assert_valid(
    report: 'pd.DataFrame'
)
plant_faults(
    df: 'pd.DataFrame'
    seed: 'int' = 0
)
```

Parameter	Default	What it does here	What goes wrong
`assert_frame_equal(check_like=...)`	`False`	ignore the order of rows and columns	`False` fails on a reordered table
`assert_frame_equal(rtol=..., atol=...)`	`1e-5`, `1e-8` when not set	the tolerance for floats	`rtol=0` alone still accepts a difference of 1e-9, because `atol` does
`assert_frame_equal(check_exact=...)`	not set	`True` compares floats exactly	—
`assert_frame_equal(check_dtype=...)`	`True`	compare the dtypes	—
`Rule(name, violated, severity=...)`	required, `'error'`	a name, a function that returns `True` for each bad row, and a severity	a range rule such as `d < 0` does not fire on a null
`check_schema(df, expected=...)`	required, the call schema	a list of problems with the names and dtypes	—
`call_rules(subs, cells, first, last)`	required, month bounds in UTC	the 10 rules for call records	—
`validate(df, rules)`	required	one row for each rule: violations, share, first three row labels	—
`assert_valid(report)`	required	raises `ValueError` when a rule of severity `error` has violations	—
`plant_faults(df, seed=...)`	required, `0`	a broken copy and the exact count that each rule must find	the counts are the same for every seed: 7, 5, 11, 4, 3, 4, 2

`validate` returns a DataFrame with the columns `rule`, `severity`, `violations`, `share` and `first_rows`.

What you have now

- You name six classes of rule and write one line for each.
- You keep rules as data, and get a report with a count for each rule.
- You test a validator with faults that you plant, and you expect exact counts.
- You stop a job on an `error` rule, and report a `warn` rule.
- You compare tables in tests with a tolerance that you chose.

The next chapter measures speed, and shows where the time of a pandas job goes.

Next: Chapter 18, Performance.

18. Performance

> **Build** — A benchmark of one task written five ways on 100,000 rows, with a check that all five give the same answer, and the measured cost of three common choices: a loop, a `category` key and a `concat` inside a loop.
>
> **Theory kept** — Measure before you change code. The cost of a pandas job is mostly Python code that runs once for each row. Move that work into whole-column operations. A data type is a speed choice as well as a memory choice. Some advice does not give a gain, and you find out by measuring.
>
> **Source** — New. Library: `pdbook/bench.py`.
>
> **Run this first** — `uv run python -m examples.ch18_performance`, which is `examples/ch18_performance.py`. Exercises: `exercises/18.md`. Solutions: `exercises/solutions/18.md`.

Measure first

A change that you believe is faster is a guess until you time it. Use `time.perf_counter`, take the best of a few runs (the fastest run has the least noise from other work on the machine), and check that the two versions give the same result. The helper in `pdbook/bench.py` does the first two:

```
import time

import pandas as pd


def best_of(fn, repeat=3):
    best, result = float("inf"), None
    for _ in range(repeat):
        t0 = time.perf_counter()
        result = fn()
        best = min(best, time.perf_counter() - t0)
    return best, result


seconds, total = best_of(lambda: pd.Series(range(1_000_000)).sum())
print(total, seconds < 1.0)
```

```
499999500000 True
```

Time the part that you want to improve, and not the load of the file. Time it on the size of the real data: the ratio between two methods changes with the number of rows (Chapter 7 shows a larger ratio at 20,000 rows than at 2,000 rows).

One task, five ways

The task: for each caller, the total talk minutes of answered calls that are longer than 60 seconds. The table has 100,000 rows, which is the call records repeated five times. Each rung of the ladder is a different way to write the same task, in

`pdbook/bench.py`:

Rung	Idea
`iterrows` loop	a Python loop over rows as Series
`itertuples` loop	a Python loop over rows as tuples
`groupby().apply`	a Python function for each group
mask + `groupby`	whole-column operations
category + mask + `groupby`	the same, with a `category` key

```
import pandas as pd

from pdbook.bench import (best_of, replicate, task_category, task_groupby_apply,
task_iterrows,
                          task_itertuples, task_mask_groupby)
from pdbook.io import load_calls

calls = replicate(load_calls(), 5)
rungs = [("iterrows", task_iterrows, 1), ("itertuples", task_itertuples, 1),
         ("groupby.apply", task_groupby_apply, 1), ("mask+groupby", task_mask_groupby, 5),
         ("category", task_category, 5)]

seconds, results = {}, {}
for name, fn, repeat in rungs:
    seconds[name], results[name] = best_of(lambda fn=fn: fn(calls), repeat)

reference = results["mask+groupby"]
for name, result in results.items():
    pd.testing.assert_series_equal(result, reference, check_names=False)

print(len(calls), len(reference))
print(" ".join(f"{n}={s:.3f}s" for n, s in seconds.items()))
assert seconds["iterrows"] / seconds["mask+groupby"] > 20
assert seconds["itertuples"] / seconds["mask+groupby"] > 5
```

```
# output varies
100000 600
iterrows=2.874s itertuples=0.575s groupby.apply=0.434s mask+groupby=0.011s category=0.014s
```

The five give the same 600 totals. The block prints the times of one run. Two assertions fail on your machine in these cases. `iterrows` is not more than 20 times slower than the mask with `groupby`.

`itertuples` is not more than 5 times slower. Read the order of magnitude and not the digits. On the machine that wrote this book, the times changed by a factor of two between runs when other work ran at the same time.

Two things are worth a second look.

First, `itertuples` was between 2.8 and 6.2 times faster than `iterrows` in every run that I made. The column form was faster than both by a large factor. `itertuples` avoids the creation of a Series for each row, and it still runs Python code for each row. If you must loop, loop over tuples.

Second, the `category` rung is not faster than the plain mask in the block. It converts two columns on every call, and the conversion costs more than it saves. The next section separates the two.

Types and speed

A `category` column compares and groups on small integer codes. The saving is real when the column is already a category. Time both on 1,000,000 rows:

```
import pandas as pd

from pdbook.bench import best_of, replicate
from pdbook.io import load_calls

big = replicate(load_calls(), 50)
t_convert, caller_cat = best_of(lambda: big["caller"].astype("category"))
as_cat = big.assign(caller=caller_cat, status=big["status"].astype("category"))

t_str, a = best_of(lambda: big.groupby("caller")["duration_s"].mean())
t_cat, b = best_of(lambda: as_cat.groupby("caller", observed=True)["duration_s"].mean())
pd.testing.assert_series_equal(a.astype("float64"), b.astype("float64"), check_index=False)

t_eq_str, n1 = best_of(lambda: int((big["status"] == "answered").sum()))
t_eq_cat, n2 = best_of(lambda: int((as_cat["status"] == "answered").sum()))
assert n1 == n2

print(len(big))
print(f"groupby  str {t_str * 1000:.1f} ms, category {t_cat * 1000:.1f} ms, ratio {t_str /
t_cat:.1f}x")
print(f"compare  str {t_eq_str * 1000:.1f} ms, category {t_eq_cat * 1000:.1f} ms, ratio
{t_eq_str / t_eq_cat:.1f}x")
print(f"convert caller to category: {t_convert * 1000:.1f} ms")
assert t_str / t_cat > 1.2 and t_eq_str / t_eq_cat > 3
```

```
# output varies
1000000
groupby  str 43.2 ms, category 18.6 ms, ratio 2.3x
compare  str 11.0 ms, category 1.0 ms, ratio 10.9x
convert caller to category: 35.4 ms
```

On a million rows, two assertions apply. A `groupby` on a category key must be at least 1.2 times faster. An equality test must be at least 3 times faster. The block prints the times and the cost of the conversion of one column.

Subtract the two `groupby` times to get the saving of one use. The conversion pays after the number of uses that you get when you divide the conversion time by that saving. This confirms Chapter 4, and it adds the condition: convert once, at load or at the start of the job, and not inside each step.

Do not grow a table in a loop

`concat` builds a new table every time. A loop that appends to a growing table copies it again at every step:

```
import pandas as pd

from pdbook.bench import best_of
from pdbook.io import load_calls

piece = load_calls().head(200)
parts = [piece] * 100


def in_a_loop():
    out = parts[0]
    for p in parts[1:]:
        out = pd.concat([out, p])
    return out


t_loop, a = best_of(in_a_loop, 1)
t_once, b = best_of(lambda: pd.concat(parts), 3)
print(len(a), len(b), a.equals(b))
print(f"loop {t_loop * 1000:.1f} ms, once {t_once * 1000:.1f} ms, ratio {t_loop /
t_once:.0f}x")
assert t_loop / t_once > 3
```

```
# output varies
20000 20000 True
loop 41.2 ms, once 2.6 ms, ratio 16x
```

100 pieces of 200 rows give 20,000 rows in both ways, and the results are equal. The loop is slower by the ratio that the block prints, and the assertion requires at least 3 times. The cost grows faster than the number of pieces, because each step copies all the rows so far. Collect the pieces in a list, and call `concat` once. The same rule holds for `df.loc[len(df)] = row` in a loop, and for `df = df.append(...)` in code that you find on the web.

Advice that did not give a gain

Two common tips are worth a test on your own data. Here they gave no gain:

```
import pandas as pd

from pdbook.bench import best_of, replicate
from pdbook.io import load_calls, load_subscribers

big = replicate(load_calls(), 50)
lookup = load_subscribers()[["subscriber_id", "plan"]]

t_q, a = best_of(lambda: big.query("duration_s > 60 and duration_s < 300 and status ==
'answered'"))
t_m, b = best_of(lambda: big[(big["duration_s"] > 60) & (big["duration_s"] < 300) &
(big["status"] == "answered")])
print(len(a) == len(b), f"query {t_q * 1000:.0f} ms, mask {t_m * 1000:.0f} ms")

def wide():
    return big.merge(lookup, left_on="caller", right_on="subscriber_id", how="left")

def narrow():
    return big[["caller", "status", "duration_s"]].merge(lookup, left_on="caller",
right_on="subscriber_id", how="left")

t_w, _ = best_of(wide)
t_n, _ = best_of(narrow)
print(f"merge wide {t_w * 1000:.0f} ms, narrow {t_n * 1000:.0f} ms")
```

```
# output varies
True query 109 ms, mask 68 ms
merge wide 173 ms, narrow 170 ms
```

`query` was slower than the mask in every run that I made. The documentation says that `query` can use the optional `numexpr` package. That package is not installed here (Chapter 2 prints the check), so this book makes no claim about its gain. Selecting three columns before the merge made no difference, because the merge cost is in the key and not in the columns that come along. A tip is a hypothesis about your data, and the only test is a timing.

Where the time goes

The five rungs, the category and the loop have one pattern. The slow versions run Python code for each row, or copy a table many times. The fast versions call compiled code a few times on whole columns. When a job is slow, ask three questions in this order:

1. **Is there a Python loop, `apply(axis=1)`, `iterrows` or `itertuples` over rows?** Replace it with a column expression.
2. **Is a table copied in a loop, or are rows added one at a time?** Collect in a list and build once.
3. **Is a column text where it can be a category, or an integer that is float?** Fix the type once, at the start.

If the answers are all no and the job is slow, you have a data size problem, and the next chapter is for that.

How it works

Most of the speed in this chapter comes from two facts. A column operation makes one call into compiled code instead of one Python call for each row (Chapter 7). And `concat` copies: it allocates a new table and copies every row of its inputs into it.

A loop that concatenates one piece at a time copies the whole table so far at each step. The rows copied are 1 + 2 + 3 + ... + n pieces. That sum is about n squared divided by 2.

You can count the rows that are copied, with no timing at all:

```
import pandas as pd

from pdbook.io import load_calls

piece = load_calls().head(200)


def rows_copied_in_a_loop(n_pieces: int) -> int:
    out, copied = piece, 0
    for _ in range(n_pieces - 1):
        out = pd.concat([out, piece])
        copied += len(out)
    return copied


for n in (100, 200, 400):
    loop = rows_copied_in_a_loop(n)
    once = n * len(piece)
    print(f"{n:>4d} pieces: loop copies {loop:>10,d} rows, one concat copies {once:>7,d},
ratio {loop / once:.0f}")

small, large = rows_copied_in_a_loop(100), rows_copied_in_a_loop(400)
print(round(large / small, 1))
```

```
 100 pieces: loop copies  1,009,800 rows, one concat copies  20,000, ratio 50
 200 pieces: loop copies  4,019,800 rows, one concat copies  40,000, ratio 100
 400 pieces: loop copies 16,039,800 rows, one concat copies  80,000, ratio 200
15.9
```

For 100 pieces the loop copies about 50 times as many rows as the single `concat`, and for 400 pieces about 200 times. When the number of pieces grows by 4, the rows copied grow by about 16. This count does not depend on the machine, and it is the cause of the quadratic time in the loop.

- **Invariant.** The time of a vectorized job is proportional to the number of rows. The time of a loop that copies is proportional to the square of the number of steps.
- **Failure.** A test with 10 pieces. The square of 10 is small, and the same code with 4,000 pieces is a different program.
- **Cost.** A copy of every row at every step. Collect pieces in a list and call `concat` once.

Reference

```
import pandas as pd

from pdbook.bench import best_of, replicate, task_mask_groupby
from pdbook.sigs import show_signature

show_signature(pd.DataFrame.iterrows)
show_signature(pd.DataFrame.itertuples, ["index", "name"])
show_signature(pd.concat, ["objs", "ignore_index"])
show_signature(pd.Series.astype, ["dtype"])
show_signature(best_of)
show_signature(replicate)
show_signature(task_mask_groupby)
```

```
DataFrame.iterrows(
    self
)
DataFrame.itertuples(
    index: 'bool' = True
    name: 'str | None' = 'Pandas'
)  # 1 more parameters
concat(
    objs: 'Iterable[Series | DataFrame] | Mapping[HashableT, Series | DataFrame]'
    ignore_index: 'bool' = False
)  # 8 more parameters
NDFrame.astype(
    dtype
)  # 3 more parameters
best_of(
    fn: 'Callable[[], object]'
    repeat: 'int' = 3
)
replicate(
    df: 'pd.DataFrame'
    times: 'int'
)
task_mask_groupby(
    df: 'pd.DataFrame'
)
```

Parameter	Default	What it does here	What goes wrong
`iterrows()`	—	yields `(label, Series)` for each row	each row is a Series: slow, and the dtype is `object` for a mixed table
`itertuples(index=..., name=...)`	`True`, `'Pandas'`	yields a named tuple for each row	still a Python call for each row
`concat(objs, ignore_index=...)`	required, `False`	stack tables	in a loop it copies 1 + 2 + ... + n pieces: 16,039,800 rows for 400 pieces of 200, against 80,000 for one call
`astype("category")`	—	convert a column	the conversion costs time; it pays only when the column is used more than once
`best_of(fn, repeat=...)`	required, `3`	`(seconds, result)` of the fastest run	the first run includes warm-up
`replicate(df, times)`	required	the table repeated, with a unique `call_id`	—
`task_mask_groupby(df)`	required	total talk minutes for each caller, answered calls over 60 s	—

`best_of` returns a tuple of a float and the result of the last run.

What you have now

- You time with the best of a few runs, on the real size, and you check that the results are equal.
- You know that `iterrows` and `itertuples` are one to two orders of magnitude slower than a column expression (the assertions of the ladder).
- You convert to `category` once, and you know when it pays.
- You collect pieces in a list and call `concat` once.
- You test a tip before you believe it.

The next chapter handles data that is larger than memory.

Next: Chapter 19, Data larger than memory.

19. Data larger than memory

> **Build** — `talk_by_caller_chunked(path)` : an aggregation that reads a large CSV file in chunks and gives exactly the result of reading the whole file, with a measure of the peak memory of three strategies.
>
> **Theory kept** — Memory is the first limit of pandas. Reduce the data before you reduce the problem: read fewer columns, use smaller types, filter at the read. When that is not enough, read in chunks and keep only an aggregate that you can add across chunks.
>
> **Source** — New. Library: `pdbook/chunked.py` .
>
> **Run this first** — `uv run python -m examples.ch19_chunked` , which is `examples/ch19_chunked.py` . It writes a 170 MB file to a temporary folder and takes about 30 seconds. Exercises: `exercises/19.md` . Solutions: `exercises/solutions/19.md` .

How much memory is a file

A table in memory needs more than the file on disk, and the read itself needs more than the table. In this book's data, the 1.56 MB call file is 2.2 MB as a table, which is 1.4 times. The peak while pandas reads a file of 170 MB is about 580 MB, which is 3.4 times (measured in the build section below). Plan for the peak and not for the table.

Look at the size before you read the whole file:

```
import pandas as pd

head = pd.read_csv("data/calls.csv", nrows=1000)
per_row = head.memory_usage(deep=True).sum() / len(head)
rows = sum(1 for _ in open("data/calls.csv")) - 1
print(rows, round(per_row), "bytes per row")
print(f"estimate for the file: {rows * per_row / 1e6:.1f} MB")
print(f"measured             : {pd.read_csv('data/calls.csv').memory_usage(deep=True).sum() /
1e6:.1f} MB")
```

```
20000 109 bytes per row
estimate for the file: 2.2 MB
measured             : 2.2 MB
```

The estimate from 1,000 rows is close to the measured size. Use this when a file has 200 million rows and you must decide what to do before you start.

Four steps, in order

1. **Read fewer columns.** `usecols` (Chapter 2). This is often the whole answer.
2. **Use smaller types.** `dtype={"status": "category"}` at the read, and integer types that fit (Chapter 4).
3. **Filter at the read.** Parquet reads only the columns and row groups that you ask for.
4. **Read in chunks.** Keep an aggregate, not the rows.

A step is enough when the result fits. Do not go to the next one before you measure.

Filter at the read with parquet

CSV must be read in full to find a row. Parquet stores the columns separately, so you can read some columns without the rest, and `read_parquet` can apply a filter at the read. The block measures both:

```
import pathlib
import tempfile
import time

import pandas as pd

from pdbook.bench import replicate
from pdbook.io import load_calls

big = replicate(load_calls(), 50)
with tempfile.TemporaryDirectory() as tmp:
    path = pathlib.Path(tmp) / "big.parquet"
    big.to_parquet(path)

    def timed(**kwargs):
        t0 = time.perf_counter()
        df = pd.read_parquet(path, **kwargs)
        return time.perf_counter() - t0, df

    t_all, everything = timed()
    t_cols, two = timed(columns=["caller", "duration_s"])
    t_filter, some = timed(columns=["caller", "duration_s"], filters=[("status", "==",
"answered")])

print(len(big), everything.shape, two.shape, some.shape)
print(round(everything.memory_usage(deep=True).sum() / 1e6),
round(two.memory_usage(deep=True).sum() / 1e6),
      round(some.memory_usage(deep=True).sum() / 1e6), "MB in memory")
print(f"read all {t_all * 1000:.0f} ms, two columns {t_cols * 1000:.0f} ms, two columns and
rows {t_filter * 1000:.0f} ms")
```

```
# output varies
1000000 (1000000, 9) (1000000, 2) (776300, 2)
101 21 16 MB in memory
read all 94 ms, two columns 36 ms, two columns and rows 48 ms
```

The table of 1,000,000 rows needs 101 MB with nine columns and 21 MB with two columns. It needs 16 MB with two columns and the answered calls only (776,300 rows). In every run that I made, the read with the filter was not faster than the read of the same two columns. The file is small, and the filter is an extra step. The saving is in memory. With a file that does not fit in memory, memory is the limit that matters.

Chunks

`read_csv(..., chunksize=n)` returns an iterator of DataFrames of `n` rows. Only one chunk is in memory at a time. The idea is to reduce each chunk to a small result and to combine the small results:

```
import pathlib
import tempfile

import pandas as pd

from pdbook.chunked import make_big_csv

with tempfile.TemporaryDirectory() as tmp:
    path = make_big_csv(pathlib.Path(tmp) / "big.csv", times=5)
    reader = pd.read_csv(path, usecols=["caller", "status", "duration_s"], chunksize=30_000)
    sizes, parts = [], []
    for chunk in reader:
        sizes.append(len(chunk))
        answered = chunk[chunk["status"] == "answered"]
        parts.append(answered.groupby("caller")["duration_s"].agg(["sum", "size"]))

    total = pd.concat(parts).groupby(level=0).sum()
    whole = pd.read_csv(path, usecols=["caller", "status", "duration_s"])
    check = whole[whole["status"] == "answered"].groupby("caller")["duration_s"].agg(["sum",
"size"])

print(len(sizes), sizes)
print(total.shape, bool((total == check).all().all()))
```

```
4 [30000, 30000, 30000, 10000]
(600, 2) True
```

100,000 rows in 4 chunks, and the combined result is equal to the result of reading the whole file. Each chunk gives one row for each caller, and the rows with the same caller are added.

The method works for aggregates that you can add across chunks: sum, count, minimum, maximum. A mean is the sum divided by the count, so keep both and divide at the end. It does not work for the median, a quantile, or the number of distinct values. Those need all the values of a group at once, or an approximation with a different structure. The mean of the means of the chunks is also wrong, because chunks hold different counts.

Build: chunked and whole

The function in `pdbook/chunked.py` does the steps and returns sum, count and mean for each caller. A second function does the same on the whole file. You can then compare the two:

```
import pathlib
import tempfile

import pandas as pd

from pdbook.chunked import make_big_csv, talk_by_caller_chunked, talk_by_caller_whole

with tempfile.TemporaryDirectory() as tmp:
    path = make_big_csv(pathlib.Path(tmp) / "big.csv", times=20)
    a = talk_by_caller_chunked(path, chunksize=70_000)
    b = talk_by_caller_whole(path)

pd.testing.assert_frame_equal(a, b)
print(a.shape, a.columns.tolist())
print(a.head(3).round(1).to_string())
print(round(float(a["sum"].sum())), int(a["size"].sum()))
```

```
(600, 3) ['sum', 'size', 'mean']
            sum  size   mean
caller
S0001   12060.0   160   75.4
S0002   19280.0   180  107.1
S0003   40520.0   360  112.6
41246960 310520
```

The two results are equal in every cell. The chunk size of 70,000 rows does not divide the file, so the last chunk is shorter, and the result is still the same. The file has 400,000 rows, and 310,520 of its calls are answered.

Now the memory. Each measure starts a new Python process, because the peak of one process includes everything that it did before. The figure is `VmHWM`, the high-water mark of the resident memory of the new process, from `/proc/self/status`, so the function works on Linux only. The block writes a file of 2,000,000 rows and 170 MB, and it takes about 30 seconds:

```
import pathlib
import tempfile
import time

import pandas as pd

from pdbook.chunked import make_big_csv, peak_rss_mb, talk_by_caller_chunked,
talk_by_caller_whole

with tempfile.TemporaryDirectory() as tmp:
    path = make_big_csv(pathlib.Path(tmp) / "big.csv", times=100)
    t0 = time.perf_counter()
    chunked_result = talk_by_caller_chunked(path)
    t_chunked = time.perf_counter() - t0
    pd.testing.assert_frame_equal(chunked_result, talk_by_caller_whole(path))

    base = peak_rss_mb("import pandas, pyarrow")
    everything = peak_rss_mb(f"import pandas as pd\ndf = pd.read_csv({str(path)!r})")
    whole = peak_rss_mb(f"from pdbook.chunked import *\nr =
talk_by_caller_whole({str(path)!r})")
    chunked = peak_rss_mb(f"from pdbook.chunked import *\nr =
talk_by_caller_chunked({str(path)!r})")

print(f"python and pandas {base:.0f} MB | all 9 columns {everything:.0f} MB | "
      f"3 columns {whole:.0f} MB | 3 columns in chunks {chunked:.0f} MB")
print(f"chunked job: {t_chunked:.1f} s for {int(chunked_result['size'].sum()):,d} answered
calls")
assert chunked < whole < everything
```

```
# output varies
python and pandas 99 MB | all 9 columns 583 MB | 3 columns 335 MB | 3 columns in chunks 178 MB
chunked job: 1.9 s for 1,552,600 answered calls
```

On the machine that wrote this book, a file of 170 MB gave these peaks. Python and pandas alone need about 100 MB. Reading all nine columns needs about 580 MB, and reading three columns needs about 340 MB. Chunks of 100,000 rows need about 180 MB.

The peaks stayed within about 10 MB between the runs that I made. Reading three columns instead of nine saves about 240 MB, and chunks save about 160 MB more. The assertion in the block fails if the order of the three is different on your machine.

The chunked version does not grow with the file. Its peak is the baseline plus one chunk. That is the property that you buy. You pay for it in more code, in aggregates that you can add, and in a read that is not faster. The block prints the time of the chunked job for 2,000,000 rows.

When the file is still too large

Other tools exist for data that does not fit in memory. `polars` and `duckdb` are two of them. They are not installed here (the check in Chapter 2 prints it), so this book did not test them and makes no claim about them. Test one on your own data before you rely on it. Appendix A lists them.

How it works

`read_csv(..., chunksize=n)` does not read the file at once. It returns a reader, an iterator, and each step of the iteration parses the next `n` rows into a DataFrame. Each chunk is parsed on its own, and the dtype of a column is chosen again from the values of that chunk. This is the source of one trap: the same column can have different dtypes in different chunks.

The reader and the trap are visible on a small text:

```
import io

import pandas as pd

text = "id,minutes\n1,5\n2,6\n3,\n4,8\n"
reader = pd.read_csv(io.StringIO(text), chunksize=2)
print(type(reader).__name__)

chunks = list(reader)
print([c["minutes"].dtype.name for c in chunks], [len(c) for c in chunks])
print(pd.concat(chunks)["minutes"].dtype.name)

fixed = pd.read_csv(io.StringIO(text), chunksize=2, dtype={"minutes": "float64"})
print([c["minutes"].dtype.name for c in fixed])
```

```
TextFileReader
['int64', 'float64'] [2, 2]
float64
['float64', 'float64']
```

The first chunk has the rows 1 and 2, all whole numbers, so `minutes` is `int64`. The second chunk has a missing value, so it is `float64`. The two chunks have different dtypes for one column, and `concat` makes `float64`. A sum over the chunks mixes the two types. With `dtype` in the call, every chunk has the type that you chose.

- **Invariant.** A chunk holds a fixed number of rows and is parsed alone. At any time one chunk and your aggregate are in memory.
- **Failure.** The dtype of a column can change from chunk to chunk. Give `dtype` for every column that you use.
- **Cost.** The parse of every row, as in a whole read. The memory is the chunk, and the time is not smaller.

Reference

```
import pandas as pd

from pdbook.chunked import make_big_csv, peak_rss_mb, talk_by_caller_chunked,
talk_by_caller_whole
from pdbook.sigs import show_signature

show_signature(pd.read_csv, ["filepath_or_buffer", "usecols", "dtype", "chunksize"])
show_signature(pd.read_parquet, ["columns", "filters"])
show_signature(make_big_csv)
show_signature(talk_by_caller_chunked)
show_signature(talk_by_caller_whole)
show_signature(peak_rss_mb)
```

```
read_csv(
    filepath_or_buffer: 'FilePath | ReadCsvBuffer[bytes] | ReadCsvBuffer[str]'
    usecols: 'UsecolsArgType' = None
    dtype: 'DtypeArg | None' = None
    chunksize: 'int | None' = None
)  # 40 more parameters
read_parquet(
    columns: 'list[str] | None' = None
    filters: 'list[tuple] | list[list[tuple]] | None' = None
)  # 7 more parameters
make_big_csv(
    path: 'str | pathlib.Path'
    times: 'int' = 50
)
talk_by_caller_chunked(
    path: 'str | pathlib.Path'
    chunksize: 'int' = 100000
)
talk_by_caller_whole(
    path: 'str | pathlib.Path'
)
peak_rss_mb(
    code: 'str'
)
```

Parameter	Default	What it does here	What goes wrong
`read_csv(chunksize=...)`	`None`	rows for each chunk; the result is an iterator	the dtype of a column is chosen again for each chunk: `int64` then `float64`
`read_csv(usecols=...)`	`None`	read only these columns	— (3 of 9 columns: about 340 MB against about 580 MB at the peak)
`read_csv(dtype=...)`	`None`	the type of the columns that you name, in every chunk	—
`read_parquet(columns=...)`	`None`	read only these columns	—
`read_parquet(filters=...)`	`None`	a filter that is applied at the read	the result has 776,300 of 1,000,000 rows
`make_big_csv(path, times=...)`	required, `50`	the call records repeated `times` over	`times=100` writes 2,000,000 rows and 170 MB
`talk_by_caller_chunked(path, chunksize=...)`	required, `100000`	sum, count and mean for each caller	the first version returned `size` as `float64` when a caller was missing from a chunk
`talk_by_caller_whole(path)`	required	the same result with the whole file in memory	—
`peak_rss_mb(code)`	required	the peak resident memory of a fresh process that runs `code`, in MB (Linux)	`ru_maxrss` includes the memory of the parent at the fork

`talk_by_caller_chunked` and `talk_by_caller_whole` return equal DataFrames with the columns `sum`, `size` and `mean`.

What you have now

- You estimate the memory of a file from a sample of rows.
- You follow four steps, in order: fewer columns, smaller types, filters at the read, chunks.
- You aggregate in chunks with sums and counts, and you check against the whole.
- You measure the peak memory in a fresh process.

The next chapter moves part of the work to a database.

Next: Chapter 20, SQL and pandas.

20. SQL and pandas

> **Build** — `monthly_report_sql(conn)` : the monthly report of Chapter 16 written as one SQL query over SQLite, with a proof that it gives the same 20 rows as the pandas chain, in every cell and every dtype.
>
> **Theory kept** — SQL and pandas answer the same questions with different tools. The database is strong at storage, indexes and joins on data that does not fit in memory. pandas is strong at the steps after the query. Pass values with parameters, and know which types the round trip changes.
>
> **Source** — New. Library: `pdbook/sqlbridge.py` . SQLite is in the Python standard library, so the chapter needs no server.
>
> **Run this first** — `uv run python -m examples.ch20_sql` , which is `examples/ch20_sql.py` . Exercises: `exercises/20.md` . Solutions: `exercises/solutions/20.md` .

Write a table, read a query

`to_sql` writes a DataFrame as a table. `read_sql_query` runs a query and returns a DataFrame:

```
import sqlite3

import pandas as pd

from pdbook.io import load_calls

conn = sqlite3.connect(":memory:")
calls = load_calls()
calls.assign(start=calls["start"].dt.strftime("%Y-%m-%dT%H:%M:%SZ")).to_sql("calls", conn,
index=False)

answered = pd.read_sql_query("SELECT COUNT(*) AS n FROM calls WHERE status = 'answered'",
conn)
print(int(answered["n"].iloc[0]))
print(pd.read_sql_query("SELECT * FROM calls LIMIT 2", conn).dtypes.astype(str).to_dict())
```

```
15526
{'call_id': 'int64', 'start': 'str', 'caller': 'str', 'callee': 'str', 'duration_s':
'float64', 'status': 'str', 'cell_id': 'str', 'codec': 'str', 'roaming': 'int64'}
```

The count is the 15,526 answered calls. The dtypes show what the round trip did: `start` is text. The code wrote it as an ISO string, and SQLite has no timestamp storage class.

`roaming` is `int64` , because SQLite stores a boolean as the integer 1 or 0. The section on how it works prints the storage classes. Convert on the way out:

```
import sqlite3

import pandas as pd

from pdbook.sqlbridge import build_database

conn = build_database()
back = pd.read_sql_query("SELECT start, roaming FROM calls LIMIT 3", conn, parse_dates=
["start"])
back["roaming"] = back["roaming"].astype(bool)
print(back.dtypes.astype(str).to_dict())
```

```
{'start': 'datetime64[us, UTC]', 'roaming': 'bool'}
```

`parse_dates` converts the text to timestamps. A column that holds 0 and 1 and means a boolean needs `astype(bool)`. Chapter 2 made the same point: a format with weak types needs types on the way out.

Parameters

Never build a query with string formatting. A value that contains a quote breaks the query, and a value from outside can change it. Pass values with `params`:

```
import sqlite3

import pandas as pd

conn = sqlite3.connect(":memory:")
pd.DataFrame({"name": ["O'Neil", "Peeters"], "n": [3, 5]}).to_sql("people", conn, index=False)

name = "O'Neil"
try:
    pd.read_sql_query(f"SELECT n FROM people WHERE name = '{name}'", conn)
except Exception as error:
    print(type(error).__name__)

safe = pd.read_sql_query("SELECT n FROM people WHERE name = ?", conn, params=(name,))
print(int(safe["n"].iloc[0]))
```

```
DatabaseError
3
```

The first query is not valid SQL after the substitution, so it fails. A value of the form `x' OR '1'='1` does not fail, and it returns every row. The second exercise of this chapter runs it. The second form passes the value separately from the query text, and the database never reads it as SQL.

The same report in two languages

The monthly report of Chapter 16 is one chain of ten steps. In SQL it is one query. The two give the same result:

```
import sqlite3

import pandas as pd

from pdbook.io import load_calls, load_cells, load_plans, load_subscribers
from pdbook.report import monthly_report
from pdbook.sqlbridge import MONTHLY_REPORT_SQL, build_database, monthly_report_sql

print(MONTHLY_REPORT_SQL.strip())
print("SQLite", sqlite3.sqlite_version)

conn = build_database()
sql_report = monthly_report_sql(conn)
pandas_report = monthly_report(load_calls(), load_subscribers(), load_cells(), load_plans())
pd.testing.assert_frame_equal(sql_report, pandas_report[sql_report.columns])
print(sql_report.shape, "equal, dtypes included")
```

```
SELECT s.plan                                              AS plan,
       s.city                                              AS home_city,
       COUNT(*)                                            AS calls,
       SUM(c.status = 'answered')                          AS answered,
       SUM(CASE WHEN c.status = 'answered'
                THEN CAST(MAX(1, CEIL(c.duration_s / 60.0)) AS INTEGER) ELSE 0 END)  AS
billed_min,
       ROUND(SUM(CASE WHEN c.status = 'answered'
                THEN ROUND(MAX(1, CEIL(c.duration_s / 60.0)) * p.rate_per_min_eur, 2)
                ELSE 0 END), 2)                            AS revenue_eur,
       ROUND(1.0 * SUM(c.status = 'answered') / COUNT(*), 3) AS answer_rate
FROM calls c
JOIN subscribers s ON s.subscriber_id = c.caller
JOIN plans p       ON p.plan = s.plan
GROUP BY s.plan, s.city
ORDER BY s.plan, s.city
SQLite 3.50.4
(20, 7) equal, dtypes included
```

Compare the two side by side. The `JOIN` lines are the two `merge` steps of `enrich`. `GROUP BY` and the aggregate functions are `groupby` and `agg`. `CASE WHEN` is `np.where`. `CEIL(duration_s / 60.0)` is `np.ceil(duration_s / 60)`.

`CEIL` is a math function of SQLite. The documentation says that these functions exist from version 3.35 and need a build with them, so the block prints the version that ran. The test says that the two agree. This is the strongest check that a report is right: two implementations in two systems that were written separately.

An index

A database makes a lookup fast with an index, in the same way as a pandas index (Chapter 11):

```
import time

from pdbook.sqlbridge import build_database

conn = build_database()
ids = list(range(1, 20001, 40))


def lookups():
    return [conn.execute("SELECT status FROM calls WHERE call_id = ?", (i,)).fetchone() for i
in ids]


t0 = time.perf_counter()
before = lookups()
t_scan = time.perf_counter() - t0

conn.execute("CREATE INDEX ix_call_id ON calls(call_id)")
t0 = time.perf_counter()
after = lookups()
t_index = time.perf_counter() - t0

print(len(ids), before == after)
print(f"without index {t_scan * 1000:.0f} ms, with index {t_index * 1000:.1f} ms, ratio
{t_scan / t_index:.0f}x")
assert before == after and t_scan / t_index > 10
```

```
# output varies
500 True
without index 365 ms, with index 2.3 ms, ratio 160x
```

The block prints the time of 500 lookups without an index, where each one scans the table, and with an index. The answers are the same. The times depend on the hardware, and the assertion fails if the index is not at least 10 times faster on yours.

Window functions

SQL has window functions, and they match Chapter 14. The top 3 answered calls of each caller, in SQL and in pandas:

```
import pandas as pd

from pdbook.io import load_calls
from pdbook.sqlbridge import build_database

conn = build_database()
query = """
SELECT caller, call_id, duration_s FROM (
    SELECT caller, call_id, duration_s,
           ROW_NUMBER() OVER (PARTITION BY caller ORDER BY duration_s DESC, call_id) AS rn
    FROM calls WHERE status = 'answered')
WHERE rn <= 3
"""
sql_top = pd.read_sql_query(query, conn)

calls = load_calls()
answered = calls[calls["status"] == "answered"]
pandas_top = (answered.sort_values(["caller", "duration_s", "call_id"], ascending=[True,
False, True])
              .groupby("caller").head(3)[["caller", "call_id", "duration_s"]])

key = ["caller", "call_id"]
a = sql_top.sort_values(key).reset_index(drop=True)
b = pandas_top.sort_values(key).reset_index(drop=True)
print(len(a), a["caller"].nunique())
pd.testing.assert_frame_equal(a, b)
print("equal")
```

```
1800 600
equal
```

Both give 1,800 rows, 3 for each of 600 callers, and the rows are the same. The tie rule is in both: longer first, then the lower `call_id`. Give a full sort key, so that the result does not depend on how an engine orders ties.

Who does what

A rule for the work:

- **In the database:** filter rows, select columns, join large tables, aggregate to a size that fits in memory, and everything that an index speeds up.
- **In pandas:** the steps after the query: reshape, pivot, rolling windows, plots, models, anything that needs your own Python function.

A query that returns 600 rows from 20,000,000 is the right size to move to pandas. A `SELECT *` that moves 20,000,000 rows to filter them in pandas pays for the move and for the memory.

How it works

SQLite is a library, not a server: the database is a file, or a memory area, in your process. It stores each value in one of five storage classes: null, integer, real, text and blob.

The class belongs to the value, and the declared type of a column is only a preference. There is no class for a timestamp or a boolean. This is why the chapter wrote the time as text and got the boolean back as an integer. An index is a B-tree of the values of a column, and the query planner decides whether to use it.

You can ask SQLite for the classes and for its plan:

```
import sqlite3

from pdbook.sqlbridge import build_database

conn = build_database()
row = conn.execute(
    "SELECT typeof(call_id), typeof(start), typeof(duration_s), typeof(roaming),
typeof(cell_id) "
    "FROM calls WHERE cell_id IS NULL AND status = 'failed' LIMIT 1").fetchone()
print(row)
print(conn.execute("SELECT typeof(1), typeof(1.5), typeof('x'), typeof(NULL),
typeof(x'00')").fetchone())

query = "SELECT status FROM calls WHERE call_id = 12345"
before = [r[3] for r in conn.execute("EXPLAIN QUERY PLAN " + query)]
conn.execute("CREATE INDEX ix_call_id ON calls(call_id)")
after = [r[3] for r in conn.execute("EXPLAIN QUERY PLAN " + query)]
print(before)
print(after)
```

```
('integer', 'text', 'null', 'integer', 'null')
('integer', 'real', 'text', 'null', 'blob')
['SCAN calls']
['SEARCH calls USING INDEX ix_call_id (call_id=?)']
```

The first line is one failed call without a cell: the id is an integer, the time is text, the duration and the cell are null, and the boolean is an integer. The second line lists the five classes. The plan before the index is a scan of the whole table, and after the index it is a search that uses it. The same query gave different plans, and the same answer.

- **Invariant.** A value has one of five classes, and a query plan is either a scan of the table or a search of an index.
- **Failure.** A type that SQLite does not have (a timestamp, a boolean) comes back as text or integer, and pandas gets `str` or `int64`.
- **Cost.** A scan costs one pass over the table for each query. An index costs space and time at each insert, and it saves the scan.

Reference

```
import pandas as pd

from pdbook.sigs import show_signature
from pdbook.sqlbridge import build_database, monthly_report_sql

show_signature(pd.DataFrame.to_sql, ["name", "con", "index", "if_exists"])
show_signature(pd.read_sql_query, ["sql", "con", "params", "parse_dates"])
show_signature(build_database)
show_signature(monthly_report_sql)
```

```
NDFrame.to_sql(
    name: 'str'
    con
    if_exists: "Literal['fail', 'replace', 'append', 'delete_rows']" = 'fail'
    index: 'bool' = True
)  # 6 more parameters
read_sql_query(
    sql
    con
    params: 'list[Any] | Mapping[str, Any] | None' = None
    parse_dates: 'list[str] | dict[str, str] | dict[str, dict[str, Any]] | None' = None
)  # 5 more parameters
build_database(
    conn: 'sqlite3.Connection | None' = None
)
monthly_report_sql(
    conn: 'sqlite3.Connection'
)
```

Parameter	Default	What it does here	What goes wrong
`to_sql(name, con, index=...)`	required, `True`	write a table	`index=True` adds a column `index`
`to_sql(if_exists=...)`	`'fail'`	what to do when the table exists	`fail` raises an error on a second run
`read_sql_query(sql, con)`	required	run a query and return a DataFrame	—
`read_sql_query(params=...)`	`None`	values passed apart from the query text	formatting the value into the text breaks on `O'Neil` and can change the query
`read_sql_query(parse_dates=...)`	`None`	columns to convert to timestamps	without it, `start` is text
`build_database(conn=...)`	`None`	write the four tables into SQLite, in memory when `conn` is not given	—
`monthly_report_sql(conn)`	required	the monthly report as one query	`CEIL` is a math function of SQLite; the chapter prints the version that ran (3.50.4)

`read_sql_query` returns a DataFrame. Booleans come back as integers and timestamps as text.

What you have now

- You write a DataFrame to SQLite, query it, and convert the types that the round trip changed.
- You pass values with parameters, and never with string formatting.
- You have one report in two languages, with a test that they agree.
- You know that an index changes a lookup from a scan to a probe, and you measured the ratio.
- You split work between the database and pandas by the size of what moves.

The next chapter draws the results.

Next: Chapter 21, Plots.

21. Plots

> **Build** — `pdbook/plots.py` : four plots of the call data (calls by hour, a traffic heat map, a KPI series with flagged samples, a duration histogram), each tested by the data that it holds, and each saved as a file in `figures/` .
>
> **Theory kept** — `DataFrame.plot` and `Series.plot` call matplotlib. A plot is a function of a table: shape the table first, then plot. Give each plot a title, axis labels with units and the zone of time. Test a plot with the numbers that are in it, and look at it once.
>
> **Source** — New. Library: `pdbook/plots.py` . Matplotlib is the one extra package that this book installs.
>
> **Run this first** — `uv run python -m examples.ch21_plots` , which is `examples/ch21_plots.py` . It writes four files to `figures/` . Exercises: `exercises/21.md` . Solutions: `exercises/solutions/21.md` .

Shape the table, then plot

pandas plots a Series or a DataFrame with `.plot` , and a method for each kind: `.plot.bar()` , `.plot.line()` , `.plot.hist()` , `.plot.scatter()` . The index becomes the x axis and each column becomes a series. This means that most of the work is in the table (Chapters 8 to 10), and the plot is one line.

```
import matplotlib

matplotlib.use("Agg")

from pdbook.io import load_calls

calls = load_calls()
hours = calls["start"].dt.tz_convert("Europe/Brussels").dt.hour
per_hour = hours.value_counts().reindex(range(24), fill_value=0).sort_index()
ax = per_hour.plot.bar()

print(type(ax).__name__, len(ax.patches))
print([int(p.get_height()) for p in ax.patches][:6])
print(per_hour.iloc[:6].tolist())
```

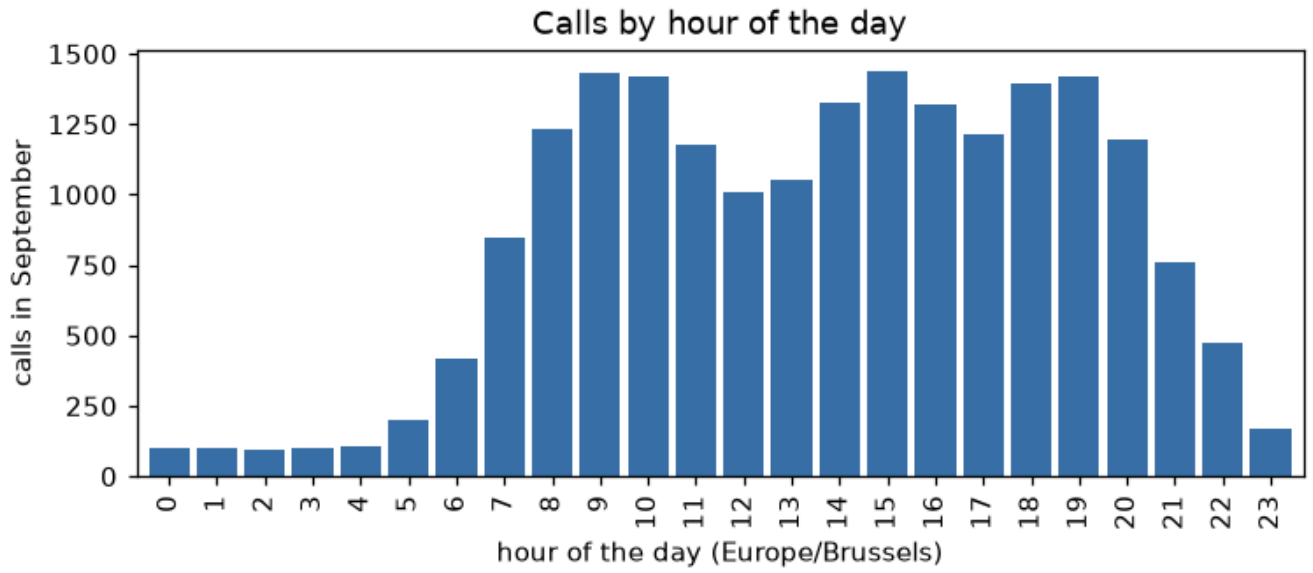
```
Axes 24
[98, 102, 94, 101, 109, 199]
[98, 102, 94, 101, 109, 199]
```

`plot.bar` returns a matplotlib `Axes` . The 24 bars are `patches` , and their heights are the numbers of the Series. Two things in the code come from earlier chapters: the conversion to local time (Chapter 12), and `reindex` so that an hour without calls is a bar of height 0 (Chapter 11).

Choose `matplotlib.use("Agg")` on a server, in a job, or in a test. It draws to a file and needs no display.

A plot has to say what it shows

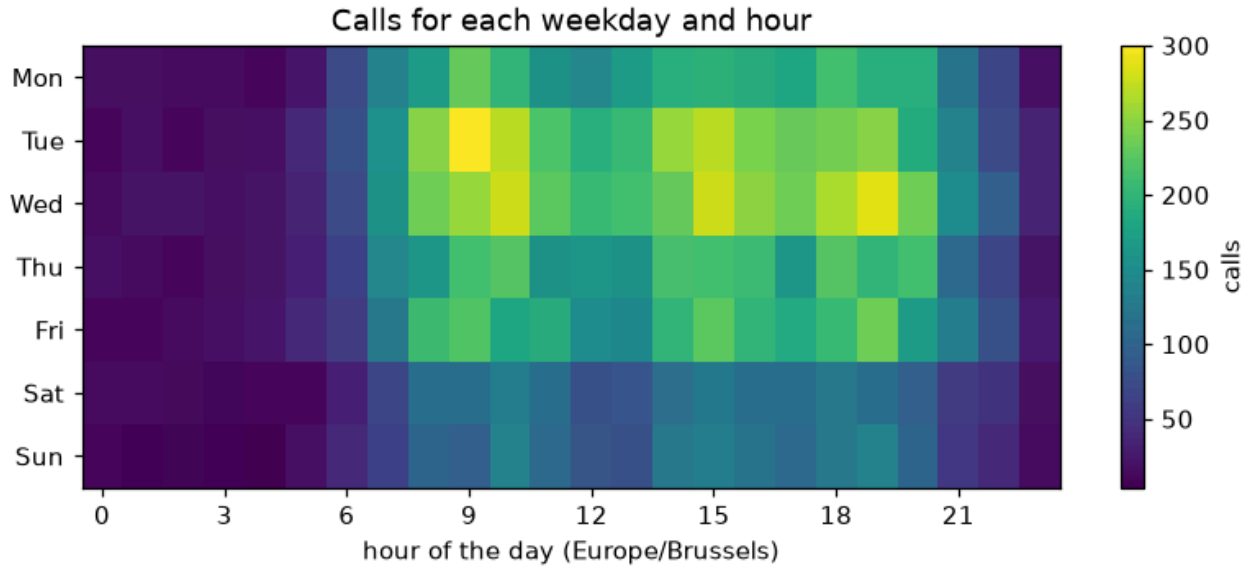
A plot without a unit or a zone is a guess. The module `pdbook/plots.py` puts four things on every figure. It gives a title that names the content and axis labels with units. It puts the time zone in the label of a time axis. It adds a legend when there is more than one mark. This is the first figure, which the example module writes to `figures/hourly.png`:



The figure has about 100 calls an hour from 00:00 to 05:00. It rises from 06:00, and it peaks at 09:00, 15:00 and 19:00. There is a dip to about 1,000 calls at 12:00. The busiest hour is 15:00 local time, the same hour that Chapter 10 found in the table. A figure that disagrees with a table that you trust is a bug in one of them.

Heat map of a grid

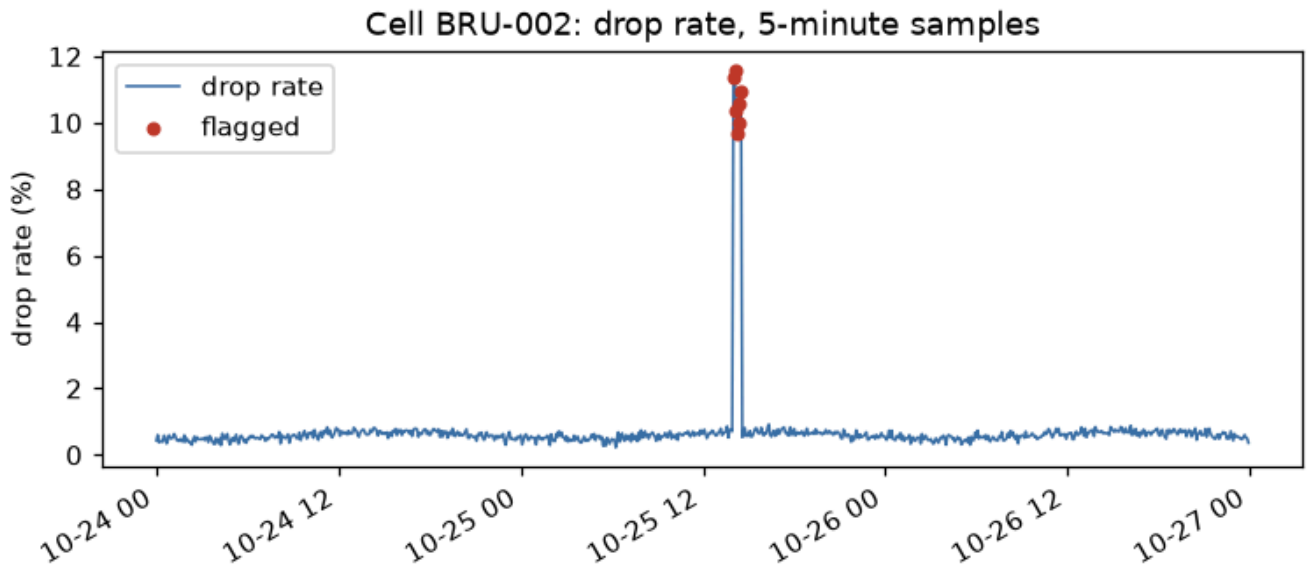
A table of counts with two dimensions is a heat map. `imshow` draws the array, and the tick labels come from the index and the columns:



From 07:00 to 21:00 the Saturday and Sunday rows are darker than the weekday rows. The Tuesday row has the brightest cell at 09:00, the 300 calls of Chapter 10. Wednesday at 19:00 is the second brightest. All cells use one colour scale, so you can compare rows by colour.

Mark the events that a rule found

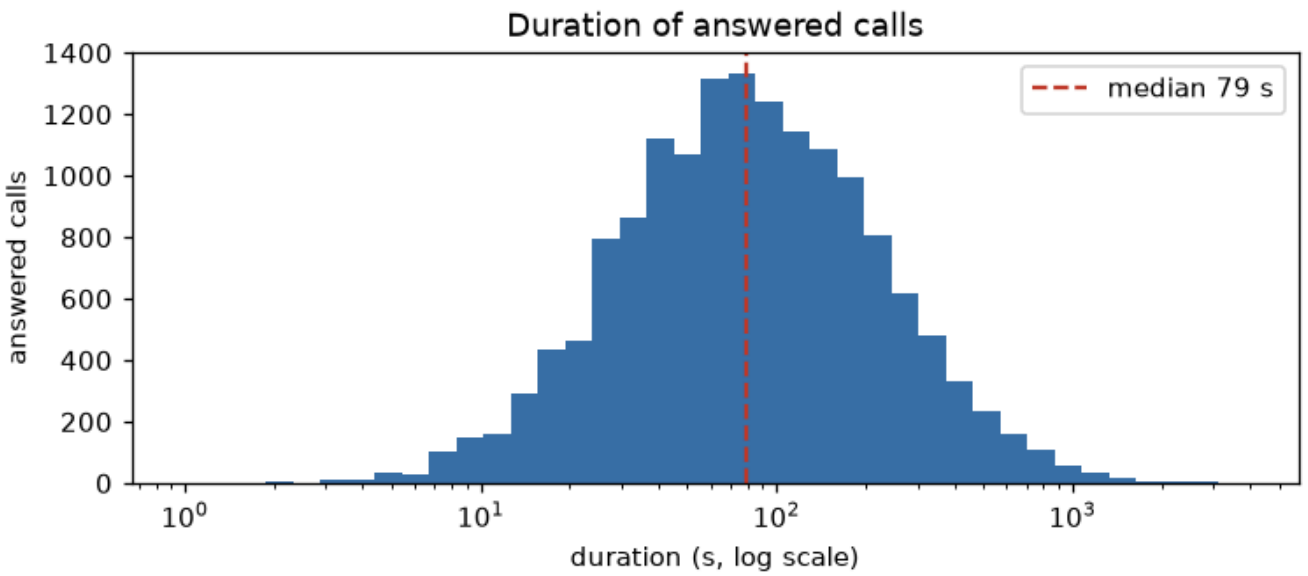
A line of a KPI is easy to read, and a short event is easy to miss in a month of data. Draw the flagged samples on top of the line, so that the figure shows what the detector of Chapter 13 found:



The spike of 25 October is 7 samples at about 10% against a normal level of about 0.6%. The marks sit on the spike, and none is on the rest of the line.

Distributions on a log axis

Call durations have a long tail: the median is 79 seconds and the longest call is 3,836 seconds. On a linear axis the bars of the long calls are not visible. A log axis with log-spaced bins shows the whole range. On this axis the shape is close to a bell, with the median of 79 seconds at its top:



Test a plot by its data

A pixel comparison breaks with every version of matplotlib. Test the numbers that are in the figure. The Axes holds them:

```
import matplotlib

matplotlib.use("Agg")

import numpy as np

from pdbook.io import load_calls, load_kpi
from pdbook.plots import plot_duration_hist, plot_hourly_calls, plot_kpi_spike,
plot_traffic_grid
from pdbook.spikes import flag_spikes
from pdbook.traffic import traffic_grid

calls = load_calls()
kpi = load_kpi().drop_duplicates(["ts", "cell_id"]).reset_index(drop=True)
grid = traffic_grid(calls)

hourly = plot_hourly_calls(calls)
print(len(hourly.patches), int(sum(p.get_height() for p in hourly.patches)) == len(calls))

heat = plot_traffic_grid(grid)
print(bool(np.array_equal(heat.images[0].get_array(), grid.to_numpy())))

spike = plot_kpi_spike(kpi, flag_spikes(kpi, z=4))
print(len(spike.collections[0].get_offsets()), spike.get_title())

hist = plot_duration_hist(calls)
print(int(sum(p.get_height() for p in hist.patches)), hist.get_xscale())
```

```
24 True
True
7 Cell BRU-002: drop rate, 5-minute samples
15526 log
```

The four tests are four lines. There are 24 bars, and they add up to 20,000 calls. The image holds the grid, cell by cell. The scatter has 7 points. The histogram holds the 15,526 answered calls, and its axis is logarithmic. These are the properties that a change in the table code breaks.

A test of the numbers does not say that the figure is readable. Open the file once, and look. A label that covers a tick, or a colour that a colour-blind person cannot read, is not in the data.

How it works

A figure is a tree of objects. A `Figure` holds one or more `Axes`. An `Axes` holds the marks as objects: a bar chart has one `Rectangle` in `patches` for each bar, a line plot has `Line2D` objects, a scatter has a collection.

`Series.plot.bar` is a short way to build this tree from a Series: it puts the index labels on the tick marks and puts the bars at the positions 0, 1, 2 and so on. The picture is made only when you save the figure, and you can read the tree before that.

```
import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt

from pdbook.io import load_calls
from pdbook.plots import plot_hourly_calls

calls = load_calls()
ax = plot_hourly_calls(calls)
fig = ax.figure
centers = [float(p.get_x() + p.get_width() / 2) for p in ax.patches]

print(type(fig).__name__, len(fig.axes), type(ax).__name__, len(ax.patches))
print(type(ax.patches[0]).__name__, centers[:3], centers[-1])
print([t.get_text() for t in ax.get_xticklabels()][:3], ax.get_xlabel())
plt.close(fig)
```

```
Figure 1 Axes 24
Rectangle [0.0, 1.0, 2.0] 23.0
['0', '1', '2'] hour of the day (Europe/Brussels)
```

The figure has one `Axes`, and the `Axes` has 24 `Rectangle` objects, one for each hour. The bar centers are the positions 0, 1, 2 and up to 23, and the tick labels are the hours as text. A test that reads `patches` reads the numbers that the bars were made from, and it needs no image.

- **Invariant.** What is drawn is in the objects of the tree, and the objects hold the numbers.

- **Failure.** A label is a string with no link to the data, so it can be wrong while the figure is complete (the exercise on a figure that lies).
- **Cost.** The tree is cheap. The cost is in the rendering when you save, and it grows with the number of marks.

Reference

```
import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt
import pandas as pd

from pdbook.plots import plot_duration_hist, plot_hourly_calls, plot_kpi_spike,
plot_traffic_grid
from pdbook.sigs import show_signature

show_signature(pd.Series.plot.bar)
show_signature(plt.subplots, ["nrows", "ncols", "fig_kw"])
show_signature(plt.Figure.savefig, ["fname", "kwargs"])
show_signature(plot_hourly_calls)
show_signature(plot_traffic_grid)
show_signature(plot_kpi_spike)
show_signature(plot_duration_hist)
```

```
PlotAccessor.bar(
    self
    x: 'Hashable | None' = None
    y: 'Hashable | None' = None
    color: 'str | Sequence[str] | dict | None' = None
    **kwargs
)
subplots(
    nrows: 'int' = 1
    ncols: 'int' = 1
    **fig_kw: 'Any'
)  # 7 more parameters
Figure.savefig(
    fname
    **kwargs
)  # 2 more parameters
plot_hourly_calls(
    calls: 'pd.DataFrame'
    tz: 'str' = 'Europe/Brussels'
    ax=None
)
plot_traffic_grid(
    grid: 'pd.DataFrame'
    ax=None
)
plot_kpi_spike(
    kpi: 'pd.DataFrame'
    flags: 'pd.Series'
    cell: 'str' = 'BRU-002'
    ax=None
)
plot_duration_hist(
    calls: 'pd.DataFrame'
    ax=None
)
```

Parameter	Default	What it does here	What goes wrong
`Series.plot.bar(color=..., **kwargs)`	`None`	a bar chart; keywords go to matplotlib (`ax` , `width` , `title` , ...)	—
`plt.subplots(nrows, ncols, **fig_kw)`	`1` , `1`	a figure and its axes; `figsize` is a keyword of `fig_kw`	—
`Figure.savefig(fname, **kwargs)`	required	write the figure; `dpi` is a keyword	—
`plot_hourly_calls(calls, tz=..., ax=...)`	required, `'Europe/Brussels'` , `None`	bars for each hour of the day in zone `tz`	the label and the conversion use the same `tz` , so they cannot disagree
`plot_traffic_grid(grid, ax=...)`	required, `None`	heat map of the 7 by 24 grid	—
`plot_kpi_spike(kpi, flags, cell=..., ax=...)`	required, required, `'BRU-002'` , `None`	the line of one cell and the flagged samples	—
`plot_duration_hist(calls, ax=...)`	required, `None`	log-scale histogram of answered durations	—

Each `plot_` function returns the `Axes` , so that a test can read its `patches` , `images` and `collections` .

What you have now

- You shape the table first and plot with `.plot` , and you set `Agg` for jobs.
- You put a title, units, a zone and a legend on each figure.
- You draw the events that a rule found on top of the series.
- You test a figure by the numbers in its Axes, and you look at it once.

The next chapter reads data that is not a table: nested JSON.

Next: Chapter 22, JSON and nested records.

22. JSON and nested records

> **Build** — `events_table(records)` and `call_summary(events, call_ids)`: a flat table with one row for each signalling event of a call trace, a summary with one row for each call, and a check of the summary against the call records.
>
> **Theory kept** — JSON nests objects and lists, and a table does not. `json_normalize` flattens nested objects into columns and a list of records into rows. A record that lacks a part gives missing values or no rows, so count the records at each step.
>
> **Source** — New. Data: `data/traces.jsonl`. Library: `pdbook/traces.py`.
>
> **Run this first** — `uv run python -m examples.ch22_json`, which is `examples/ch22_json.py`. Exercises: `exercises/22.md`. Solutions: `exercises/solutions/22.md`.

A record with a list inside

The file `data/traces.jsonl` has one JSON object on each line, one for each of 500 sampled calls. Each object has the call id, the caller, a `meta` object and a list of `events`:

```
import json

from pdbook.traces import read_traces

records = read_traces()
print(len(records))
print(json.dumps(records[0])[:150])
print(sorted(records[0]))
print(sum("meta" not in r for r in records), sum(len(r["events"]) == 0 for r in records))
```

```
500
{"call_id": 2572, "caller": "S0127", "events": [{"t_ms": 0, "type": "INVITE"}, {"t_ms": 138,
"type": "RESPONSE", "code": 100}, {"t_ms": 248, "type": "
['call_id', 'caller', 'events', 'meta']
23 8
```

The structure is not the same for every line. 23 of the 500 records have no `meta`, and 8 have an empty list of events. A loader that assumes the structure fails on the first one, or gives a wrong answer.

`read_json(lines=True)` reads the file as a table, and it keeps the nested parts as Python objects:

```
import pandas as pd

from pdbook.datasets import DATA

df = pd.read_json(DATA / "traces.jsonl", lines=True)
print(df.shape, df.dtypes.astype(str).to_dict())
print(type(df["meta"].iloc[0]).__name__, type(df["events"].iloc[0]).__name__)
print(df["meta"].isna().sum(), df["events"].map(len).eq(0).sum())
```

```
(500, 4) {'call_id': 'int64', 'caller': 'str', 'events': 'object', 'meta': 'object'}
dict list
23 8
```

The columns `meta` and `events` have the dtype `object`: each cell is a dict or a list. You cannot group by a cell that is a list, and you cannot filter on a key of a dict with a mask. The next steps turn them into columns and rows.

Flatten the objects, then the list

`json_normalize` turns nested dicts into columns, with the path in the name:

```
import pandas as pd

from pdbook.traces import read_traces

records = read_traces()
flat = pd.json_normalize(records, sep=".")
print(flat.columns.tolist())
print(flat.shape, flat[["meta.cell", "meta.codec", "meta.roaming"]].isna().sum().to_dict())
```

```
['call_id', 'caller', 'events', 'meta.cell', 'meta.codec', 'meta.roaming']
(500, 6) {'meta.cell': 30, 'meta.codec': 126, 'meta.roaming': 23}
```

The three keys of `meta` are columns now, and a record with no `meta` has missing values in all three: 23 for `meta.roaming`. `meta.codec` has 126 missing values: the records with no `meta`, and the records where the codec is `None` because the call was not answered.

To get one row for each event, give the path to the list in `record_path` and the fields to carry along in `meta`:

```
import pandas as pd

from pdbook.traces import read_traces

records = read_traces()
events = pd.json_normalize(records, record_path="events",
                           meta=["call_id", "caller", ["meta", "cell"]], errors="ignore")
print(events.shape, events.columns.tolist())
print(events["type"].value_counts().to_dict())
print(events["code"].value_counts(dropna=False).sort_index().to_dict())

exploded = pd.DataFrame(records).explode("events")
print(len(exploded), int(exploded["events"].isna().sum()))

try:
    pd.json_normalize(records, record_path="events", meta=["call_id", "caller", ["meta",
"cell"]])
except KeyError as error:
    print("KeyError:", str(error)[:28])
```

```
(2652, 6) ['t_ms', 'type', 'code', 'call_id', 'caller', 'meta.cell']
{'RESPONSE': 1394, 'INVITE': 492, 'ACK': 383, 'BYE': 383}
{100.0: 472, 180.0: 430, 200.0: 383, 408.0: 47, 486.0: 42, 503.0: 20, nan: 1258}
2660 8
KeyError: "Key 'meta' not found. To re
```

There are 2,652 events for 500 calls. The last two lines show a difference between two tools. `json_normalize` with `record_path` gives no row for a call with an empty list: 492 calls have at least one event, as the 492 `INVITE` rows show. `explode` gives one row for such a call, with a missing value:

2,660 rows, 8 of them missing. Choose by the question. "Every event" wants the first. "Every call, with its events if it has any" wants the second.

The argument `errors="ignore"` matters. The last line of the block is the same call without it: a record that has no `meta` raises a `KeyError`. With it, the field is `NaN`.

Build: the events table and the summary

`events_table` does the flattening, names the columns and sorts the events of each call in time. `call_summary` keeps one row for each call id that you pass, so that a call with no events is a row with `n_events` 0:

```
import pandas as pd

from pdbook.traces import call_summary, events_table, read_traces

records = read_traces()
events = events_table(records)
summary = call_summary(events, [r["call_id"] for r in records])

print(events.shape, events.dtypes.astype(str).to_dict())
print(summary.shape, int(summary["established"].sum()), int((summary["n_events"] == 0).sum()))
print(summary.dropna().head(3).to_string())
print(summary["ring_ms"].describe().round(0).to_dict())
```

```
(2652, 7) {'call_id': 'int64', 'caller': 'str', 'cell': 'str', 'codec': 'str', 't_ms':
'int64', 'type': 'str', 'code': 'float64'}
(500, 5) 383 8
         n_events  ring_ms  answer_ms  final_code  established
call_id
13749           6    232.0     8752.0       200.0         True
19802           6    914.0     4375.0       200.0         True
384             6    955.0     3264.0       200.0         True
{'count': 430.0, 'mean': 935.0, 'std': 363.0, 'min': 155.0, '25%': 677.0, '50%': 928.0, '75%':
1193.0, 'max': 1753.0}
```

`code` is `float64` because the `INVITE` and `ACK` events have no code, which is a missing value (Chapter 5). 383 calls were established. The time to ringing has 430 values. The next check shows that they are the 383 answered calls and the 47 `noanswer` calls. Its median is 927.5 ms, which the table above rounds to 928.

Check the summary against the call records

The trace and the call record describe the same calls, so they must agree. An established call, one with a `200` response, must be `answered`:

```
import pandas as pd

from pdbook.io import load_calls
from pdbook.traces import call_summary, events_table, read_traces

records = read_traces()
summary = call_summary(events_table(records), [r["call_id"] for r in records])
status = load_calls().set_index("call_id").loc[summary.index, "status"]

print(pd.crosstab(status, summary["established"]).to_string())
print(bool((status[summary["established"]] == "answered").all()))
print(pd.crosstab(status, summary["ring_ms"].notna()).to_string())
empty = summary[summary["n_events"] == 0].index
print(status.loc[empty].value_counts().to_dict())
```

```
established  False  True
status
answered         7    383
busy            43      0
failed          20      0
noanswer        47      0
True
ring_ms   False  True
status
answered      7    383
busy         43      0
failed       20      0
noanswer      0     47
{'answered': 7, 'busy': 1}
```

All 383 established calls are `answered`, and no call of another status is established. The other side is a finding: 7 calls are `answered` and have no `200` in the trace. All 7 are among the 8 traces with an empty list of events.

The missing `200` is a missing trace and not evidence about the call. The 8th empty trace is a `busy` call. A check in one direction found no fault. The other direction found 7 records to send to the owner of the trace.

How it works

`json_normalize` is a recursive walk over dicts. For each record it visits the keys. A value that is a dict is entered, and its keys are joined to the path with the separator (`meta` and `cell` give `meta.cell`). Any other value becomes a cell.

With `record_path`, the walk goes into that list. It makes one row for each element, and it copies the fields in `meta` onto each of those rows. The result is the same as nested Python loops, and you can write them:

```
import pandas as pd

from pdbook.traces import read_traces

records = read_traces()

rows = []
for record in records:
    for event in record["events"]:
        meta = record.get("meta") or {}
        rows.append({"t_ms": event["t_ms"], "type": event["type"], "code": event.get("code"),
                     "call_id": record["call_id"], "meta.cell": meta.get("cell")})
by_hand = pd.DataFrame(rows)

by_pandas = pd.json_normalize(records, record_path="events", meta=["call_id", ["meta",
"cell"]], errors="ignore")
print(by_hand.shape, by_pandas.shape)
print(bool((by_hand["t_ms"].to_numpy() == by_pandas["t_ms"].to_numpy()).all()))
print(by_hand["meta.cell"].isna().sum(), by_pandas["meta.cell"].isna().sum())
print(bool((by_hand["call_id"].to_numpy() == by_pandas["call_id"].to_numpy()).all()))
```

```
(2652, 5) (2652, 5)
True
149 149
True
```

The two loops and `json_normalize` give the same number of rows and the same times and call ids. A missing `meta` is a missing cell in both, with `get` in the loop and `errors="ignore"` in pandas.

- **Invariant.** The number of rows is the number of elements in the lists, summed over the records. A record with an empty list adds none.
- **Failure.** A key that is missing in some records. Without `errors="ignore"` it raises, and with it, it is `NaN`.
- **Cost.** A Python loop over every element, in both versions. `json_normalize` is not a compiled operation, so a very large file needs a streaming reader.

Reference

```
import pandas as pd

from pdbook.sigs import show_signature
from pdbook.traces import call_summary, events_table, read_traces

show_signature(pd.json_normalize, ["data", "record_path", "meta", "errors", "sep"])
show_signature(pd.read_json, ["path_or_buf", "orient", "lines", "dtype", "convert_dates"])
show_signature(pd.DataFrame.explode, ["column", "ignore_index"])
show_signature(read_traces)
show_signature(events_table)
show_signature(call_summary)
```

```
json_normalize(
    data: 'dict | list[dict] | Series'
    record_path: 'str | list | None' = None
    meta: 'str | list[str | list[str]] | None' = None
    errors: 'IgnoreRaise' = 'raise'
    sep: 'str' = '.'
)  # 3 more parameters
read_json(
    path_or_buf: 'FilePath | ReadBuffer[str] | ReadBuffer[bytes]'
    orient: 'str | None' = None
    dtype: 'DtypeArg | None' = None
    convert_dates: 'bool | list[str]' = True
    lines: 'bool' = False
)  # 13 more parameters
DataFrame.explode(
    column: 'IndexLabel'
    ignore_index: 'bool' = False
)  # 1 more parameters
read_traces(
    path: 'str | pathlib.Path | None' = None
)
events_table(
    records: 'list[dict]'
)
call_summary(
    events: 'pd.DataFrame'
    call_ids: 'list[int]'
)
```

Parameter	Default	What it does here	What goes wrong
`json_normalize(data)`	required	a list of dicts	—
`json_normalize(record_path=...)`	`None`	the path to a list: one row for each element	a record with an empty list gives no row
`json_normalize(meta=...)`	`None`	fields to copy onto each row; a list in a list is a path	—
`json_normalize(errors=...)`	`'raise'`	`'ignore'` gives `NaN` for a missing meta field	`'raise'` raises `KeyError` for the 23 records with no `meta`
`json_normalize(sep=...)`	`'.'`	the joiner of the path	—
`read_json(lines=...)`	`False`	`True` reads one JSON object for each line	`meta` and `events` stay as dicts and lists
`read_json(dtype=..., convert_dates=...)`	`None`, `True`	type inference and date conversion	—
`explode(column, ignore_index=...)`	required, `False`	one row for each element of a list column	an empty list gives one row of missing values: 2,660 rows against 2,652
`read_traces(path=...)`	`None`	the list of records in `data/traces.jsonl`	—
`events_table(records)`	required	one row for each event, sorted by call and time	calls with no events have no row
`call_summary(events, call_ids)`	required	one row for each id: events, ring time, answer time, last code	—

`events_table` returns a DataFrame with the columns `call_id`, `caller`, `cell`, `codec`, `t_ms`, `type` and `code`.

What you have now

- You read JSON lines, and you count the records that lack a part.
- You flatten dicts with `json_normalize`, and lists with `record_path` or `explode`.
- You know that `json_normalize` drops a call with an empty list and `explode` keeps it.
- You check a derived table against a table that you trust, in both directions.

The next chapter uses the tables of this book to build features without leaking the future.

Next: Chapter 23, Features and evaluation.

23. Features and evaluation

> **Build** — `past_features` with `assert_no_leak`: features for each subscriber that use only calls before a cutoff, and a test that proves it by changing the future. `compare_configs`: a paired comparison of two configurations on the same tasks, with a bootstrap interval, written as a table of results.
>
> **Theory kept** — A feature must be computable at the moment of the decision. A group statistic over the whole table uses the future. Test it by changing the future and checking that the features do not move. Compare two systems on the same tasks, and make the task, not the run, the unit.
>
> **Source** — New. Data: `data/runs.jsonl`. Libraries: `pdbook/features.py` and `pdbook/evals.py`.
>
> **Run this first** — `uv run python -m examples.ch23_features`, which is `examples/ch23_features.py`. Exercises: `exercises/23.md`. Solutions: `exercises/solutions/23.md`.

The question and the cutoff

The question: how many calls will each subscriber make from 22 September on, given what we know before? The cutoff is local midnight at the start of 22 September. Everything before it is the past, and everything from it on is the future:

```
import pandas as pd

from pdbook.features import TZ, past_features, target
from pdbook.io import load_calls, load_subscribers

calls, subs = load_calls(), load_subscribers()["subscriber_id"]
cutoff = "2026-09-22"
limit = pd.Timestamp(cutoff, tz=TZ)
print(limit, "=", limit.tz_convert("UTC"))
print(int((calls["start"] < limit).sum()), int((calls["start"] >= limit).sum()))

X = past_features(calls, subs, cutoff)
y = target(calls, subs, cutoff)
print(X.shape, int(y.sum()))
print(X.describe().round(2).loc[["mean", "min", "max"]].to_string())
```

```
2026-09-22 00:00:00+02:00 = 2026-09-21 22:00:00+00:00
13905 6095
(600, 5) 6095
       calls  answered_share  talk_min  active_days  days_since_last
mean   23.18            0.77     39.40        11.72             1.35
min     3.00            0.43      0.90         2.00             0.01
max   450.00            1.00    731.02        21.00            13.73
```

The cutoff in UTC is 22:00 on 21 September, because Brussels is two hours ahead. A cutoff at UTC midnight is two hours off. Calls between 22:00 and 24:00 then fall on the wrong side. The past has 13,905 calls and the future 6,095.

The feature table has one row for each of the 600 subscribers. Every subscriber has at least 3 calls in the past, so there are no missing features here. In another table a subscriber with no past calls has a missing `answered_share`, and that must be a decision and not an accident.

The easy feature leaks

The shortest code for "calls of each subscriber" is a `groupby` over the whole table. It includes the future, which is the thing that we want to predict:

```
import numpy as np
import pandas as pd

from pdbook.features import leaky_features, past_features, target
from pdbook.io import load_calls, load_subscribers

calls, subs = load_calls(), load_subscribers()["subscriber_id"]
cutoff = "2026-09-22"
X, y, L = past_features(calls, subs, cutoff), target(calls, subs, cutoff),
leaky_features(calls, subs)


def score(x: pd.Series) -> tuple[float, float]:
    prediction = x * (y.sum() / x.sum())
    return float(np.corrcoef(x, y)[0, 1]), float((prediction - y).abs().mean())


for name, feature in [("past features", X["calls"]), ("leaky feature", L["calls"])]:
    corr, mae = score(feature)
    print(f"{name:14s} correlation {corr:.3f}  mean absolute error {mae:.2f} calls")
```

```
past features  correlation 0.955  mean absolute error 2.87 calls
leaky feature  correlation 0.978  mean absolute error 2.00 calls
```

The leaky feature looks better: a higher correlation and a lower error. It is better because it contains the answer. A person who reports the second line reports a result that nobody can reproduce when the model runs for real.

The future is not there on that day. The difference here is small, 2.87 against 2.00 calls, because most of the signal is the difference between heavy and light subscribers. A leak can be small, and it is still wrong.

Test for the leak

Reading the code is not a test. The test is a property. A feature that uses only the past does not change when the future changes. `assert_no_leak` changes the future in two ways and compares the features before and after:

```
import pandas as pd

from pdbook.features import assert_no_leak, leaky_features, past_features
from pdbook.io import load_calls, load_subscribers

calls, subs = load_calls(), load_subscribers()["subscriber_id"]
cutoff = "2026-09-22"

assert_no_leak(lambda c, s: past_features(c, s, cutoff), calls, subs, cutoff)
print("past_features: no change when the future changes")

try:
    assert_no_leak(leaky_features, calls, subs, cutoff)
except AssertionError as error:
    print("leaky_features:", error)
```

```
past_features: no change when the future changes
leaky_features: features change when the future changes (future removed)
```

The function that uses the past passes. The function that groups over the whole table fails, and the message names the change that exposed it. The two changes are: remove the future, and give every future call to one subscriber.

My first version of this test shuffled the callers of the future, and it did not find the leak. A shuffle keeps the number of calls of each caller, so a count does not move. A test of a property must change what the feature reads.

Compare two configurations

The file `data/runs.jsonl` holds the results of an evaluation: 150 tasks, run with two configurations, `baseline` and `improved`, with 3 seeds each. Each line has a nested `tokens` object, and `load_runs` flattens it:

```
import numpy as np
import pandas as pd

from pdbook.evals import load_runs

runs = load_runs()
print(runs.shape, runs.columns.tolist())
print(runs.groupby("config")["passed"].agg(["mean", "size"]).round(4).to_dict("index"))

per_task = runs.groupby(["task_id", "config"])["passed"].mean().unstack()
diff = per_task["improved"] - per_task["baseline"]
print(len(diff), round(float(diff.mean()), 4))
print(int((diff > 0).sum()), int((diff < 0).sum()), int((diff == 0).sum()))
```

```
(900, 8) ['task_id', 'category', 'config', 'seed', 'passed', 'latency_ms', 'tokens.prompt',
'tokens.completion']
{'baseline': {'mean': 0.5756, 'size': 450}, 'improved': {'mean': 0.64, 'size': 450}}
150 0.0644
67 42 41
```

The pass rate is 0.576 for `baseline` and 0.640 for `improved`, from 450 runs each: a difference of 0.064. The second part is the paired table: for each task, the mean over its three seeds for each configuration. 67 tasks are better with `improved`, 42 are worse and 41 are equal. The mean of the 150 differences is the same 0.064.

A claim that one configuration is better needs an interval. `compare_configs` resamples the *tasks* with replacement 2,000 times, and takes the mean difference of each resample:

```
import pandas as pd

from pdbook.evals import compare_configs, load_runs

runs = load_runs()
r = compare_configs(runs)
for key in ["tasks", "rate_a", "rate_b", "diff", "paired_se", "unpaired_se", "ci_low",
"ci_high"]:
    print(f"{key:12s} {r[key]:.4f}")

by_category = {}
for category, part in runs.groupby("category"):
    c = compare_configs(part)
    by_category[category] = (round(c["diff"], 3), round(c["paired_se"], 3), int(c["tasks"]))
print(by_category)
```

```
tasks        150.0000
rate_a       0.5756
rate_b       0.6400
diff         0.0644
paired_se    0.0308
unpaired_se  0.0325
ci_low       0.0022
ci_high      0.1244
{'billing': (-0.022, 0.07, 30), 'device': (0.078, 0.071, 30), 'outage': (0.022, 0.06, 30),
'roaming': (0.144, 0.069, 30), 'routing': (0.1, 0.075, 30)}
```

The interval of the difference is 0.002 to 0.124. The lower end is close to zero, so the data are compatible with a very small effect and with an effect of twelve points. The paired standard error is 0.031 and the unpaired one is 0.033, so pairing helped by about 5% here. The reason that it helps little is in the construction of these data. A run passes with a probability between about 0.2 and 0.9. The result of one run is mostly chance, and the difficulty of the task is a small part of it.

The breakdown by category is the trap of the table. Each category has 30 tasks, and the standard error of its difference is between 0.06 and 0.075. Four of the five differences are inside two standard errors of zero. `roaming` is 0.144, which is 2.1 standard errors.

Five categories give five chances. Suppose that no category has a real effect. The chance that at least one of five is beyond 1.96 standard errors is $1 - 0.95^5$, which is 23%. A report that says "the improvement is in `roaming`" can be a report of noise. Report the interval for the whole set, and name the number of tasks behind every cell.

Build: the results as a table

The numbers that go in a report are the ones in `compare_configs`. A report that shows a difference without its interval, or without the number of tasks, is not finished:

```
import pandas as pd

from pdbook.evals import compare_configs, load_runs

r = compare_configs(load_runs())
table = pd.DataFrame({
    "baseline": [r["rate_a"]],
    "improved": [r["rate_b"]],
    "difference": [r["diff"]],
    "95% low": [r["ci_low"]],
    "95% high": [r["ci_high"]],
    "tasks": [int(r["tasks"])],
    "better": [int(r["tasks_better"])],
    "worse": [int(r["tasks_worse"])],
}).round(3)
print(table.to_string(index=False))
```

```
 baseline  improved  difference  95% low  95% high  tasks  better  worse
    0.576      0.64       0.064    0.002     0.124    150      67     42
```

How it works

The bootstrap is a way to ask "how much would the mean change with other tasks?" without a formula. It draws tasks with replacement from the tasks that you have, and makes many new data sets of the same size. It takes the mean difference of each, and it reads the spread of those means. The 95% interval is the 2.5th and the 97.5th percentile. The unit of the draw is the task, because the three seeds of a task are not three independent tasks.

You can run it by hand with another seed, to see how much the interval moves with the draw itself:

```
import numpy as np

from pdbook.evals import compare_configs, load_runs

runs = load_runs()
per_task = runs.groupby(["task_id", "config"])["passed"].mean().unstack()
diff = (per_task["improved"] - per_task["baseline"]).to_numpy()

ends = []
for seed in range(5):
    rng = np.random.default_rng(100 + seed)
    means = np.array([rng.choice(diff, size=len(diff), replace=True).mean() for _ in
range(2000)])
    ends.append(np.percentile(means, [2.5, 97.5]))
ends = np.array(ends)
print(ends.round(3).tolist())
print(round(float(ends[:, 0].max() - ends[:, 0].min()), 3), round(float(ends[:, 1].max() -
ends[:, 1].min()), 3))
print(round(float(diff.mean() - 1.96 * diff.std(ddof=1) / np.sqrt(len(diff))), 3),
      round(float(diff.mean() + 1.96 * diff.std(ddof=1) / np.sqrt(len(diff))), 3))
```

```
[[0.004, 0.122], [0.004, 0.122], [0.004, 0.124], [0.0, 0.124], [0.007, 0.124]]
0.007 0.002
0.004 0.125
```

Five seeds give five intervals. The lower ends are 0.000 to 0.007. The upper ends are 0.122 to 0.124. The second decimal is stable, and the third is noise. The normal-theory interval in the last line, the mean plus and minus 1.96 standard errors, agrees with the bootstrap to the second decimal.

- **Invariant.** The bootstrap interval is a statement about the tasks that you have. It says how the mean difference varies when you draw other tasks like them.
- **Failure.** The unit of the draw is wrong: the runs, and not the tasks. The interval is then too narrow if runs of one task are alike.
- **Cost.** 2,000 resamples of 150 numbers each, which is a fraction of a second. The cost is small, so use enough resamples that the ends do not move in the second decimal.

Reference

```
import pandas as pd

from pdbook.evals import compare_configs, load_runs
from pdbook.features import assert_no_leak, leaky_features, past_features, target
from pdbook.sigs import show_signature

show_signature(pd.json_normalize, ["data", "sep"])
show_signature(pd.DataFrame.groupby, ["by", "observed"])
show_signature(past_features)
show_signature(target)
show_signature(leaky_features)
show_signature(assert_no_leak)
show_signature(load_runs)
show_signature(compare_configs)
```

```
json_normalize(
    data: 'dict | list[dict] | Series'
    sep: 'str' = '.'
)  # 6 more parameters
DataFrame.groupby(
    by=None
    observed: 'bool' = True
)  # 6 more parameters
past_features(
    calls: 'pd.DataFrame'
    subscribers: 'pd.Series'
    cutoff: 'str'
)
target(
    calls: 'pd.DataFrame'
    subscribers: 'pd.Series'
    cutoff: 'str'
)
leaky_features(
    calls: 'pd.DataFrame'
    subscribers: 'pd.Series'
)
assert_no_leak(
    feature_fn
    calls: 'pd.DataFrame'
    subscribers: 'pd.Series'
    cutoff: 'str'
)
load_runs(
    path: 'str | pathlib.Path | None' = None
)
compare_configs(
    runs: 'pd.DataFrame'
    a: 'str' = 'baseline'
    b: 'str' = 'improved'
    n_boot: 'int' = 2000
    seed: 'int' = 0
)
```

Parameter	Default	What it does here	What goes wrong
`past_features(calls, subscribers, cutoff)`	required	one row for each subscriber id, from the calls before the cutoff in `Europe/Brussels`	a UTC midnight puts calls from 22:00 to 24:00 on the wrong side
`target(calls, subscribers, cutoff)`	required	calls from the cutoff on, 0 for a subscriber with none	—
`leaky_features(calls, subscribers)`	required	the same count over the whole table	fails `assert_no_leak`
`assert_no_leak(feature_fn, calls, subscribers, cutoff)`	required	raises `AssertionError` when the features move if the future changes	a shuffle of the future callers does not move a count
`load_runs(path=...)`	`None`	the runs, with `tokens.prompt` and `tokens.completion` as columns	—
`compare_configs(runs, a, b, n_boot, seed)`	`'baseline'`, `'improved'`, `2000`, `0`	a dict: rates, paired difference, standard errors, interval, tasks better and worse	the interval ends move by less than 0.01 with the seed

`compare_configs` returns a dict of floats. Its interval is for the mean over tasks of the per-task difference.

What you have now

- You split by time, in the zone of the business, and build features from the past only.
- You test a feature for leakage by changing the future, and you know a shuffle is not enough.
- You compare two configurations on the same tasks, with the task as the unit and an interval.
- You report the number of tasks next to every cell, and you do not read a breakdown that has 30 tasks.

The last chapter collects what the book measured into one field guide.

Next: Chapter 24, Field guide.

24. Field guide

> **Build** — One page of lookups for the whole book: which idiom answers which question, which behavior is new in pandas 3.0, and which first-draft claims of this book the machine contradicted, each one reproduced by a block that runs.
>
> **Theory kept** — Most pandas errors are not syntax errors. They are a result that looks right. The defense is a habit: name the question, state what must add up, and test the result against a second computation.
>
> **Source** — The measurements of Chapters 1 to 23. Library: all of `pdbook/`.
>
> **Run this first** — `uv run python scripts/check_snippets.py`, which runs every block of the book. For this chapter alone: `uv run python -m examples.ch24_field_guide`, which is `examples/ch24_field_guide.py`. Exercises: `exercises/24.md`. Solutions: `exercises/solutions/24.md`.

From question to idiom

You want to	Use	Check that	Chapter
read a file with the right types	`read_csv(dtype=..., parse_dates=...)`, one loader function	identifiers are text, timestamps have a zone	2
keep a table small	`category`, small integers, `usecols`	`same_content` is `True`	4
know why a value is missing	`null_report`, `determinants`	`groupby` totals add up to `len(df)`	5
clean text and dates	`.str`, `to_datetime(format="mixed")`	the rejects are listed, not guessed	6
add a rule as a column	`where`, `np.select`, `map`	no row loop, `isna().sum()` after a `map`	7
one row for each key	`groupby(...).agg(name=(col, func))`	the sum of the groups equals the table	8
add columns from a lookup	`merge(how="left", validate="m:1")`	the row count did not change	9
reshape	`pivot_table`, `melt`, `unstack`	the total of the cells equals the input	10
a day or an hour in local time	`tz_convert`, then `dt`	the zone is in the label	12
a moving statistic	`rolling("1h", closed="left")`	the count of samples is kept	13
sessions, ranks, running totals	sort, mark, `cumsum`	the result does not depend on row order	14
a readable pipeline	method chain, `pipe`, `require`	a shuffled input gives the same report	16
trust an input	rules, `validate`, `plant_faults`	the validator finds the planted counts	17
speed	column operations, one `concat`	time on the real size, and check equality	18
a file larger than memory	`usecols`, filters, `chunksize`	chunked equals whole	19
a database	`to_sql`, parameters, one query	the SQL report equals the pandas report	20
nested JSON	`json_normalize`, `explode`	records in, rows out, in both directions	22
features for a model	cutoff, past only, `assert_no_leak`	the future can change and the features do not	23

What changed in pandas 3.0

This book ran every line on pandas 3.0.6. A change that the book only describes is in Appendix A.

Behavior in 3.0	Where it shows	Chapter
text columns have the dtype `str`, stored with Arrow	`calls["status"].dtype` is `str`	1, 4
timestamps parse to microseconds (`datetime64[us]`), nanoseconds only when the text needs them	`to_datetime` on three strings	2
copy-on-write: a derived object is a copy, chained assignment does nothing and warns	`ChainedAssignmentError`	3, 15
`.str` methods on `str` return `False` for a missing value, a `bool` mask	`startswith` on `cell_id`	3
`groupby(observed=True)` is the default for categories	an empty category has no row	4
`to_json` warns about the default epoch dates with `Pandas4Warning`	`date_format="iso"`	2
the option `mode.copy_on_write` is deprecated: copy-on-write is always on	reading it warns	15
the first parameter of `Series.map` is named `func`	printed signature	7

What went wrong while writing this book

A book that shows only its results is an advertisement. Each row below is a claim that I wrote down before I ran the code, and what the code printed. The block reproduces eight of them in one run, so that they stay true when the library changes:

```
import io

import numpy as np
import pandas as pd

rows = []

calls = pd.read_csv("data/calls.csv")
rows.append(("ch1", "callee is text", f"callee is {calls['callee'].dtype}"))

try:
    pd.DataFrame({"k": [1, 2]}).merge(pd.DataFrame({"k": ["1", "2"]}), on="k")
except ValueError:
    rows.append(("ch2", "int and text keys match with no rows", "ValueError"))

try:
    pd.Series(["a"]) == "a" & pd.Series([1.0]) > 0
except Exception as error:
    rows.append(("ch3", "missing parentheses give ValueError", type(error).__name__))

missing = pd.Series(["BRU-001", None]).str.startswith("BRU")
rows.append(("ch3", "startswith keeps NaN for a missing value", f"{missing.dtype},
{missing.tolist()}"))

a = pd.DataFrame({"k": pd.Categorical(["x", "y"])})
b = pd.DataFrame({"k": pd.Categorical(["y", "z"])})
rows.append(("ch4", "concat of two categoricals is a category", pd.concat([a, b])
["k"].dtype.name))

rows.append(("ch6", "two missing keys are not duplicates", str(pd.Series([None,
None]).duplicated().tolist())))

try:
    pd.MultiIndex.from_tuples([(1, "a")]).isna()
except NotImplementedError:
    rows.append(("ch11", "MultiIndex has isna()", "NotImplementedError"))

x, y = pd.DataFrame({"v": [1.0]}), pd.DataFrame({"v": [1.0 + 1e-9]})
try:
    pd.testing.assert_frame_equal(x, y, rtol=0)
    rows.append(("ch17", "rtol=0 rejects a 1e-9 difference", "accepted"))
except AssertionError:
    rows.append(("ch17", "rtol=0 rejects a 1e-9 difference", "rejected"))

print("pandas", pd.__version__, "| numpy", np.__version__)
for chapter, claim, printed in rows:
    print(f"{chapter:5s} {claim:44s} -> {printed}")
```

```
pandas 3.0.6 | numpy 2.5.4
ch1   callee is text                              -> callee is int64
ch2   int and text keys match with no rows        -> ValueError
ch3   missing parentheses give ValueError         -> TypeError
ch3   startswith keeps NaN for a missing value    -> bool, [True, False]
ch4   concat of two categoricals is a category    -> str
ch6   two missing keys are not duplicates         -> [False, True]
ch11  MultiIndex has isna()                       -> NotImplementedError
ch17  rtol=0 rejects a 1e-9 difference            -> accepted
```

Read each line as "I wrote the first half, and the machine printed the second". Eight lines in one run, and none of the eight was what I expected. The full list of the corrections that the book needed is longer, and it has the cause for each:

Chapter	First draft said	The machine said	Cause
5	the lost cells are independent of everything	the loss rate is 0.48% for `EVS` and 2.09% for `AMR-NB`	the generator picks the codec from the cell, and a missing cell takes the 4G branch
6	`N/A` stays a string with `dtype=str`	`N/A` and the empty string are `NaN` after the read, only `-` stays text	the default list of missing markers applies to text columns
6	a bad regex raises `re.error`	`ArrowInvalid`	Arrow strings use another regular expression engine
16	a plan missing from the lookup stops the report	the report runs and the revenue is 3,681.87 instead of 4,245.09	`groupby().sum()` skips `NaN`
17	`rtol=0` makes the comparison exact	a difference of 1e-9 passes	`atol=1e-8` is a second tolerance
18	a category column is faster in one step	the conversion costs more than one use saves	the conversion is in the timed function
19	the peak memory of a child is its own	it includes the parent at the fork	`ru_maxrss` is inherited; use `VmHWM`
19	the chunked count is an integer	`float64` when a caller is missing in a chunk	`add(fill_value=0)` upcasts
19	`17 13 MB` in a block	`21 16 MB`	the numbers were typed, not printed
20	this SQLite has no `CEIL`	it has `ceil`	the limit was assumed and never tried
23	shuffling the future finds a count leak	it passes	a shuffle keeps each caller's count
23	450 tasks with 1 seed has a smaller standard error	0.0302 against 0.0308: the same	the noise is in the runs, and three seeds cancel it

The pattern is the same in every row. The error was in a sentence that I wrote before I ran something, or in a number that I carried from one run to another. The checks that the book ships catch each of them.

How it works

This chapter has no new function. Its mechanism is a lookup. Each row of the first table names a chapter. The chapter must exist, and it must contain the function or the behavior that the row claims. The block checks the first part for every chapter link in this file:

```
import pathlib
import re

root = pathlib.Path(".")
text = (root / "chapters" / "24-field-guide.md").read_text(encoding="utf-8")
links = sorted(set(re.findall(r"\]\((\d\d-[a-z0-9-]+\.md)\)", text)))
missing = [name for name in links if not (root / "chapters" / name).exists()]
print(len(links), missing)

numbers = sorted({int(n[:2]) for n in links})
print(numbers[0], numbers[-1], len(numbers))
```

```
21 []
1 23 21
```

- **Invariant.** Every chapter in the tables exists, and the numbers cover Chapters 1 to 23 except those that have no row of their own.
- **Failure.** A renamed chapter file leaves a dead link. The checker of the book resolves every link and fails on it.
- **Cost.** Nothing at run time. The cost is the discipline of putting the check in the table and not in a comment.

Reference

This chapter has no API of its own. The functions of the book are listed here, by module, from the code, so that you can find them by name:

```
import importlib
import inspect
import pkgutil

import pdbook

for info in sorted(pkgutil.iter_modules(pdbook.__path__), key=lambda i: i.name):
    if info.name in {"datasets", "sigs"}:
        continue
    module = importlib.import_module(f"pdbook.{info.name}")
    names = [n for n, f in inspect.getmembers(module, inspect.isfunction)
             if f.__module__ == module.__name__ and not n.startswith("_")]
    print(f"pdbook.{info.name}: {', '.join(names)}")
```

```
pdbook.bench: best_of, replicate, task_category, task_groupby_apply, task_iterrows,
task_itertuples, task_mask_groupby
pdbook.chunked: make_big_csv, peak_rss_mb, talk_by_caller_chunked, talk_by_caller_whole
pdbook.clean: clean_contacts, e164, money, name_key
pdbook.enrich: enrich
pdbook.evals: compare_configs, load_runs
pdbook.features: assert_no_leak, leaky_features, past_features, target
pdbook.indexcheck: index_audit
pdbook.io: load_calls, load_cells, load_kpi, load_plans, load_subscribers
pdbook.kpi: hourly_kpi
pdbook.looks: first_look, headline
pdbook.nulls: determinants, null_report
pdbook.plots: plot_duration_hist, plot_hourly_calls, plot_kpi_spike, plot_traffic_grid
pdbook.rating: rate_apply, rate_loop, rate_vector
pdbook.report: monthly_report, require
pdbook.sessions: session_table, sessionize
pdbook.shrink: memory_report, same_content, shrink
pdbook.spikes: flag_spikes
pdbook.sqlbridge: build_database, monthly_report_sql
pdbook.summary: subscriber_summary
pdbook.traces: call_summary, events_table, read_traces
pdbook.traffic: to_long, to_wide, traffic_grid
pdbook.traps: live
pdbook.validate: assert_valid, call_rules, check_schema, plant_faults, validate
```

The helpers `pdbook.datasets` (the generator of the data) and `pdbook.sigs` (the signature printer) are left out of the list. The parameters of each function are in the Reference section of the chapter that builds it.

What you have now

- You can find the idiom for a question, and the check that goes with it.
- You know which behaviors are new in pandas 3.0, and which of them this book ran.
- You have the list of what went wrong while this book was written, and the reason in each case.
- You have a method: write the claim, run it, and let the machine correct the sentence.

This is the last chapter. The appendices list what the book describes and does not run, and what Python already gives you.

Next: Appendix A, Described, not built.

Appendix A. Described, not built

Everything in this book that a block of code did not measure is listed here. Each item is a statement that the book makes or leans on, and the reason that no block tests it. Read these as the documentation says, not as measured. Where an item says "checked", the book printed the evidence in the chapter that is named.

Packages that this book does not install

The chapters mention four optional packages. Chapter 2 prints that none of them is installed.

Package	Where it is mentioned	What the book says, from the documentation
`openpyxl`	Chapter 2	pandas reads and writes `.xlsx` files with it. The book runs no Excel code.
`numexpr`	Chapter 18	`query` and `eval` can use it. The book makes no claim about the gain.
`polars`	Chapter 19	a tool for data that does not fit in memory. Not tested.
`duckdb`	Chapter 19	the same. Not tested.

Behavior of other versions of pandas

The book was written on pandas 3.0.6 only. Statements about earlier versions are history, taken from the documentation and from the changes that 3.0 announced.

Statement	Chapter
In pandas 2.x, a column of text had the dtype `object` by default.	1, 2, 3, 4
In pandas 2.x, timestamps were stored with nanosecond resolution.	2
In pandas 2.x, `.str` methods on an `object` column returned a missing value for a missing element. Chapter 3 shows this on an `object` column in 3.0, and does not run 2.x.	3
Before 3.0, a selection was sometimes a view and sometimes a copy, and `SettingWithCopyWarning` tried to tell you.	15
In 2.x, `observed` of `groupby` on categories was `False` by default.	4

Other systems

Statement	Chapter	Why it is not measured
SQLite has math functions such as `CEIL` from version 3.35, and a build can be made without them.	20	The book prints the version that ran, 3.50.4, and the query ran. Other builds are not tested.
An index is a B-tree in SQLite.	20	The book shows the query plan before and after (`SCAN` and `SEARCH ... USING INDEX`), not the structure.
`ru_maxrss` of a child process includes the parent's memory at the fork.	19	While writing, the baseline of the first method moved between runs, and the method was changed. The mechanism is the documented behavior of Linux.
`VmHWM` in `/proc/self/status` is the high-water mark of the new process.	19	Linux only. The three baselines of the new method agreed within 1 MB.

What the data are

The data are synthetic and made by `pdbook/datasets.py`. Two consequences:

- The statements about what a real network does (retries, bursts, roaming) are examples of what a real table can contain, and not findings. The data of this book have no bursts.
- A result that depends on the generator is marked as such where it appears: the loss of the cell identifier (Chapter 5) and the pass rates of the evaluation (Chapter 23).

Time

Timings depend on the machine and on its load. The blocks that print a time have `# output varies` above the output, and an assertion that checks an order of magnitude. The chapters quote only what the assertion proves. The values printed in the book come from one run on one machine.

Appendix B. What Python already gives you

Pandas is not the only tool in this book. These pieces of the standard library did work in the chapters, and each of them is the right tool for one kind of problem.

Need	Tool	Where it was used
a database with no server	`sqlite3`	Chapter 20
the signature of a function	`inspect.signature`	every Reference section, through `pdbook/sigs.py`
a time, with a monotonic clock	`time.perf_counter`	Chapter 18
a temporary file or folder	`tempfile`	Chapters 2, 19, 20
a child process for a clean measure	`subprocess`	Chapter 19
a warning that you want to see, not print	`warnings.catch_warnings(record=True)`	Chapters 2, 3, 15
JSON, one object for each line	`json.loads`	Chapter 22
a record type with named fields	`dataclasses`	Chapters 15, 17
checking that a package is installed	`importlib.util.find_spec`	Chapter 2

A rule for choosing: when the question is about tables of data, use pandas. When the question is about the program (a process, a file, a clock, a warning), use the standard library. Its tools are older and smaller.

Exercises for Chapter 1

Chapter: Series and DataFrame. Code: `examples/ch01_first_look.py`. Solutions: `solutions/01.md`.

1. [measure] Where the nulls are

The column `duration_s` in `data/calls.csv` has null values.

Count the null values. Then find which values of `status` the rows with a null duration have.

Expected: 1,033 nulls. All of them have one status. State which. Your count must match exactly.

2. [break] Labels against positions

Make two Series, `s` and `t`, with the same three values but with the index of `t` in reverse order. Add them with pandas. Add the same two arrays with NumPy.

Change one thing about `t` so that the two results are equal, and explain what you changed.

Expected: the pandas result is `[4, 4, 4]` and the NumPy result is `[2, 4, 6]` for `s = [1, 2, 3]` over `a, b, c` and `t = [1, 2, 3]` over `c, b, a`.

3. [judge] What should first_look show?

`first_look` prints the first value of each column. A table can have 400 columns and a few million rows.

Decide which three columns to add to `first_look` for such a table, and which one to remove. Say what question each column answers. A good answer weighs three points:

- the cost of the computation.
- the width of the output.
- the risk of a misleading summary.

Exercises for Chapter 2

Chapter: Input and output. Code: `examples/ch02_input_output.py`. Solutions: `solutions/02.md`.

1. [measure] The subscriber numbers

Read `data/subscribers.csv` with the default settings of `read_csv`. Report the dtype of `msisdn`. Then read it again with `dtype={"msisdn": "str"}`.

Expected: the first dtype is `int64`. In the second read, all 600 values have the length 12 (the characters `+324` and eight digits).

2. [break] A column that parquet refuses

Make a DataFrame with one column that holds the values `1`, `"x"` and `3.5`. Write it to parquet. The write fails.

Find the error class and change one thing so that the write succeeds. State what you lose.

Expected: the error class is `ArrowInvalid`. After your change, the values are all strings: `'1'`, `'x'` and `'3.5'`.

3. [judge] The nightly handover

Another team receives a nightly extract of the call records. They use a spreadsheet and a database loader. They ask for CSV. You want parquet.

Decide which format you send, or whether you send both. A good answer weighs three points:

- who pays when a type is wrong.
- who can open the file.
- how you find out that the receiving side parsed it incorrectly.

Exercises for Chapter 3

Chapter: Selection. Code: `examples/ch03_selection.py`. Solutions: `solutions/03.md`.

1. [measure] Both ends of a range

Count the calls with a `duration_s` from 60 to 120 seconds. Do it three times: with `between(60, 120)`, with `>= 60` and `< 120`, and with `> 60` and `< 120`.

Expected: 4,226, 4,179 and 4,072. Your counts must match exactly. Say what the differences are.

2. [break] The missing row

Make `calls` with the index `call_id` (`set_index("call_id")`). Select the calls `4` to `10` with `.loc[4:10]`. The call with `call_id` 7 is in the table, and it is not in the result.

Find out why, and give two ways to get all seven rows.

Expected: `.loc[4:10]` returns 6 rows (4, 5, 6, 8, 9, 10). A correct selection returns 7.

3. [judge] Masks or query

A team writes the filters for a nightly job. One engineer wants `query` strings everywhere because they read like SQL. Another wants masks everywhere because the editor checks the names.

Decide which rule the team adopts. A good answer weighs three points:

- readability.
- what a typo costs.
- who maintains the code a year from now.

Exercises for Chapter 4

Chapter: Types and memory. Code: `examples/ch04_types_memory.py`. Solutions: `solutions/04.md`.

1. [measure] The saving, column by column

Load the calls with `load_calls()`. Shrink them with `shrink(calls, float_to_int=True)`. Use `memory_report` and report the sum of the bytes of the columns, before and after.

Then find the column that gives the largest saving in bytes.

Expected: 2,017,566 bytes before and 788,158 after (39%). Your numbers must match exactly. The largest saving is in one of the text columns: say which.

2. [break] A type that is too small

Convert `call_id` to `int8` with `astype("int8")`. Count the rows where the new value differs from the old value.

Explain the result with arithmetic, and give the conversion that is safe.

Expected: 19,873 rows differ. Only the identifiers 1 to 127 survive.

3. [judge] Where to shrink

A nightly pipeline loads 40 million call records, joins them with subscribers and writes a report. You can call `shrink` after the load, or after the join, or never.

Decide where it goes. A good answer weighs three points:

- the peak memory.
- the effect on the join.
- the risk of a wrong type that the next stage does not expect.

Exercises for Chapter 5

Chapter: Missing data. Code: `examples/ch05_missing.py`. Solutions: `solutions/05.md`.

1. [measure] Is the loss random?

Compute the share of calls with a null `cell_id` among the answered calls, and among all calls. Then compute it for each `codec` value (the nulls of `codec` are left out of the groups).

Expected: 1.37% among answered calls and 1.38% among all calls (to two decimals). The shares for the three codecs are 2.09% (`AMR-NB`), 1.61% (`AMR-WB`) and 0.48% (`EVS`). Say what the codec shares tell you about the statement "the cells are lost at random".

2. [break] A determinant that is not a cause

Make a copy of the calls in which `cell_id` is null for exactly the roaming calls and has a value for every other call. Run `determinants` on it.

State what `determinants` now reports for `cell_id`, and explain why the report is correct and still does not tell you whether the data are lost.

Expected: `determinants` reports `['roaming']` for `cell_id`.

3. [judge] The outage in the KPI table

The cell `BRU-004` has no KPI rows for two hours on 25 October. A dashboard shows `prb_util_pct` for each cell as a line. An operations manager asks you to fill the hole, because the gap in the line looks like a mistake.

Decide what you put in the hole, if anything. A good answer weighs three points:

- what the manager does with the picture.
- what a filled value claims.
- who else reads the series.

Exercises for Chapter 6

Chapter: Text and date cleanup. Code: `examples/ch06_cleanup.py`. Solutions: `solutions/06.md`.

1. [measure] How many formats?

Count the rows of `data/contacts_messy.csv` that match each of the four date formats: `2024-03-01`, `01/03/2024`, `March 01, 2024` and `01-Mar-24`. Use `str.match` with one regular expression for each. Then count the rows that match none.

Expected: the counts are 78, 66, 67 and 59, and 1 row matches none. The row with `31/02/2024` matches the second pattern, so it is inside the 66.

2. [break] The wrong key

Take the rows of the cleaned contacts that have a phone. Remove duplicates with `name_key` as the key, instead of the phone. Count the rows that remain. Then count how many removed rows have a phone that is different from the phone of every other row.

Expected: 244 rows have a phone, and 170 remain when the name is the key. So 74 rows are removed. Each removed row has a phone that no other row has, so each one is a different subscriber.

3. [judge] Reject or repair

A person repairs the balance `twelve euros` in one second. A function can repair it with a table of number words. Your cleaner rejects it.

Decide whether the cleaner must convert number words. A good answer weighs three points:

- the cost of a wrong repair.
- how often this input arrives.
- the effect on the person who reads the rejects.

Exercises for Chapter 7

Chapter: New columns. Code: `examples/ch07_new_columns.py`. Solutions: `solutions/07.md`.

1. [measure] Speed against size

Time `rate_loop` and `rate_vector` on the first 2,000 rows of the calls, and on all 20,000 rows. Use the best of 5 runs for `rate_vector`. Report the ratio of the two times for each size.

Expected: both ratios are above 10, and the ratio for 20,000 rows is larger than the ratio for 2,000 rows. Your numbers depend on your hardware and on the load of the machine.

2. [break] A rounding rule that changes the bill

Replace `np.ceil` in `rate_vector` by `np.round`, and then by `np.floor`. Report the total bill for each.

Expected: 3,709.03 euros with `np.round` and 3,307.96 euros with `np.floor`, against 4,245.09 euros with `np.ceil`. Explain which calls cause the difference.

3. [judge] One call at a time

A web service prices one call when a customer asks "what did this call cost?". The service receives one record per request. A teammate wants to call `rate_vector` on a one-row DataFrame, so that there is one rule in the code base. Another wants a plain function on a dict.

Decide which. A good answer weighs three points:

- the cost of a DataFrame for one row.
- the risk of two rules that drift apart.
- how you test that the two give the same price.

Exercises for Chapter 8

Chapter: Group by. Code: `examples/ch08_group_by.py`. Solutions: `solutions/08.md`.

1. [measure] Who makes the calls?

Build `subscriber_summary` and report the median number of calls for each subscriber. Then report the number of calls and the share of all calls that the ten busiest subscribers make.

Expected: the median is 22 calls. The ten busiest subscribers make 2,957 calls, which is 14.8% of the 20,000 calls.

2. [break] A count that skips the nulls

In `subscriber_summary`, the column `calls` is `("call_id", "size")`. Change it to `("duration_s", "count")`. Run the four checks from the chapter.

Expected: the total of `calls` is 18,967 and not 20,000. State which check fails, and why the other checks still pass.

3. [judge] The average subscriber

A manager asks for "the average number of calls for each subscriber" for a slide. The mean is 33.3 and the median is 22.

Decide what you put on the slide. A good answer weighs three points:

- what the manager does with the number.
- how the distribution looks.
- what a reader concludes from each choice.

Exercises for Chapter 9

Chapter: Joins. Code: `examples/ch09_joins.py`. Solutions: `solutions/09.md`.

1. [measure] Left against inner

Remove the first five subscribers from the subscriber table. Join the calls to this shorter table on the caller, once with `how="inner"` and once with `how="left"`. Report the number of rows of each result, and the number of calls without a plan in the left join.

Expected: 19,856 rows for `inner`, 20,000 rows for `left`, and 144 calls with a missing plan. Your counts must match exactly.

2. [break] A key in another form

Change the `msisdn` column of the subscribers so that it has the national form, `0` and nine digits, instead of `+32` and nine digits. Run `enrich`.

Report the number of on-net calls and the contents of the report. Then say why the report did not warn you, and add a check that does.

Expected: the call runs without an error, `on_net` is `True` for 0 calls, and the report shows 0 unmatched keys for the three lookups.

3. [judge] Inner or left

The nightly billing job joins the calls to the subscribers. A developer uses `how="inner"` because "a call without a subscriber is not billable anyway".

Decide which join the job uses. A good answer weighs three points:

- what happens to the revenue.
- who finds out.
- what you do with the calls that do not match.

Exercises for Chapter 10

Chapter: Pivot and melt. Code: `examples/ch10_pivot.py`. Solutions: `solutions/10.md`.

1. [measure] The same grid in two zones

Build the traffic grid with `tz="Europe/Brussels"` and with `tz="UTC"`. Report the busiest hour of the day, and its number of calls, for each grid. Count how many of the 168 cells have a different value in the two grids.

Expected: the busiest local hour is 15 with 1,441 calls. The busiest UTC hour is 13 with 1,441 calls. 164 of the 168 cells differ.

2. [break] A duplicate row

Add a copy of the first row of `to_long(grid)` to the long table. Convert it back with `to_wide`. Then convert it with `pivot_table(..., aggfunc="sum")`.

Report what each one does, and the total of the cells of the result.

Expected: `to_wide` raises a `ValueError`. `pivot_table` returns a table without an error, and its total is larger than 20,000.

3. [judge] The format of a report

The operations team gets the traffic grid every Monday. A dashboard team wants the same numbers every night, as a feed. One person proposes a single format for both: the long form, one row for each weekday and hour.

Decide whether one format serves both. A good answer weighs three points:

- what each reader does with the file.
- what breaks when a column is added.
- what the cost of two formats is.

Exercises for Chapter 11

Chapter: The index and MultiIndex. Code: `examples/ch11_index.py`. Solutions: `solutions/11.md`.

1. [measure] The repeated label

The audit says that the index `(caller, start)` has one duplicate label. Find the two rows. Report the subscriber, the second, and the two `call_id` values.

Expected: the subscriber is `S0069`, and the second is 2026-09-19 18:29:11 UTC. Two rows share the label.

2. [break] A ratio with a missing hour

Compute the share of calls that failed in each local hour, for the roaming calls only. Divide the count of failed roaming calls for each hour by the count of roaming calls for each hour. Count the hours that have `NaN`.

Then fix the result so that an hour with roaming calls and no failures shows 0, and an hour without roaming calls shows `NaN`.

Expected: the plain division has 23 values, and 8 of them are `NaN` although they mean "no failures". After your fix, none of the 23 hours is `NaN`, and 8 are 0. When you reindex to all 24 hours, one hour is `NaN`: the hour without any roaming call.

3. [judge] An index for ten million rows

A job looks up one call by `call_id` for 50,000 ids in a table of ten million rows. A colleague proposes `set_index("call_id")` at load time. Another says that the extra step and the memory are not worth it, and wants `merge`.

Decide which one. A good answer weighs three points:

- the cost of the index.
- how often the table is read for lookups.
- what you do when the ids are not unique.

Exercises for Chapter 12

Chapter: Dates and resampling. Code: `examples/ch12_dates.py`. Solutions: `solutions/12.md`.

1. [measure] Complete hours and the busiest hour

Use `hourly_kpi`. Count the complete hours of each cell. Then find the local hour of the day (0 to 23) with the highest mean of `prb_util_pct`, taken over all cells and all days.

Expected: the complete hours for each cell are `ANT-001` 60, `ANT-002` 61, `BRU-001` 60, `BRU-002` 58, `BRU-003` 61 and `BRU-004` 54. The busiest local hour is 16, with a mean of 73.4%.

2. [break] Rows are not time

A colleague computes a "24-hour rolling count" of KPI samples for the cell `BRU-001` as `rolling(288).count()`, because there are 288 samples of 5 minutes in a day. Compare it with `rolling("1D")` at 12:00 local time on 26 October. Report both values, and the time span of the 288 rows that end at that moment.

Expected: `rolling("1D")` gives 283 and `rolling(288)` gives 288. The span of the 288 rows is longer than 24 hours.

3. [judge] Which zone in the warehouse

A warehouse table holds the KPIs for a mobile network in one country. The analysts want local time, because a "busy hour" means the clock hour that a person lives in. The engineers want UTC, because it has no summer time.

Decide which time the table stores. A good answer weighs three points:

- the two days each year that break local time.
- how many people query the table.
- what you do about the other columns.

Exercises for Chapter 13

Chapter: Windows, shifts and gaps. Code: `examples/ch13_windows.py`. Solutions: `solutions/13.md`.

1. [measure] The length of the baseline

Run `flag_spikes` with `z=4` and three windows: `"1h"` with `min_periods=6`, `"6h"` with `min_periods=50` and `"1D"` with `min_periods=50`. For each, report the number of the 7 injected samples that are found, and the number of other samples that are flagged.

Expected: `1h` finds 1 and flags 5 others. `6h` finds 4 and flags 0. `1D` finds 7 and flags 0.

2. [break] A baseline that contains the sample

Write a copy of the detector with `closed="right"` (the sample is in its own baseline), and compare it with `closed="left"` for `z` of 4, 8 and 12. Report the number of injected samples found.

Expected: at `z=4` both find 7. At `z=8`, `left` finds 5 and `right` finds 3. At `z=12`, `left` finds 2 and `right` finds 1. No false flags in any of the six runs.

3. [judge] Who is woken at night

An alert from `flag_spikes` pages an engineer. A spike of one sample, 5 minutes, is common in this kind of data and ends by itself. A spike of 7 samples is an incident.

Decide the rule that decides when a person is woken. A good answer weighs three points:

- the number of false pages.
- the delay of the real alert.
- what the engineer does in the first 5 minutes of a page.

Exercises for Chapter 14

Chapter: Ranks, running totals and sessions. Code: `examples/ch14_sessions.py`. Solutions: `solutions/14.md`.

1. [measure] Sessions with more than one call

Use the gap of 30 minutes. Report the number of sessions and the number of sessions with more than one call. Report the number of calls in those sessions and their share of all calls.

Expected: 18,420 sessions. 1,224 of them have more than one call, and they hold 2,804 calls, which is 14.0% of the 20,000 calls.

2. [break] No sort

Write the three steps of `sessionize` without the `sort_values`: take the file order of `load_calls()`. Count the sessions for the gap of 30 minutes, and count the negative gaps.

Expected: 403 negative gaps, and 18,151 sessions where the correct number is 18,420.

3. [judge] The size of the gap

An analyst uses a gap of 30 minutes because "that is what the web analytics tools use". The calls are phone calls of one subscriber.

Decide how you choose the gap for phone calls. A good answer weighs three points:

- what a session is for.
- what the data show about the gaps between calls.
- how a wrong gap changes the result.

Exercises for Chapter 15

Chapter: Copy-on-Write and traps. Code: `examples/ch15_traps.py`. Solutions: `solutions/15.md`.

1. [measure] Which write reaches the table

For each of five ways to set the value 0 in the column `x` of `df = pd.DataFrame({"x": [1, 2, 3]})`, report whether `df` changed. The five ways are:

- `df["x"] = 0`
- `df.loc[df["x"] > 1, "x"] = 0`
- `df["x"].iloc[0] = 0`
- `sub = df[["x"]]` and then `sub.iloc[0, 0] = 0`
- `s = df["x"]` and then `s[:] = 0`

Expected: the first two change `df`. The last three do not.

2. [break] A function with a side effect

Write `add_share(df)`. It adds a column `share = x / x.sum()` with a plain assignment to its argument. Call it twice on the same table, from a function that first checks that the column is absent. Report what the second call does. Then change one line so that the second call works.

Expected: the second call fails the check because the column is already there. After the change, the table of the caller has no `share` column, and the function returns a new table.

3. [judge] Add a trap

The catalog has ten traps. A teammate says that ten is arbitrary, and that a longer list will not be read.

Decide how long the list must be and who maintains it. A good answer weighs three points:

- the cost of a wrong result in your reports.
- how a person finds the right trap at the right time.
- how the list stays correct when pandas changes.

Exercises for Chapter 16

Chapter: Method chains and pipelines. Code: `examples/ch16_chain.py`. Solutions: `solutions/16.md`.

1. [measure] Where the revenue is

Use `monthly_report`. Report the revenue for each home city, from the largest, and the share of the largest city in the total. Then find the plan and city with the lowest answer rate, and the number of calls behind it.

Expected: Brussels 1,647.41, Antwerp 1,049.60, Liège 803.75, Ghent 457.36, Leuven 286.97. Brussels has 38.8% of the revenue. The lowest answer rate is 0.737, for `business` in Leuven, from 205 calls.

2. [break] A plan that is not in the plans table

Remove the row of the plan `plus` from the plans table, and run `monthly_report`. Report the total calls and the total revenue, and compare the revenue with the total of the full table.

Then add one `require` step to the chain that stops the report in this case. State where in the chain it goes.

Expected: the report runs without an error. The calls total is 20,000, and the revenue is 3,681.87 against 4,245.09.

3. [judge] One chain or three functions

The chain of `monthly_report` has ten steps. A reviewer asks you to split it into three named functions: `enrich`, `add_billing` and `summarize`. A colleague says that one chain is easier to read from top to bottom.

Decide. A good answer weighs three points:

- what each choice does to testing.
- to a person who reads the code for the first time.
- to the next change in the billing rule.

Exercises for Chapter 17

Chapter: Data validation. Code: `examples/ch17_validation.py`. Solutions: `solutions/17.md`.

1. [measure] Is the validator exact for other seeds?

Plant the faults with `seed` 0, 1, 2, 3 and 4. For each seed, compare the counts that the validator finds with the expected counts. Report the number of seeds for which all seven rules match exactly.

Expected: all 5 seeds match. If one does not, look at the faults of that seed. Two faults on the same row are the likely cause.

2. [break] A fault that no rule sees

Take the call records. Set `duration_s` to 90,000 seconds (25 hours) for 30 answered calls. Run the validator with `call_rules`. Report the number of violations that it finds for these rows.

Then write one more rule that finds them, and report how many rows it finds on the clean data and on the broken data.

Expected: the validator finds no violation for the 30 rows. Your new rule finds 0 rows on the clean data and 30 on the broken data.

3. [judge] Stop or drop

The nightly report job finds 7 rows (of 20,000) with a repeated `call_id`. A developer wants the job to drop the 7 rows and continue, because "the report is due at 07:00". Another wants the job to stop.

Decide. A good answer weighs three points:

- who reads the report at 07:00.
- what a wrong number costs.
- what the 7 rows are likely to be.

Exercises for Chapter 18

Chapter: Performance. Code: `examples/ch18_performance.py`. Solutions: `solutions/18.md`.

1. [measure] Does the gap grow with the size?

Time `task_itertuples` and `task_mask_groupby` on 100,000 rows (`replicate(calls, 5)`) and on 200,000 rows (`replicate(calls, 10)`). Report the ratio of the two times for each size.

Expected: both ratios are above 10, and the two ratios are within a factor of 3 of each other. Your numbers depend on your hardware and on the load of the machine.

2. [break] A loop that is quadratic

Time the loop with `concat` from the chapter for 100, 200 and 400 pieces of 200 rows. Report the three times and the ratio of the time for 400 pieces to the time for 100 pieces.

Expected: the ratio is above 6. A cost that is linear in the number of pieces gives 4, and a cost that is quadratic gives 16.

3. [judge] Is it worth it

A nightly job takes 30 minutes and finishes at 01:30. Nobody reads the output before 08:00. A teammate wants to spend two days to make it 10 times faster.

Decide whether to do it. A good answer weighs four points:

- what the 30 minutes cost.
- what a failure.
- a rerun cost.
- what else the two days can do.

Exercises for Chapter 19

Chapter: Data larger than memory. Code: `examples/ch19_chunked.py`. Solutions: `solutions/19.md`.

1. [measure] A chunk size that does not divide the file

Run `talk_by_caller_chunked` and `talk_by_caller_whole` on three cases. For each case, report whether `assert_frame_equal` passes and how many chunks were read:

- the first 2,000 rows of the calls, with chunks of 10 rows
- the call records repeated 20 times (400,000 rows), with chunks of 70,000 rows
- the same file, with one chunk of 400,000 rows

Expected: all three pass. The numbers of chunks are 200, 6 and 1. In the first run, a caller is often missing from a chunk. Say what that did to the dtype of the count before the function was repaired.

2. [break] The mean of the means

Compute the mean duration for each caller in two ways from the chunks: the correct way (sum of the sums divided by sum of the counts), and the wrong way (the mean of the mean of each chunk). Use chunks of 70,000 rows on the file of 400,000 rows. Report the largest difference over the callers.

Expected: the correct way equals the whole-file mean. The wrong way differs for many callers. State the largest absolute difference in seconds.

3. [judge] Chunks or a database

A colleague has a 40 GB CSV file of call records and a laptop with 16 GB of memory. You can write the chunked job in an afternoon. Another colleague says: put the file into a database once, and then everything is a query.

Decide. A good answer weighs three points:

- how often the file is analysed.
- how many different questions you ask.
- what is not additive.

Exercises for Chapter 20

Chapter: SQL and pandas. Code: `examples/ch20_sql.py`. Solutions: `solutions/20.md`.

1. [measure] Answered calls for each plan

Write one SQL query that returns the number of answered calls for each plan, and the same count in pandas. Report both.

Expected: basic 5,153, business 1,763, plus 3,497, prepaid 5,113. The two agree.

2. [break] A value that changes the query

Search the table `people` of the chapter (names "O'Neil" and "Peeters") with the name `x' OR '1'='1`. Do it once with string formatting and once with `params`. Report the number of rows that each returns.

Expected: string formatting returns 2 rows, which is every row. `params` returns 0 rows.

3. [judge] Where does the report live

The monthly report exists as a pandas chain and as an SQL query, and the two agree. The team wants to keep only one.

Decide which one to keep. A good answer weighs three points:

- who can read it.
- where the data live.
- what you lose when you delete the other.

Exercises for Chapter 21

Chapter: Plots. Code: `examples/ch21_plots.py`. Solutions: `solutions/21.md`.

1. [measure] The data inside a figure

Draw `plot_hourly_calls(calls, tz="UTC")` and read the 24 bar heights from the Axes. Report the hour with the tallest bar and its height, and compare with the same figure in `Europe/Brussels`.

Expected: the tallest bar is hour 13 in UTC with 1,441 calls, and hour 15 in Brussels with 1,441 calls. The bar heights are the same numbers, shifted by two hours.

2. [break] A figure that lies

Draw the hourly bars with `calls["start"].dt.hour` instead of the converted hour (the zone is UTC), and keep the label "hour of the day (Europe/Brussels)". Report which hour the figure shows as busiest.

Expected: the figure shows hour 13 as the busiest, under a label that says Brussels. Nothing in the code or the figure signals the fault.

3. [judge] Test or look

A colleague says that the tests of plots in the chapter are enough, and that nobody needs to look at a figure. Another says that plots cannot be tested.

Decide what you do before a figure goes into a monthly report. A good answer weighs three points:

- what the number tests catch.
- what they cannot catch.
- how often the report changes.

Exercises for Chapter 22

Chapter: JSON and nested records. Code: `examples/ch22_json.py`. Solutions: `solutions/22.md`.

1. [measure] The shape of the traces

Use `events_table` and `call_summary`. Report the number of events of each type. Report the number of calls that have a `180` response (ringing). Report the median time to ringing in milliseconds.

Expected: `RESPONSE` 1,394, `INVITE` 492, `ACK` 383, `BYE` 383. 430 calls have a ringing response. The median time to ringing is 927.5 ms.

2. [break] explode counts a call with no events as one

Build the events with `DataFrame(records).explode("events")` and count the events of each call with `groupby("call_id").size()`. Compare with `call_summary`.

Expected: 8 calls have a size of 1 although they have no event. Report the fix.

3. [judge] Flatten at load or keep it nested

The traces arrive every night as JSON lines. An analyst wants a flat table in the warehouse. An engineer wants to keep the original JSON and flatten in each query, because the structure of the events changes.

Decide. A good answer weighs three points:

- what changes when a new event field appears.
- who reads the data.
- what you can recompute.

Exercises for Chapter 23

Chapter: Features and evaluation. Code: `examples/ch23_features.py`. Solutions: `solutions/23.md`.

1. [measure] A cutoff in the wrong zone

Build the features and the target with the cutoff `2026-09-22` in `Europe/Brussels`, and again with the cutoff as UTC midnight (`2026-09-22 00:00` UTC). Report the number of calls before the cutoff in each case, and how many calls fall between the two cutoffs.

Expected: 13,905 calls before the Brussels cutoff. The UTC cutoff is two hours later, so more calls are before it. State the number of calls between the two.

2. [break] A feature that looks safe

Write `recent_share(calls, subscribers, cutoff)`: the share of each subscriber's calls in the whole month that are before the cutoff. Run `assert_no_leak` on it.

Expected: the test fails. Say what the feature uses that it must not.

3. [judge] How many tasks

An evaluation has 150 tasks with 3 seeds each. A colleague proposes 450 tasks with 1 seed each, at the same cost.

Decide which design you prefer. A good answer weighs three points:

- what the bootstrap interval is for.
- what the seeds measure.
- what limits the number of tasks in practice.

Exercises for Chapter 24

Chapter: Field guide. Code: `examples/ch24_field_guide.py`. Solutions: `solutions/24.md`.

1. [measure] Your own list of mistakes

Choose three results from your own work with pandas that you once believed and that were wrong. For each one, write a block of at most six lines. The block reproduces the wrong result and the right result, in the form of the block in the chapter.

Expected: each block runs and prints two different values. Your own cases are not in the book, so there is no number to match. Run each one twice.

2. [break] A trap that no longer exists

The module `pdbook.traps` has ten traps, and `live` checks that each one still gives a wrong answer. Pick one trap and change its `correct` function so that it gives the same result as `naive`. Run `live` on it.

Expected: `live` returns `False` for that trap. State what this means for a trap list, and what you do when it happens after an upgrade of pandas.

3. [judge] The order of reading

A new colleague has one afternoon with this book. They will use pandas next week on call data and reports.

Decide which three chapters they read, and in which order. A good answer weighs three points:

- what a wrong result costs.
- which chapters change the habits that matter most.
- what they can look up later.

Solutions for Chapter 1

Exercises: `../01.md`. Chapter: Series and DataFrame.

1. [measure] Where the nulls are

```
import pandas as pd

calls = pd.read_csv("data/calls.csv")
missing = calls[calls["duration_s"].isna()]
print(len(missing))
print(missing["status"].value_counts().to_dict())
print(int((calls["status"] == "failed").sum()))
```

```
1033
{'failed': 1033}
1033
```

All 1,033 null durations belong to calls with the status `failed`. A call that failed never started, so it has no duration. The number of `failed` calls is also 1,033, so every failed call has a null duration, and no other call has one. A call that rang with no answer has a duration of 0, not null. The two values mean different things: 0 is a measurement, null is the absence of one.

2. [break] Labels against positions

```
import pandas as pd

s = pd.Series([1, 2, 3], index=["a", "b", "c"])
t = pd.Series([1, 2, 3], index=["c", "b", "a"])
print((s + t).tolist())
print((s.to_numpy() + t.to_numpy()).tolist())

t_fixed = t.set_axis(["a", "b", "c"])
print((s + t_fixed).tolist())
```

```
[4, 4, 4]
[2, 4, 6]
[2, 4, 6]
```

pandas matches `a` with `a`, so the value 1 of `s` meets the value 3 of `t`. NumPy matches by position, so 1 meets 1. The two answers differ because the data differ: the two Series are not the same data in the same order. The fix is to replace the labels of `t` with the labels of `s`.

With `set_axis`, the labels no longer describe the values, and the sum becomes the positional sum. This is also the risk: after `set_axis`, the labels of `t` can be wrong. Use it only when you know that the order is right.

3. [judge] What should first_look show?

Position. Add the minimum and maximum for numeric columns. Add the most frequent value with its share. Show the share of nulls instead of the count. Remove the first value.

The first value of one row is an anecdote. The minimum and maximum show whether a column holds a sentinel such as `-1` or `9999`. The most frequent value with its share shows a column that is 99% one value, and such a column carries almost no information. The share of nulls is comparable across tables of different length, and the count is not.

Strongest objection. The maximum and minimum of a column are one extreme row each. One bad row moves them, so they report the bad row and tell you nothing about the typical row. Quantiles are more robust.

What changes this position. Suppose that on your own tables the first value showed a format problem (for example a date as text) more often than the extremes did. Then keep it. You can measure this. Take ten tables that gave you trouble, and count how often each column of the report pointed at the problem.

Solutions for Chapter 2

Exercises: `../02.md`. Chapter: Input and output.

1. [measure] The subscriber numbers

```
import pandas as pd

default = pd.read_csv("data/subscribers.csv")
print(default["msisdn"].dtype)

fixed = pd.read_csv("data/subscribers.csv", dtype={"msisdn": "str"})
print(fixed["msisdn"].str.len().value_counts().to_dict())
print(fixed["msisdn"].iloc[0])
```

```
int64
{12: 600}
+32427521358
```

The default read gives `int64`, and the plus sign is gone: the first value is the number 32427521358, and the text is `+32427521358`. All 600 values have the length 12 when read as text. A join of `calls.callee` as integer with `subscribers.msisdn` as integer still matches, because both sides lost the plus sign in the same way. A join of one integer column with one text column stops with a `ValueError`.

That is the lucky case: you see it. The silent case is two integer columns where only one side lost leading zeros. Chapter 9 shows the stop.

2. [break] A column that parquet refuses

```
import tempfile

import pandas as pd

df = pd.DataFrame({"a": [1, "x", 3.5]})
print(df.dtypes.astype(str).to_dict())
with tempfile.TemporaryDirectory() as tmp:
    try:
        df.to_parquet(f"{tmp}/a.parquet")
    except Exception as error:
        print(type(error).__name__)
    fixed = df.astype({"a": "str"})
    fixed.to_parquet(f"{tmp}/a.parquet")
    print(pd.read_parquet(f"{tmp}/a.parquet")["a"].tolist())
```

```
{'a': 'object'}
ArrowInvalid
['1', 'x', '3.5']
```

The column has the dtype `object`, which holds any Python value. Parquet has one type for each column, and pyarrow cannot choose one for `1`, `"x"` and `3.5`. The change is `astype("str")`. You lose the number type: `1` is now `'1'`, and `3.5` is now `'3.5'`.

This is the right outcome. A column with three types is a data problem, and the error forced you to look at it. Some people avoid the error with `astype(str)` in a loop over all columns. That hides the problem and is the worse choice.

3. [judge] The nightly handover

Position. Send parquet as the main file. Send a CSV file with a schema description beside it, until the other team confirms in writing that they read the parquet file.

The people who read the extract pay for a wrong type, usually without knowing the cause. A CSV file that lost the leading zeros is a failure that shows up later, in a report. The parquet file carries the types, so the receiving side can read them without a guess. The CSV file stays for one reason. The receiving tools can fail to read parquet, and "we cannot open it" stops the work on the first day.

Strongest objection. Two files are two sources of truth. When they differ, nobody knows which one is right. A single CSV with a documented schema (a `dtype` mapping that the other team uses in their loader) is simpler.

What changes this position. Compare the two files on one night: load both with the receiving team's own tools and count the differing cells. If the count is zero for a month, drop the CSV. If it is not zero, the CSV file is the cause, and you must find out why before you send it again.

Solutions for Chapter 3

Exercises: `../03.md`. Chapter: Selection.

1. [measure] Both ends of a range

```
from pdbook.io import load_calls

d = load_calls()["duration_s"]
print(int(d.between(60, 120).sum()))
print(int(((d >= 60) & (d < 120)).sum()))
print(int(((d > 60) & (d < 120)).sum()))

print(int((d == 120).sum()), int((d == 60).sum()))
```

```
4226
4179
4072
47 107
```

`between` includes both ends by default. The second count drops the calls of exactly 120 seconds, and there are 47 of them (4,226 minus 4,179). The third count also drops the calls of exactly 60 seconds, and there are 107. Durations are whole seconds, so a boundary value is common. When you write a range, write the rule for the ends in the name of the variable or in a comment. `between(..., inclusive="left")` states the rule in the call.

2. [break] The missing row

```
from pdbook.io import load_calls

calls = load_calls().set_index("call_id")
print(calls.index.is_monotonic_increasing)

result = calls.loc[4:10]
print(result.index.tolist())
print(int(calls.index.get_loc(7)), len(calls))

by_list = calls.loc[list(range(4, 11))]
print(by_list.index.tolist())

by_sort = calls.sort_index().loc[4:10]
print(by_sort.index.tolist())
```

```
False
[4, 5, 6, 8, 9, 10]
19000 20000
[4, 5, 6, 7, 8, 9, 10]
[4, 5, 6, 7, 8, 9, 10]
```

The file ends with 1,000 late-arriving records, so the index is not sorted. Call 7 is one of the late records, and it sits at position 19,000, near the end of the file. A label slice on an index that is not sorted uses the positions of its two end labels (here the positions of 4 and 10) and returns everything between them. Call 7 lies outside that range of positions, so it is missing. There is no error and no warning.

Two fixes work. A list of labels asks for each row by name, and the order of the file does not matter. A sorted index makes the slice follow the order of the labels. The second fix also protects every later `.loc` on this table, so it is the one to prefer. Chapter 11 returns to this.

3. [judge] Masks or query

Position. Use masks in functions that other code calls, and `query` in one-off analysis and in notebooks. Write the rule down in one line in the team guide.

A function that a nightly job calls must fail early and clearly. A mask uses names that an editor and a type checker can see, so a renamed column is found before the job runs. A `query` string is checked only when it runs. In a notebook the feedback is immediate, so the cost of a typo is small and the shorter text has more value.

Strongest objection. Two styles in one code base are a cost for every new person. A rule with exceptions needs a judgement for each case, and teams drift toward whichever style the last author liked.

What changes this position. Count the incidents over a quarter. If the nightly job failed because of a name inside a string, use masks everywhere. If nobody has a typo in a notebook query in three months, stop discussing it.

Solutions for Chapter 4

Exercises: `../04.md`. Chapter: Types and memory.

1. [measure] The saving, column by column

```
from pdbook.io import load_calls
from pdbook.shrink import memory_report, shrink

calls = load_calls()
small = shrink(calls, float_to_int=True)
rep = memory_report(calls, small)
print(int(rep["bytes_before"].sum()), int(rep["bytes_after"].sum()))
saving = (rep["bytes_before"] - rep["bytes_after"]).sort_values(ascending=False)
print(saving.head(3).to_dict())
```

```
2017566 788158
{'status': 292163, 'cell_id': 283877, 'codec': 221243}
```

The column bytes sum to 2,017,566 before and 788,158 after. The chapter shows 132 bytes more on each side, because `memory_usage` of the whole table also counts the index, and `memory_report` leaves the index out. The largest saving is in `status`: 292,163 bytes, for a text column with 4 distinct values. The next two are `cell_id` (283,877 bytes, 12 distinct values) and `codec` (221,243 bytes, 3 distinct values).

All three are text columns with few distinct values. The integer columns save less each: `call_id` saves 120,000 bytes, because it goes from 8 bytes to 2 bytes for each of 20,000 rows. A text column pays for every character of every row, so text with few distinct values is where the memory is.

2. [break] A type that is too small

```
from pdbook.io import load_calls

ids = load_calls()["call_id"].sort_values().reset_index(drop=True)
wrapped = ids.astype("int8")
print(int((wrapped != ids).sum()))
at = [126, 127, 128, 299]
print(ids.iloc[at].tolist())
print(wrapped.iloc[at].tolist())
print(300 % 256, int(ids.max()))
```

```
19873
[127, 128, 129, 300]
[127, -128, -127, 44]
44 20000
```

An `int8` holds the numbers from −128 to 127. The identifier 127 survives. The identifier 128 does not fit and becomes −128, and 129 becomes −127. Only the 127 identifiers from 1 to 127 are unchanged, so 20,000 minus 127 is 19,873 changed rows.

The identifier 300 becomes 44, because 300 is 44 more than 256, and 256 is the number of values that 8 bits can hold. All the arithmetic is arithmetic modulo 256, in the same way as a counter that wraps. The safe conversion is `pd.to_numeric(ids, downcast="integer")`, which chooses `int16` because the largest identifier is 20,000.

3. [judge] Where to shrink

Position. Call `shrink` on each input table right after the load. Give both sides of the join one shared `CategoricalDtype` for the key, if the key is a category. Test the content with `same_content` once for each run, on a sample.

The peak memory of a pipeline is where the load and the join both hold their tables. Shrinking after the load makes both tables small before the join, and that is the point of the peak. Shrinking after the join does not help the peak, because the join has already happened with the big tables.

Strongest objection. Take a key column that is a `category` in one table and a different `category` in the other. It falls back to `str` in the join (Chapter 4, when not to shrink). The saving is lost at the moment that you need it most. Shrinking only after the join avoids this.

What changes this position. Measure the peak memory of the process for the three options, on a 10% sample, as Chapter 19 does. Suppose that the peak is the same for "after the load" and "after the join". Then the load is not the bottleneck, and the simpler pipeline (never shrink) wins.

Solutions for Chapter 5

Exercises: `../05.md`. Chapter: Missing data.

1. [measure] Is the loss random?

```
from pdbook.io import load_calls

calls = load_calls()
lost = calls["cell_id"].isna()
print(round(100 * lost[calls["status"] == "answered"].mean(), 2))
print(round(100 * lost.mean(), 2))
print((100 * lost.groupby(calls["codec"]).mean()).round(2).to_dict())
print(lost.groupby(calls["codec"]).agg(["sum", "size"]).to_dict("index"))
```

```
1.37
1.38
{'AMR-NB': 2.09, 'AMR-WB': 1.61, 'EVS': 0.48}
{'AMR-NB': {'sum': 70, 'size': 3343}, 'AMR-WB': {'sum': 119, 'size': 7392}, 'EVS': {'sum': 23,
'size': 4791}}
```

The shares are 1.37% among answered calls and 1.38% among all calls. The two are close, so the status does not change the rate of loss. The codec shares are not close: 0.48% for `EVS`, 1.61% for `AMR-WB` and 2.09% for `AMR-NB`.

A call that uses `AMR-NB` is four times as likely to have a lost cell as a call that uses `EVS`. The statement "the cells are lost at random" is false for this table. The loss is independent of the status and associated with the codec.

The cause is in the data generator. It draws the codec from the technology of the cell, and a call with no cell gets the draw for a 4G cell. In a real pipeline, look for the step that reads the cell and chooses a default. The counts are small (277 calls in all). Check the size of the effect before you build on it: 23 of the 4,791 `EVS` calls have a lost cell, and 70 of the 3,343 `AMR-NB` calls.

2. [break] A determinant that is not a cause

```
from pdbook.io import load_calls
from pdbook.nulls import determinants

calls = load_calls()
changed = calls.copy()
changed["cell_id"] = changed["cell_id"].where(~changed["roaming"], None)
changed["cell_id"] = changed["cell_id"].where(changed["roaming"] | changed["cell_id"].notna(),
"UNKNOWN")
print(int(changed["roaming"].sum()), int(changed["cell_id"].isna().sum()))
print(determinants(changed)["cell_id"])
```

```
563 563
['roaming']
```

The report is `['roaming']`. In the copy, 563 calls roam and 563 calls have no cell. They are the same calls. In each group of `roaming`, the cell is all null or none null, so `roaming` decides `cell_id` by the rule of the function. The report is correct.

The report does not say whether the data are lost. There are two readings. A roaming subscriber is on a foreign network, and the home operator has no cell identifier for the call, so the value does not exist. Or the export job drops the cell for roaming calls by mistake, so the value exists and is lost. The data cannot tell the two apart.

A decision needs a document: the specification of the record format, or the person who wrote the export. A test finds an association. It does not find a cause.

3. [judge] The outage in the KPI table

Position. Leave the hole in the line, and mark it. Draw the line with a visible break and put a label "no data, 10:00 to 12:00" over it.

The manager wants a picture that looks complete. A filled line says "the cell carried this load". Nobody measured that. If the cell was down, the real load was zero for the subscribers on it.

An interpolated line of 60% load for two hours is a false statement about a failure. The manager can take a wrong decision from it, for example to add capacity to a cell that was not overloaded. The next person who reads the dashboard can do the same.

Strongest objection. A line with a break makes every alert rule and every rolling average on the series fail or skip. A person who looks at 40 cells must now look at 40 gaps. A short gap of one sample (5 minutes) is noise and not information. Leaving it hurts more than it helps.

What changes this position. Fill gaps of up to three steps (15 minutes) with `interpolate(limit=3)` and mark them in a separate column `filled`. Keep longer gaps open. Then count how often a person asks about a gap. If the question never comes for the filled ones, and always comes for the long ones, the split is right.

Solutions for Chapter 6

Exercises: `../06.md`. Chapter: Text and date cleanup.

1. [measure] How many formats?

```
import pandas as pd

raw = pd.read_csv("data/contacts_messy.csv", dtype=str)
signup = raw["signup"].str.strip()
patterns = {
    "2024-03-01": r"^\d{4}-\d{2}-\d{2}$",
    "01/03/2024": r"^\d{2}/\d{2}/\d{4}$",
    "March 01, 2024": r"^[A-Z][a-z]+ \d{2}, \d{4}$",
    "01-Mar-24": r"^\d{2}-[A-Z][a-z]{2}-\d{2}$",
}
hits = {name: signup.str.match(pat) for name, pat in patterns.items()}
for name, mask in hits.items():
    print(f"{name:16s} {int(mask.sum())}")
none = ~pd.concat(hits, axis=1).any(axis=1)
print("none", int(none.sum()), signup[none].tolist())
print("31/02/2024" in signup[hits["01/03/2024"]].tolist())
```

```
2024-03-01       78
01/03/2024       66
March 01, 2024   67
01-Mar-24        59
none 1 ['someday']
True
```

The four patterns match 78, 66, 67 and 59 rows. Together that is 270 rows, and one row of 271 matches none: `someday`. The date `31/02/2024` matches the second pattern, because a pattern checks the shape of the text and not the calendar.

Only `to_datetime` knows that there is no 31 February. This is the reason that the cleaner needs both: patterns to count the formats, and the parser to find the invalid dates. A count of formats that adds up to the number of rows is not proof that every date is valid.

2. [break] The wrong key

```
import pandas as pd

from pdbook.clean import clean_contacts

raw = pd.read_csv("data/contacts_messy.csv", dtype=str)
clean, _ = clean_contacts(raw)
by_phone = clean.dropna(subset=["phone"])
by_name = by_phone.drop_duplicates(subset="name_key")
print(len(by_phone), len(by_name), len(by_phone) - len(by_name))

removed = by_phone.loc[~by_phone.index.isin(by_name.index)]
print(len(removed), int(removed["phone"].nunique()))
```

```
244 170 74
74 74
```

There are 244 rows with a phone. With `name_key` as the key, 170 remain, and 74 are removed. The second line shows that the 74 removed rows have 74 distinct phones. After the cleaning, no phone repeats, so no removed row has a phone that another row has. Each of the 74 removed rows is a different subscriber that has the same name as a kept row. The name key merges 74 people into others.

The number 170 is lower than the 172 distinct name keys that the chapter reports for the whole file. The two missing rows are the two rows with a missing phone, and they have their own names.

3. [judge] Reject or repair

Position. Reject it. Do not add a table of number words to the cleaner.

A wrong repair is worse than a rejected value, because nobody sees it. Number words have traps: "twelve fifty", "a hundred", "12 euro 50", and in Dutch and French the same words with other rules, because the export contains Belgian data. The input `twelve euros` appears once in 271 rows. The list of rejects gives a person the raw text and the row, and one second of work repairs it.

Strongest objection. One rejected row in 271 is a small cost. At a thousand rows a day, the rejects take an hour a day, and the person stops reading them. A reject list that nobody reads is the same as silent guessing, with more work.

What changes this position. Count the rejects of one month and sort them by the raw text. If three spellings cover 90% of the rejects, add exactly those three rules, with a test for each. If the rejects are all different, the table of number words will not help.

Solutions for Chapter 7

Exercises: `../07.md`. Chapter: New columns.

1. [measure] Speed against size

```
import time

from pdbook.io import load_calls, load_plans, load_subscribers
from pdbook.rating import rate_loop, rate_vector

calls, subs, plans = load_calls(), load_subscribers(), load_plans()
ratios, loop_times, vector_times = {}, {}, {}
for n in (2000, 20000):
    part = calls.head(n)
    t0 = time.perf_counter()
    rate_loop(part, subs, plans)
    t_loop = time.perf_counter() - t0
    t_vec = float("inf")
    for _ in range(5):
        t0 = time.perf_counter()
        rate_vector(part, subs, plans)
        t_vec = min(t_vec, time.perf_counter() - t0)
    ratios[n] = t_loop / t_vec
    loop_times[n], vector_times[n] = t_loop, t_vec
    print(n, f"loop {t_loop:.3f}s vector {t_vec:.4f}s ratio {ratios[n]:.0f}")
assert ratios[2000] > 10 and ratios[20000] > ratios[2000]
assert vector_times[20000] / vector_times[2000] < 6 and loop_times[20000] / loop_times[2000] >
5
```

```
# output varies
2000 loop 0.060s vector 0.0040s ratio 15
20000 loop 0.581s vector 0.0086s ratio 67
```

The block prints a ratio for each size, and the assertion requires that the ratio for 20,000 rows is larger. The second assertion explains why. The rows grow 10 times. The time of the vectorized form grows by less than 6 times. The time of the loop grows by more than 5 times. Two points fit a line:

the vectorized form has a fixed cost (building the two lookup dicts and calling `map` twice) and a small cost for each row. The loop has almost no fixed cost and a larger cost for each row. The ratio keeps growing until the fixed cost is small against the cost of the rows. Test a vectorized rule on the size of the real data, not on a sample of ten rows.

2. [break] A rounding rule that changes the bill

```
import numpy as np

from pdbook.io import load_calls, load_plans, load_subscribers

calls, subs, plans = load_calls(), load_subscribers(), load_plans()
plan_of = dict(zip(subs["subscriber_id"], subs["plan"]))
rate_of = dict(zip(plans["plan"], plans["rate_per_min_eur"]))
rate = calls["caller"].map(plan_of).map(rate_of)
answered = calls["status"] == "answered"
minutes_exact = calls["duration_s"] / 60

total = {}
for name, fn in [("ceil", np.ceil), ("round", np.round), ("floor", np.floor)]:
    minutes = np.maximum(1, fn(minutes_exact))
    total[name] = round(float((minutes * rate).where(answered, 0.0).round(2).sum()), 2)
print(total)

whole = answered & (calls["duration_s"] % 60 == 0)
print(int(answered.sum()), int(whole.sum()))
```

```
{'ceil': 4245.09, 'round': 3709.03, 'floor': 3307.96}
15526 225
```

The bill is 4,245.09 euros with `np.ceil`, 3,709.03 with `np.round` and 3,307.96 with `np.floor`. The rule bills each started minute, which is `ceil`. A call of 61 seconds is two minutes under the rule. `round` gives one minute for 61 to 89 seconds, and `floor` gives one minute for 61 to 119 seconds.

Only 225 of the 15,526 answered calls last a whole number of minutes, and for those three functions agree. For the other 15,301 calls, `floor` is always lower than `ceil`, and `round` is lower for about half of them. The bill falls by 12.6% with `round` and by 22.1% with `floor`.

A change of one function name gave a bill that is lower by a quarter, and no test of types or shapes finds it. Only a test with a known price for a known call finds it. Add two or three such calls to the test of the rating function: a call of 59, 60 and 61 seconds.

3. [judge] One call at a time

```
import math
import time

from pdbook.io import load_calls, load_plans, load_subscribers
from pdbook.rating import rate_vector

calls, subs, plans = load_calls(), load_subscribers(), load_plans()
plan_of = dict(zip(subs["subscriber_id"], subs["plan"]))
rate_of = dict(zip(plans["plan"], plans["rate_per_min_eur"]))


def price_one(status, duration_s, caller):
    if status != "answered":
        return 0.0
    return round(max(1, math.ceil(duration_s / 60)) * rate_of[plan_of[caller]], 2)


row = calls[calls["status"] == "answered"].head(1)
args = (row["status"].iloc[0], row["duration_s"].iloc[0], row["caller"].iloc[0])
assert price_one(*args) == rate_vector(row, subs, plans).iloc[0]

def timed(fn, repeat):
    t0 = time.perf_counter()
    for _ in range(repeat):
        fn()
    return (time.perf_counter() - t0) / repeat

t_plain = timed(lambda: price_one(*args), 2000)
t_frame = timed(lambda: rate_vector(row, subs, plans), 20)
print(f"plain {t_plain * 1e6:.1f} us, one-row frame {t_frame * 1e6:.0f} us, ratio {t_frame /
t_plain:.0f}x")
assert t_frame / t_plain > 50
```

```
# output varies
plain 0.8 us, one-row frame 3564 us, ratio 4556x
```

Position. Write the rule once as a plain function on scalars. Use it in the web service. Keep `rate_vector` for batch work, and test the two against each other on a sample in the test suite.

The timing shows why. The block prints both times, and the assertion requires that the one-row DataFrame costs more than 50 times as much as the plain function. The cost is the overhead of the table, not the rule.

For a service that answers one request at a time, the overhead is paid on every request. The rule is short. The test in the last block of the chapter already compares a vectorized form with a loop on 1,000 rows.

Strongest objection. Two implementations of one rule are the classic source of drift. One day a new plan type is added to one and not to the other. The teammate who wants one rule is right that a single implementation cannot disagree with itself.

What changes this position. Use a single implementation if the plain function is the one source. The batch job can call it through `map` on a column. At 20,000 rows `apply` takes a fraction of a second (the block of Chapter 7 prints it),

which is acceptable for a nightly job. At 200 million rows the two-implementation design is needed, and then the test between the two is not optional.

Solutions for Chapter 8

Exercises: `../08.md`. Chapter: Group by.

1. [measure] Who makes the calls?

```
from pdbook.io import load_calls, load_subscribers
from pdbook.summary import subscriber_summary

calls, subs = load_calls(), load_subscribers()
summary = subscriber_summary(calls, subs)
top = summary["calls"].nlargest(10)
print(float(summary["calls"].median()))
print(int(top.sum()), round(float(top.sum() / len(calls)), 4))
print(top.head(3).tolist())
```

```
22.0
2957 0.1479
[628, 458, 329]
```

The median is 22 calls. The ten busiest subscribers make 2,957 calls, which is 14.8% of the table. Ten of 600 subscribers is 1.7% of the subscribers. The busiest one alone makes 628 calls, which is 3.1% of all calls and 29 times the median.

The distribution has a long tail, and a few subscribers decide the total. The consequence for the work: a sample of subscribers that leaves out the heavy ones gives a wrong total.

2. [break] A count that skips the nulls

```
from pdbook.io import load_calls, load_subscribers

calls, subs = load_calls(), load_subscribers()
good = calls.groupby("caller").agg(calls=("call_id", "size"))
bad = calls.groupby("caller").agg(calls=("duration_s", "count"))
print(int(good["calls"].sum()), int(bad["calls"].sum()), len(calls))
print(len(good) == len(bad), good.index.equals(bad.index))
print(int((good["calls"] != bad["calls"]).sum()), "subscribers have a different count")
print(int((bad["calls"] == 0).sum()), "subscribers have a count of zero")
```

```
20000 18967 20000
True True
439 subscribers have a different count
0 subscribers have a count of zero
```

The good count adds up to 20,000. The bad count adds up to 18,967, which is 1,033 less: one for each failed call, because `count` skips the null `duration_s`. The check "calls add up to the table" fails.

The three other checks pass. The table still has one row for each subscriber. The `answered` column does not depend on the change, and the talk time does not depend on it either.

The fault is local to 439 subscribers, the ones with at least one failed call. No subscriber has a count of zero, so the fault does not show as an empty row. The `answer_rate` of those 439 subscribers is now too high, because the denominator is too small. A subscriber with 10 calls and 1 failed call shows 9 calls.

Without the check on the total, the table looks right. This is the use of the four checks: each one guards a different part, and the one on the total finds a fault that the others do not.

3. [judge] The average subscriber

Position. Put the median, the 25th and 75th percentiles, and the share of calls that the top ten subscribers make. Write "typical subscriber: 22 calls".

The mean of 33.3 is larger than the value for three subscribers in four. A reader who sees "33 calls on average" thinks of a normal subscriber, and no subscriber is like that. The median says what half of the subscribers do. The share of the top ten says what the tail does, and a manager who plans capacity needs the tail.

Strongest objection. The mean has a property that the median does not have: the mean multiplied by the number of subscribers is the total. A manager who plans the load on the network needs the total, and the mean gives it in one step. A slide with only the median hides the volume.

What changes this position. Ask what the number is for. If it is a description of a typical subscriber, give the median. If it is an input to a total (capacity, cost, revenue), give the mean, and state that the tail drives it. If the manager uses the number for both, put both and name them: "median 22, mean 33".

Solutions for Chapter 9

Exercises: `../09.md`. Chapter: Joins.

1. [measure] Left against inner

```
from pdbook.io import load_calls, load_subscribers

calls, subs = load_calls(), load_subscribers()
short = subs[~subs["subscriber_id"].isin(subs["subscriber_id"].head(5))]
lookup = short[["subscriber_id", "plan"]]

inner = calls.merge(lookup, left_on="caller", right_on="subscriber_id", how="inner")
left = calls.merge(lookup, left_on="caller", right_on="subscriber_id", how="left")
print(len(inner), len(left), int(left["plan"].isna().sum()))
print(len(calls) - len(inner))
```

```
19856 20000 144
144
```

The inner join returns 19,856 rows. The left join returns 20,000 rows, and 144 of them have no plan. The difference, 144 calls, is exactly the number of calls that the five removed subscribers made. The inner join dropped them without a message. The result of the left join has the same 144 rows with `NaN`, so you can count them, look at them and decide.

The point is where the information goes. After an inner join, 144 calls are not in the table and no column says so. The total of the calls is 144 calls too low, and the report looks complete. After a left join, a count of `isna()` on the new column tells you the same number.

2. [break] A key in another form

```
from pdbook.enrich import enrich
from pdbook.io import load_calls, load_cells, load_plans, load_subscribers

calls, subs, cells, plans = load_calls(), load_subscribers(), load_cells(), load_plans()
national = subs.assign(msisdn=subs["msisdn"].str.replace("+32", "0", regex=False))
print(national["msisdn"].iloc[0], subs["msisdn"].iloc[0])

enriched, report = enrich(calls, national, cells, plans)
print(len(enriched), int(enriched["on_net"].sum()))
print(report)

share = enriched["on_net"].mean()
print(round(float(share), 3))
try:
    assert 0.2 < share < 0.4, f"on-net share {share:.3f} is outside the range 0.2 to 0.4"
except AssertionError as error:
    print("AssertionError:", error)
```

```
0427521358 +32427521358
20000 0
{'caller -> subscribers': 0, 'cell_id -> cells': 0, 'plan -> plans': 0}
0.0
AssertionError: on-net share 0.000 is outside the range 0.2 to 0.4
```

The call runs and returns 20,000 rows. No call is on-net, and the report has three zeros. The report has no entry for the callee, because an external number is correctly unmatched. The function cannot tell a wrong form from a normal external number. Nothing failed, and the result is wrong.

The check in the block is on the result and not on the join: the share of on-net calls is outside the range 0.2 to 0.4. The range comes from knowing the data. In the chapter the share was 29.9%.

A check of this kind is a statement of what you know about the data. It needs a number that you can defend. Revise it when the business changes.

A second, stronger check compares the form of the keys before the join: every `msisdn` and every `callee` must match one regular expression (`^\+32\d{9}$`). That check fails on the national form at once, and it needs no assumption about the share.

3. [judge] Inner or left

Position. Use a left join. Report the calls that have no subscriber as a separate output of the job, with their count and their duration.

A call without a subscriber is a fact about the network. The network carried it, and someone must explain it: a SIM that was removed from the subscriber table, a test number, or a key in a wrong form. An inner join turns the fact into a missing row. If 144 calls fall out, nobody sees it, and the revenue is lower by an amount that nobody computes.

Strongest objection. The billing job has no price for a call without a plan. Every step after the join must then handle a row with `NaN` in the price column. The inner join removes that burden. The left join moves the problem to the next step. That step can fail, or it can sum a `NaN` as zero.

What changes this position. Put the split at the join: `matched = joined[joined["plan"].notna()]` goes on to billing, and `unmatched` goes to a file that is mailed to a person. Then the billing code sees only matched rows, as with the inner join, and the unmatched rows have an owner. A job that does this needs one more output and gains one number each night: the count of unmatched calls. A rise in that number is the alarm.

Solutions for Chapter 10

Exercises: `../10.md`. Chapter: Pivot and melt.

1. [measure] The same grid in two zones

```
from pdbook.io import load_calls
from pdbook.traffic import traffic_grid

calls = load_calls()
local = traffic_grid(calls, tz="Europe/Brussels")
utc = traffic_grid(calls, tz="UTC")

for name, grid in [("local", local), ("utc", utc)]:
    by_hour = grid.sum(axis=0)
    print(name, int(by_hour.idxmax()), int(by_hour.max()))
print(int((local.to_numpy() != utc.to_numpy()).sum()), "of", local.size)
print(int(local.to_numpy().sum()), int(utc.to_numpy().sum()))
```

```
local 15 1441
utc 13 1441
164 of 168
20000 20000
```

The busiest hour is 15 in Brussels and 13 in UTC, and both have 1,441 calls. The two grids hold the same 20,000 calls and the same peak. They differ in 164 of 168 cells, because every call moves by two hours. September has summer time, so Brussels is UTC+2. The four cells that do not differ are chance: in each of them, the count two hours earlier is equal to the count of the cell itself.

A report that says "the peak is at 13:00" is wrong for a person in Brussels by two hours. The zone is part of the number. Write it in the title of the table and in the name of the column.

2. [break] A duplicate row

```
import pandas as pd

from pdbook.io import load_calls
from pdbook.traffic import WEEKDAYS, to_long, to_wide, traffic_grid

grid = traffic_grid(load_calls())
long = to_long(grid)
dirty = pd.concat([long, long.head(1)], ignore_index=True)
print(len(long), len(dirty))

try:
    to_wide(dirty)
except ValueError as error:
    print("to_wide: ValueError:", error)

summed = dirty.pivot_table(index="weekday", columns="hour", values="calls",
aggfunc="sum").reindex(WEEKDAYS)
print(int(summed.to_numpy().sum()), int(grid.to_numpy().sum()))
print(int(summed.loc["Mon", 0]), int(grid.loc["Mon", 0]))
```

```
168 169
to_wide: ValueError: Index contains duplicate entries, cannot reshape
20016 20000
32 16
```

`to_wide` raises `ValueError: Index contains duplicate entries`, because the cell Monday at 00:00 now occurs twice. `pivot_table` with `aggfunc="sum"` does not raise. It adds the two values: the cell Monday 00:00 shows 32 and not 16, and the table total is 20,016 and not 20,000.

The two functions make a different promise. `pivot` promises that each cell occurs once, and it stops when that is false. `pivot_table` promises a number for each cell, and it makes one by aggregating. For a count table, "sum" is the right aggregate for a legitimate duplicate (two files of the same day), and the wrong aggregate for a copy of one row. The code cannot tell the two cases apart. Use `pivot` after a step that must give unique keys, so that a copy is a stop and not a wrong total.

3. [judge] The format of a report

Position. Two formats. Send the long form to the dashboard team as the feed. Keep the wide grid for the Monday report.

The dashboard team reads the feed with a program. A program reads long data easily: one row has the same columns every night, and a new column, for example `roaming`, is a new field that no consumer must change for. A wide feed has 24 columns that are hours. The meaning is in the column names, and a new dimension needs a new set of columns, and every consumer must change. The person who reads the Monday report wants to compare the hours with the eye, and a long table of 168 rows prevents that.

Strongest objection. Two formats are two things to keep correct. When the rule for the zone or for the hour changes, someone changes it twice and forgets one. The functions `to_long` and `to_wide` exist because the two forms are one

dataset.

What changes this position. Build the Monday report from the feed. Then the long form is the only source, and the wide form is a view made by `pivot_table`. The round trip test in the chapter shows that the conversion loses nothing. This design has one set of numbers and two presentations, which is what the objection wants.

Solutions for Chapter 11

Exercises: `../11.md`. Chapter: The index and MultiIndex.

1. [measure] The repeated label

```
from pdbook.io import load_calls

calls = load_calls()
same = calls[calls.duplicated(subset=["caller", "start"], keep=False)]
print(same[["call_id", "caller", "start", "status", "callee"]].to_string(index=False))
```

```
 call_id caller                     start   status       callee
   12699  S0069 2026-09-19 18:29:11+00:00 answered +32496811868
   12700  S0069 2026-09-19 18:29:11+00:00 answered +32427080945
```

Two calls of the subscriber `S0069` start in the same second, 18:29:11 UTC on 19 September. They have consecutive `call_id` values, 12699 and 12700, and different callees, and both were answered. Two calls from one number in one second are possible, for example a conference service or a call forwarded in two legs. They are not an error in the file.

The consequence is for the key. The pair `(caller, start)` looks like a key, and for 19,998 of 20,000 rows it is one. A `set_index(["caller", "start"])` followed by `loc` for a pair gives two rows for this pair. Code that expects one row breaks here, and only on this pair.

2. [break] A ratio with a missing hour

```
from pdbook.io import load_calls

calls = load_calls()
hour = calls["start"].dt.tz_convert("Europe/Brussels").dt.hour
roaming = calls[calls["roaming"]]
h = hour[calls["roaming"]]

n_all = roaming.groupby(h).size()
n_failed = roaming[roaming["status"] == "failed"].groupby(h[roaming["status"] ==
"failed"]).size()
print(len(n_all), len(n_failed))

plain = n_failed / n_all
print(int(plain.isna().sum()), len(plain))

fixed = n_failed.reindex(n_all.index, fill_value=0) / n_all
print(int(fixed.isna().sum()), int((fixed == 0).sum()))

full = fixed.reindex(range(24))
print(int(full.isna().sum()), full.dropna().round(3).max())
```

```
23 15
8 23
0 8
1 0.5
```

Roaming calls started in 23 of the 24 hours, and failed roaming calls in 15 hours. The plain division aligns the two counts on the union of their labels, which is 23 hours. In 8 of them, the numerator has no label, so the result is `NaN`. These 8 hours had roaming calls and no failure, and the right value is 0. A reader of the table sees `NaN` and reads "unknown".

The fix is `reindex(n_all.index, fill_value=0)` on the numerator: a missing count of failures, in an hour that has a count of calls, is 0. After the fix, no hour among the 23 is `NaN`, and 8 are 0. The last step puts the result on all 24 hours. One hour has no roaming call at all, so the share has no meaning, and `NaN` is the right value there.

The highest share is 0.5. It is for an hour with a small count, so read it with the count. A share of 1 failure in 2 calls is not a rate that you can compare with a share from 400 calls.

3. [judge] An index for ten million rows

Position. Do not set the index. Use `isin` on the 50,000 ids, or a `merge` with the ids as a left table, in a single operation.

The index has a cost: building a hash table for ten million labels takes time and memory, and you pay it at load time. It pays back when you do many separate lookups. In this job there is one batch of 50,000. A single `isin` or `merge` is one pass over the column. An index is built for one use only.

Strongest objection. The table is read many times in a day by other jobs. The index is built once and used by all. Then the cost is shared, and the lookups are faster than any scan. A service that answers one id at a time must have the index.

What changes this position. Count the lookups over a day. The break-even is the number of lookups at which building the index costs less than the scans that it replaces. The block of Chapter 11 prints the time of 500 single lookups by index and by mask. Divide the build time of the index by the saving of one lookup, and you have the break-even.

A single batch merge wins when you have only one batch. If the ids are not unique, a `merge` and an indexed lookup both return every match. Both need the check from Chapter 9.

Solutions for Chapter 12

Exercises: `../12.md`. Chapter: Dates and resampling.

1. [measure] Complete hours and the busiest hour

```
from pdbook.io import load_kpi
from pdbook.kpi import hourly_kpi

hourly = hourly_kpi(load_kpi())
print(hourly.groupby("cell_id")["complete"].sum().to_dict())

by_hour = hourly.assign(hour=hourly["ts"].dt.hour).groupby("hour")["prb_util_pct"].mean()
print(int(by_hour.idxmax()), round(float(by_hour.max()), 1))
print(by_hour.round(1).loc[[3, 10, 16, 22]].to_dict())
```

```
{'ANT-001': 60, 'ANT-002': 61, 'BRU-001': 60, 'BRU-002': 58, 'BRU-003': 61, 'BRU-004': 54}
16 73.4
{3: 15.6, 10: 39.0, 16: 73.4, 22: 47.5}
```

A cell has at most 72 hours. The number of complete hours is 60 for `ANT-001`, 54 for `BRU-004` and 58 to 61 for the others. The cell `BRU-004` has the fewest, because of its two-hour outage and the gaps around it. In all, 354 of the 432 hours are complete, which is 82%.

The busiest local hour is 16, with a mean of 73.4% of the radio resources. The mean is 15.6% at 03:00, 39.0% at 10:00 and 47.5% at 22:00. The day has one clear peak in the afternoon. The mean is over three days and six cells. Eighteen values give a rough ranking of hours. The difference between 16:00 and the next hour is not a finding from this much data.

2. [break] Rows are not time

```
import pandas as pd

from pdbook.io import load_kpi

kpi = load_kpi().drop_duplicates(["ts", "cell_id"])
cell = kpi[kpi["cell_id"] == "BRU-001"].set_index("ts")
["prb_util_pct"].tz_convert("Europe/Brussels").sort_index()
moment = pd.Timestamp("2026-10-26 12:00", tz="Europe/Brussels")

by_time = cell.rolling("1D").count().loc[moment]
by_rows = cell.rolling(288).count().loc[moment]
print(int(by_time), int(by_rows))

pos = cell.index.get_loc(moment)
window_start = cell.index[pos - 287]
print(window_start, "->", moment)
print(moment - window_start)
```

```
283 288
2026-10-25 11:40:00+01:00 -> 2026-10-26 12:00:00+01:00
1 days 00:20:00
```

The rolling count by time at 12:00 on 26 October is 283. The rolling count by rows is 288, which is a number that you read as "samples in the last day". It is not. The 288 rows end at the same moment and begin at 11:40 on 25 October, so they span 24 hours and 20 minutes. Five 5-minute samples are missing inside the day, and the window must reach back 25 minutes to find 288 rows.

The window of 288 rows has no fixed length in time. It is 24 hours when the data are complete, and it is longer when samples are missing. A rolling mean over 288 rows then mixes more than a day. A window by time (`rolling("1D")`) has a fixed length, and it has a count that varies. That variation is the information: 283 of 288 samples, so the data are 98% complete.

3. [judge] Which zone in the warehouse

Position. Store UTC, and add the local time as a second column that is computed from the UTC column with the zone database. Analysts query the local column.

UTC has one property that nothing else has: every timestamp is a unique instant, in order, with no gaps and no repeats. A table in local time has an hour that occurs twice in October and an hour that does not exist in March. Joins between tables, differences between timestamps and rolling windows are correct in UTC and wrong for one night each year in local time. The local column gives the analysts what they want, and it is derived, so a change in the rules of the zone is a recomputation.

Strongest objection. Two columns are two chances for an analyst to group by the wrong one. A person who groups by the UTC day gets a day that begins at 02:00. The "daily total" is not the day of the business. The fault does not show.

What changes this position. Name the columns so that the wrong choice is hard: `ts_utc` and `ts_local`, and a view for analysts that has only `ts_local` and the date. The UTC column stays in the base table. If all queries are made through the view, the objection loses its force, and the engineers keep the property that they need.

Solutions for Chapter 13

Exercises: `../13.md`. Chapter: Windows, shifts and gaps.

1. [measure] The length of the baseline

```
from pdbook.io import load_kpi
from pdbook.spikes import flag_spikes

kpi = load_kpi().drop_duplicates(["ts", "cell_id"]).reset_index(drop=True)
truth = (kpi["cell_id"] == "BRU-002") & kpi["ts"].between("2026-10-25 14:00", "2026-10-25
14:30")

for window, min_periods in [("1h", 6), ("6h", 50), ("1D", 50)]:
    flags = flag_spikes(kpi, window=window, z=4, min_periods=min_periods)
    print(window, int((flags & truth).sum()), int((flags & ~truth).sum()))
```

```
1h 1 5
6h 4 0
1D 7 0
```

A longer baseline gives a better result. With one hour (12 samples at most, and at least 6), the detector finds 1 of the 7 samples and flags 5 normal samples. With six hours it finds 4 and flags none. With one day it finds all 7 and flags none.

Two explanations fit these numbers. The book did not test them one by one, so read them as hypotheses. First, a short window has few samples, so its standard deviation is itself noisy. A normal sample can be 4 deviations from a mean of 6 values. That fits the 5 false flags.

Second, a short window fills with the spike. After the first spike samples enter the one-hour baseline, the later ones look normal against it. That fits the result of 1 found of 7.

The cost of the long window is the memory of a long past. A cell that changes its normal level at noon is compared with a baseline from yesterday afternoon. The detector then flags the change for hours. Test the window on the changes that you expect, not only on spikes.

2. [break] A baseline that contains the sample

```
import pandas as pd

from pdbook.io import load_kpi

kpi = load_kpi().drop_duplicates(["ts", "cell_id"]).reset_index(drop=True)
truth = (kpi["cell_id"] == "BRU-002") & kpi["ts"].between("2026-10-25 14:00", "2026-10-25
14:30")


def flag(kpi, closed, z):
    flags = pd.Series(False, index=kpi.index)
    for _, g in kpi.sort_values("ts").groupby("cell_id"):
        s = g.set_index("ts")["drop_rate_pct"]
        roll = s.rolling("1D", min_periods=50, closed=closed)
        flags.loc[g.index] = ((s - roll.mean()) / roll.std() > z).to_numpy()
    return flags


for z in (4, 8, 12):
    for closed in ("left", "right"):
        f = flag(kpi, closed, z)
        print(z, closed, int((f & truth).sum()), int((f & ~truth).sum()))
```

```
4 left 7 0
4 right 7 0
8 left 5 0
8 right 3 0
12 left 2 0
12 right 1 0
```

At `z=4` the two forms are the same, 7 found. At `z=8`, `closed="left"` finds 5 and `closed="right"` finds 3. At `z=12`, they find 2 and 1. In all six runs there are no false flags.

A baseline that contains the sample makes the sample look less extreme, because its own value raises the mean and the standard deviation. For a sample that is 25 deviations away from the normal level, the effect is large: the standard deviation of a window that holds the sample is a multiple of the standard deviation without it. The more extreme the sample is, the more it hides itself. This is the reverse of what a detector must do.

The effect is also a reason to be careful with the result of `rolling().mean()` and `rolling().std()` in any "z-score" that you find in a tutorial. Look for `closed="left"`, or a `shift(1)` before the window.

3. [judge] Who is woken at night

Position. Page when the score is above the threshold for 3 samples in a row (15 minutes) or when one sample is above twice the threshold. Send a message, and not a page, for one sample above the threshold.

The data in the chapter give the base of the argument. At `z=4` the detector has no false flag in three days and six cells, which is 5,079 samples. The exercise says that a one-sample spike is common in this kind of data. The sustained spike

of 7 samples is an incident. A rule that needs 3 samples in a row waits 10 minutes more than a rule that pages at once. It removes the one-sample events from the night.

Strongest objection. The first 15 minutes of an incident are the most useful for the engineer, and the rule gives them away. For a cell that serves an emergency number, a delay of 10 minutes is not acceptable. The one-sample spikes are then not noise. They are the first sample of an incident.

What changes this position. Count the incidents of one quarter, and for each, the number of samples above the threshold in the first 15 minutes. If incidents always have 3 samples above the threshold within 10 minutes, the delay is real and small. If many incidents start with a single large sample and then wait, use the "twice the threshold" branch. Set the rule from the data of your own incidents.

Solutions for Chapter 14

Exercises: `../14.md`. Chapter: Ranks, running totals and sessions.

1. [measure] Sessions with more than one call

```
from pdbook.io import load_calls
from pdbook.sessions import session_table, sessionize

calls = load_calls()
table = session_table(sessionize(calls, "30min"))
multi = table[table["calls"] > 1]
print(len(table), len(multi))
print(int(multi["calls"].sum()), round(float(multi["calls"].sum() / len(calls)), 3))
print(table["calls"].value_counts().sort_index().head(4).to_dict())
```

```
18420 1224
2804 0.14
{1: 17196, 2: 988, 3: 163, 4: 43}
```

There are 18,420 sessions, and 1,224 of them have more than one call. Those sessions hold 2,804 calls, which is 14.0% of the calls. The rest, 17,196 sessions, hold one call each. The counts by size are 17,196 sessions of one call, 988 of two, 163 of three and 43 of four.

The count of sessions is a count of "visits". For this generator, with calls that are independent in time, a subscriber with 22 calls in a month rarely calls twice within 30 minutes. A table with retries has a larger share of calls in sessions of more than one call. An example is a subscriber who calls again after a failed call. This number is a first check on whether a data set has such behavior at all.

2. [break] No sort

```
import pandas as pd

from pdbook.io import load_calls

calls = load_calls()
gap = calls.groupby("caller")["start"].diff()
print(int((gap < pd.Timedelta(0)).sum()))

new = gap.isna() | (gap > pd.Timedelta("30min"))
session = new.groupby(calls["caller"]).cumsum()
wrong = int(session.groupby(calls["caller"]).max().sum())
print(wrong)

ordered = calls.sort_values(["caller", "start"])
gap = ordered.groupby("caller")["start"].diff()
new = gap.isna() | (gap > pd.Timedelta("30min"))
right = int(new.groupby(ordered["caller"]).cumsum().groupby(ordered["caller"]).max().sum())
print(right, right - wrong)
```

```
403
18151
18420 269
```

The file order has 403 negative gaps. They come from the late-arriving calls at the end of the file. A late call is compared with the row before it in the file, which is a later call of the same subscriber. The gap is negative, and a negative gap is not larger than 30 minutes, so the late call does not start a session. Its place in time is ignored.

The result is 18,151 sessions against the correct 18,420, which is 269 sessions too few. The wrong code merges sessions that are separate in time. No error is raised, and the table looks normal. A test that sorts the rows before and after, as the test of the chapter does, finds the fault. The test `a.equals(b)` for a shuffled input fails for this code.

3. [judge] The size of the gap

Position. Take the gap from the data. Plot or tabulate the gaps between consecutive calls of one subscriber, on a log scale. Put the threshold in the valley between the short gaps and the long gaps. Write the threshold with the evidence.

The analyst took the 30 minutes from web analytics, a field with another kind of event. A phone call can have a different rhythm. A retry after a failed call can come in a minute. The next independent call can come in hours. The data show where one rhythm ends and the other begins. A fixed number from another field is a guess.

Strongest objection. The valley can be absent. The chapter data have a median gap of 10 hours and no valley at all, because the generator has no bursts. Then every threshold is arbitrary, and a team that cannot agree on one loses time. A standard number is a good tool for agreement.

What changes this position. Run the table of the chapter on your data. If the number of sessions changes by 1% between 20 and 60 minutes, the choice does not matter, so take 30 and move on. If it changes by 20%, the choice decides the result, and the number needs the evidence from the gaps.

Solutions for Chapter 15

Exercises: `../15.md`. Chapter: Copy-on-Write and traps.

1. [measure] Which write reaches the table

```
import warnings

import pandas as pd


def fresh():
    return pd.DataFrame({"x": [1, 2, 3]})


results = {}

df = fresh()
df["x"] = 0
results["df['x'] = 0"] = df["x"].tolist()

df = fresh()
df.loc[df["x"] > 1, "x"] = 0
results["df.loc[mask, 'x'] = 0"] = df["x"].tolist()

df = fresh()
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    df["x"].iloc[0] = 0
results["df['x'].iloc[0] = 0"] = df["x"].tolist()

df = fresh()
sub = df[["x"]]
sub.iloc[0, 0] = 0
results["sub = df[['x']]; sub.iloc[0, 0] = 0"] = df["x"].tolist()

df = fresh()
s = df["x"]
s[:] = 0
results["s = df['x']; s[:] = 0"] = df["x"].tolist()

for name, value in results.items():
    print(f"{name:42s} -> df is {value}")
```

```
df['x'] = 0                                -> df is [0, 0, 0]
df.loc[mask, 'x'] = 0                      -> df is [1, 0, 0]
df['x'].iloc[0] = 0                        -> df is [1, 2, 3]
sub = df[['x']]; sub.iloc[0, 0] = 0        -> df is [1, 2, 3]
s = df['x']; s[:] = 0                      -> df is [1, 2, 3]
```

Only the first two forms change `df`. The first assigns to the table itself. The second writes through `df.loc` with the row selection and the column in one call, and only rows 1 and 2 (the values above 1) change.

The three other forms write to an object that pandas made from `df`: the column `df["x"]`, the derived table `sub`, and the name `s` for the column. Under copy-on-write, each one is a separate object from the first write, so the write does not reach `df`. The form `s[:] = 0` is the sharpest case: it looks like "set the column to 0", and the table does not change.

A rule for reading code: a write changes a table only when the left side of the `=` starts at the table itself. It must name the table, the rows and the columns in one `loc` or in one bracket.

2. [break] A function with a side effect

```
import pandas as pd


def add_share_bad(df):
    df["share"] = df["x"] / df["x"].sum()
    return df


def add_share_good(df):
    return df.assign(share=df["x"] / df["x"].sum())


def strict_caller(df, add):
    assert "share" not in df.columns, "share is already there"
    return add(df)


table = pd.DataFrame({"x": [1, 3]})
for name, add in [("bad", add_share_bad), ("good", add_share_good)]:
    table = pd.DataFrame({"x": [1, 3]})
    first = strict_caller(table, add)
    try:
        strict_caller(table, add)
        second = "second call worked"
    except AssertionError as error:
        second = f"second call failed: {error}"
    print(f"{name:5s} columns of the caller: {table.columns.tolist()} | {second}")
```

```
bad   columns of the caller: ['x', 'share'] | second call failed: share is already there
good  columns of the caller: ['x'] | second call worked
```

The bad function adds `share` to the table of the caller. The second call fails on the check, as the exercise says. The good function returns a new table, so the table of the caller keeps only `x`, and both calls work.

The failure of the bad function is the lucky kind: an `assert` stopped it. The unlucky kind has no check. The second call then runs and replaces `share` with a value that has the same name. Or a later step reads a column that an earlier step was not meant to add. The error is far from its cause.

The change is one line: `return df.assign(...)` in place of an assignment and a `return df`. The alternative is `df = df.copy()` at the top of the function. Both are fine. The `assign` form has the advantage that it reads as an expression and it chains (Chapter 16).

3. [judge] Add a trap

Position. Keep the list short, about ten, give every entry a test, and add an entry only after a wrong result reached someone. The team lead owns the list, and everyone proposes entries with a pair of functions.

A short list is read, and it is used as a checklist in a code review. A trap without a test goes out of date: in this book, the ninth line of the list is a fact about floating point, and the first eight are facts about pandas 3.0. A test is what tells you when a version of pandas fixes one. An entry that comes from a real wrong result has the right priority, because it has already cost something.

Strongest objection. A list that grows only after a failure is a list of past failures. The cost of the first occurrence is paid every time. A longer list that comes from the documentation prevents some of them.

What changes this position. Count the review comments of a quarter. Count how many name a trap that is in the list, and how many name a trap that is not. If most are in the list, it works, and it needs only a few more entries. If most are not, the list has the wrong entries. Look at what the reviewers see.

Solutions for Chapter 16

Exercises: `../16.md`. Chapter: Method chains and pipelines.

1. [measure] Where the revenue is

```
from pdbook.io import load_calls, load_cells, load_plans, load_subscribers
from pdbook.report import monthly_report

calls, subs, cells, plans = load_calls(), load_subscribers(), load_cells(), load_plans()
report = monthly_report(calls, subs, cells, plans)

by_city = report.groupby("home_city")
["revenue_eur"].sum().round(2).sort_values(ascending=False)
print(by_city.to_dict())
worst = report.loc[report["answer_rate"].idxmin()]
print(worst["plan"], worst["home_city"], worst["answer_rate"], int(worst["calls"]))
print(round(float(by_city.iloc[0] / by_city.sum()), 3))

best = report.loc[report["answer_rate"].idxmax()]
se = lambda p, n: (p * (1 - p) / n) ** 0.5
diff = best["answer_rate"] - worst["answer_rate"]
se_diff = (se(best["answer_rate"], best["calls"]) ** 2 + se(worst["answer_rate"],
worst["calls"]) ** 2) ** 0.5
print(best["plan"], best["home_city"], best["answer_rate"], int(best["calls"]))
print(round(100 * se(worst["answer_rate"], worst["calls"]), 1), round(100 * diff, 1),
      round(100 * se_diff, 1), round(float(diff / se_diff), 1))
```

```
{'Brussels': 1647.41, 'Antwerp': 1049.6, 'Liège': 803.75, 'Ghent': 457.36, 'Leuven': 286.97}
business Leuven 0.737 205
0.388
plus Brussels 0.792 1733
3.1 5.5 3.2 1.7
```

Brussels brings 1,647.41 euros, which is 38.8% of the total of 4,245.09. Antwerp follows with 1,049.60, and Leuven, the smallest city, brings 286.97.

The lowest answer rate, 0.737, is for `business` in Leuven. It comes from 205 calls, and its standard error is 3.1 points. The highest rate in the table is 0.792, for `plus` in Brussels, from 1,733 calls. The two differ by 5.5 points.

The standard error of the difference is 3.2 points, so they are 1.7 standard errors apart. With 20 cells in the table, a pair that far apart is likely to occur by chance. Do not report "business in Leuven has a problem" from this table.

2. [break] A plan that is not in the plans table

```
from pdbook.enrich import enrich
from pdbook.io import load_calls, load_cells, load_plans, load_subscribers
from pdbook.report import monthly_report

calls, subs, cells, plans = load_calls(), load_subscribers(), load_cells(), load_plans()
few = plans[plans["plan"] != "plus"]

_, unmatched = enrich(calls, subs, cells, few)
print(unmatched)

report = monthly_report(calls, subs, cells, few)
full = monthly_report(calls, subs, cells, plans)
print(int(report["calls"].sum()), round(float(report["revenue_eur"].sum()), 2),
round(float(full["revenue_eur"].sum()), 2))
print(sorted(report["plan"].unique()))
```

```
{'caller -> subscribers': 0, 'cell_id -> cells': 0, 'plan -> plans': 4475}
20000 3681.87 4245.09
['basic', 'business', 'plus', 'prepaid']
```

The report runs and shows no error. It lists all four plans, and the calls total is 20,000. The revenue is 3,681.87 euros, and the full table gives 4,245.09. The 563.22 euros that are missing are exactly the revenue of the plan `plus`. The `enrich` step counts 4,475 unmatched plans, and the number is in the report that `monthly_report` throws away.

The cause is the `NaN` that `groupby().sum()` skips. The rate of each `plus` call is `NaN`, so its revenue is `NaN`, and the sum of the group is 0. The row for `plus` is in the report with calls and no revenue. A reader who looks at totals sees a lower revenue and has no reason to suspect an error.

The check goes right after the `enrich` call and before the grouping: `.pipe(require, lambda d: d["rate_per_min_eur"].notna().all(), "a call has no price")`. It stops at the first step where the fault exists. A check at the end finds it too, but the message is farther from the cause.

3. [judge] One chain or three functions

Position. Three named functions, and a short chain in the top function that calls them: `enrich(...)`, then `.pipe(add_billing)`, then `.pipe(summarize)`.

The report has three parts with three different owners of change. The billing rule changes when the tariff changes. The enrichment changes when a lookup table changes. The summary changes when a manager asks for a different table.

A function for each part is a unit with its own test and its own docstring. A change in one does not need a review of the others. The top chain keeps the property that the colleague wants. It reads from top to bottom, in three lines.

Strongest objection. Three functions mean that a reader must jump between three places to see the whole. For a ten-step report that one person writes and reads, the jump is a cost with no benefit. Names such as `add_billing` can also hide a side effect.

What changes this position. Count how many people change the file and how often. If one person owns the report and it changes twice a year, keep one chain. If two or three people change different parts, or if a second report uses a part, split it. The reuse test is the strongest: the second time that you copy a block of the chain, it is a function.

Solutions for Chapter 17

Exercises: `../17.md`. Chapter: Data validation.

1. [measure] Is the validator exact for other seeds?

```
from pdbook.io import load_calls, load_cells, load_subscribers
from pdbook.validate import call_rules, plant_faults, validate

calls, subs, cells = load_calls(), load_subscribers(), load_cells()
rules = call_rules(subs, cells)

exact = 0
for seed in range(5):
    broken, expected = plant_faults(calls, seed=seed)
    report = validate(broken, rules)
    found = dict(zip(report["rule"], report["violations"]))
    ok = all(found[rule] == n for rule, n in expected.items())
    exact += ok
    print(seed, ok, [found[rule] for rule in expected])
print(exact)
```

```
0 True [7, 5, 11, 4, 3, 4, 2]
1 True [7, 5, 11, 4, 3, 4, 2]
2 True [7, 5, 11, 4, 3, 4, 2]
3 True [7, 5, 11, 4, 3, 4, 2]
4 True [7, 5, 11, 4, 3, 4, 2]
5
```

All five seeds match, and the seven counts are the same for every seed: 7, 5, 11, 4, 3, 4 and 2. The counts are the same because `plant_faults` chooses a fixed number of rows for each fault, from separate pools. A different seed changes only which rows. The validator does not depend on which rows are broken.

This is a weak form of the test. It shows that the rules are exact for these six faults. It does not show that the rules find faults of other kinds, and the next exercise is one of those. Five seeds on one data set say nothing about a data set with other properties. An example is a data set with many calls that have a duration of 0.

2. [break] A fault that no rule sees

```
import numpy as np
import pandas as pd

from pdbook.io import load_calls, load_cells, load_subscribers
from pdbook.validate import Rule, call_rules, validate

calls, subs, cells = load_calls(), load_subscribers(), load_cells()
rng = np.random.default_rng(0)
answered = calls.index[calls["status"] == "answered"].to_numpy()
rows = rng.choice(answered, size=30, replace=False)

broken = calls.copy()
broken.loc[rows, "duration_s"] = 90_000.0

old = validate(broken, call_rules(subs, cells))
print(int(old.loc[old["severity"] == "error", "violations"].sum()))

too_long = Rule("a call lasts at most 4 hours", lambda d: d["duration_s"] > 4 * 3600)
print(int(validate(calls, [too_long])["violations"].iloc[0]), int(validate(broken, [too_long])
["violations"].iloc[0]))
print(float(calls["duration_s"].max()))
```

```
0
0 30
3836.0
```

The seven rules of `error` severity find no violation for the 30 rows with a duration of 25 hours. Each of the seven rules is true for these rows: the duration is not negative, it is above 0, the status is `answered`, and the codec is present. A value that is valid in type and in sign, and wrong in size, is invisible to a rule that checks type and sign.

The new rule `a call lasts at most 4 hours` finds 0 rows on the clean data and 30 on the broken data. The longest call in the clean data is 3,836 seconds, a little more than an hour. The limit of 4 hours has a margin of more than three times. A limit that is too tight gives false alarms on real long calls. A limit that is too loose, such as 24 hours, misses a fault of 5 hours.

The lesson is about how rules are found. A rule is a statement of what you know about the data. You write the rules that you think of, and the faults that you did not think of go through. The usual source of the next rule is the last incident.

3. [judge] Stop or drop

Position. Stop the job, and send the 7 rows with the message. A billing or a revenue report does not drop rows it cannot explain.

A repeated `call_id` has two possible causes, and the job does not know which one it has. The row can be a duplicate delivery of the same call, and then dropping it is right. The row can be a different call with a colliding id, and then dropping it removes a real call. The seven rows are 0.035% of the data, and the difference to the total is small. The decision is not about the size. It is about whether a number that cannot be explained goes in a report that people trust.

Strongest objection. A report that is late is also a failure, and the people who read it at 07:00 make decisions with no report. A job that stops for every small fault is a job that people learn to restart with the check turned off.

What changes this position. Split the output. Produce the report at 07:00 with the 7 rows excluded and a line at the top, "7 rows with a repeated id are excluded, 0.035% of the calls". Send the 7 rows to the owner of the data. The report is on time, and the fault is visible and has an owner.

The check stays on, and the job does not stop, because the rule has a defined action: drop and say so. Decide which rules drop and which stop. Write the decision down, with a name against each rule.

Solutions for Chapter 18

Exercises: `../18.md`. Chapter: Performance.

1. [measure] Does the gap grow with the size?

```
from pdbook.bench import best_of, replicate, task_itertuples, task_mask_groupby
from pdbook.io import load_calls

calls = load_calls()
ratios = []
for times in (5, 10):
    data = replicate(calls, times)
    t_loop, _ = best_of(lambda: task_itertuples(data), 1)
    t_col, _ = best_of(lambda: task_mask_groupby(data), 3)
    ratios.append(t_loop / t_col)
    print(len(data), f"loop {t_loop:.3f}s column {t_col:.4f}s ratio {t_loop / t_col:.0f}")
assert min(ratios) > 10 and max(ratios) / min(ratios) < 3
```

```
# output varies
100000 loop 0.589s column 0.0126s ratio 47
200000 loop 1.133s column 0.0224s ratio 51
```

The block prints a ratio for each size. The assertion requires that both are above 10 and within a factor of 3 of each other. This is the point of the exercise. In Chapter 7 the ratio at 20,000 rows is larger than at 2,000 rows, because the column form has a fixed cost.

Here both sizes are far above that fixed cost. Both methods take time in proportion to the number of rows, and the ratio is about constant. The constant is the cost of Python code for one row against the cost of compiled code for one row.

Quote ratios and not seconds when you compare two methods. The seconds belong to one machine at one moment.

2. [break] A loop that is quadratic

```
import pandas as pd

from pdbook.bench import best_of
from pdbook.io import load_calls

piece = load_calls().head(200)
times = {}
for n in (100, 200, 400):
    parts = [piece] * n

    def in_a_loop():
        out = parts[0]
        for p in parts[1:]:
            out = pd.concat([out, p])
        return out

    times[n], _ = best_of(in_a_loop, 3)
    print(n, f"{times[n] * 1000:.0f} ms")
print(f"ratio {times[400] / times[100]:.1f}")
assert times[400] / times[100] > 6
```

```
# output varies
100 41 ms
200 112 ms
400 383 ms
ratio 9.4
```

The block prints the time for 100, 200 and 400 pieces, and the ratio of the last to the first. The assertion requires that the ratio is above 6. A linear cost gives 4 and a quadratic cost gives 16, so a ratio above 6 is a cost that grows faster than linear. The cause is in the code:

each `concat` copies all the rows so far, so the copies add up to 1 + 2 + ... + n pieces, which grows with n squared. A busy machine can change the ratio by a factor of two, so the assertion uses a threshold and not an exact value.

The practical result: the cost is invisible in a test with 10 pieces and it is large in production, because it grows faster than the number of pieces.

3. [judge] Is it worth it

Position. Do not do it now. Write down the cost of a rerun, and decide from that.

The job finishes at 01:30 and nobody reads the output before 08:00, so the 30 minutes have no cost today. Two days of work have a cost. A faster job is a benefit only when something depends on the time: a rerun after a failure that must finish before 08:00, or a deadline that moves, or a data volume that grows.

Strongest objection. The job will grow. If the input doubles each year, 30 minutes become an hour and then two hours. The work becomes urgent at the worst time, after a failure at 05:00.

What changes this position. Measure where the 30 minutes go with a profile before you decide. Suppose that one step takes 25 of the 30 minutes and it is a loop over rows. The rewrite is small, and a large gain is available. Do it then.

Suppose that the time is spread over 40 steps. The two days give a small gain, and the answer is no. Decide from the profile and not from the total.

Solutions for Chapter 19

Exercises: `../19.md`. Chapter: Data larger than memory.

1. [measure] A chunk size that does not divide the file

```
import pathlib
import tempfile

import pandas as pd

from pdbook.chunked import make_big_csv, talk_by_caller_chunked, talk_by_caller_whole

with tempfile.TemporaryDirectory() as tmp:
    small = make_big_csv(pathlib.Path(tmp) / "small.csv", times=1)
    head = pd.read_csv(small, nrows=2000)
    head.to_csv(small, index=False)
    large = make_big_csv(pathlib.Path(tmp) / "large.csv", times=20)

    for path, chunk in [(small, 10), (large, 70_000), (large, 400_000)]:
        a = talk_by_caller_chunked(path, chunksize=chunk)
        b = talk_by_caller_whole(path)
        rows = sum(1 for _ in open(path)) - 1
        pd.testing.assert_frame_equal(a, b)
        print(rows, chunk, -(-rows // chunk), "chunks, equal")
```

```
2000 10 200 chunks, equal
400000 70000 6 chunks, equal
400000 400000 1 chunks, equal
```

All three runs give the same table as the whole file, for 200, 6 and 1 chunks. The chunk size of 70,000 does not divide 400,000, so the last of the six chunks has 50,000 rows. The result is not affected.

The first run found a fault in the first version of the function. With chunks of 10 rows, a caller is missing from most chunks. The addition `total.add(part, fill_value=0)` aligns two tables on the caller, and where one table has no row for a caller, pandas works in floats. The count came back as `float64` and the whole-file count was `int64`, so `assert_frame_equal` raised an error on the dtype and not on a value. With chunks of 70,000 rows every caller is in every chunk, and the fault did not show. The function now casts the count back to `int64`.

The lesson is about the test and not about the function. A test with one chunk size passed for a fault that depended on the chunk size. Test a chunked job with a chunk size that is far too small, because small chunks make every edge case happen.

2. [break] The mean of the means

```
import pathlib
import tempfile

import pandas as pd

from pdbook.chunked import make_big_csv, talk_by_caller_whole

with tempfile.TemporaryDirectory() as tmp:
    path = make_big_csv(pathlib.Path(tmp) / "large.csv", times=20)
    chunks = pd.read_csv(path, usecols=["caller", "status", "duration_s"], chunksize=70_000)
    means, sums, counts = [], [], []
    for chunk in chunks:
        answered = chunk[chunk["status"] == "answered"].groupby("caller")["duration_s"]
        means.append(answered.mean())
        sums.append(answered.sum())
        counts.append(answered.size())
    whole = talk_by_caller_whole(path)

wrong = pd.concat(means).groupby(level=0).mean()
right = pd.concat(sums).groupby(level=0).sum() / pd.concat(counts).groupby(level=0).sum()
print(round(float((right - whole["mean"]).abs().max()), 9))
diff = (wrong - whole["mean"]).abs()
print(round(float(diff.max()), 2), int((diff > 1e-6).sum()), len(diff))
print(diff.idxmax(), round(float(whole.loc[diff.idxmax(), "mean"]), 1),
round(float(wrong[diff.idxmax()]), 1))
```

```
0.0
3.94 599 600
S0297 194.5 198.4
```

The correct way matches the whole-file mean to nine decimals: the largest difference is 0.0. The wrong way, the mean of the chunk means, is different for 599 of 600 callers, and the largest difference is 3.94 seconds. For `S0297` the true mean is 194.5 seconds and the mean of means is 198.4.

The mean of the means gives each chunk the same weight, although the chunks contain different numbers of calls from the caller. A chunk with one call counts as much as a chunk with fifty. The error is not large here, 2% for the worst caller, because the chunks have similar counts. With data sorted by time, where a caller is busy in one chunk and quiet in another, it is large. The right rule: keep what you can add (sum and count), and divide at the end.

3. [judge] Chunks or a database

Position. Put the file into a database once, and ask the questions there. Keep the chunked job for one question that is run on a schedule.

The chunked job of the chapter read 170 MB in a few seconds. At that speed a file of 40 GB takes minutes for each question. Every new question is a new job that reads all 40 GB. A database reads the file once and stores it. It can have an index on the columns that you filter by, and then a new question does not read all the rows (Chapter 20 measured this for SQLite). The questions that are not additive (the median, the number of distinct callers, a rolling window) are easy in SQL and hard in chunks.

Strongest objection. A database is a system to install, to keep and to pay for. If the 40 GB file is delivered once and you ask two questions, the afternoon for the chunked job is cheaper. A database also needs the types to be right at the load, which is the work of Chapter 2 again.

What changes this position. Count the questions. One or two questions, or questions that are all sums and counts, use chunks. Three or more different questions, or any question that is not additive, load the data. A file-based engine (the chapter names two) is the middle way: it has no server, and it runs SQL over the file. Chapter 20 uses SQLite, which is in the Python library, for the same idea at a smaller size.

Solutions for Chapter 20

Exercises: `../20.md`. Chapter: SQL and pandas.

1. [measure] Answered calls for each plan

```
import pandas as pd

from pdbook.io import load_calls, load_subscribers
from pdbook.sqlbridge import build_database

conn = build_database()
sql = pd.read_sql_query(
    "SELECT s.plan, COUNT(*) AS n FROM calls c JOIN subscribers s ON s.subscriber_id =
c.caller "
    "WHERE c.status = 'answered' GROUP BY s.plan ORDER BY s.plan", conn).set_index("plan")
["n"]

calls, subs = load_calls(), load_subscribers()
plan = calls["caller"].map(dict(zip(subs["subscriber_id"], subs["plan"])))
pandas = calls[calls["status"] == "answered"].groupby(plan)["call_id"].size().sort_index()
print(sql.to_dict())
print(bool((sql.to_numpy() == pandas.to_numpy()).all()))
```

```
{'basic': 5153, 'business': 1763, 'plus': 3497, 'prepaid': 5113}
True
```

The two counts are equal for every plan. The SQL form joins and groups in the database. The pandas form maps the plan onto the calls and groups the result.

2. [break] A value that changes the query

```
import sqlite3

import pandas as pd

conn = sqlite3.connect(":memory:")
pd.DataFrame({"name": ["O'Neil", "Peeters"], "n": [3, 5]}).to_sql("people", conn, index=False)
value = "x' OR '1'='1"

formatted = pd.read_sql_query(f"SELECT n FROM people WHERE name = '{value}'", conn)
safe = pd.read_sql_query("SELECT n FROM people WHERE name = ?", conn, params=(value,))
print(len(formatted), len(safe))
```

```
2 0
```

With string formatting, the value closes the quote and adds a condition that is always true, so the query returns every row. With `params`, the database compares the whole text with each name and finds none. Use `params` for every value that you did not write yourself, and for every value that you did.

3. [judge] Where does the report live

Position. Keep the SQL query as the report, and keep the pandas chain as the test of it, until the team trusts the query. After that, keep the one that the people who change the report can read.

The two agree today. The value of having both is the test: two separate implementations that give the same 20 rows. Deleting one removes the check.

Strongest objection. Two implementations double the cost of every change to the tariff, and they drift. A test that nobody runs is not a test.

What changes this position. Run the comparison in the nightly job. If it fails once in a quarter for a real reason, it earns its cost. If it never fails and the report changes often, keep one.

Solutions for Chapter 21

Exercises: `../21.md`. Chapter: Plots.

1. [measure] The data inside a figure

```
import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt

from pdbook.io import load_calls
from pdbook.plots import plot_hourly_calls

calls = load_calls()
for tz in ("UTC", "Europe/Brussels"):
    ax = plot_hourly_calls(calls, tz=tz)
    heights = [int(p.get_height()) for p in ax.patches]
    print(tz, heights.index(max(heights)), max(heights), sum(heights))
    plt.close(ax.figure)
```

```
UTC 13 1441 20000
Europe/Brussels 15 1441 20000
```

The tallest bar is at hour 13 with 1,441 calls in UTC, and at hour 15 with 1,441 calls in Brussels. The two figures hold the same 20,000 calls, and the same height for the tallest bar. The bars are two hours apart. That is the offset of Brussels in September.

The conclusion for a reader is different. "The peak of the day is 13:00" is false for a person in Brussels, and the first figure shows it with a correct label for UTC. A reader who sees only the shape will not see the difference. The zone is in the label for this reason.

2. [break] A figure that lies

```
import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt

from pdbook.io import load_calls

calls = load_calls()
wrong = calls["start"].dt.hour.value_counts().reindex(range(24), fill_value=0).sort_index()
ax = wrong.plot.bar()
ax.set_xlabel("hour of the day (Europe/Brussels)")
heights = [int(p.get_height()) for p in ax.patches]
print(heights.index(max(heights)), max(heights), ax.get_xlabel())
plt.close(ax.figure)
```

```
13 1441 hour of the day (Europe/Brussels)
```

The figure shows hour 13 as the busiest, under a label that says "Europe/Brussels". The correct hour is 15. The code ran without an error, the figure is complete, and the 24 bars add up to 20,000. A test of the total passes.

The fault is in the connection between the label and the data. The label is a string, and nothing links it to the conversion. The remedy is structural: make the function take the zone as one argument and use it for both the conversion and the label, as `plot_hourly_calls` does. Then a wrong label cannot occur by itself.

3. [judge] Test or look

Position. Do both, and put each where it is cheap. The numbers go in the test that runs every night. The look happens once when the figure is made or changed, and again when a library is upgraded.

The tests catch what a change in the table code breaks: a wrong total, a missing hour, a lost series. They are fast and they run for ever. They cannot catch a label that is wrong for the data, as exercise 2 shows, or a figure that is unreadable.

Strongest objection. "Look once" does not scale. A report with 40 figures that changes every month needs a person who looks at 40 figures. In practice no one does, and the figure that is wrong goes out.

What changes this position. Write down, for each figure, the question it answers and the number that you expect for one known case. The chapter does this for the 7 flagged samples. Then the look is a comparison with a number. It takes seconds, and it can become part of the review of a change.

Solutions for Chapter 22

Exercises: `../22.md`. Chapter: JSON and nested records.

1. [measure] The shape of the traces

```
from pdbook.traces import call_summary, events_table, read_traces

records = read_traces()
events = events_table(records)
summary = call_summary(events, [r["call_id"] for r in records])

print(events["type"].value_counts().to_dict())
print(int(summary["ring_ms"].notna().sum()), float(summary["ring_ms"].median()))
```

```
{'RESPONSE': 1394, 'INVITE': 492, 'ACK': 383, 'BYE': 383}
430 927.5
```

The events have four types. There are 492 `INVITE` events, one for each trace that has any events, and 1,394 `RESPONSE` events. The 383 `ACK` and 383 `BYE` events are the same 383 calls that were established. 430 calls have a ringing response (`180`), and the median time from the `INVITE` to ringing is 927.5 ms.

The counts hold together, which is the check: 492 invitations is 500 traces minus the 8 traces that have no events, and 383 acknowledgements is the number of calls with a `200`.

2. [break] explode counts a call with no events as one

```
import pandas as pd

from pdbook.traces import call_summary, events_table, read_traces

records = read_traces()
ids = [r["call_id"] for r in records]
summary = call_summary(events_table(records), ids)

exploded = pd.DataFrame(records).explode("events")
wrong = exploded.groupby("call_id").size()
print(len(exploded), int(exploded["events"].isna().sum()))
print(int((wrong != summary["n_events"].reindex(wrong.index)).sum()))
print(wrong[summary["n_events"].reindex(wrong.index) == 0].unique().tolist())

right = exploded.dropna(subset=["events"]).groupby("call_id").size().reindex(ids,
fill_value=0)
print(bool((right.to_numpy() == summary["n_events"].reindex(ids).to_numpy()).all()))
```

```
2660 8
8
[1]
True
```

The exploded table has 2,660 rows, 8 of which have a missing event: the 8 calls with an empty list. The size of each of those calls is 1, not 0, and it is 1 for all 8. Those 8 calls are the whole difference between the wrong sizes and the summary: the second line is 8.

`size` counts rows, and the row of an empty list is a row. The fix is to drop the rows with a missing event before the count (`dropna(subset=["events"])`) and to put the calls back with `reindex(ids, fill_value=0)`. The last line is `True`: the repaired counts equal the summary for every call.

The trap is general. A function that counts rows counts the placeholder that `explode` makes. Use `count()` on a column that is never missing in a real event, such as `t_ms`, or drop the placeholders first.

3. [judge] Flatten at load or keep it nested

Position. Keep the raw JSON lines as the source of record, and build the flat tables from them in a job that you can rerun. Give the analyst the flat tables, and give the engineer the raw file.

The structure of an event changes, and a flat table is a decision about which fields to keep. Suppose that you flatten at load and throw the JSON away. A new field is lost until you change the loader. The past nights are lost for that field. If you keep the raw file, a new field needs one change in the job and a rerun over the nights that you need.

Strongest objection. Two copies of the data need a rule for which one is true. The flatten job is code, and it can have a fault. An analyst who queries the raw JSON in each query repeats the work. The repeated work can differ between queries.

What changes this position. Test the flatten job as in this chapter: compare it with a source that you trust, in both directions. If the check finds no difference for a month, the flat table is a safe view of the raw file. If the structure changes more than once a month, watch the cost of a rerun. Then flatten only what the analysts use, and keep the rest nested as one JSON column.

Solutions for Chapter 23

Exercises: `../23.md`. Chapter: Features and evaluation.

1. [measure] A cutoff in the wrong zone

```
import pandas as pd

from pdbook.features import TZ
from pdbook.io import load_calls

calls = load_calls()
brussels = pd.Timestamp("2026-09-22", tz=TZ)
utc = pd.Timestamp("2026-09-22", tz="UTC")
before_brussels = int((calls["start"] < brussels).sum())
before_utc = int((calls["start"] < utc).sum())
print(brussels.tz_convert("UTC"), utc)
print(before_brussels, before_utc, before_utc - before_brussels)
between = calls[(calls["start"] >= brussels) & (calls["start"] < utc)]
print(len(between), between["start"].min().isoformat(), between["start"].max().isoformat())

local_hour = calls["start"].dt.tz_convert(TZ).dt.hour
busy = int(local_hour.isin([15, 16]).sum())
print(busy, round(busy / 30))
```

```
2026-09-21 22:00:00+00:00 2026-09-22 00:00:00+00:00
13905 13913 8
8 2026-09-21T22:01:27+00:00 2026-09-21T23:55:59+00:00
2761 92
```

The Brussels cutoff is 22:00 UTC on 21 September, and the UTC cutoff is 00:00 UTC on 22 September, two hours later. 13,905 calls are before the Brussels cutoff and 13,913 before the UTC one, so 8 calls fall between them. They are the calls from 22:01 to 23:55 UTC on 21 September. In Brussels that is 00:01 to 01:55 on 22 September.

Eight calls in 20,000 is a small error because the hours after midnight are quiet: about 100 calls an hour in the whole month (Chapter 21), which is about 3 an hour on one day. A cutoff at 15:00 has a larger error: the hours 15 and 16 hold 2,761 calls in the month, which is about 92 for one day.

The size of the error depends on the hour of the cutoff. The cutoff in the wrong zone is also a leak: 8 calls that belong to the future are in the features.

2. [break] A feature that looks safe

```
import pandas as pd

from pdbook.features import assert_no_leak
from pdbook.io import load_calls, load_subscribers

calls, subs = load_calls(), load_subscribers()["subscriber_id"]
cutoff = "2026-09-22"


def recent_share(calls, subscribers, cutoff):
    limit = pd.Timestamp(cutoff, tz="Europe/Brussels")
    total = calls.groupby("caller").size()
    before = calls[calls["start"] < limit].groupby("caller").size()
    share = (before / total).reindex(pd.Index(subscribers, name="caller"))
    return pd.DataFrame({"recent_share": share})


print(recent_share(calls, subs, cutoff).describe().round(3).loc[["mean", "min",
"max"]].to_dict())
try:
    assert_no_leak(lambda c, s: recent_share(c, s, cutoff), calls, subs, cutoff)
    print("passed")
except AssertionError as error:
    print("AssertionError:", error)
```

```
{'recent_share': {'mean': 0.697, 'min': 0.308, 'max': 1.0}}
AssertionError: features change when the future changes (future removed)
```

The feature fails the test on the first change, the removal of the future. It uses the total of the calls of each subscriber in the whole month as its denominator, so the future is inside every value. The share looks harmless. It is a number between 0.31 and 1.0 with a mean of 0.70. A person who reads the code sees the filter `start < limit` on the numerator.

The leak is in the denominator. The numerator is the past and the denominator is the past plus the future. For a subscriber with many future calls the share is lower, so the feature carries the answer. This is the common form of a leak: a ratio, a rank or a z-score that is computed over the whole table. Compute every statistic that uses a group of rows on the past rows only.

3. [judge] How many tasks

Measure the two designs at the same cost, 450 runs for each configuration. The first is the data that you have: 150 tasks with 3 seeds. The second cannot be run, because there are only 150 tasks.

So the block uses one seed of the 150 tasks and projects the standard error to 450 tasks. The projection divides by the square root of 3. It is correct when the new tasks vary as these do.

```
import numpy as np

from pdbook.evals import load_runs

runs = load_runs()
three = runs.groupby(["task_id", "config"])["passed"].mean().unstack()
d3 = three["improved"] - three["baseline"]
one = runs[runs["seed"] == 0].set_index(["task_id", "config"])
["passed"].astype(float).unstack()
d1 = one["improved"] - one["baseline"]

se_150_tasks_3_seeds = d3.std(ddof=1) / np.sqrt(len(d3))
se_150_tasks_1_seed = d1.std(ddof=1) / np.sqrt(len(d1))
se_450_tasks_1_seed = se_150_tasks_1_seed / np.sqrt(3)
print(round(float(se_150_tasks_3_seeds), 4), round(float(se_150_tasks_1_seed), 4),
round(float(se_450_tasks_1_seed), 4))
print(round(float(d3.var(ddof=1)), 3), round(float(d1.var(ddof=1)), 3))
```

```
0.0308 0.0523 0.0302
0.143 0.41
```

The paired difference has three standard errors in the block. It is 0.0308 for 150 tasks with 3 seeds and 0.0523 for 150 tasks with 1 seed. It is a projected 0.0302 for 450 tasks with 1 seed. At the same cost, 450 runs for each configuration, the two designs are the same to the second decimal, 0.031 and 0.030.

The variance of the per-task difference explains it: 0.143 with three seeds and 0.41 with one. One seed has about three times the variance. Three times as many tasks cancel it.

Position. The standard error does not decide this. Choose the design by what else the tasks give you. 450 tasks cover more of the cases that you care about, and they allow a breakdown by category with more than 30 tasks each. 150 tasks with 3 seeds tell you more about the stability of one task. If the system is deterministic, seeds give nothing, and 450 tasks is the only choice.

Strongest objection. 450 tasks cost more to write than 150, and the cost of a task is not the cost of a run. At equal cost of runs, the tasks can differ in the cost of people. A task of lower quality adds noise that a seed does not add.

What changes this position. Count the cost of a task in hours of a person, and the cost of a run in money. If a task takes a day and a run takes a cent, buy seeds. If tasks come from a log that you already have, buy tasks. Check that the claim "the same standard error" holds for your system: compute the variance of the per-task difference with one seed and with three, as the block does.

Solutions for Chapter 24

Exercises: `../24.md`. Chapter: Field guide.

1. [measure] Your own list of mistakes

There is no single answer. This is the form. It has one case from this book that is not in the chapter, so that you can check your own blocks against it:

```
import numpy as np
import pandas as pd

s = pd.Series([100, 100], dtype="int8")
rows = [
    ("sum of two int8 values", "200", int((s + s).iloc[0])),
    ("sum() of the same column", "200", int(s.sum())),
    ("mean of the int8 column", "100.0", float(s.mean())),
]
for claim, expected, got in rows:
    print(f"{claim:28s} expected {expected:>6s}  got {got}")
```

```
sum of two int8 values       expected    200  got -56
sum() of the same column     expected    200  got 200
mean of the int8 column      expected  100.0  got 100.0
```

The first line is the wrap of Chapter 4: the sum of two `int8` values is computed in `int8`. The second line shows that `sum()` adds in a wider type. The third shows that `mean` does too. One column gives three results from three methods, and only one of them is wrong. A test with a known answer for each method finds it.

2. [break] A trap that no longer exists

```
from dataclasses import replace

from pdbook.traps import TRAPS, live

trap = TRAPS[0]
broken = replace(trap, correct=trap.naive)
print(trap.name)
print(live(trap), live(broken))
print(sum(live(t) for t in TRAPS), len(TRAPS))
```

```
a Series adds by label, not by position
True False
10 10
```

The first trap is live: the naive code and the intended code give different results. The copy with the same function for both is not live, as it must be. The count over the ten original traps is 10 of 10.

What it means for a trap list: a trap that is not live is a sentence in the book that is no longer true. After an upgrade of pandas, run `live` for every trap. A trap that turns `False` has one of two causes.

The library fixed the behavior, and then you remove the trap or move it to the history. Or your two functions drifted, and then you repair the test. Do not leave a trap in the list because it was true once. The example module of Chapter 15 stops with a message that names the traps that are not live, for this reason.

3. [judge] The order of reading

Position. Read Chapter 2 (input), Chapter 5 (missing data) and Chapter 9 (joins), in this order, and keep Chapter 15 (the traps) open as a checklist.

Those three produce wrong totals without an error, and the book measured the size of each. A phone number is read as an integer. A `groupby` drops 277 of 20,000 calls from a report. Ten duplicate keys in a lookup add 284 rows. A wrong total is the most expensive result in a report. The other chapters add speed, convenience and structure, and a colleague can look them up when the need comes.

Strongest objection. A colleague first selects and groups, and Chapters 3 and 8 teach those. A reader who skips them writes the code of Chapters 2, 5 and 9 without the vocabulary.

What changes this position. Ask what the colleague does first. If the work is a time series (a KPI, a rate by hour), put Chapter 12 in the place of Chapter 9. If it is a model, put Chapter 23 first. Choose the three chapters that cover the first wrong total that the colleague can produce.